





VulnLab: Risk-Aware Automated Web Application Vulnerability Assessment System with Scheduled Scanning, Trend Analysis, Smart Scan Modes, and Email-Based Reporting.

By

Ahmed Khalid Abdul Baki Al Malalla

APD3F2511CS(CYB)

A report submitted in partial fulfillment of the requirements for the degree of B. Sc. (Hons) Computer Science (Cyber Security) at Asia Pacific University of Technology and Innovation

Acknowledgement

First and foremost, the developer would like to express his sincere gratitude to his supervisor, Mrs. Noris Binti Ismail, for her continuous guidance, valuable feedback, and support throughout the Investigation Report process. Her insightful comments and constructive suggestions have greatly assisted the developer in improving the quality of this report. The developer is especially thankful for her dedication, availability, and encouragement, which played a crucial role in ensuring the successful completion of this Investigation Report.

Secondly, the developer would like to express his deepest appreciation to his parents, as well as his siblings for their unwavering support, encouragement, and guidance throughout this journey. Their constant motivation, advice, and belief in the developer provided him with the strength and determination to overcome challenges faced during the development of this report.

Thirdly, the developer would like to extend his utmost appreciation to the Final Year Project manager, Ts. Umapathy Eaganathan, for providing clear guidance and regular briefings throughout all stages of the Final Year Project. His structured explanations and well-organized briefings helped the developer gain a clear understanding of the project requirements and milestones, making the overall Investigation Report process smoother and more manageable.

Lastly, the developer would like to express his sincere thanks to all the participants who took part in the survey conducted for this project. Their willingness to participate and share their opinions contributed valuable insights and data, which were essential in supporting the analysis and objectives of this study. The developer greatly appreciates their time and cooperation, as their contributions significantly enhanced the quality and relevance of this project.

Abstract

Web application vulnerabilities continue to pose a significant security risk, particularly for small organisations and individual users who often lack access to advanced security assessment tools. This project presents the design, implementation, and evaluation of VulnLab, a continuous, risk-aware automated web application vulnerability assessment system aimed at improving the accessibility of security testing. The work was motivated by the limitations of existing tools, which survey respondents identified as complex, expensive, and difficult to interpret without specialist expertise. A survey of 30 respondents was conducted to establish user requirements, which informed the system design. The implemented system integrates automated vulnerability scanning built on OWASP ZAP, a rule-based risk-aware scoring engine that converts coarse severity labels into a continuous 0–10 score using exploitability and exposure signals, three smart scan modes (quick, deep, and targeted), scheduled unattended scanning, scan comparison for tracking security posture over time, an AI explanation layer that translates findings into plain language, and email-based reporting. The system was evaluated using three complementary testing techniques: white-box testing of internal logic across six backend modules, black-box testing of the REST API and user interface across eight features, and user acceptance testing with three participants representing the target user groups. All designed test cases returned a Pass result, and UAT participants completed all twelve scenarios unaided, rating the system's clarity, prioritisation, and trustworthiness between 4 and 5 out of 5. The results indicate that risk-aware prioritisation combined with plain-language explanation measurably improves the accessibility of vulnerability assessment for non-expert users. This project aligns with **Sustainable Development Goal (SDG) 9: Industry, Innovation and Infrastructure**, by promoting secure and innovative digital infrastructure.

Keywords — Web Application Security, Vulnerability Assessment, Risk-Aware Scanning, Automation, Trend Analysis, Cybersecurity

Table of Contents

Acknowledgement 2

Abstract 2
List of Figures 8
List of Tables 11
Chapter 1: Introduction 12
1.1 Introduction 12
1.2 Problem Background 13
1.3 Project aim 15
1.4 Objectives 15
1.5 Scope 16
1.6 Potential benefits 17
1.7 Overview of the Report 17
1.8 Project Plan 20
Chapter 2: Literature review 22
2.1 Domain Research 22
2.1.1 Cybersecurity 22
2.1.2 Weaknesses in Web Applications 24
2.1.3 Technologies Used to Detect Vulnerabilities 30
2.1.4 Vulnerability Detection, Scoring, and Trend Analysis 37
2.1.5 Mechanisms for Continuous Monitoring 41
2.1.6 Risk-Aware Prioritization 43
2.2 Similar Systems / Works 44
2.2.1 Burp suite 44
2.2.2 Nessus 48
2.2.3 OWASP ZAP 50
2.2.4 Comparing the systems 54
2.3 Technical Research 55
2.3.1 System Requirement Analysis 55
2.3.2 Hardware Requirements 56
2.3.3 Software Requirements 57
2.3.4 System Architecture Reflections. 59
Chapter 3: Methodology 60

3.1.1 System Development Methodology 60
3.1.2 Waterfall Model 61
3.1.3 Scrum Model 65
3.1.4 Spiral Model 69
3.1.5 Comparison of Models 73
3.1.6 Selected Methodology and Justification 74
3.2 Data Gathering Design 76
3.2.1 Data Gathering techniques 76
3.2.2 Survey questions 77
3.3 Analysis 90
Chapter 4: Design and Implementation 104
4.1 Introduction 104
4.2 Design 105
4.2.1 Design Overview and Principles 105
4.2.2 System Architecture 107
4.2.3 Backend Module and Component Design 108
4.2.4 Frontend Application Design 110
4.2.5 Use Case Diagram and Use Case Specifications 112
4.2.6 Scan Mode Design 114
4.2.7 Risk-Aware Scoring Design 115
4.2.8 AI Explanation Layer Design 117
4.2.9 Scheduling Design 118
4.2.10 Email Reporting Design 120
4.2.11 Sequence Diagrams — Scan Execution Flow 120
4.2.12 Security and Legal Design Considerations 121
4.3 Database Design 122
4.4 Interface Design 124
4.4.1 Authentication Screens (Login.jsx, Signup.jsx) 124
4.4.2 Application Shell (Sidebar.jsx, Layout.jsx) 125
4.4.3 Dashboard (Dashboard.jsx) 125
4.4.4 New Scan (ScanModal.jsx) 125

4.4.5 Scan Results and Detail (Results.jsx, ScanDetail.jsx) 126
4.4.6 Scheduled Scans (Scheduled.jsx, ScheduleModal.jsx) 126
4.4.7 Compare Scans (Trends.jsx) 127
4.4.8 Reports (Reports.jsx) 127
4.4.9 Settings (Settings.jsx) 127
4.4.10 AI Assistant (Assistant.jsx) 128
4.5 Execution 128
4.5.1 Code Screenshots 130
4.6 Screenshot 138
4.7 Summary 151
Chapter 5: Result and Discussion 151
5.1 Introduction 151
5.2 Testing Design 152
5.2.1 White-Box Testing Design 153
5.2.2 Black-Box Testing Design 157
5.2.3 User Acceptance Testing (UAT) Design 161
5.3 System Testing and Discussion 164
5.3.1 White-Box Testing Results 165
5.3.2 Black-Box Testing Results 170
5.3.3 User Acceptance Testing Results 175
5.3.4 Cross-Technique Synthesis 187
5.4 Summary 188
Chapter 6: Conclusion 189
6.1 Critical Evaluation 189
6.1.1 Achievement of the Project Aim and Objectives 189
6.1.2 Contribution to Community, Industry, and SDG 9 190
6.1.3 Strengths of the Project 192
6.2 Limitation 193
6.2.1 Architectural and Scalability Limitations 193
6.2.2 Feature Completeness Limitations 194
6.2.3 Testing Scope Limitations 195

6.3 Recommendation 195
6.3.1 Recommendations Addressing Architectural Limitations 196
6.3.2 Recommendations Addressing Feature Completeness 196
6.3.3 Recommendations Addressing Testing Scope 197
References 199
Appendices 203
Project Title Proposal 203
Ethics Form 204
Log Sheets 207
Gantt Chart 208
Respondence Demographic Profile 209

List of Figures

Figure 1 Growth of publicly disclosed vulnerabilities recorded in the National Vulnerability Database 12
Figure 2 Comparison between periodic vulnerability assessments and continuous security monitoring. 14
Figure 3 The three core principles of cybersecurity 22
Figure 4 NIST Cybersecurity Framework. The five core cybersecurity functions: Identify, Protect, Detect, Respond, and Recover. 23
Figure 5 Comparison of OWASP Top 10 (2017) and OWASP Top 10 (2021) 25
Figure 6 SQL Injection Attack Process 26
Figure 7 Cross-Site Scripting (XSS) Attack 27
Figure 8 Broken Access Control 28
Figure 9 CVSS Risk Scoring 29
Figure 10 Web Crawler Architecture 32
Figure 11 Signature-Based Detection Architecture 34
Figure 12 Static Application Security Testing (SAST) Process 35
Figure 13 Comparison of Static and Dynamic Application Security Testing 36
Figure 14 Burp suite logo 44
Figure 15 Burp suite main dashboard 45
Figure 16 Burp Suite Proxy – HTTP History 46
Figure 17 Burp Suite Vulnerability Scanner Report 47
Figure 18 Nessus Logo 48
Figure 19 Nessus Vulnerability Scan Results 49
Figure 20 Nessus Scan Configuration Interface 50
Figure 21 ZAP logo 51
Figure 22 ZAP main dashboard 52
Figure 23 ZAP Automated Scan Interface 53
Figure 24 ZAP Vulnerability Scan Report 53
Figure 25 Waterfall model 62
Figure 26 Scrum model 66
Figure 27 Spiral Model 70
Figure 28 Survey introduction 77
Figure 29 Gender question 78
Figure 30 Age question 79
Figure 31 Educational background question 79
Figure 32 Occupation question 80
Figure 33 Trusting security of websites question 80
Figure 34 Verifying website security question 81
Figure 35 General website security question 81

Figure 36 Use of vulnerability scanner question 82
Figure 37 Challenges faced with vulnerability scanners question 83
Figure 38 If scanners were simpler question 83
Figure 39 Ease of use for scanner question 84
Figure 40 Scanners accessibility question 84
Figure 41 Tools with addition features question 85
Figure 42 Confidence in reading scan reports question 85
Figure 43 Automated email-based scanners question 86
Figure 44 Clear risk scoring question 86
Figure 45 Current tools review question 87
Figure 46 Simplified risk-aware dashboard question 87
Figure 47 Automated scanners question 88
Figure 48 Importance of scheduled scanning question 88
Figure 49 Concerns about automated scanning question 89
Figure 50 Open ended question on improving scanners 89
Figure 51 Survey thank you message 90
Figure 52 Gender analysis 91
Figure 53 Age analysis 91
Figure 54 Educational background analysis 92
Figure 55 Occupation analysis 92
Figure 56 Trusting website security analysis 93
Figure 57 Verifying if a website is safe analysis 94
Figure 58 Websites general security analysis 94
Figure 59 Familiarity with scanners analysis 95
Figure 60 Challenges experienced with scanners analysis 96
Figure 61 If scanners were more simple analysis 96
Figure 62 Rate ease of scanner use analysis 97
Figure 63 Are scanners accessible by non-experts analysis 97
Figure 64 Value of extra features analysis 98
Figure 65 Confidence in scanning reports analysis 98
Figure 66 Can automated email-based reporting improve scanners analysis 99
Figure 67 Importance of clear risk scoring analysis 99
Figure 68 Rating current tools analysis 100
Figure 69 Simplified risk-aware dashboard analysis 101
Figure 70 Automating scanners analysis 101
Figure 71 Importance of scheduling for website security analysis 102
Figure 72 Concerns about automating scanning analysis 102
Figure 73 Respondent feedback highlighting the need for improved usability, automation, and clearer risk prioritization in current vulnerability scanning tools. 103

Figure 74 Respondent suggestion emphasizing the importance of reducing false positives and improving detection accuracy for business logic and API-related vulnerabilities. 103
Figure 75 Respondent opinion stressing the need for better prioritization and reporting features to reduce the workload caused by false positives in vulnerability scanning results. 103
Figure 76 Respondent recommendation focusing on improved contextual validation, enhanced vulnerability detection, and collaborative vulnerability management features. 103
Figure 77 VulnLab System Architecture (traced from the implemented backend) 107
Figure 78 VulnLab Backend Module Dependency Diagram 109
Figure 79 VulnLab Frontend Route Map 110
Figure 80 VulnLab Use Case Diagram 112
Figure 81 Risk-Aware Scoring Decision Flow (scanner/risk_scoring.py) 116
Figure 82 Schedule Row State Machine (schedules.status / schedules.paused) 119
Figure 83 Sequence Diagram of a Scheduler-Triggered Scan 119
Figure 84 Sequence Diagram of the Manual Scan Execution Flow, Including Client Polling 120
Figure 85 VulnLab Entity Relationship Diagram (traced from db.py) 122
Figure 86 risk_scoring.py — score_finding() 131
Figure 87 scanner/zap_client.py — run_scan() 132
Figure 88 scanner/runner.py — _worker() 133
Figure 89 auth.py — password_problem() and require_auth 134
Figure 90 scheduler.py — _check_due_schedules() 135
Figure 91 ai_explainer.py — generate_explanations() 136
Figure 92 db.py — SQLAlchemy models 136
Figure 93 ScheduleModal.jsx — wallClockToEpoch() / epochToWallClock() 137
Figure 94 Trends.jsx — the Compare Scans diff logic 137
Figure 95 Login screen and Sign Up screen, including the live password-policy checklist 138
Figure 96 Application shell showing the sidebar navigation and active-route highlighting 140
Figure 97 Dashboard with summary cards, findings-over-time chart, and top risks panel 141
Figure 98 New Scan modal — configuration state 141
Figure 99 New Scan modal — live progress state 142
Figure 100 Scan Results grid with search, mode filter, and sort 143
Figure 101 Scan Detail page with AI plain-English explanations 144
Figure 102 Additional view of the Scan Results / Scan Detail screen 144
Figure 103 Scheduled Scans list showing pending, running (with progress ring), and paused states 145
Figure 104 Delete-schedule confirmation dialog 146
Figure 105 Compare Scans screen showing Fixed / New / Still-present findings 147
Figure 106 Reports screen with date-range and risk filtering 147
Figure 107 Settings screen, including the timezone selector 148
Figure 108 Emailed vulnerability report received in the test inbox 149
Figure 109 AI Assistant answering a VulnLab-related question 150

Figure 110 PPF 203
Figure 111 Ethics Form Page 1 204
Figure 112 Ethics Form Page 2 205
Figure 113 Ethics Form Page 3 206
Figure 114 Ethics Form Page 4 206
Figure 115 Meeting Log Sheet 207
Figure 116 Project Timeline Gantt Chart 208
Figure 117 FYP poster 211

List of Tables

Table 1 Project Plan Table 21
Table 2 Comparing similar systems 54
Table 3 Comparing Models 73
Table 4 Backend Module Responsibilities 108
Table 5 Frontend Page Components and Their Backend Dependencies 111
Table 6 Selected Use Case Specifications 113
Table 7 Smart Scan Mode Configuration (scanner/scan_modes.py) 114
Table 8 Worked Examples of the Risk-Aware Scoring Algorithm 117
Table 9 Database Table Descriptions (db.py) 123
Table 10 Technology Stack by Component 128
Table 11 Overview of Testing Techniques Applied to VulnLab 153
Table 12 White-Box Test Case Template 154
Table 13 White-Box Test Cases 154
Table 14 Black-Box Test Case Template 157
Table 15 Black-Box Test Cases 158
Table 16 UAT Tester Profiles 161
Table 17 UAT Test Case Template 162
Table 18 UAT Test Scenarios 162
Table 19 UAT Post-Task Feedback Questionnaire 164
Table 20 White-Box Testing Results 165
Table 21 Black-Box Testing Results 170
Table 22 UAT Tester Profiles (as executed) 176
Table 23 UAT Post-Task Feedback Questionnaire Results 186
Table 24 Achievement of Project Objectives 189
Table 25 Respondence Demographic Profile 210

Chapter 1: Introduction

1.1 Introduction

The rapid advancement of digital technology has resulted in increased dependence on web-based software applications in virtually every sector, including finance, healthcare, education, and e-commerce. Web applications, however, create a highly critical infrastructure component, and manage sensitive consumer information, conduct financial transactions, and manage business operations. With that said, web applications have also created a greater opportunity in terms of cyber threats, and the risk of web application vulnerabilities, including SQL injection, cross-site scripting (XSS), authentication mistakes, and misconfigurations, are increasingly being exploited by attackers which leads to, at a minimum, data breaches and loss of a business's financial resources and a loss in consumer confidence in a business's reputation.

This is because the number of publicly reported vulnerabilities has improved over the years. One can observe this trend in the National Vulnerability Database (NVD), which contains entries of Common Vulnerabilities and Exposures (CVE) that are reported every year. Since the CVE data set keeps growing, it is indicative of the rising vulnerability count in the various software systems, which demonstrates the continually growing vulnerability of contemporary digital infrastructures to security threats.

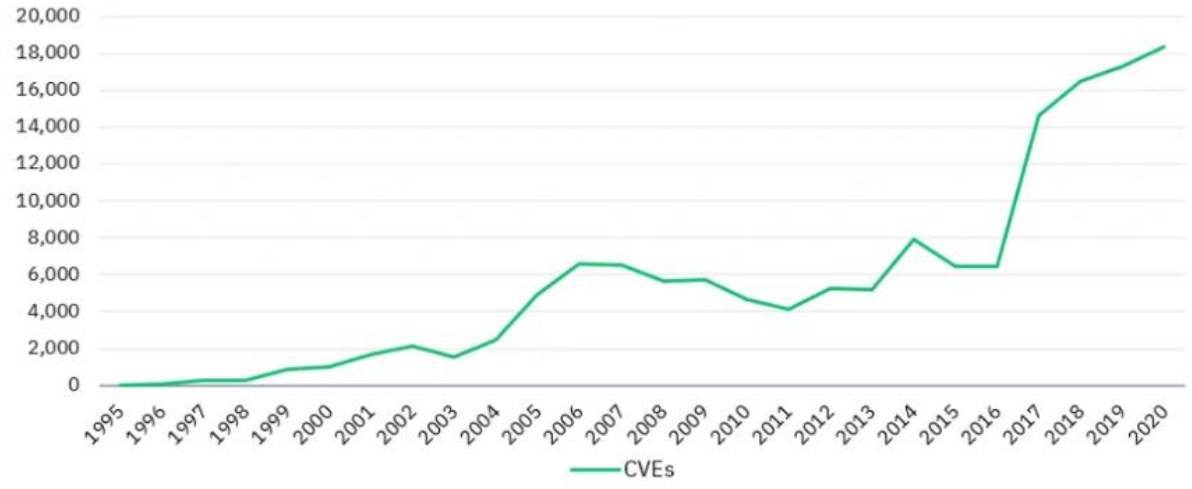


Figure 1 Growth of publicly disclosed vulnerabilities recorded in the National Vulnerability Database

Many traditional vulnerability management processes rely on manual evaluation, expert opinion or static scanners, where even though there are commercial security solutions, most are either too expensive, have no monitoring, or do not offer good risk-aware prioritization solutions for

students, educators, researchers, startups, and small and medium enterprises (SMEs). Moreover, the methodology for cyber risk assessment reports is mostly due to the expertise of the author, rather than using a fully automated, data-driven methodology. (Kia et al., 2024)

This in turn means that a major section of the available vulnerability information is not being reshaped into operational security intelligence because in most cases organizations fail to adequately assess and rank the risks by vulnerabilities identified.

Recent works have proven that cyber risk can be quantified using historical vulnerability data if the data is coupled with a measure of risk, which is defined as the combination of likelihood (i.e. the frequency of the occurrence of the event) and impact (i.e. the severity of the event as defined using CVSS base scores, etc). (Kia et al., 2024)

These methods show the usefulness of combining vulnerability data with risk scoring and predictive analytics to demonstrate leading security risks. The majority of research, however, continues to describe large scale practical risk prediction models or theories rather than establishing practical, continuous, user-friendly vulnerability assessment systems.

Thus, an increasing demand exists for an automated, continuous, and risk-based approach to web application vulnerability assessment systems that incorporate scheduled scanning, configurable scanning intelligence, time-based trend analysis and reporting, and prioritized reporting features. Automated vulnerability detection systems combined with risk scoring and longitudinal analysis are available for organizations, educational institutions, students, researchers, and small and medium-sized enterprises (SMEs) to provide actionable and affordable cybersecurity solutions. This project aims to design and develop a continuous, risk-based automated web application vulnerability assessment system that improves security monitoring and decision support and enhances the nation’s digital immunity.

1.2 Problem Background

As a result of the plethora of security tools made available to the public, as well as vulnerability databases, many organizations, educational institutions, development teams, and individual developers fail to successfully manage vulnerabilities while web applications undergo near-constant change and integration of new features, third-party components (i.e., new security vulnerabilities), and new updates. This adds even more pressure to the vulnerability management cycle, yet security assessments and testing are often performed as a series of discrete, one-off

assessments as opposed to dynamic security monitoring of applications throughout development and after initial deployment. New vulnerabilities introduced through updates go undetected and unmanaged for as long as assessments are not continuously performed.

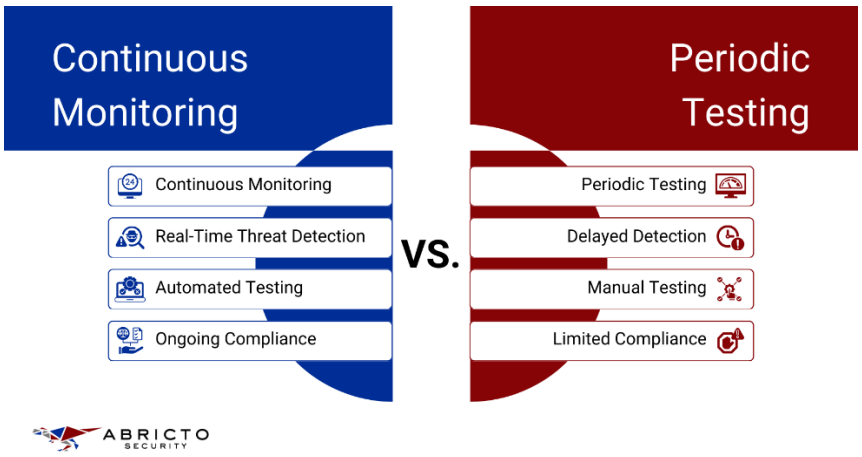


Figure 2 Comparison between periodic vulnerability assessments and continuous security monitoring.

The traditional vulnerability assessment methods are mostly based on manual testing techniques which are occasionally accompanied by periodic penetration testing, or on automated vulnerability scanning tools alone. Although penetration testing may give very reliable findings, it is usually costly and only carried out as a single exercise and thus it is hard to do on a regular basis. Instead, automated scanning tools can produce many findings with a large number of findings without much prioritization. Due to this, the security teams and developers, particularly the less experienced ones, might fail to determine the vulnerabilities that have to be addressed with urgency. This may cause lack of association between the vulnerabilities identified and the real security risk associated with them.

Moreover, current studies on the modeling of cyber risks indicate that data regarding vulnerabilities can be measured and risks can be prioritized using the value of loss models that incorporate the two dimensions of likelihood and impact. (Kia et al., 2024)

Nevertheless, these models are usually applied at a macro scale with CVE datasets, large scale vulnerability data, and predictive models, as opposed to being used at the practical security assessment of individual web applications. Consequently, the trend analysis and longitudinal analysis are rarely considered by many web application developers during application security

assessment. Trend analysis may be used to determine the change in the security posture of an application as time goes by, so that organizations can have a better insight into the change in vulnerability and the risks.

The need for adaptive scanning is another problem that is apparent. While deep scanning can be very resource intensive, quick scans provide coverage and depth for a lengthy period of time. Security is often a balance for less negligence. Companies often do not receive guidance from staff to control the depth of the scan and to balance the performance. In the absence of intelligent scan modes or scanning settings, security analysis can be overly default, or, in a productive sense, almost no analysis can be performed.

Reporting on vulnerabilities is often very technical and complex and is not easy to understand for developers who do not have a dedicated security team. There is a need for automated solutions to report vulnerabilities that includes risk-based prioritization and streamlined notifications, such as email notifications, to facilitate quick access to critical issues.

The primary concern this project tackles is the absence of an easily accessible continuous, risk-informed web application vulnerability assessment system that combines automated scanning, risk assessment prioritization, automated scheduling of vulnerability assessments, and trend analysis within one cohesive and user-friendly framework. Closing this gap is crucial for advancing proactive security management, especially for resource-constrained organizations.

1.3 Project aim

To design and implement a continuous, risk-aware automated web application vulnerability assessment system that integrates scheduled scanning, smart scan modes, trend analysis, and email-based reporting to support proactive and prioritized identification of web application security weaknesses.

1.4 Objectives

- To study existing web application vulnerabilities, automated scanning techniques, and risk-based vulnerability assessment models.
- To design and develop a functional automated web application vulnerability scanning system capable of detecting common web vulnerabilities.

- To implement a quantitative risk scoring mechanism that prioritizes identified vulnerabilities based on severity and impact levels.
- To evaluate changes in web application security posture over time through trend analysis of repeated scan results

1.5 Scope

VulnLab is developed based on the local web application which will automate the vulnerability scan of web applications that are legally open to the general public. The system is oriented to the detection of the most popular web application vulnerabilities such as SQL Injection, Cross-site Scripting (XSS), and the absence of security HTTP headers, configuration, and the vulnerability of simple basic authentication systems. The scanner works on an automated crawling that is used to search web pages, web URLs, input fields, and other possible attack points on a target application. Once the crawling process has traced the application structure, a predefined set of pre-established vulnerability tests are run to identify the presence of security weaknesses.

The system has different scanning options (Quick, Deep and Targeted) whereby the user can control the depth of scans as well as the scans area based on the security needs and time available. The other important capability of the system is that the scans can be scheduled automatically according to specific time schedule and the system monitors the vulnerability continuously and without the attention.

VulnLab has also a built-in quantitative risk scoring system that assesses identified vulnerabilities and approximates the possible impact of vulnerabilities on the system. The system creates a comprehensive vulnerability report after every vulnerability assessment and sends email notifications in order to update the users on the results of the scan.

Besides, the system assesses the variability of the security posture of monitored web applications. This is done by performing the simplest trend analysis on the findings of the regular vulnerability tests.

The project's scope does not include advanced exploit development, malware testing, network-level penetration testing, or mobile application testing. VulnLab does not perform automatic exploitation of vulnerabilities beyond proof-of-concept validation and does not offer any sort of intrusion capability. The system is designed for ethical security assessments only. Furthermore, the system offers no long-term cloud-based storage for scanned data, and is primarily designed for testing in controlled and/or authorized environments.

1.6 Potential benefits

Tangible Benefits:

- Offers a local web application vulnerability assessment tool that is functional and able to identify common security vulnerabilities.
- Adopts automated risk-based prioritization to identify critical vulnerabilities which need immediate redressing.
- Provides scheduled scanning and smart scan (quick, deep and targeted) to enhance the flexibility of the assessment.
- Provides real-time e-mail messages with vulnerability reports and threat severity.
- Minimizes the use of manual or single security checks.

Intangible Benefits:

- Enhances active security in web applications development.
- Increases the level of security awareness among developers and small organizations.
- Enhances better decision-making, as vulnerabilities are presented in a prioritized and organized way.
- Provides more resilient digital infrastructure that is in line with **Sustainable Development Goal 9** (Industry, Innovation and Infrastructure).

Target Users:

- Web application developers
- Students, lecturers, and researchers in cybersecurity and software development
- Small and medium-sized firms (SMEs), startups, and small organizations
- Cybersecurity entry-level and junior security analysts
- Organizations and individuals with limited cybersecurity budgets

1.7 Overview of the Report

Chapter 1 provides the context and rationale for the work, discussing the importance of web applications and their security in an increasingly digital environment. It describes how one-off manual assessments fail to reveal newly introduced weaknesses as web applications change over

time. It also sets out the aim, objectives, scope, and potential benefits of the project, which centres on the design and development of VulnLab — a continuous, risk-aware, automated web application vulnerability assessment system featuring scheduled scanning, smart scan modes, and email-based reporting.

Chapter 2 reviews the literature on cybersecurity and web application vulnerabilities. It examines common vulnerability classes such as SQL Injection and Cross-Site Scripting, the technologies currently used to detect them, and the mechanisms available for continuous monitoring. It critically evaluates CVSS-based severity scoring against empirical research on real-world exploitation, establishing the argument that static severity scores do not adequately represent operational risk. The chapter also compares three existing tools — Burp Suite, Nessus, and OWASP ZAP — and concludes with the technical research and requirements analysis that informed the system design.

Chapter 3 describes the research methodology. It begins by comparing the Waterfall, Scrum, and Spiral development models and justifying the selection of Scrum as the most appropriate framework for developing VulnLab. It then presents the data gathering design, including the survey instrument, the ethical considerations observed, and the target respondent profile. The chapter closes with an analysis of the thirty survey responses collected and the user requirements derived from them, which directly informed the design decisions taken in Chapter 4.

Chapter 4 explains the design and implementation of VulnLab. It presents the overall system architecture, the backend module structure, and the frontend application design, together with the use case specifications, database schema, and interface design. It then details the implementation of each major feature: the three smart scan modes, the risk-aware scoring engine that constitutes the project's original technical contribution, the AI explanation layer, the scheduling subsystem, and email-based reporting. Each design decision is traced back to a specific requirement established in Chapters 2 and 3, and the chapter concludes with code extracts and screenshots demonstrating the working system.

Chapter 5 presents the testing and evaluation of the completed system. It describes the design and rationale for three complementary testing techniques: white-box testing of internal logic across six backend modules, black-box testing of the REST API and React interface across eight features, and user acceptance testing conducted with participants representing the target user groups defined in Section 1.6. The chapter reports the results obtained from each technique, discusses what they demonstrate about the system's correctness and usability, and synthesises the findings across all three approaches to assess whether the objectives set out in Section 1.4 were met.

Chapter 6 critically evaluates the project as a whole. It assesses each of the four objectives against the evidence produced in Chapters 4 and 5, and discusses the project's contribution to the community and industry, including its alignment with Sustainable Development Goal 9. It then presents an honest account of the system's limitations — architectural and scalability constraints, gaps in feature completeness, and limitations in the scope of testing conducted — before offering targeted recommendations for future work that address each limitation in turn.

1.8 Project Plan

Week	Activity	Deliverable
Week 1	Project topic confirmation and discussion with supervisor	Approved project topic
Week 2	Literature search and collection of research papers on web vulnerabilities, vulnerability scanners, and cybersecurity frameworks	Initial literature collection
Week 3	Continue literature search and organize collected research papers, datasets, and references	Compiled literature sources
Week 4	Review and analyze selected research papers and vulnerability datasets (CVE, OWASP Top 10)	Literature analysis notes
Week 5	Write Chapter 1 including introduction, problem statement, objectives, scope, and project plan	Draft of Chapter 1
Week 6	Write Chapter 2 sections on web application vulnerabilities and vulnerability scanning technologies	Chapter 2 draft (Part 1)
Week 7	Complete remaining sections of Chapter 2 including risk scoring, detection mechanisms, and trend analysis	Completed Chapter 2
Week 8	Select and justify the System Development Methodology (SDM) and design system architecture	Chapter 3 methodology draft
Week 9	Design system workflow including scanning modules, risk scoring model, and trend analysis approach	System design diagrams
Week 10	Conduct analysis and evaluation of vulnerability scanning approaches and system design	Evaluation data
Week 11	Write evaluation and discussion sections and refine system design explanation	Draft of Chapter 4

Week 12	Final report editing, formatting, figure insertion, reference verification, and preparation for submission	Final FYP report
Week 13	Set up development environment, Flask backend skeleton, React frontend scaffold, and OWASP ZAP daemon integration	Working project skeleton with ZAP connectivity
Week 14	Design and implement the database schema, user authentication, and password policy logic	Database schema and working auth module
Week 15	Implement the core scan runner, ZAP spider and active scan integration, and asynchronous job handling	Functional single-scan pipeline
Week 16	Implement the three smart scan modes (Quick, Deep, Targeted) and scan cancellation	Scan mode configuration module
Week 17	Design and implement the risk-aware scoring engine, including base severity mapping, exploitability and exposure adjustments, and band promotion	Risk scoring module
Week 18	Implement the scheduling subsystem, including recurring schedule logic, pause/resume, and backlog handling	Scheduler module
Week 19	Implement the AI explanation layer, in-app assistant, and rule-based fallback behaviour	AI explanation module
Week 20	Implement email reporting, scan comparison, CSV export, and the settings screen	Reporting and comparison features
Week 21	Complete frontend dashboard, results and detail views, and interface refinement	Completed user interface
Week 22	Design and execute white-box and black-box test cases; record results and address defects	White-box and black-box test results
Week 23	Recruit UAT participants, conduct user acceptance testing sessions, and collect post-task questionnaire feedback	UAT results and participant feedback
Week 24	Write Chapters 5 and 6, complete critical evaluation and recommendations, final formatting and submission	Final FYP report

Table 1 Project Plan Table

Chapter 2: Literature review

2.1 Domain Research

2.1.1 Cybersecurity

The term cybersecurity can be described as technology combined with a process and practice to combat hostile and malicious acts targeting information systems and networks, as well as the data contained within, and the unauthorized access or use, interception, disruption, or destruction of that data. The National Institute of Standards and Technology (NIST) describes cyber security as defending or protecting the use of cyberspace (NIST, 2018). In a more general and basic sense, Cybersecurity revolves around the protective measures to the three core and foundational basic security objectives of any information system security, known as the Confidentiality, Integrity, and Availability (CIA) triad. While this is a technical answer to the question of what cybersecurity is, Confidentiality involves the ability to keep unauthorized persons from having access to the information, Integrity refers to the accuracy and reliability of the data, and Availability refers to the ability of the system to be utilized and the services to be rendered whenever needed.

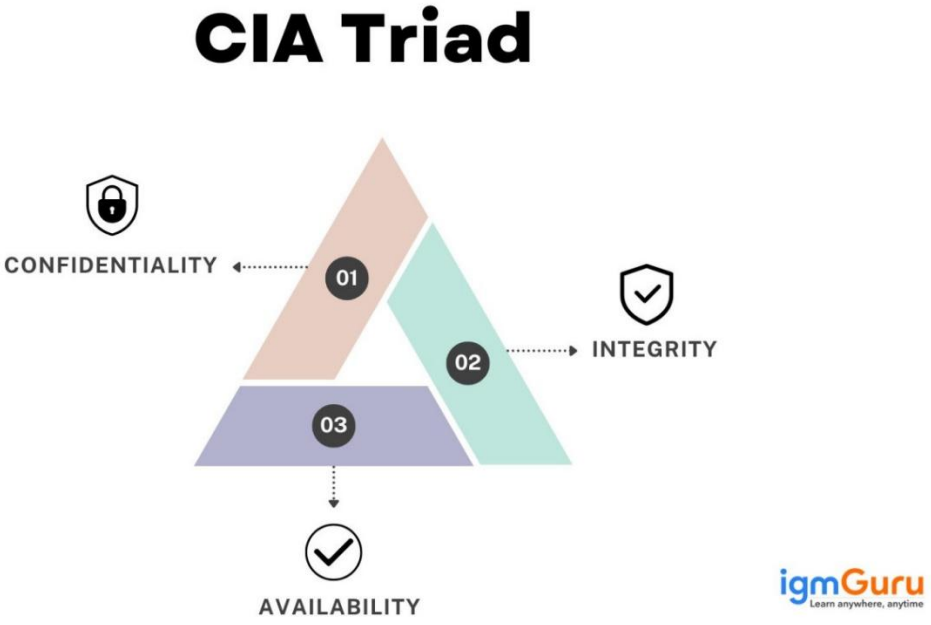


Figure 3 The three core principles of cybersecurity

With the global digital revolution and the rapid digitization of, interlinked systems, as well as the rapid adoption of distributed systems and cloud services, There has been an unprecedented and rapid increase in the global cyber vulnerability and distributed systems, and cloud services. The Internet and cloud services have also made it easy to commit crimes that have previously been very difficult to carry out. Cyber crimes and cyber threats have also become very sophisticated, automated, and economically driven, It can range from data breaches, ransomware attacks, and large scale abuse of vulnerabilities that have been made publicly available.

For many years, cybersecurity has concentrated on an organization's reactive and incident response posture. The NIST Cybersecurity Framework focuses on five core functions - Identify, Protect, Detect, Respond, and Recover (NIST, 2018). When breaking down the functions, vulnerability management is the Detect and Identify. This allows an organization to defend against an adversary before a weakness is exploited. Cybersecurity is truly reliant on more than a perimeter defense and security. It also relies on the ongoing assessment, monitoring, and prioritization of vulnerabilities across the systems and applications.



Figure 4 NIST Cybersecurity Framework. The five core cybersecurity functions: Identify, Protect, Detect, Respond, and Recover.

Cybersecurity is an ever evolving field, and therefore reactive security is not sufficient. Organizations need to have well defined and articulated structures, and the posture has to be automated and risk based. The mechanisms that end up serving the greater cybersecurity goals

need to be constructed to ops lead to the identification of weaknesses, and to the assessment of the weaknesses.

2.1.2 Weaknesses in Web Applications

Web applications are now the leading method of service delivery across financial systems, e-commerce, healthcare, education, and even critical infrastructure. Their direct exposure to the internet, as well as their integration with backend databases, APIs, and third-party services, makes them one of the largest and most continuously evolving attack surfaces in today's digital ecosystem.

The **OWASP Top 10:2021** serves as the leading industry benchmark for identifying the most critical web application security threats. According to OWASP, **Broken Access Control (A01:2021)** is the most serious web application threat, followed by **Injection (A03:2021)**, which includes SQL Injection and Cross-Site Scripting (XSS), and **Security Misconfiguration (A05:2021)**. The report indicates that broken access control vulnerabilities were the most prevalent among reviewed applications, highlighting the systemic nature of authorization weaknesses.

These industry findings are supported by empirical academic research. A large-scale analysis of 22,521 CVE entries in the National Vulnerability Database revealed that injection vulnerabilities, cross-site scripting, buffer overflows, file inclusion, and access control weaknesses consistently appear among the most frequently disclosed vulnerabilities (Chang et al., 2011). Furthermore, more than 80% of vulnerabilities were found to be remotely exploitable without authentication, reinforcing the importance of external attack vectors in web application exposure (Chang et al., 2011).

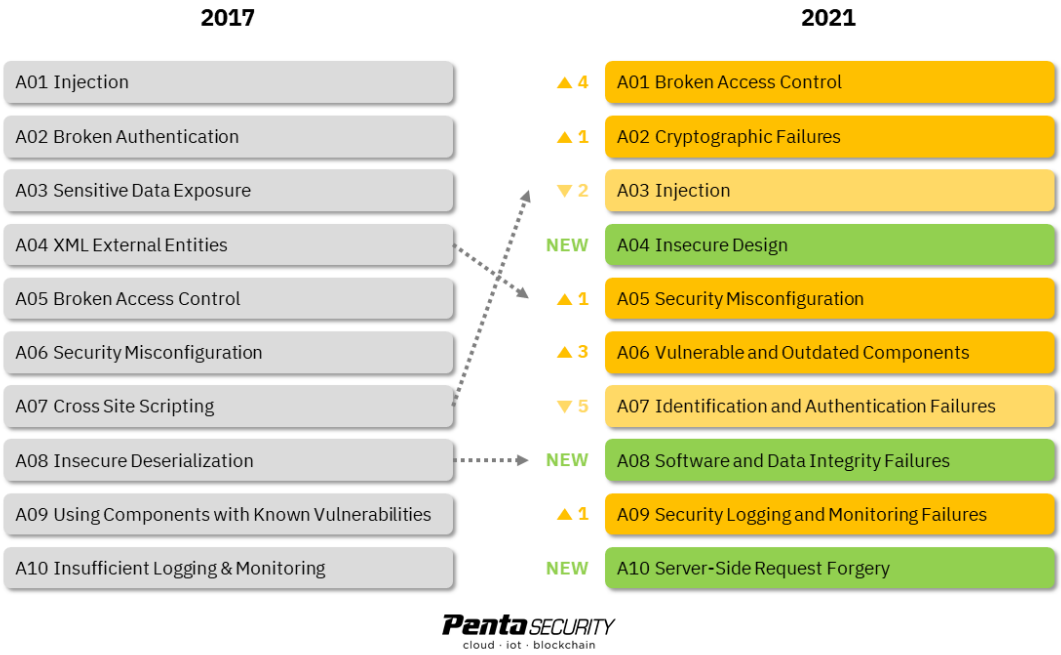


Figure 5 Comparison of OWASP Top 10 (2017) and OWASP Top 10 (2021)

Injection Vulnerabilities

Among web application vulnerabilities, SQL injection remains one of the most extensively studied and historically destructive attack classes. Injection vulnerabilities stem from failures in the proper construction of backend queries that incorporate user-controllable input. In such cases, attackers can modify the syntactic structure of SQL commands, enabling unauthorized data retrieval, authentication bypass, or even remote command execution.

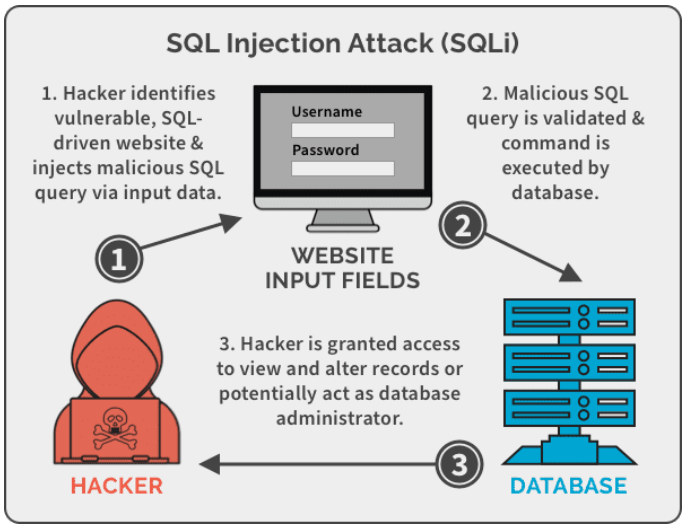


Figure 6 SQL Injection Attack Process

Vulnerability trend analyses have documented the continued persistence of injection flaws in CVE disclosures, consistent with OWASP ranking Injection as a top-tier risk (Chang et al., 2011). However, frequency alone does not capture the technical depth exhibited by injection vulnerabilities.

Halfond and Orso proposed **AMNESIA**, a model-based technique that integrates static analysis and runtime monitoring to mitigate SQL injection attacks (Halfond & Orso, 2006). Their approach constructs a model of legitimate SQL queries and validates dynamically generated queries against this model at runtime. The effectiveness of this technique demonstrates that injection vulnerabilities can be formally analyzed and prevented through structured verification rather than relying solely on heuristic input filtering.

Complementing runtime defenses, Wassermann and Su introduced a static analysis technique utilizing context-free grammars and language transducers to detect injection vulnerabilities prior to deployment (Livshits & Lam, 2007). Their work formally verifies whether user input can alter the syntactic structure of SQL queries, addressing limitations of simple taint-based detection mechanisms. The ability to apply such static analysis techniques to real-world systems demonstrates the feasibility of incorporating meaningful injection detection into secure software development processes.

Collectively, these studies demonstrate that injection vulnerabilities are not merely coding mistakes but reflect deeper structural issues in input handling and query construction logic.

Skewed Exploitation and Operational Risk

Although injection vulnerabilities are prevalent, exploitation studies reveal that vulnerability impact is unevenly distributed. Allodi analyzed 75.7 million recorded attacks and found that fewer than 10% of vulnerabilities accounted for more than 90% of exploitation attempts (Allodi, 2015). In certain cases, approximately 5% of vulnerabilities were responsible for 95% of observed attacks (Allodi, 2015).

This heavy-tailed exploitation phenomenon illustrates that vulnerability management strategies that rely solely on disclosure counts or static severity ratings may misrepresent real-world

operational risk. Injection flaws and access control weaknesses frequently exhibit high risk due to their remote exploitability and the availability of automated exploitation tools. Consequently, risk-based prioritization frameworks are necessary to ensure effective remediation.

Cross-Site Scripting (XSS)

Cross-Site Scripting (XSS) is one of the most persistent vulnerabilities in web applications. XSS vulnerabilities arise when applications allow user-controlled input to be inserted into dynamically generated web pages without proper contextual output encoding. Attackers can inject malicious scripts into victims' browsers, enabling credential theft, session hijacking, and client-side data manipulation.

CVE trend analyses consistently identify XSS as a common vulnerability class (Chang et al., 2011). While inadequate input validation contributes to XSS, the underlying issue is structural and more closely associated with improper contextual output encoding and fragmented client-side security enforcement.

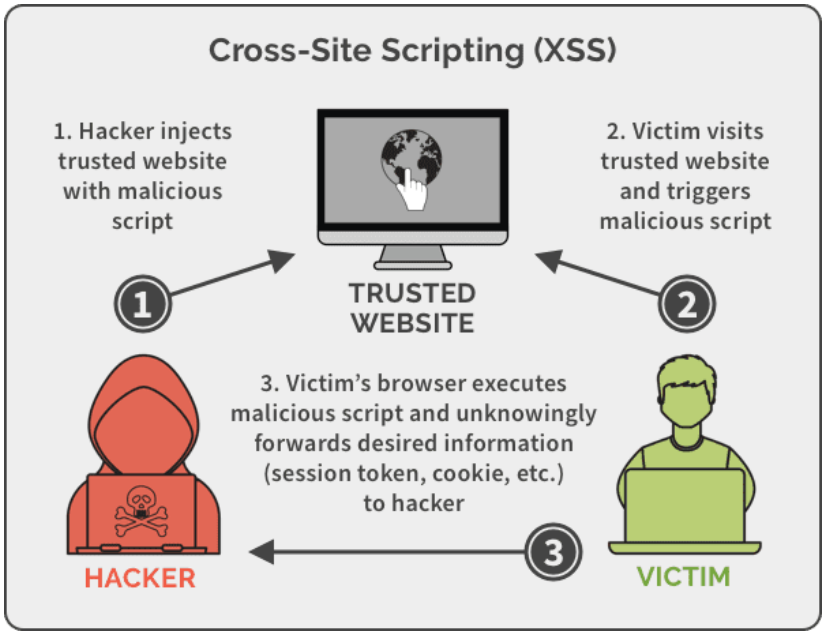


Figure 7 Cross-Site Scripting (XSS) Attack

Modern single-page applications, asynchronous requests, and dynamic manipulation of the Document Object Model (DOM) further increase exposure, particularly for DOM-based XSS variants. These architectural complexities amplify the attack surface and complicate detection.

Broken Access Control

Broken Access Control has been identified as the most critical web application vulnerability in OWASP Top 10:2021. Unlike injection vulnerabilities, access control weaknesses originate from logical failures in enforcing authorization policies defined within the system. Such failures allow users to bypass role-based restrictions, access unauthorized resources, or escalate privileges.

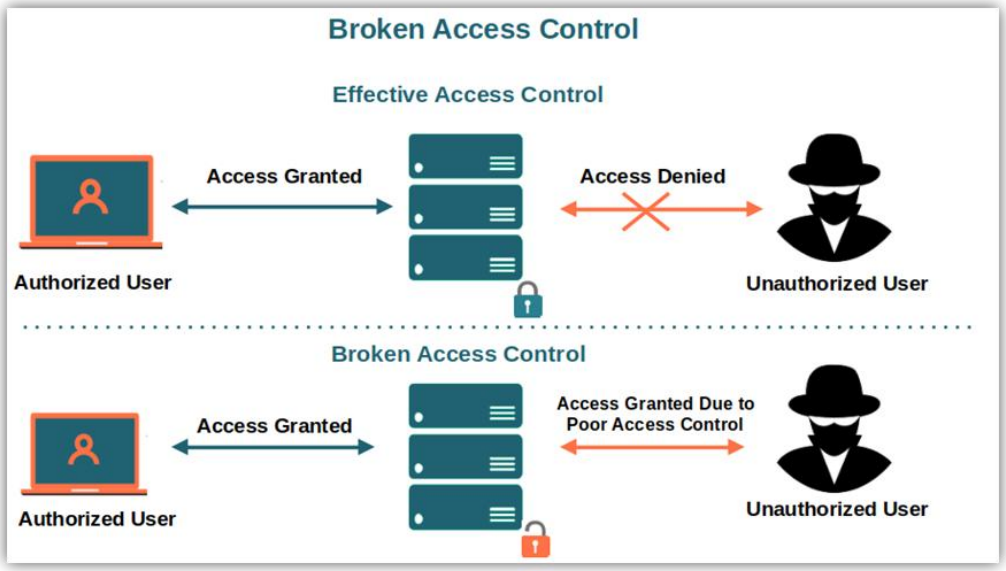


Figure 8 Broken Access Control

A comprehensive study categorizes mitigation strategies for access control vulnerabilities into three areas: (1) prevention during system design and development; (2) detection after deployment; and (3) runtime enforcement mechanisms (Anas, Elgamal, & Youssef, 2024). These vulnerabilities are inherently tied to application logic and contextual authorization rules, making them more difficult to detect using traditional automated scanning techniques.

Configuration Errors, File Inclusion, and Information Disclosure

Insecure configuration practices and insufficient parameter restrictions frequently lead to file inclusion and information leakage vulnerabilities. CVE analyses demonstrate the continued presence of configuration weaknesses and file inclusion flaws as active threats.

Although often perceived as lower severity individually, such vulnerabilities frequently facilitate reconnaissance by exposing system structure, configuration details, or sensitive data. This reconnaissance capability enables attackers to perform more efficient and targeted exploitation.

The inclusion of Security Misconfiguration in OWASP Top 10:2021 reinforces the conclusion that improper deployment practices remain a systemic root cause of web application exposure.

Limitations of CVSS-Based Risk Assessment

Web application vulnerability management has traditionally relied on CVSS-based severity scoring models. However, research examining CVSS-based prioritization models has identified inconsistencies across CVSS versions and limitations in using static metrics to accurately represent exploitation likelihood (Balsam et al., 2025).

Risk Ranking	CVSS Score
Critical	8.0-10.0
High	6.0-7.9
Medium	4.0-5.9
Low	2.0-3.9
Very Low	0.0-1.9

Figure 9 CVSS Risk Scoring

When considered alongside Allodi's exploitation findings (Allodi, 2015), it becomes evident that severity scores alone do not fully reflect operational risk. As web applications evolve through feature updates, third-party integrations, and configuration changes, vulnerabilities and their associated risks remain dynamic rather than static.

Implications for Vulnerability Management

The collective evidence from OWASP benchmarking, CVE trend analyses, exploitation behavior studies, static and dynamic detection research, and prioritization modeling supports several key observations:

1. Injection and access control weaknesses remain structurally dominant and persistent.
2. Exploitation is disproportionately concentrated on a small subset of vulnerabilities.
3. Static severity scoring does not fully represent real-world operational risk.
4. Traditional vulnerability scanners struggle to detect logic-based weaknesses such as broken access control.

These findings justify the need for continuous, risk-aware vulnerability assessment mechanisms that incorporate detection, exploitability evaluation, and dynamic prioritization. Without ongoing monitoring and structured prioritization, organizations remain vulnerable to high-impact weaknesses despite remediating lower-risk findings.

2.1.3 Technologies Used to Detect Vulnerabilities

The methods used to uncover web application weaknesses have transitioned from manual assessments to fully automated evaluations utilizing increasingly sophisticated methodologies. According to OWASP testing methodologies, detection strategies can broadly be categorized into static, dynamic, and hybrid approaches. Each methodology focuses on a distinct portion of the application's attack surface. In recent years, vulnerability assessment systems have increasingly adopted combinations of these methodologies in order to improve detection accuracy, reduce undiscovered weaknesses, and enhance overall assessment effectiveness.

Dynamic Application Security Testing (DAST)

Dynamic Application Security Testing (DAST) operates in a black-box manner and interacts with a running web application in real time to simulate the behavior of an attacker. Research on active and passive scanning systems (Li et al., 2020) explains that a typical web application scanner performs the following sequence:

- Asset discovery through web crawling
- Identification of URLs, forms, and input parameters
- Creation and injection of malicious payloads
- Analysis of server responses
- Validation of detected vulnerabilities

Active Crawling Techniques

Active scanning systems rely heavily on crawler modules to map and expand the application's attack surface. (Li et al., 2020) outlines several crawling strategies, including Breadth-First Search (BFS), Depth-First Search (DFS), and Best-First Search. Traditional crawlers extract hyperlinks from web pages and queue them for traversal; however, such crawlers are often inefficient and suffer from blind spots.

Advanced crawler frameworks utilize headless browser engines that enable:

- Rendering of DOM and CSSOM
- Evaluation of AJAX requests
- Traversal of events and form submissions
- Monitoring of XMLHttpRequest calls
- Activation of dynamic elements

(Li et al., 2020)

This multi-stage traversal approach increases attack surface coverage by capturing dynamically generated content that static crawlers typically fail to detect.

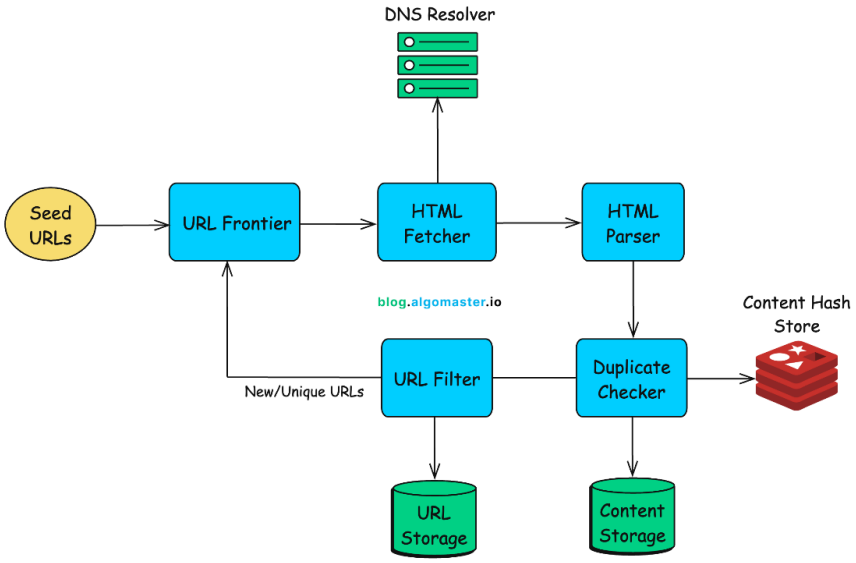


Figure 10 Web Crawler Architecture

Despite these advancements, active scanning still faces limitations (Li et al., 2020). High-intensity crawling may trigger defensive mechanisms, such as rate limiting or firewall responses, which can interfere with the scanning process.

Passive Scanning Techniques

In passive scanning, malicious payloads are not injected. Instead, communication flows are analyzed based on previously generated or user-initiated traffic. Passive scanning relies on three primary data sources:

- Traffic mirroring
- Web server logs
- Proxy-based interception

(Li et al., 2020)

Passive scanning can identify endpoints, file paths, parameter usage, and request patterns. When implemented through a controlled proxy environment, encrypted HTTPS traffic can also be analyzed, provided decryption occurs at the proxy layer.

Proxy-based passive scanning captures user-initiated interactions that active crawlers may miss, thereby improving coverage in authenticated or restricted sections of applications. However, passive scanning remains limited to observed traffic and cannot guarantee coverage of unvisited functionality.

To address blind spots and improve functional coverage, active and passive scanning modules are often combined, enhancing overall detection effectiveness (Li et al., 2020).

Data Processing and Detection Engines

Modern vulnerability scanners incorporate intermediary processing layers between crawling and detection. Y0081 describes a structured data processing framework that includes:

- Preprocessing of HTTP requests
- Deduplication of requests
- Classification by request method, file type, and content type
- Decryption of encoded parameters

This preprocessing improves efficiency by eliminating redundant data and organizing requests for targeted analysis. Deduplication prevents repeated evaluation of identical request structures, while classification enables focused vulnerability testing based on request characteristics.

Rule-Based Detection

One detection approach described in (Li et al., 2020) is rule-based vulnerability matching. In this model, HTTP request components are structured as key-value pairs and compared against predefined vulnerability signatures. Detection decisions are often based on response status codes, error messages, and content-length variations.

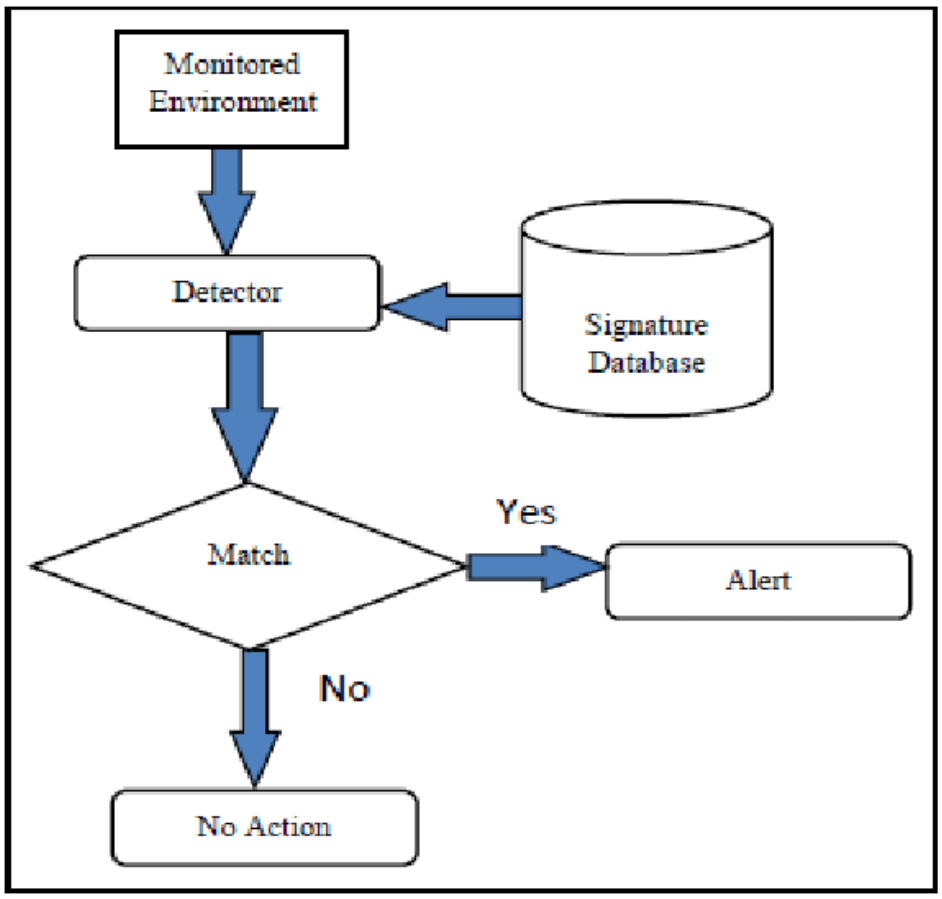


Figure 11 Signature-Based Detection Architecture

The effectiveness of rule-based detection is dependent on the completeness of its rule library. Broad heuristics may lead to false positives, while insufficient rule coverage may result in missed vulnerabilities.

Proof-of-Concept (POC) Validation Techniques

To improve detection accuracy, scanners may incorporate Proof-of-Concept (POC) validation techniques. POC-based detection executes targeted, often CVE-specific, exploitation logic to confirm the existence of a vulnerability. This method can significantly reduce false positives.

However, POC-based validation does not scale effectively to previously unknown vulnerabilities, as it depends on predefined exploit logic.

Static Application Security Testing (SAST)

Unlike dynamic scanning, Static Application Security Testing (SAST) analyzes an application's source code without executing it. Wassermann and Su proposed a formal method for detecting SQL injection vulnerabilities using context-free grammars and language transducers. Their approach defines conditions under which user input can alter the syntactic structure of SQL queries, thereby overcoming limitations associated with simple taint-tracking mechanisms.

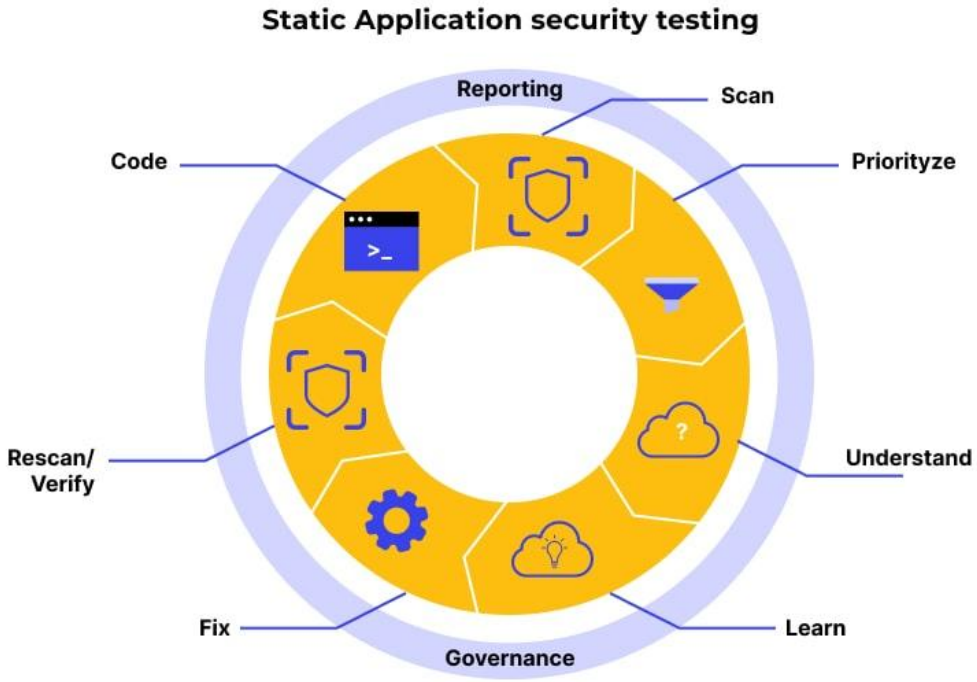


Figure 12 Static Application Security Testing (SAST) Process

Although static analysis during development can identify vulnerabilities at an early stage, it depends on source code availability and may struggle to address runtime-dependent behaviors or dynamically constructed queries.

Hybrid and Runtime Monitoring Approaches

Hybrid approaches combine static and dynamic analysis techniques. Halfond and Orso's AMNESIA system integrates static modeling of legitimate SQL queries with runtime monitoring of dynamically generated queries (Halfond & Orso, 2006). This approach verifies that runtime behavior conforms to predefined structural models.

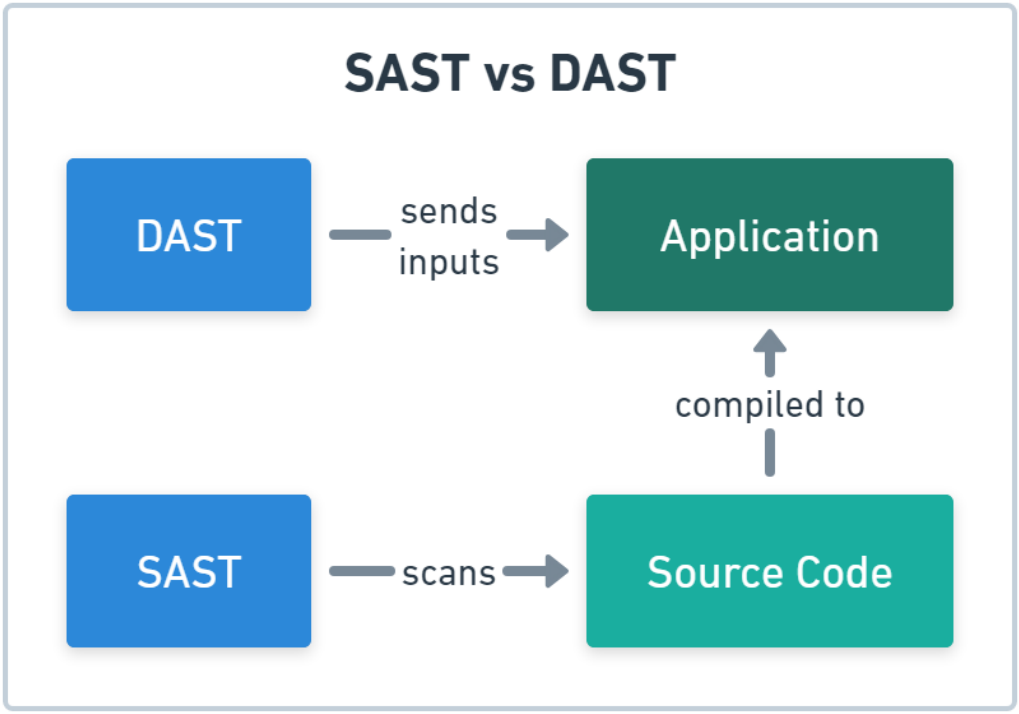


Figure 13 Comparison of Static and Dynamic Application Security Testing

While hybrid systems provide increased confidence through runtime validation, they may introduce performance overhead and require instrumentation of the target system.

Limitations of Automated Scanning

Automated scanning systems face inherent limitations. Logical vulnerabilities, such as broken access control, are particularly difficult to detect because they depend on contextual authorization logic rather than malformed input structures. The literature categorizes mitigation strategies for such vulnerabilities into three approaches: design-time prevention, post-deployment detection, and runtime enforcement (Anas, Elgamal, & Youssef, 2024).

This classification highlights the difficulty of identifying logic-based flaws through context-independent scanning rules.

Furthermore, CVSS-based prioritization models have been criticized for inconsistencies in predicting exploitation probability. Machine learning–based prioritization research demonstrates

that static score-based prioritization may lack reliability and that risk-oriented approaches are necessary (Balsam et al., 2025).

Automated scanning also faces challenges including false positives, incomplete coverage, blind spots in authenticated areas, JavaScript rendering complexity, encrypted traffic handling, and firewall interference.

Technological Advancement in Intelligent Scanning

The evolution from basic rule-based scanners toward more adaptable assessment systems is reflected in the integration of structured preprocessing, intelligent crawling, POC-based verification, static code modeling, and prioritization frameworks.

Current research trends emphasize:

- Intelligent attack surface expansion
- Reduced disruption to operational systems
- Dynamic risk-sensitive prioritization
- Continuous monitoring

These developments indicate a shift toward systems that integrate detection, validation, and contextual prioritization rather than relying solely on signature-based scanning.

2.1.4 Vulnerability Detection, Scoring, and Trend Analysis

After vulnerabilities are detected, effective vulnerability management requires systematic evaluation, prioritization, and time-based monitoring. Detection alone does not quantify operational risk; instead, vulnerabilities must be assessed based on severity, likelihood of exploitation, and potential impact.

The **Common Vulnerability Scoring System (CVSS)** is the most widely used framework for measuring vulnerability severity. CVSS provides a standardized assessment methodology through three metric groups:

- **Base Metrics** – intrinsic attributes of a vulnerability, including attack vector, attack complexity, required privileges, user interaction, and impacts on scope, confidentiality, integrity, and availability.
- **Temporal Metrics** – dynamically changing factors such as exploit code maturity and availability of remediation.
- **Environmental Metrics** – organization-specific contextual factors based on infrastructure and the value of critical assets.

This structured scoring system enables quantitative classification of vulnerabilities as low, medium, high, or critical. However, most CVSS assessments focus primarily on theoretical severity rather than actual operational risk.

Empirical Trends in Vulnerability Disclosures

A first-order critical analysis of CVE data provides important structural insights into web application vulnerabilities. A large-scale study of 22,521 CVE entries demonstrated that a significant proportion of vulnerabilities are remotely exploitable without requiring authentication (Chang et al., 2011). The study also showed that vulnerability categories evolve over time, with injection, cross-site scripting, buffer overflow, and access control weaknesses consistently dominating disclosures.

The evolving nature of vulnerability disclosures indicates that risk is dynamic rather than static. Within a software ecosystem characterized by continuous integration, architectural modification, and feature updates, newly exposed weaknesses accumulate alongside previously disclosed vulnerabilities. Therefore, trend analysis becomes essential for understanding long-term security posture.

Trend analysis serves several important purposes:

1. Identifying recurring vulnerability classes.
2. Detecting shifts in attack surface characteristics.
3. Measuring the rate at which specific vulnerability types increase over time.
4. Evaluating the effectiveness of patching and remediation efforts.

Without longitudinal analysis, systemic weaknesses may persist across multiple development cycles and remain undetected until comprehensive review is conducted.

Exploitation Distribution and Operational Risk

Although CVSS assigns severity scores to vulnerabilities and provides a baseline for evaluation, empirical research into real-world exploitation demonstrates that severity alone does not reliably represent operational risk. Allodi analyzed 75.7 million attacks and found that vulnerability exploitation follows a heavy-tailed distribution (Allodi, 2015). In this distribution, a relatively small proportion of vulnerabilities accounts for the majority of observed attacks.

Specifically, fewer than 10% of vulnerabilities were responsible for over 90% of exploitation activity. This disproportionate distribution suggests that many vulnerabilities classified as critical may never be exploited in practice. Conversely, a small subset of remotely accessible vulnerabilities tends to attract sustained attention from malicious actors.

This research supports several conclusions:

1. Operational risk cannot be determined solely by assigned severity values.
2. The availability of exploit code and the ability to automate exploitation significantly influence real-world impact.
3. Exploitability must be incorporated into any scoring or prioritization framework that aims to assess operational risk accurately.

If vulnerability management priorities are determined exclusively through CVSS scoring, organizations risk misallocating limited remediation resources.

Limitations of CVSS Scoring

Many limitations of CVSS stem from its relatively static nature. Although temporal metrics attempt to incorporate exploit maturity and remediation status, the scoring system does not account for real-time exploitation activity. This represents a structural weakness in the model.

The heavy-tailed exploitation phenomenon (Allodi, 2015) demonstrates that vulnerability prioritization requires contextual intelligence beyond severity scores. For example:

- A medium-severity vulnerability with publicly available exploit code may present greater operational risk than a high-severity vulnerability without observed exploitation.
- Vulnerabilities that are remotely exploitable without authentication generally carry higher exposure risk (Chang et al., 2011).

Therefore, vulnerability scoring systems should be supplemented by:

1. Exploit availability analysis.
2. Risk exposure assessment.
3. Trend analysis.
4. Continuous monitoring of vulnerability evolution.

The Role of Continuous Trend Analysis

Trend analysis extends beyond simply counting vulnerability disclosures. It enables organizations to:

1. Track the frequency of specific vulnerabilities over time.
2. Identify recurring patterns of weaknesses.
3. Monitor the security posture of target applications.
4. Measure the effectiveness of remediation strategies.

Because vulnerabilities evolve alongside application development, risk assessment must be iterative rather than static. Vulnerability management should not be treated as a one-time scanning activity, but as an ongoing process that integrates structured scoring, exploitability evaluation, and longitudinal trend monitoring.

Implications for a Risk-Based System

The combined evidence from CVSS scoring models, CVE trend analyses (Chang et al., 2011), and exploitation distribution research (Allodi, 2015) demonstrates that vulnerability risk is multidimensional.

Key observations include:

- Many vulnerabilities are remotely exploitable and externally exposed.

- Vulnerability categories evolve and continue to expand over time.
- Exploitation activity is heavily concentrated on a small subset of weaknesses.
- Fixed severity scores do not fully capture operational risk.

These findings establish a strong foundation for improving vulnerability assessment methodologies by integrating structured scoring, exploitability awareness, and continuous trend analysis. To achieve effective risk-based prioritization, assessment systems must move beyond static severity scoring and incorporate dynamic evaluation of evolving threats and vulnerability behavior over time.

2.1.5 Mechanisms for Continuous Monitoring

Traditional vulnerability assessment frameworks tend to be based on a single time or periodic assessment, e.g., periodic penetration tests or security audits (driven by compliance). As much as these tests will give a picture of the security posture at a given point of time, they tend to be inadequate in highly dynamic environments where applications, infrastructure, and threat environment are constantly in flux. With the current web ecosystem which is dynamic and marred with frequent updates, third-party integrations, and dynamic configurations, new vulnerabilities could be identified or become exploitable in between planned tests. Consequently, periodic checking is usually insufficient to ensure that there is the right and current picture of the security threats to the existence of a system.

Continuous monitoring systemically overcomes these challenges through persistent evaluation mechanisms, which for the most part is automated and done in near real time. Continuous risk monitoring is a term used to describe the evaluation of systems, networks, and applications to detect and assess the threat of risk as it continues to develop (Awan et al., 2024). Unlike snapshot reports of assessments done, frameworks of continuous monitoring sustain the visibility of the security posture of the system and gives room for changes in risk to be assessed as needed. This is particularly true for the discovery of new vulnerabilities to be exploited, and for the techniques of exploitation to evolve. Such an approach is targeted at an ever changing cyber environment as opposed to a static risk posture.

Automated systems are a new growing field where devices are able to collect, visualize, and analyze massive amounts of information regarding security measurements, and support the field of continuous monitoring, which activates systems for detecting and responding to potential security breaches. A key component of these systems is the Intrusion Detection Systems (IDS) and Intrusion Prevention Systems (IPS) that check for malicious activity in the network and at the host level. Detection can happen using one of three methods: (1) signature-based detection, (2) anomaly detection, and (3) behavior-based heuristics. The advantage of IDPS is that they run analyzes in

real time, and therefore can notify the systems administrators of malicious activity as soon as a potential target is compromised, therefore reducing the window of time of a security breach.

SIEM (Security Information and Event Management) Systems improve the process of automated monitoring by combining data and event logs from different components of the network such as web, application, and authentication servers, as well as network devices and endpoints. As stated in (Awan et al., 2024), the SIEM systems are able to present an overall view of the different components in a distributed system and apply correlation logic to determine a type of attack that is not visible by simply analyzing the logs from one component. The continuous process of log data collection and analysis allows organizations to identify coordinated attacks and to determine if the identified vulnerabilities are actively being targeted by the attack.

Mixing structured methodologies of vulnerability assessments with continuous monitoring tools can increase security posture. Automated scanners can detect certain weaknesses in your applications, while monitoring tools look for ultimate behavior and attempts of exploitations. When combined, these systems can detect and assess new vulnerabilities, increase prioritization of an exploit, and assess unvalidated scans after certain configurations. By integrating both an exploit detection and monitoring system, your systems vulnerabilities will be reduced and limitations of the systems will be exposed. The time from the disclosure of a vulnerability and its mitigation will be significantly reduced.

When compared to continuous monitoring, traditional assessments, while touted for their ease of implementation, have many structural constraints. They can only be accessed in an limited time frame and don’t incorporate new vulnerabilities in their risk assessments. They do not have the ability to change risk based on new data provided about exploitation attempts. In many risk scenarios monitoring tools are far superior. In environments of frequent software deployment and update cycles the exposure to risk can change significantly. In many cases the risk conditions that an organization operates under will be largely unknown until the next traditional assessment.

Continuous monitoring reconfigures vulnerability management as a process that is ongoing and adaptive instead of a set of disconnected scanning activities. Continuous monitoring tools provide accurate and real-time context-dependent assessments of security posture by supporting dynamic reassessment, real-time anomaly detection, and automated risk correlation. In order to keep vulnerability management practices effective and responsive, persistent and automated continuity is critical in developing threat environments.

2.1.6 Risk-Aware Prioritization

The disparity in the exploitation of vulnerability during cyber attacks has contributed to the vulnerability management making risk-aware prioritization a key area of interest. Research has indicated that a very small portion of vulnerabilities is actually used in most attacks. To illustrate, Allodi used 75.7 million cyber attacks and discovered that the 10 percent of vulnerabilities caused the majority of attempted exploitation (over 90 percent) (Allodi, 2015). This is a dispensation that means that any vulnerability remediation plan that addresses all vulnerabilities on equal grounds may result in wasteful utilization of organizational resources. Security teams should instead focus on the vulnerabilities according to their real probability of exploitation and the potential effect it can produce.

The Common Vulnerability Scoring System (CVSS) is one of the most popular scheme of vulnerability evaluation. CVSS offers a standardized approach to assess the severity of vulnerability using three groups of metrics, namely, base metrics, temporal metrics, and environmental metrics. Base metrics explain the inherent technical features of a vulnerability, whereas environmental metrics enable organizations to modify the score regarding the exposure of the impacted asset and its significance in the infrastructure (FIRST, 2019). Despite the fact that CVSS provides a systematic methodology on vulnerability scoring, studies have revealed that severity scores do not reflect on the actual exploitation risk in the real world. It has been shown that the vulnerabilities of closely similar CVSS scores may actually have a vastly different exploitation behavior in practice (Balsam et al., 2025). The reason behind this limitation is that CVSS is more of a technical impact analysis and lacks sufficient consideration of contextual threat intelligence, availability of the exploits, and the behavior of the attackers.

Due to these constraints, other methods have been investigated to forecast the exploitation of vulnerability. Bozorgi et al. (2010) proved that machine learning models that are trained off of historical vulnerability data are able to perform better than CVSS-based prioritization in forecasting vulnerabilities that are likely to be compromised. The elements that were included in their work were the description of vulnerabilities in text form, disclosure time frames, and indicators of the exploit development. In the same manner, Sabottke et al. (2015) demonstrated that social signals in the form of conversations in social networks like Twitter could be used as early warning about the following exploitation activity. Such findings indicate that by adding dynamic sources of information instead of just relying on static scores of severity, vulnerability prioritization can be enhanced.

Other studies have also considered the vulnerability exploitation lifecycle. Frei et al. (2009) have shown that the risk related to a vulnerability changes over time as the exploit code gets available and has become commonly spread. Their model demonstrates the temporal connection among the vulnerability disclosure, the development of the exploits and the general accessibility of the exploit tools. With easier availability of exploit code, there are higher chances that attacks will be successful. This shows vulnerable risk is dynamic, and it depends on factors like availability of exploits and incentives by attackers.

These results indicate that the use of solely static severity scoring systems is limited when it comes to prioritization of vulnerabilities. In its place, vulnerability management must be carried out through an approach that is risk-conscious and includes contextual controls like the availability of exploits, threat intelligence, vulnerability lifecycle relations, and environmental exposure. When these factors are combined, the vulnerability assessment systems will have a more accurate answer to the question of what the weaknesses are the most operationally risky. There has therefore been a shift towards using automated risk-sensitive prioritization tools that integrate vulnerability detection with contextual risk assessment and temporal analysis as an alternative to simple recourse to the often-stagnant CVSS severity ratings in modern vulnerability management strategies.

2.2 Similar Systems / Works

2.2.1 Burp suite

Burp Suite is a popular web application security testing software that was created by PortSwinger. It is useful in helping security professionals, penetration testers and developers in detecting vulnerabilities in web applications. The tool offers a total package of the analysis of web system security since it enables the user to monitor, intercept, and manipulate traverse of web between a browser and an application. Burp Suite has emerged as a tool that is widely used in web application penetration testing and vulnerability testing and assessment because of its broad capabilities and reliability.



Figure 14 Burp suite logo

Burp Suite is a set of tools that facilitate users to find and explore the security vulnerabilities in web applications. The interception proxy is one of its key components since it enables users to intercept and view HTTP and HTTPS traffic between the web browser and the web server. It allows testers to amend requests, study responses, and note how the application acts on various inputs. Besides that, the Burp Suite has the option of an automatic vulnerability scanner, an automatic application page and parameter web crawler, authentication testing, session testing, and input validation tests. Such features assist security testers in detecting the usual security vulnerabilities in web applications like SQL injection, cross-site scripting (XSS), and misconfigurations.

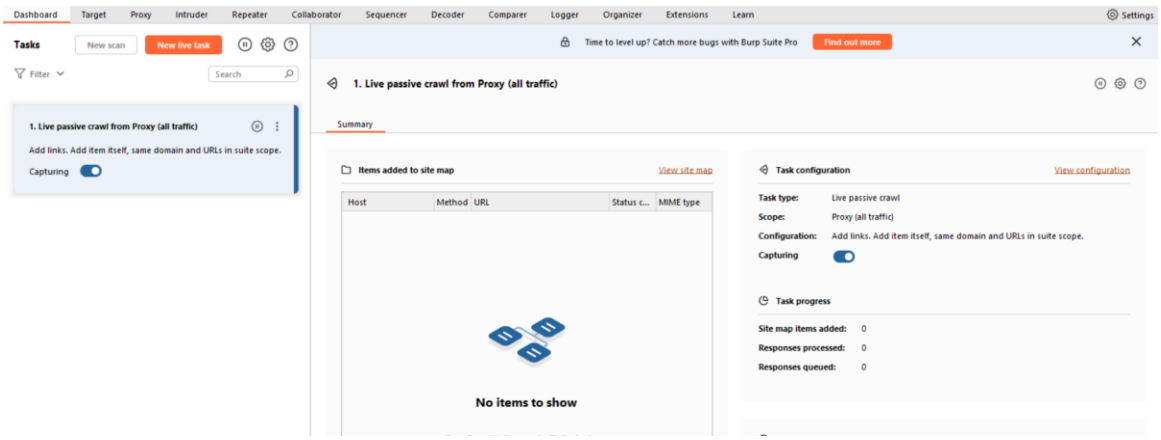


Figure 15 Burp suite main dashboard

The figure displays the primary dashboard of Burp Suite where one can manage and monitor scanning activities. Active tasks are shown in the dashboard e.g. the live passive crawl of proxy traffic which is an automatic processing of the HTTP and HTTPS traffic that the proxy captures. It also gives an overview of the items found, such as hosts, URLs and response status that are included in the site map. This interface enables users to monitor the progress of scanning and specify task configurations as well as view how Burp Suite finds and explores the structure of the target web application when security testing.

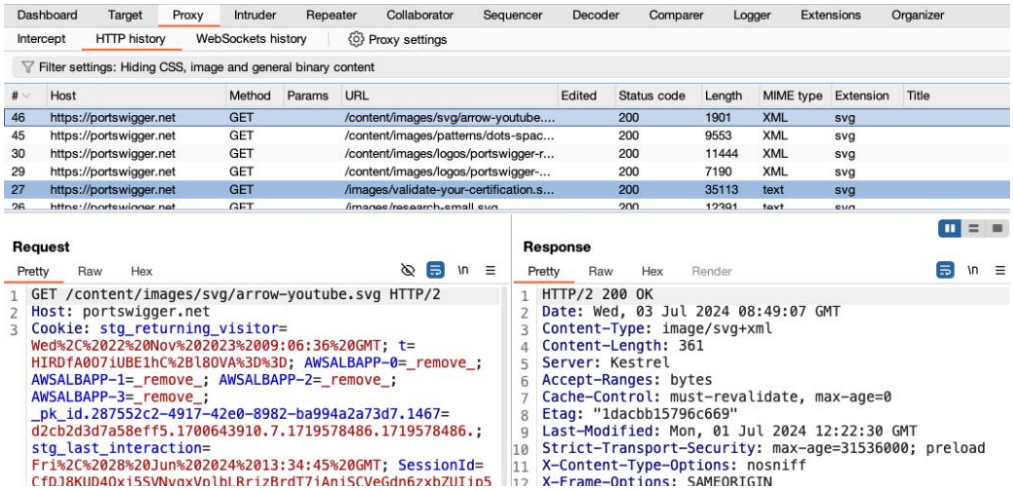
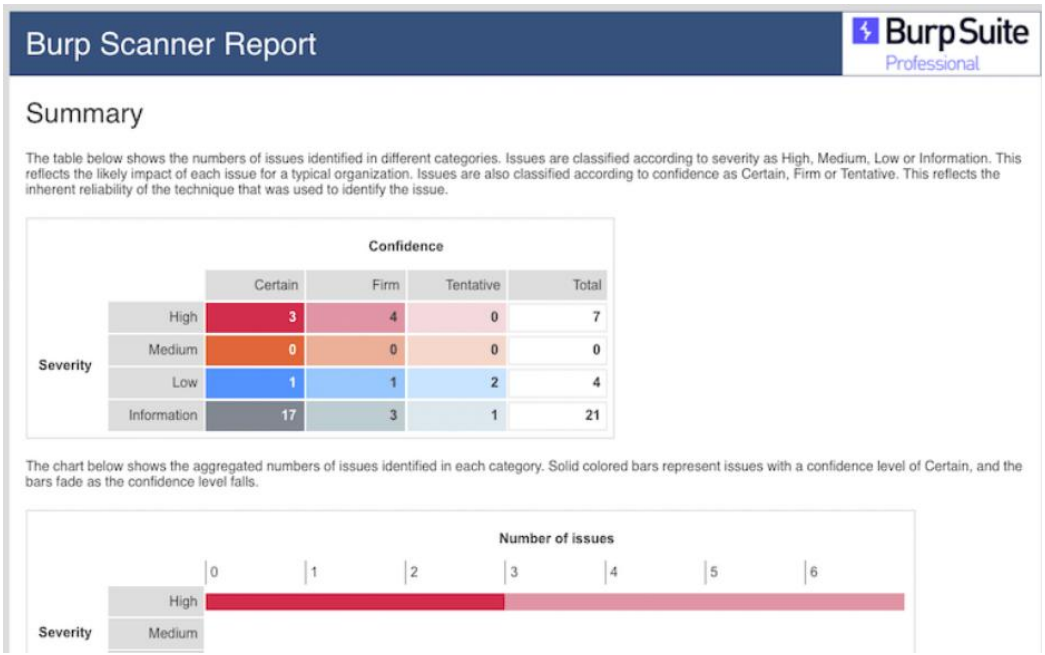


Figure 16 Burp Suite Proxy – HTTP History

The figure is going to showcase the HTTP History functionality of the Burp Suite Proxy tool. All HTTP and HTTPS requests that are relayed through the proxy interface by the browser and the web application are captured by this interface. Every request will have host, HTTP method, URL, status code, response length and file type information. With this traffic, testers will be able to see the communication between the web application and the server and attempt to find potentially vulnerable requests.

Besides, the bottom part of the interface displays the request and response information of a chosen entry. Headers, cookies, parameters, and server response can be analyzed by the security testers to gain insight into how the application accepts input and handles the session. This capability enables users to examine and control the web traffic and thus they can more easily identify vulnerabilities like invalid input validation, insecure session processing and other security flaws.



Burp Scanner Report

Burp Suite Professional

Summary

The table below shows the numbers of issues identified in different categories. Issues are classified according to severity as High, Medium, Low or Information. This reflects the likely impact of each issue for a typical organization. Issues are also classified according to confidence as Certain, Firm or Tentative. This reflects the inherent reliability of the technique that was used to identify the issue.

Severity	Confidence: Certain	Firm	Tentative	Total
High	3	4	0	7
Medium	0	0	0	0
Low	1	1	2	4
Information	17	3	1	21

The chart below shows the aggregated numbers of issues identified in each category. Solid colored bars represent issues with a confidence level of Certain, and the bars fade as the confidence level falls.

Figure 17 Burp Suite Vulnerability Scanner Report

The image presents a sample Burp suite vulnerability scanner report, which is the overview of the security problems revealed in a scan of a web application. The report lists identified vulnerabilities according to their level of severity as High, Medium, Low, and Informational. It also categorizes the issues based on confidence level of the detection like Certain, Firm or Tentative which would show the degree of reliability of the vulnerability that is identified. This formal overview can assist security testers to grasp the security posture of the tested application in general.

Furthermore, the report introduces a visual site of the quantity of the vulnerabilities identified in every category. The chart shows the issues by their severity and gives the user the opportunity of ensuring that the most critical vulnerabilities are dealt with first. These reporting features must be vital in vulnerability assessment tools since they assist organizations to examine scan findings, interpret the possible threats and make relevant measures aimed at enhancing security of their web applications.

2.2.2 Nessus

Nessus is a very popular vulnerability assessment program created by Tenable, Inc. It is intended to assist an organization in recognizing security risks, misconfigurations, and possible vulnerabilities of the systems, networks, and web applications. Nessus is widely applied by the security experts, system administrators and the penetration testers to programme vulnerability scans of different types of infrastructure in an automated manner. Nessus has since proven to be one of the most reliable and most popular vulnerability scanners in the cybersecurity industry because of its built-in vulnerability database and frequent security patches.



Figure 18 Nessus Logo

Nessus will carry out automated security checks to identify security vulnerabilities in operating systems, network services, applications and web environments. The tool operates by scanning target systems and making a comparison with a huge database of identified vulnerabilities, configuration problems, and security vulnerabilities. Once a scan is done, Nessus will produce detailed reports that rank the vulnerabilities by their severity level i.e. critical, high, medium, and low. Such reports assist security teams to focus on the remediation efforts and enhance the overall security stance of the infrastructure of the organization.

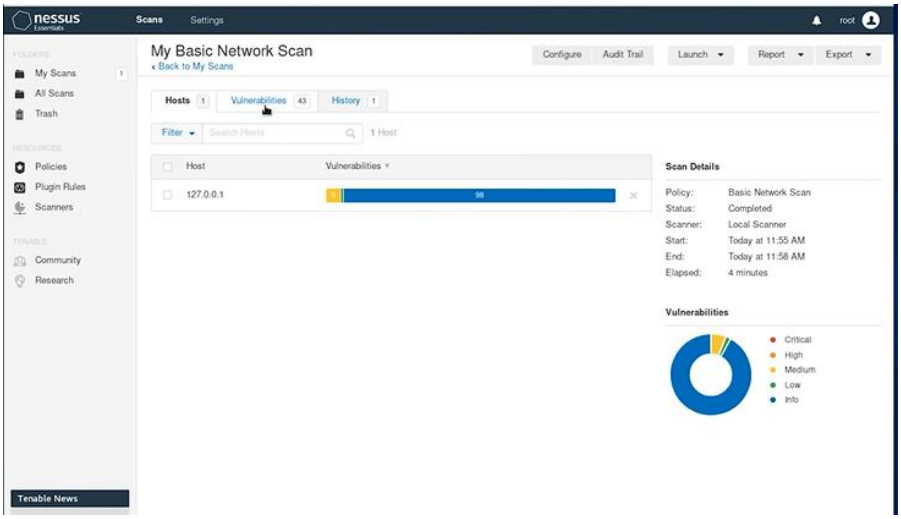


Figure 19 Nessus Vulnerability Scan Results

The diagram demonstrates the scan results window of Nessus, in which the vulnerabilities identified in the process of network scan are presented. The interface includes the list of the scanned host and the number of vulnerabilities that have been detected. The findings are classified according to the degree of severity including critical, high, medium, low, and informational findings. Such classification enables security professionals to find the most dangerous vulnerabilities within a short period of time which need urgent measures.

Moreover, the interface gives a visual overview of the findings of the scan in the form of charts and statistics. These visual tools allow users to see how vulnerabilities are spread throughout the system and determine the general security attitude of the scanned host. These reporting capabilities allow companies to focus on the remediation work and enhance the infrastructure security of the companies by helping to cover the most significant vulnerabilities first.

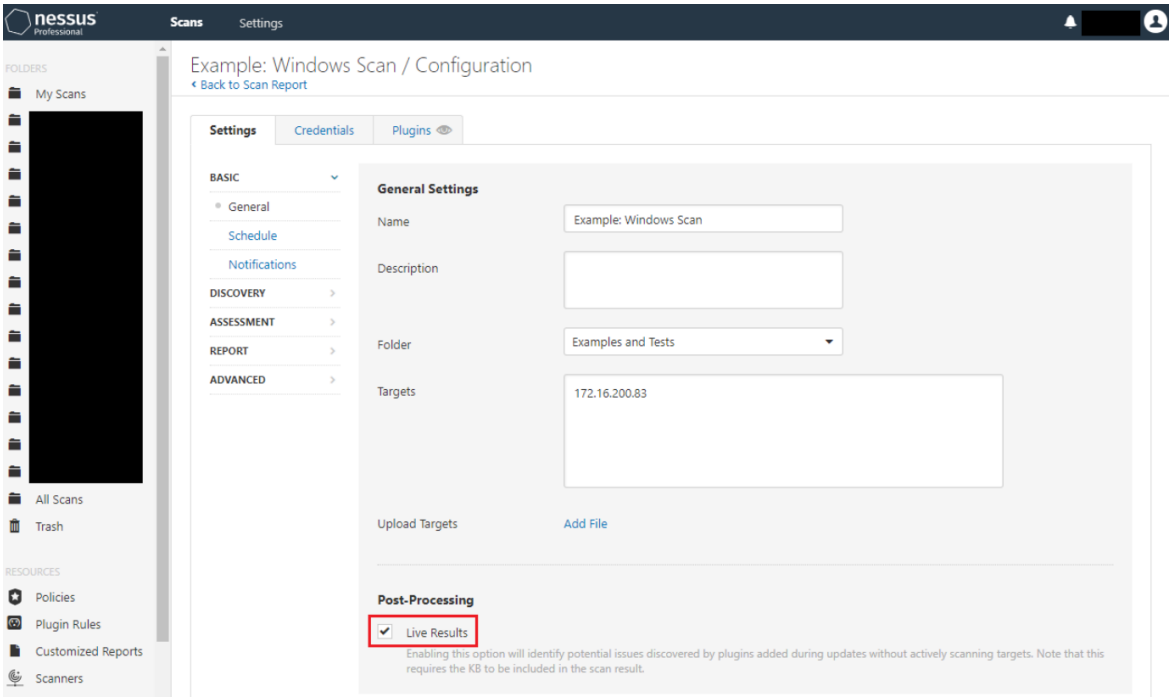


Figure 20 Nessus Scan Configuration Interface

The figure represents the screen that displays the scan configuration of the Nessus where the user can set the parameters of a vulnerability scan. Some of the general settings that can be specified by the users in this interface are the scan name, description, target systems and scan folder. The target field enables the user to type the IP address or domain of the system the user wants to scan to detect any vulnerability. This arrangement step is significant in that it defines the areas and causes of the vulnerability analysis.

Also, there are various settings areas that include credentials, plugins, scheduling, and reporting, which are available in the interface. These environments enable individuals to tailor the way the scan is going to be conducted and the vulnerability tests that will be carried out. By modifying these parameters, security experts are able to customize the scan accordingly to meet certain security needs in such a way that the assessment would be comprehensive and applicable to the target environment.

2.2.3 OWASP ZAP

OWASP ZAP (Zed Attack Proxy) is an open source web application security testing tool created by the OWASP (Open Web Application Security Project). It is built to assist developers, security testers and students to detect weaknesses in web application during development and testing.

OWASP ZAP is popular since it is free, and offers a variety of advanced functionality to identify vulnerabilities. It is one of the most popular open-source vulnerability assessment tools used in a web application because of its accessibility and the support of the community.



Figure 21 ZAP logo

OWASP ZAP operates by becoming a proxy between the user browser and the web application which allows it to intercept and scan through the HTTP and HTTPS traffic. The tool will be able to crawl on web applications automatically in order to identify pages, forms and parameters susceptible to attacks. It also contains auto scanning capabilities that scan the application against a known vulnerability including SQL injection, cross-site scripting (XSS), and security misconfigurations. Upon the completion of the scan, ZAP produces detailed reports that identify Web vulnerabilities found and present information to the developer and security people in order to enhance the security of their web applications.

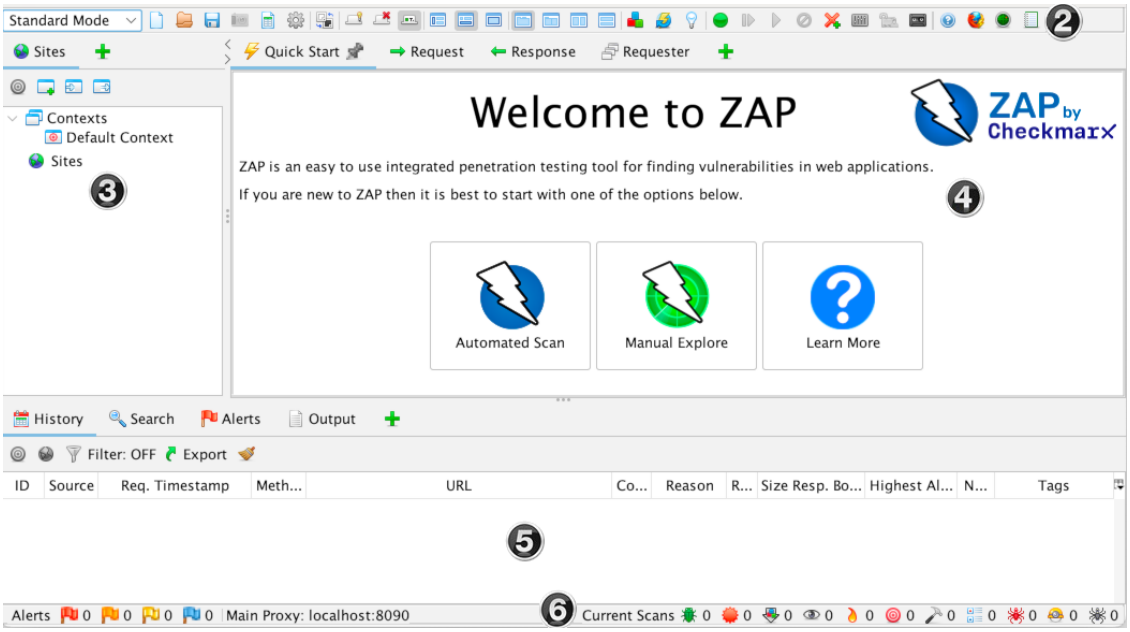


Figure 22 ZAP main dashboard

The following figure presents the main interface of OWASP ZAP, which provides the main workspace to perform the web application security testing. The interface has various sections through which users can control their scanning activities and analyze the web traffic. The left panel shows the site tree which is the list of found web application structure including domains, pages, and resources found during the scanning or browsing process. The central panel gives quick access to important functions such as Automated Scan, Manual Explore and other help options for initiating a security assessment.

In addition, at the bottom of the interface appear tabs like History, Alerts and Output, which allow the user to track the captured HTTP requests or examine the detected vulnerabilities. The alerts tab shows identified security issues divided by severity levels, which helps the testers to quickly find potential weaknesses within the application. This structured interface allows security analysts to navigate efficiently through the scanning process and examine the vulnerabilities found during the evaluation process.

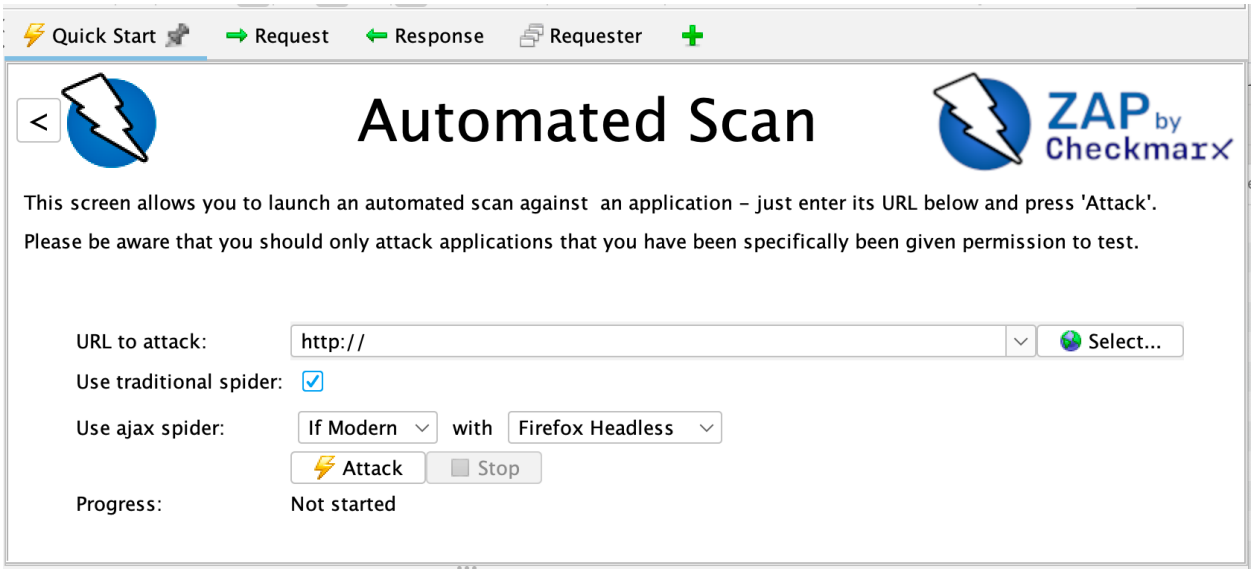
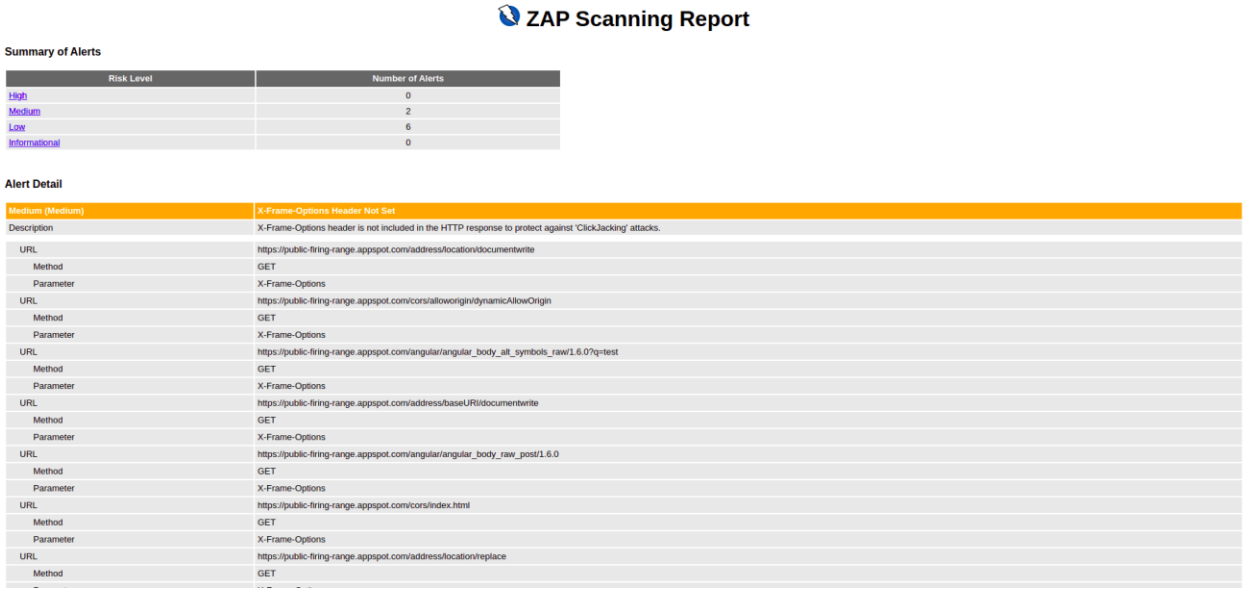


Figure 23 ZAP Automated Scan Interface

The figure presents the automated scan functionality in OWASP ZAP, where a user can easily start a vulnerability scan against a target web application by entering the URL of the web application. The interface offers options to use the scanning component like the traditional spider and the AJAX spider to automatically discover the web pages and the application inputs. Once the scan is initiated, ZAP automatically tests the target application for common vulnerabilities and weaknesses in the security of the application.



ZAP Scanning Report

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	2
Low	6
Informational	0

Alert Detail

Medium (Medium)	X-Frame-Options Header Not Set
Description	X-Frame-Options header is not included in the HTTP response to protect against 'ClickJacking' attacks.
URL	https://public-firing-range.appspot.com/address/location/documentwrite
Method	GET
Parameter	X-Frame-Options
URL	https://public-firing-range.appspot.com/cors/alloworigin/dynamicAllowOrigin
Method	GET
Parameter	X-Frame-Options
URL	https://public-firing-range.appspot.com/angular/angular_body_alt_symbols_raw/1.6.0?q=test
Method	GET
Parameter	X-Frame-Options
URL	https://public-firing-range.appspot.com/address/baseURI/documentwrite
Method	GET
Parameter	X-Frame-Options
URL	https://public-firing-range.appspot.com/angular/angular_body_raw_post/1.6.0
Method	GET
Parameter	X-Frame-Options
URL	https://public-firing-range.appspot.com/cors/index.html
Method	GET
Parameter	X-Frame-Options
URL	https://public-firing-range.appspot.com/address/location/replace
Method	GET

Figure 24 ZAP Vulnerability Scan Report

The figure displays an example of a vulnerability report from OWASP ZAP after a security scan is complete. The report presents a summary of the detected vulnerabilities based on the risk level (high, medium, low and informational alerts). It also provides detailed information about each of the issues identified, such as the affected URL, the request method used and the specific security weakness detected. This report helps security testers to understand the vulnerabilities available in the web application and prioritize the needful remediation activities.

2.2.4 Comparing the systems

Feature	VulnLab	Burp Suite	Nessus	OWASP ZAP
Type	Automated Web Vulnerability Assessment System	Web Application Security Testing Tool	Network & Vulnerability Scanner	Web Application Security Scanner
Target Users	Small businesses, developers, researchers, students	Security professionals and penetration testers	Enterprise security teams and system administrators	Developers and security testers
Automated Scanning	Yes	Yes	Yes	Yes
Risk-Based Vulnerability Prioritization	Yes	Limited	Yes	Limited
Scheduled Scanning	Yes	Limited (requires configuration)	Yes	Limited
Smart Scan Modes	Yes (Quick, Deep, Targeted)	No predefined modes	No	No
Trend Analysis Over Time	Yes	No	Limited	No
Email-Based Reporting	Yes	Limited	Yes	Limited
Ease of Use	Designed to be simple and accessible	Requires technical expertise	Moderate complexity	Moderate
Cost	Intended to be accessible / low cost	Paid (Professional version)	Paid	Free / Open Source

Table 2 Comparing similar systems

2.3 Technical Research

2.3.1 System Requirement Analysis

The suggested system VulnLab is aimed at being a risk-conscious and continuous automated system of web application vulnerability assessment. The system will help in automated vulnerability detection, periodic scanning, monitoring vulnerability trend, and prioritization of risks based on context. To meet these goals, both non-functional and functional requirements should be specified in order to inform the system implementation and design.

In functional terms, the system should offer a number of fundamental functions that would enable people to determine, examine, and trace vulnerabilities in web applications. These facilities are automated vulnerability detection, scheduling of scanning activities, analysis of historical scan results and automatic generation of vulnerability reports. The system will also enable the administrators to set scanning parameters, and control system settings using a web-based interface.

The major functionality needs of the system are:

- Automate vulnerability scan of target web applications.
- Favor active scanning and passive scanning methods.
- Make vulnerability scans be scheduled at the discretion of their administrators.
- archive previous scan outcomes to monitor in the long run.
- Indicate the trend of vulnerability analysis using the past data.
- Deploy vulnerability prioritization systems that are risk conscious.
- Automatic generation of vulnerability reports.
- Email vulnerability information to administrators.
- Give administrators a possibility to set the scanning targets and parameters.

Detection, monitoring, and prioritization features should be integrated into a single system, and it has brought a lot of advantages as it allows users to handle the web application security more efficiently.

The system should also meet a number of non-functional requirements in order to be reliable and usable besides the functional capabilities. Such needs are associated with the system performance, its security, and stability in operation.

The non-functional requirements are:

- Scalability to facilitate scanning process of various web applications.
- Effective work under the planned vulnerability scanning processes.
- Safe storing of the vulnerability scan findings and system data.
- Consistency to operate continuously with little failure.
- User-friendlyness to administrators with insufficient cybersecurity knowledge.

These non-functional properties make sure that the system can be successfully functioning within real-life settings without compromising on the performance and security levels.

2.3.2 Hardware Requirements

The hardware need of the system proposed is determined by the magnitude of the deployment environment and the numbers of the web application being monitored. In the case of small to medium size settings, the system may be run using a single server or virtual machine with adequate processing power and memory to handle vulnerability scanning and data processing activities.

The hardware resources that a typical deployment should consist of are:

- A single server or virtual machine having multi core processor.
- Not less than 8GB RAM to facilitate the scanning and analysis process.
- Storage facilities of the vulnerability scan report and history information.
- Good internet connectivity to conduct the web application scanning activities.

These hardware specifications enable the system to effectively execute the vulnerability scanning activities without any significant changes in the performance. When scanning many web

applications in a larger configuration environment, more resources or distributed scanning nodes might be needed in order to serve more workloads.

2.3.3 Software Requirements

VulnLab system will be deployed in a modular software structure comprising of vulnerability detecting mechanism, data processing modules, risk prioritization models and automated reporting functions. The system will be designed and implemented on a windows operating system like windows 10 or windows 11 which will serve as a local server where the web application will be hosted.

The environment used in its development is windows-based, however, the VulnLab itself is a web-based platform that can be used with a regular web browser. This architecture enables the administrator and users to communicate with the system by using easy to use web interface without the need to install any extra software.

The primary programming language that will be used to develop the system will be Python. It has found extensive application in cybersecurity systems because it has powerful support features in the area of automation, networking, and data processing activities. Besides Python, the web-based user interface will be developed by use of JavaScript, and the database operations and data management will be performed using SQL.

The development setting and the software tools are:

- Python as the main backend programming language.
- JavaScript to the Web-based user interface.
- SQL database management/data storage.
- The main integrated development environment is Visual Studio Code (VS Code).
- The version control and source code management Git.
- API testing and development, postman.

The system will be based on the relational database management system, e.g., PostgreSQL or MySQL, where the results of vulnerability scans, configuration settings, and historical

vulnerability information will be stored. Historical data is essential to understand change in security posture of monitored web applications over time by allowing the platform to compare trend on vulnerability.

The system will also incorporate the open-source vulnerability scanning frameworks instead of creating a vulnerability scanning engine in its entirety. Such frameworks offer ready-made methods of identifying typical web applications vulnerabilities that include SQL Injection, Cross-Site Scripting, and security misconfigurations. VulnLab will be an orchestration platform which controls scanning activities, scan results processing and uses risk prioritization and reporting mechanisms.

To facilitate continuous monitoring, the system will have a scheduling component which will enable vulnerability scans to be automatically run at scheduled times. This is a scheduling mechanism that will carry out security assessments on a schedule without the need of manual intervention.

There will also be the built-in system of automated email reporting, which will transfer the vulnerability scan reports to administrators. This aspect will make sure that vulnerabilities identified will be reported in a timely manner and will enable administrators to effectively address security threats.

The system will also include the OpenAI API to improve vulnerability information analysis and presentation. The OpenAI service will help in a number of activities which include:

- Producing human readable vulnerability reports.
- Demonstrating identified weaknesses and their possible effect.
- Helping make risk-conscious prioritization of vulnerabilities.

It is this integration that enables the system to transform technical vulnerability data into understandable explanations and application insights to enhance the usability of vulnerability reports to administrators and developers.

Mechanisms of logging and monitoring will also be put in place to monitor the activities of the system, scanning process and possible error. Such logs are useful in maintaining the system and assist in ensuring the platform is reliable as long as the process of carrying out the vulnerability monitoring is in progress.

2.3.4 System Architecture Reflections.

The VulnLab system is designed based on a modular design that breaks the system into a number of functional components. This type of architecture makes it easier to maintain, scale, and be flexible with because, each module will be dedicated to a particular purpose and communicate with the rest of the parts through clearly defined interfaces.

The key elements of the system entail:

- Scanning Module - active and passive vulnerability scanning of the target web applications is carried out.
- Data Processing and Storage Module - processes, stores and structures vulnerability data gathered in scans.
- Risk Prioritization Engine - determines the severity and context of risk on the identified vulnerabilities and allocates the level of risks.
- Trend Analysis Component - compares previous vulnerability data with the current security posture to determine how current security posture has changed over a period of time.
- Scheduling Module - handles automated vulnerability scans in accordance to the predetermined time periods.
- Reporting and Notification Module - creates vulnerability reports and delivers the notifications to administrators.

The modular architecture will also make sure that every component of the system can be upgraded or enhanced without necessarily impacting on the whole platform. It also facilitates future improvements, including incorporating machine learning models to enhance vulnerability risks prediction and minimizing false positive detection.

VulnLab architecture offers a complete web application vulnerability assessment system by integrating automated scanning, intelligent prioritization, historical analysis, and automated reporting into one platform.

Chapter 3: Methodology

3.1.1 System Development Methodology

System development Methodology (SDM) is a systematic process that helps in planning, analyzing, designing, implementing, testing, deploying and maintaining different elements of an information system in a structured manner. An SDM provides a step-by-step breakdown of the various stages in system development and also gives the activities to be performed in those stages. This makes the development of the system to be methodical, organized, and effective. SDMs have the benefit of ensuring that software is not written haphazardly or in an unstructured way and they have rules, standards and documentation of the deliverables and assessment at different stages of the development life cycle. Royce (1970) suggests that when the development practices are structured, the level of ambiguity in the software engineering practice is even less and, similarly, development practices are the most appropriate measure of controlling the degree of sophistication and the risk associated with the technology in the software systems (Boehm, 1988).

Choosing an appropriate Software Development Methodology (SDM) is pivotal to the success of the project. The selected methodology will affect how requirements will be captured, how changes will be controlled, how risk is handled, and how stakeholder perspectives will be integrated during the course of development. A methodology that does not fit the project at best will delay the project, cause costs to escalate, results will be aligned poorly to requirements and defects will be found late. A good fit will increase transparency, and improve quality control and the system is more likely to help achieve the intended purpose. For development of technical systems with security considerations, and evolving requirements, the methodology chosen has to balance flexibility and control.

Selecting the right SDM also contributes to achieving Sustainable Development Goal (SDG) 9: Industry, Innovation and Infrastructure which focuses on resilient infrastructure, and the promotion of inclusive and sustainable industrialisation and innovation. The developing of new innovative systems of cybersecurity requires the right balance between rigidity and flexibility in the engineering processes to achieve reliability, scalability, and sustainability over the long-term. This project will contribute to SDG 9 by developing a system that practises innovative digital engineering, and applying the right SDM, we will strengthen the digital infrastructure resilience by developing a web application vulnerability assessment system.

This project will analyze and compare three established system development methodologies:

- The Waterfall Model (Royce, 1970), which is a classic linear and sequential development method.
- Scrum (Schwaber & Sutherland, 2020), which is an Agile framework with a focus on iterative development and the continuous delivery of value to stakeholders.
- The Spiral Model (Boehm, 1988), which is an iterative and risk-driven methodology that focuses on continuous risk assessment and refinement.

Each method will be examined with respect to its structure, its flexibility, its ability to manage risk, and its overall applicability to the project's technical and academic constraints. The outcome of this assessment will enable the selection of the most appropriate methodology to ensure that the system's development process will foster flexibility, effective, and innovative project delivery within the set scope and timeline.

3.1.2 Waterfall Model

Description of the Model

The Waterfall Model exemplifies the methodical and pioneer approach to system development focusing on a structured and methodical approach to software engineering. Software development is decomposed into a number of steps taken by the engineers, and these steps are taken one at a time or in a sequential manner. The popularity of this method is from Royce (1970) describing the linear flow of development with distinct stages of requirements, design, implementation, and testing. Although Royce (1970) described the Implementation stage as a sequence of instructions, this model has been interpreted to mean a single, rigid, single-way approach.

The Waterfall Model focuses on reaching a clear understanding of system requirements and documenting these requirements to the greatest extent possible before development begins. Following the capture of the requirements, the project moves on to the design stage, then the implementation and testing stages, with very little iterative cycles. The nature of the model is geared towards producing and then documenting the results of each stage, to obtain formal reviews and final approvals, before the next stage begins. The structured and documentation-focused design of the Waterfall Model makes it ideal for projects where the requirements are known and where changes to requirements are highly unlikely (Pressman & Maxim, 2010).

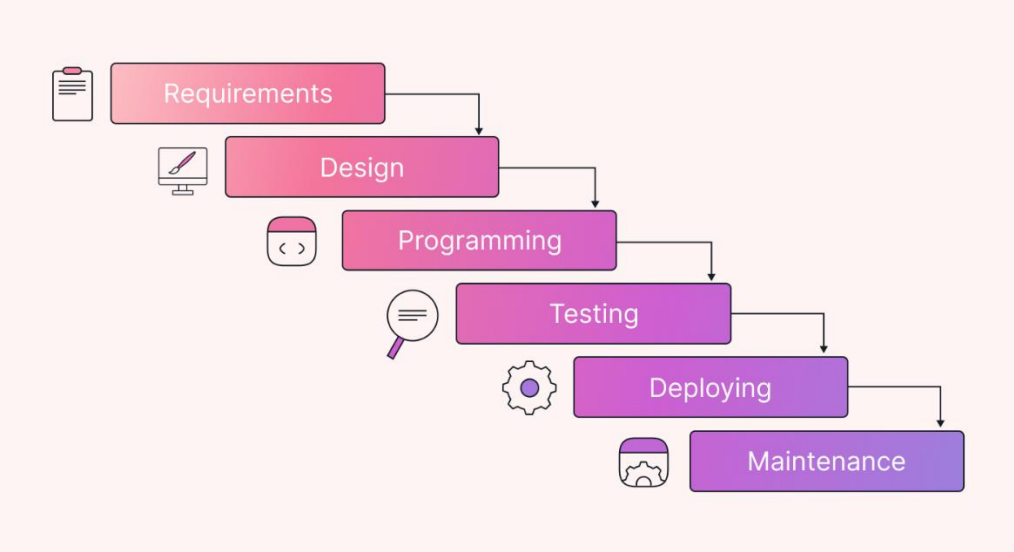


Figure 25 Waterfall model

Stages of the Waterfall Model

The Waterfall model comprises five key stages:

1. Requirement Analysis

During this stage, the system's requirements are collected, analyzed, documented, and finalized. Typically, a formal requirements specification document is produced. This document outlines the system's objectives, constraints, and functionalities. A robust presumption of this methodology is that the requirements remain clear and unchanged during the entire project lifecycle (Sommerville, 2016).

2. System and Software Design

In this stage, the system's design is established based on the requirements that have been previously documented. The system's architecture and the intricate details of each component are defined. This includes the various modules, data structures, interfaces, and the system’s overall workflow. The design document acts as a construction blueprint (Pressman & Maxim, 2019).

3. Implementation (Coding)

In this stage, the system is developed according to the previously established specifications. The Design/Document that has been developed is transformed into executable code. Implementation can only commence after design has been completed and approved in a strict Waterfall model (Sommerville, 2016).

4. Verification (Testing)

The testing process starts after the total implementation of the system. Here, the aim is to confirm if the system meets the requirements that have been outlined in the system's documentation. This stage of the process may encompass unit testing, integration testing, and system testing. A primary critique that has been leveled against the waterfall model is that all the testing occurs late in the cycle of development (Royce, 1970).

5. Maintenance

The system is maintained once it has been deployed. This is the stage where improvements to defects that have been identified, modifications that are minor in nature, and issues relating to performance, are all addressed. Maintenance, as the term suggests, is the process whereby the system is ensured to continue to function after it has been released (Pressman & Maxim, 2019).

Strengths of the Waterfall Model

The waterfall model offers a streamlined process, guided by documentation at every phase. This is clearly evident from the fact that every phase is required to obtain approval from project stakeholders before movement to the next phase. This structure makes it possible for project managers to identify how far project goals have been achieved, and perform effective control over that element (Pressman & Maxim, 2019). Where requirements are stable, this process simplifies planning and management greatly.

The documentation and control requirements of the model are suited to projects where the requirements are clearly laid out and will remain constant (Sommerville, 2016). Furthermore, projects that take place in settings with a high degree of formal documentation and control requirements will benefit from the model's documentation emphasis.

The Waterfall Model's Shortcomings

The Waterfall model's infrequency is famously documented in literature due to its bureaucratic tendencies. Each phase requires a formal sign-off to proceed, slowing approvals at all phases (Pressman & Maxim, 2019). Providing developers less control over the development process. Notably, requirements are volatile, increasing the inaccuracy of any scheduled or costed estimates.

This model does not suit any projects in which requirements are to continuously change throughout the development process (Sommerville, 2016). Requirements documentation is defined early in the process so any changes in the later phases are considered very costly and disruptive.

In addition, the late testing problem manifests as design issues are found too late in the development process, pushing the cost of rework and increasing project risk (Royce, 1970). Unpredictable experimental systems exacerbate this problem as systems are meant to behave in a manner that is not fully defined

Paired with the lack of development engagement from stakeholders, a greater distance to the development process increases the likelihood that the resulting system does not meet user expectations (Pressman & Maxim, 2019).

Suitability Analysis for VlunLab

The waterfall model starts to show its limitations with VlunLab. VlunLab is an automated web application vulnerability assessment system that continuously evolves and is risk aware, with a number of continuously changing features that include risk scoring, smart scanning (quick, deep, and targeted), scheduled scanning, trend detection, and email reporting.

These features call for continuous evaluation and iterative improvements. For example,

- Updating the risk scoring mechanism to account for scan results and integrate identified false positives.
- Optimizing the smart scanning modes to improve performance and detection.
- Adjusting trend analysis to account for the structured presentation and historical data.
- Security testing may show unforeseen behavior, requiring design revisions.

The waterfall model is not right for iterative refinement. Changes made after the requirements or design phases may necessitate going back to one of the earlier phases, which will lengthen the overall development (Sommerville, 2016).

Also, the nature of cyber security systems is that they are iterative and require continual validation. Because the waterfall model places a reliance on late testing, significant issues could remain undetected until the later phases of development (Royce, 1970).

Although the Waterfall approach has its advantages, such as developing detailed documentation and providing organized structured planning, it lacks the flexibility and support for VlunLab's needs for iterative refinements. Here, the Waterfall approach will be utilized as a baseline for comparison with other potential methodologies that may better facilitate iterative development and the optimization of risk.

3.1.3 Scrum Model

Understanding Scrum

Scrum is an iterative and incremental framework that is based on Agile methodology and is used for building and developing complex systems. Scrum is not a traditional linear methodology and encourages a different approach when developing a product so that an adaptable methodology can be used. Scrum is described in The Scrum Guide as an easy framework used to assist individuals, teams, and companies to create value through adaptable solutions to complex problems (Schwaber & Sutherland, 2020).

The Scrum framework integrates principles from Agile that value customer collaboration, delivery of working software in short timeframes, and positive response to changes (Beck et al., 2001). Scrum allows both new and modified requirements to be defined as a project progresses. The project aims to create an incremental improvement through development, which is divided into short, defined time intervals known as sprints. During sprints, project teams design, develop, and test specific features. The iterative nature of sprints allows teams to potentially inspect the project progress and improve their approach towards the project (Schwaber & Sutherland, 2020).

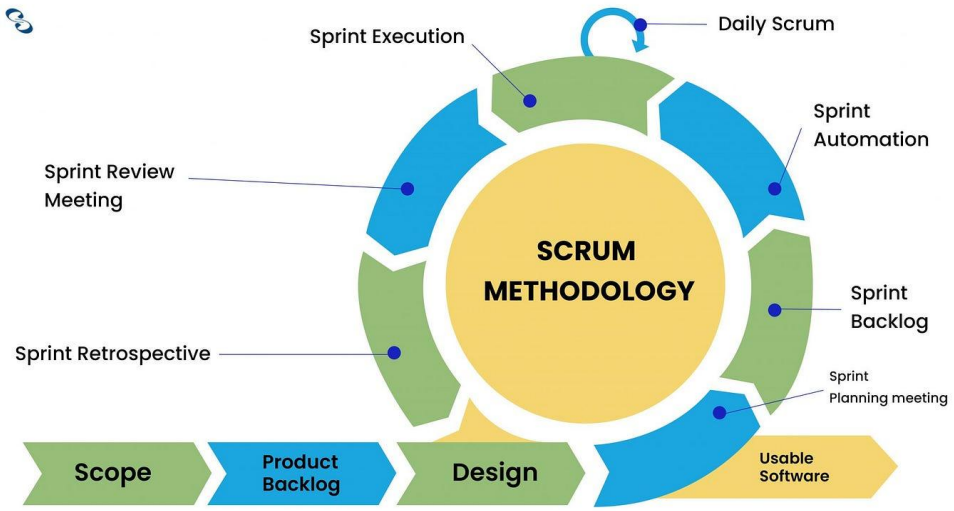


Figure 26 Scrum model

Components of Scrum

The Scrum framework uses defined processes, roles, and events to organize development.

1. Scrum Roles

There are three Scrum defined main roles (Schwaber & Sutherland, 2020):

- Product Owner - The role is responsible for product value maximization and for the Product Backlog. The Product Owner decides on which features to include and attempts to align them to the needs of stakeholders.
- Scrum Master— Provides support as a facilitator and guides Scrum principles. The Scrum Master alleviates blockers and improves team effectiveness.
- Development Team— A group of self-organized individuals paying effort to present increments of functional software at the closure of each sprint.

2. Scrum Artifacts.

Scrum is centered and highly dependent on the (three) artifacts (Schwaber & Sutherland, 2020):

- Product Backlog— A catalog describing the system's features, enhancements, fixes, and requirements organized in order.
- Sprint Backlog— A certain selection of developed items listed as part of the Product Backlog within a defined sprint duration.

- Increment— The product resulting from the sprint that is functional and satisfies the previously agreed Definition of Done.

3. Scrum Events (Sprints and Meetings).

The Sprint is the most significant event within the framework of Scrum; it is an iteration that has a fixed duration of two to four weeks (Schwaber & Sutherland, 2020). Each Sprint has the following events:

- Sprint Planning— This outlines the work to be done in the sprint.
- Daily Scrum— This is a brief daily meeting for coordination of work.
- Sprint Review— This event presents the complete increment and collects the input of various stakeholders.
- Sprint Retrospective— This event assesses the processes and identifies ways the processes can be improved in the subsequent sprint.Scrum embraces structured cycles to improve the continuous review and modification process in order to lessen the ambiguity that comes with intricate assignments.

Strengths of Scrum

Scrum is lauded for its high degree of adaptability. From the standpoint of the project as a whole, the fact that the development process is divided up into brief phases and that new requirements or changes are fully accommodated within new development cycles is a great advantage to this process (Sommerville, 2016).

The willingness of stakeholders to be involved in the process at every stage is another strong point in favor of Scrum. Given the fact that stakeholders can assess and critique where the Sprint Review process is at in the development process, there is a great deal of assurance that what is developed will actually be of value (Schwaber & Sutherland, 2020).

The early and consistent presenting of functional software that Scrum demands is another critical strength that helps facilitate the process of software development. From the perspective of the software development life cycle, allowing development to establish a point where development can actually begin software testing is a strong logistical advantage for software development. Development cycles can actually be supported, in part, by iterative testing (Pressman & Maxim, 2019).

The factual basis from which to develop a software system to ensure that no serious flaws will be discovered in the last stages of development is the iterative testing method.

The last silogism concludes that when Scrum is effectively implemented, its demand for high levels of productivity and creativity from teams is encouraging (Beck et al., 2001).

Weaknesses of Scrum

Scrum also has its share of detractors and disadvantages. It is obvious that one of the weaknesses of Scrum is the requirement for a very high degree of corporate or operational discipline and a great deal of involvement from stakeholders in a very active way (Sommerville, 2016).

Scrum may also pose weaknesses in projects that require elaborate or rigidly structured documentation because Agile frameworks place less emphasis on heavy documentation and more on working code (Beck et al., 2001).

The Agile approach makes scope control more difficult. If there is no clear prioritization of backlog items, items may keep getting added, and the scope is likely to keep creeping if there is insufficient control on the Product Owner's part (Pressman & Maxim, 2019).

Suitability Analysis for VlunLab

In the particular context of VlunLab, the application of Scrum appears to be an appropriate choice. VlunLab is an automated web application vulnerability assessment system that is continuous and risk-aware, with components that are currently evolving, including risk scoring, smart scan modes, scheduled scanning, trend analysis, and reporting via email. Each of these elements requires incremental development in addition to iterative development.

For example:

- The risk scoring mechanism may necessitate further tuning and adjustments after analyzing the results of scans and rates of false positives.
- The smart scan modes may also require iterative adjustments in order to find the right trade-off between performance and detection.

- The trend analysis component may require further evolution once the historical scan data has been structured and appropriately analyzed.
- In the context of cybersecurity, continuous testing is necessary because of the system's constantly changing vulnerabilities

The sprint-oriented model of Scrum enables manageable increase in the development of specific components. Each sprint can develop a specific feature or improvement, which will then be followed by testing and evaluation. This will reduce the probability of having to rework the entirety of a component, allowing the team to identify issues in the early stages of component development (Schwaber & Sutherland, 2020).

Moreover, innovation and continuous improvement are also supported by the iterative and flexible processes in Scrum. This matches the project's aim to achieve SDG 9: Industry, Innovation and Infrastructure, which focuses on the development of resilient management and rapid modernization of technology.

Scrum supports the iterative refinement characteristic of cybersecurity tool development. This is in contrast to the old linear models of project management. Therefore, it is quite clear that VlunLab would benefit from this methodology the most.

3.1.4 Spiral Model

What is the Spiral Model

The spiral model is a combination of the iterative model and the risk management model (Boehm, 1988). Also formulated in the 1980s, the model was made to combine the pros of the waterfall models while working on the cons of the model, adding elements of risk throughout each step of the validation refinement loop, as well as continuously throughout the whole development cycle of the model. This model helps systems development through multiple models in a structured way.

The Spiral Model shows emphasis on developing and mitigating risks rather than working in a linear way of just developing or going step by step in the spiral. F. W. Boehm (1988) best describes it as \'an overall integration of risk assessment and incremental system development.\' This describes the model very well in terms of developing system models that are on the more

complicated and riskier sides. Every spiral is considered a phase, or cycle of a phase, in the overall system project, and helps the system as a whole through the steps of development by refining the system before moving further.



Figure 27 Spiral Model

Risk-Based Circles of the Spiral Model

The Spiral Model is structured in a series of repeats, otherwise known as a spiral. Every spiral is usually made up of four sections (Boehm, 1988).

1. Sets of Objectives: This section of the cycle is used to define the goals of the cycle, define the system requirements, define the system constraints, and define the system alternatives. Setting objectives for the cycle is the most important system of the section.

2. Identification & Mitigation of Risk: This is the fundamental building block of the Spiral Model. It involves identifying and evaluating possible risks, including technical, operational, cost, and security risks. Before proceeding to full-scale development, prototyping, simulation, or other experimental work may be done to eliminate uncertainties (Boehm, 1988).

3. Development and Verification: Based on the analyzed risks, the system or one of its increments is developed and verified. This may include some combination of design, programming, integration, and testing.

4. Planning for Further Iterations: Once verification is complete, the outcome of that cycle is analyzed. Pertaining decisions are made for the following development stage, including allocation of resources and the adjustment of scope. The spiral then advances into the following cycle.

Each of the spiral routes expands progressively, and for uncertainties to dissipate, the system is refined. This cyclical spiral process continues until the system is complete.

Benefits of the Spiral Model

A primary benefit of the Spiral Model is its vigorous focus on risk analysis. The model optimizes the chances of success and minimizes the chances of failure by looking at and attempting to eliminate the risk at each cycle (Boehm, 1988).

The ability to progressively modify system requirements and design is an additional advantage of the Spiral Model. Because of this, it is effective for larger, more intricate, and more advanced systems in which the requirements may not be fully comprehended at the start (Sommerville, 2016).

The Spiral Model is also beneficial for early stage prototyping, which allows stakeholders to assess incomplete versions of the system and offer revisions prior to the complete implementation of the system. This decreases uncertainty and helps the system align better with what the stakeholders want (Pressman & Maxim, 2019).

The Spiral Model's Weaknesses

The Spiral Model does, however, have its disadvantages. Its complexity is one of the major obstacles. The continuous risk analysis process has the potential to increase project overhead because it requires more structured management and technical expertise (Sommerville, 2016).

The model may also demand an excessive amount of resources. Because small projects and projects with limited time are less likely to be appropriate, the time and costs associated with the numerous risk assessments, and the associated prototypes and evaluations may also be significant (Pressman & Maxim, 2019).

In order to reap the best benefits of the Spiral Model, one must have the ability to pinpoint potential risks. If risks are not accurately identified and assessed and the risks are identified, the advantages of using the Spiral Model are greatly diminished (Boehm, 1988).

Suitability analysis for VlunLab

The Spiral Model for VlunLab shows a few positive aspects. VlunLab is a risk-aware automated web application system that assesses the vulnerabilities of a web application. Cybersecurity projects include technical and operational risks. The Spiral Model focuses more on the risks of system assessment.

For example:

- The risk scoring mechanism can be improved with more cycles of iterative evaluations.
- Security-related risks such as false positive, performance bottleneck, and incomplete detection of vulnerabilities can be diminished with early cycles.
- Different scanning strategies can be prototyped to minimize technical uncertainties before the overall implementation.

Nonetheless, the Spiral Model is aligned with the risk management system, but for an academic prototype like VlunLab, it could add unnecessary complexity. Given that the project is time-bound and that it is an individual undertaking, the extensive risk management and iterative cycles of formal evaluations may be excessive (Sommerville, 2016) .

While the Spiral Model is applicable and suited for extensive industrial systems that are large and carry high risks, the time management and documentation that is known to be high for the scope and time available to VlunLab may mean that the alignment that is risk driven with the nature of Cybersecurity Development Spiral Model is conceptually the case, but practically the iterative frameworks that are more simplistic and that are less intensive may actually be a more efficient option than applying the Spiral Model.

3.1.5 Comparison of Models

In order to ascertain the development methodology best suited for VlunLab, the Waterfall, Scrum, and Spiral models will be evaluated pertaining to the specifics of the project. Comparisons will be made regarding the models in relation to project flexibility, stakeholder engagement, risk mitigation, cyber system development, evolving requirements, and the level of documentation required by each model

Criteria	Waterfall	Scrum (Agile Framework)	Spiral Model
Development Approach	Linear and sequential	Iterative and incremental	Iterative and risk-driven
Flexibility to Change	Low	High	High
Risk Management	Limited, mostly late-stage	Managed through iteration	Strong emphasis on risk analysis
Testing Approach	Primarily after implementation	Continuous within sprints	Conducted in every cycle
Stakeholder Involvement	Limited after requirements phase	Continuous through reviews	Moderate, especially during risk evaluation
Documentation Level	High and formal	Moderate and adaptive	High and structured
Complexity	Low to moderate	Moderate	High
Suitability for Innovative Systems	Low	High	High
Suitability for Small Academic Projects	Moderate	High	Moderate to Low

Table 3 Comparing Models

The Discussion about VlunLab

VlunLab is being constructed as an automated web application vulnerability assessment system that is continuous and risk-aware; it incorporates dynamic features such as risk scoring and smart scanning, undergoing scheduled scans, performing trend analyses, and emailing reports. Each of these elements is going to require ongoing development, testing, and real scan results refinement, plus performance feedback.

The Waterfall model offers solid documentation and organized preemptive processes, but the rigidity of its linear sequence is a primary shortcoming. Since VlunLab's risk scoring and smart

scan mode components will need iterative refinement, later stages of testing and inflexible phase shifts in the Waterfall model make it a poor fit for this project (Sommerville, 2016).

Because the Spiral model's primary focus is on risk analysis and mitigation, it is a good fit for the development of cybersecurity components (Boehm, 1988). Given the focus on security issues present in vulnerability assessment systems, the Spiral model aligns well with the overall development of VlunLab. However, for an individual academic project with a limited amount of time, the increased risk management complexity and burdensome overhead are not ideal (Pressman & Maxim, 2019).In contrast, Scrum tends to be more flexible and allows for more structure with its iterative sprint-based system and processes that can incrementally be built and considered and evaluated over time (Schwaber & Sutherland, 2020), such as vulnerability scanning, risk scoring, and reporting modules. With the ability to assess and refine a feature before the product gets released, the risk of having to do large scale retrofits on the product in the future is lessened. In the case of a product that is focused on cybersecurity, the benefit of having each sprint cycle be able to incorporate testing is particularly important. This is because validating protection mechanisms early on in development is critical.

From the perspective of sustainable innovation, choosing an adaptive method also aligns with Sustainable Development Goal (SDG) 9 (Industry, Innovation and Infrastructure). SDG 9 focuses on innovative and resilient technological infrastructure. The development of VlunLab with an iterative and adaptive approach will allow the system to be more resilient, continuously innovative through iterative refinements, and provide a solution that enhances the digital security infrastructure.

In conclusion, while the Waterfall methodology offers good structural clarity, and the Spiral method offers good management of risk by having good built in mechanisms to manage those risks, Scrum offers a more balanced methodology that would work for the VlunLab. It promotes innovation, allows iterative refinements, is good at containing complexity, and aligns best with the project's technical needs as well as the academic scope.

3.1.6 Selected Methodology and Justification

Through the comparison of the three methodologies consisting of Waterfall, Scrum, and Spiral, the most appropriate development framework for VlunLab is Scrum.

VlunLab is an automated web application that assesses vulnerabilities through a continuous and risk-aware approach. It aims to incorporate evolving features that include risk scoring, smart scanning modes (which can be quick, deep, targeted), scheduled scanning, trend analysis, and email-based reporting. In order to achieve this, Scrum methodology will be utilized to meet the incremental development, continuous validation, and iterative refinements. These are all a part of the sprint-based, iterative structure of Scrum (Schwaber & Sutherland, 2020).

Scrum is most appropriate for VlunLab development as it will be built feature by feature. Each sprint can be concentrated on one functional component. The first 4 or 5 sprints for example, can be focused on implementing and integrating the core vulnerability scanning engine, risk scoring, smart scanning modes, trend analysis, and reporting functionalities. This incremental approach also allows for manageable development cycles and reduces the chances of experiencing large-scale rework (Pressman & Maxim, 2019).

Project requirements will likely change throughout the project. For example, the mechanism for determining risk may need adjustment based on the results of the scans, and the smart scan modes may need to be adjusted to better detect and enhance the performance. As stated, adjustments can be made within sprints in the Scrum framework without altering the entire project plan (Sommerville, 2016).

Another important aspect of the Scrum methodology for project management is the continuous testing within the framework. In the case of the cybersecurity system, for example, validation for accurate detection of vulnerabilities and to minimize false positives is of utmost importance. As a methodology that emphasizes iterative development, Scrum allows testing to be done throughout the sprint of a project, rather than pushing validation to the end, and helps to minimize issues in a more timely manner.

Besides, for the time being, VlunLab is being designed as an academic prototype system within a set time. Where the complexity of the Spiral Model involves cycle(s) of risk analysis to a greater extent, Scrum is more simplistic while still offering the framework to iterative development and collaboration without excessive complexity. In risk-prone environments, the Spiral Model has its strengths (Boehm, 1988). For this reason, the complexity of it is likely to be greater than the scope of this singular academic project.

On a macro scale, the adoption of Scrum allows for innovation and adaptive system design, which can be linked to Sustainable Development Goal 9 (Industry, Innovation and Infrastructure). Scrum allows for the iterative refinement and practice of resilience system development, which assists in digital infrastructure development and technological innovation in the field of cybersecurity solutions. Thus, based on flexibility, iterative capability, continuous validation, manageable complexity, and alignment with project objectives, Scrum was chosen as the development methodology for VlunLab.

3.2 Data Gathering Design

3.2.1 Data Gathering techniques

The results of collecting the data in this project were done through online survey that was administered using Google Forms. The survey was structured in a way that gathers the opinions of the target group, consisting of people who work in the web development, cybersecurity and IT-related industries as well as small business owners who use and/or operate web applications. This survey was aimed at gaining a better insight into how users are aware of the vulnerabilities of web applications, how they are experienced when using vulnerability scanning tools that are available, and what problems they experience when conducting the security assessment.

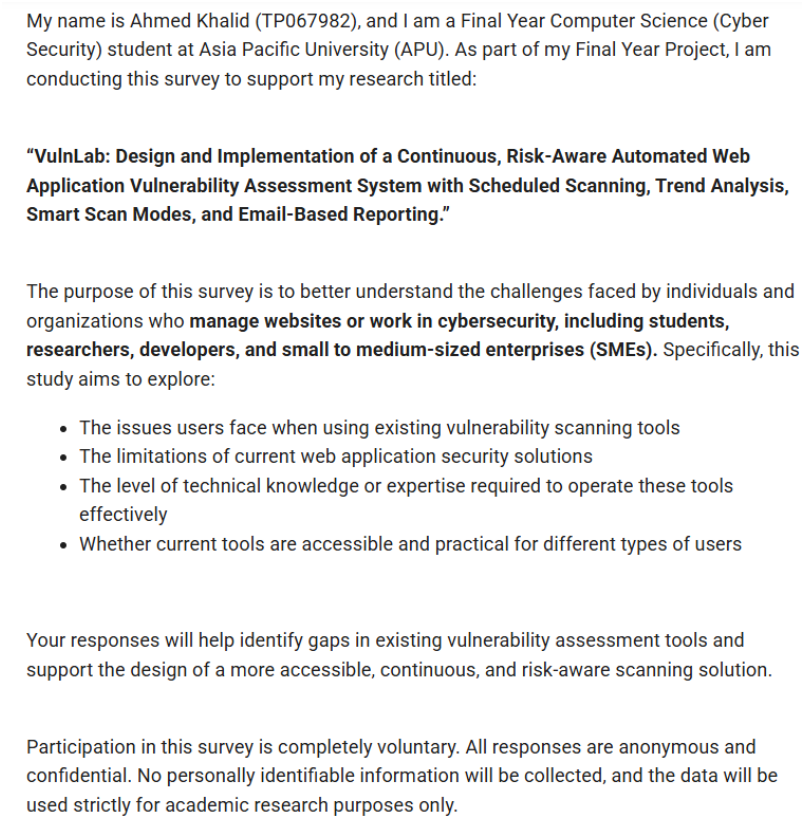
The questionnaire had 22 structured questions that included multiple-choice and opinion-based questions that were intended to assess the usability, awareness and expectations of vulnerability assessment tools. The questionnaire was made publicly available on the internet in a bid to capture more people as part of the target market of this survey.

The data collection process was done ethically. The survey was conducted on a voluntary basis and the respondents were notified of the purpose of the study prior to giving their responses. All the answers were anonymous and the data given by the respondents is confidential. Moreover, informed consent was obtained by the respondents prior to filling the questionnaire and this is one of the reasons why the research adheres to the principle research ethics.

The information gathered in the form of the responses will be processed to determine frequent difficulties, user requirements, and expectations towards automated vulnerability scanning

systems. These will be used in designing and developing the proposed system, VulnLab, to ensure that it meets the real-life usability and security testing requirements.

3.2.2 Survey questions



My name is Ahmed Khalid (TP067982), and I am a Final Year Computer Science (Cyber Security) student at Asia Pacific University (APU). As part of my Final Year Project, I am conducting this survey to support my research titled:

"VulnLab: Design and Implementation of a Continuous, Risk-Aware Automated Web Application Vulnerability Assessment System with Scheduled Scanning, Trend Analysis, Smart Scan Modes, and Email-Based Reporting."

The purpose of this survey is to better understand the challenges faced by individuals and organizations who **manage websites or work in cybersecurity, including students, researchers, developers, and small to medium-sized enterprises (SMEs).** Specifically, this study aims to explore:

- The issues users face when using existing vulnerability scanning tools
- The limitations of current web application security solutions
- The level of technical knowledge or expertise required to operate these tools effectively
- Whether current tools are accessible and practical for different types of users

Your responses will help identify gaps in existing vulnerability assessment tools and support the design of a more accessible, continuous, and risk-aware scanning solution.

Participation in this survey is completely voluntary. All responses are anonymous and confidential. No personally identifiable information will be collected, and the data will be used strictly for academic research purposes only.

Figure 28 Survey introduction

This introduction was put to give the participants a clear explanation of the purpose and context of the survey. It gives the respondents a notion that the survey is a Final Year Project that is being undertaken in Asia Pacific University and briefly presents the subject of research, which is a vulnerability research on automated web application assessment systems. The background provided will give the participants an idea of the relevancy of the study and the role their response would play in the development of the proposed system, VulnLab.

Moreover, the section will describe the target population of the survey, which will consist of persons that run websites or work in the area of cybersecurity, including students, developers, researchers, and small and medium-sized businesses. This makes the responses obtained to be relevant to the objectives of the research.

The introduction also describes the areas that are being studied the most such as the difficulties in using the existing vulnerability scanning tools, the shortcomings of the present security solutions, and the usability of the tools to the various users.

Ethically, the participants were made aware of the fact that taking part in the survey is in no way compulsory, and all the answers would remain anonymous and confidential. No personal identifiable data were gathered and the information is utilized with the purpose of academic research. Furthermore, a consent confirmation question was added to the survey subsequently, in which the respondents would have to assert that they are willing to participate voluntarily and that the survey data could be utilized in the research. This makes the data collection exercise adhere to the normal research ethics and informed consent practices.

Section A: Demographic Information: This section will document the demographic data of the participants who will engage in the research.

The inclusion of this section was aimed at collecting demographic basic data concerning the respondents who took part in the survey. Demographic data of the participants can serve to give a background of interpreting the results of the survey and also to ensure the answers will be used to represent a wide range of users within the targeted population.

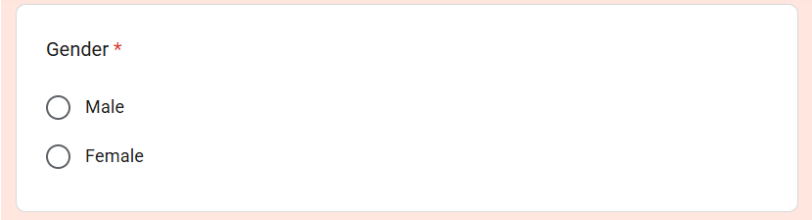


Figure 29 Gender question

The question on gender was added to know the gender proportions of the respondents who took part in the survey. This assists in giving some form of background information on the respondents and this enables the researcher to interpret whether the responses given are a representative sample of the target audience.

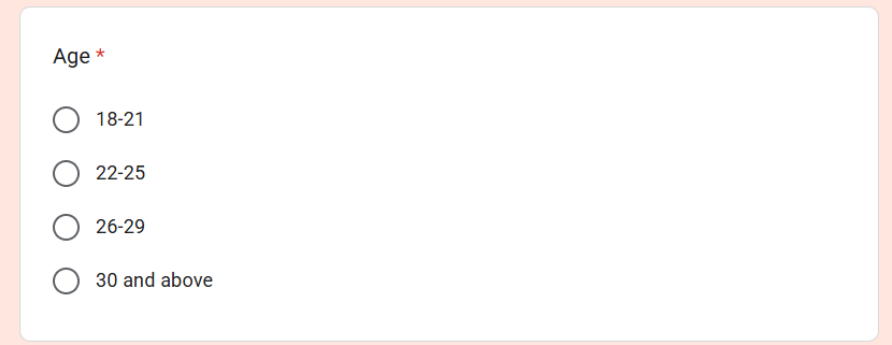


Figure 30 Age question

The question on age was also added to determine the age group of respondents who took part in the survey. This will aid in giving background details about the respondents and the researcher will know the age groups that are best represented in the responses.

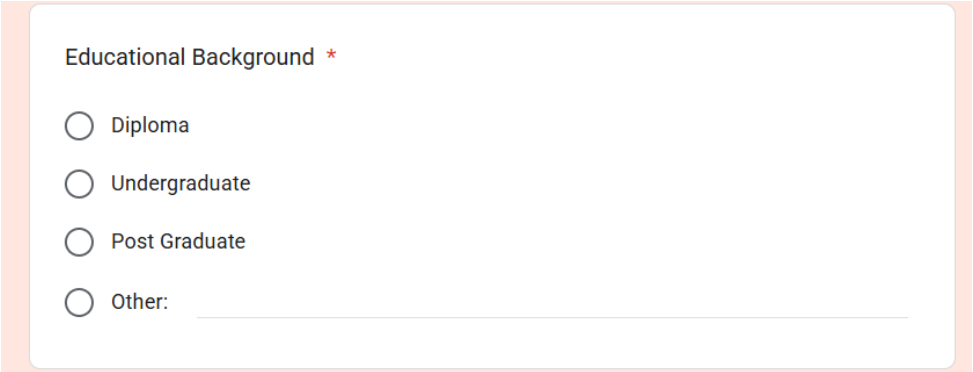


Figure 31 Educational background question

The question of the educational background was included in order to determine the educational level of those who participated in the survey. This helps to establish whether the respondents have a technical or educational background that can impact on their knowledge and experience in regards to cybersecurity and vulnerability scanning tools.

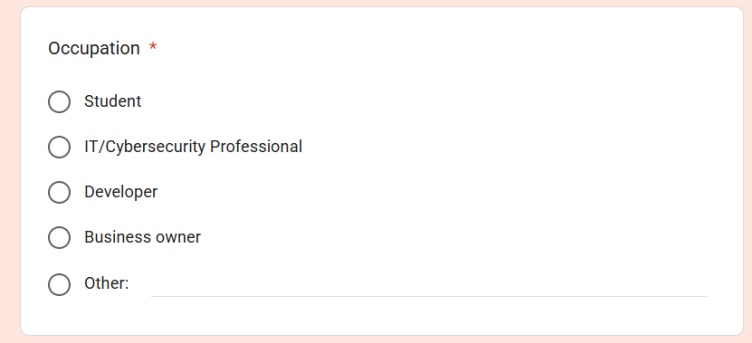


Figure 32 Occupation question

The occupation question was incorporated to determine the professional background of the respondents. It can aid in identifying the participants as students, cybersecurity professionals or developers, or business owners, which will assist the researcher to get a clearer idea of their exposure to web application security and vulnerability assessment tools.

Section B: Website Security Awareness and Perception

The section has been added to determine the level of awareness and perception of the respondents on the security of websites. The questions will be used to comprehend the level of trust of the users with the security of the websites, the frequency of the process of verification of the safety of the websites prior to entering the sensitive information and what opinion users have concerning the security of the modern web applications. This knowledge is useful to determine the level of security awareness and behaviors of the users, which is pertinent to comprehending the necessity of the availability of easy-to-use vulnerability assessment tools like VulnLab.

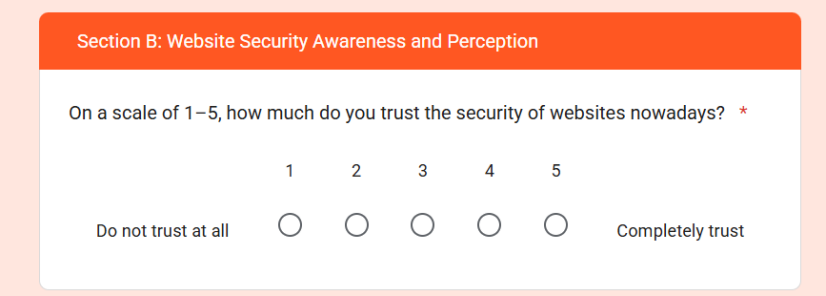


Figure 33 Trusting security of websites question

This question was also added to gauge how much the users have trust with security of websites these days. The perception of users on trust assists in determining whether or not people consider that modern web sites are not very dangerous or they have doubts about the security risks.

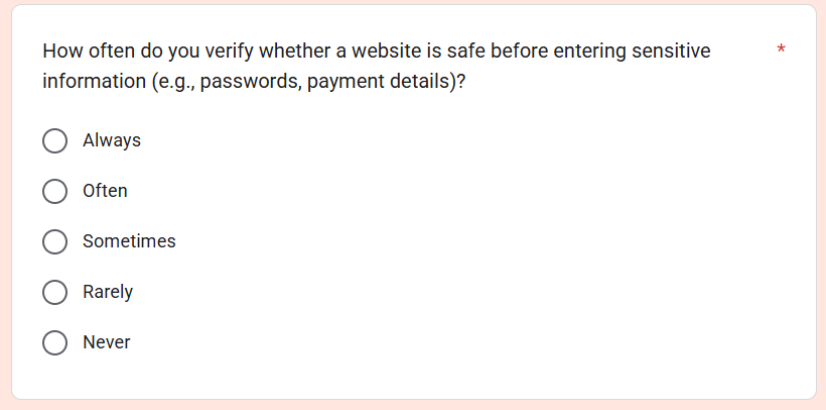


Figure 34 Verifying website security question

This question was to determine how the users behave in relation to security when using websites, whether they go the extra mile to find out the security of a site before submitting sensitive information to the site. The answers aid in showing the degree of caution and security consciousness among the users of online services.

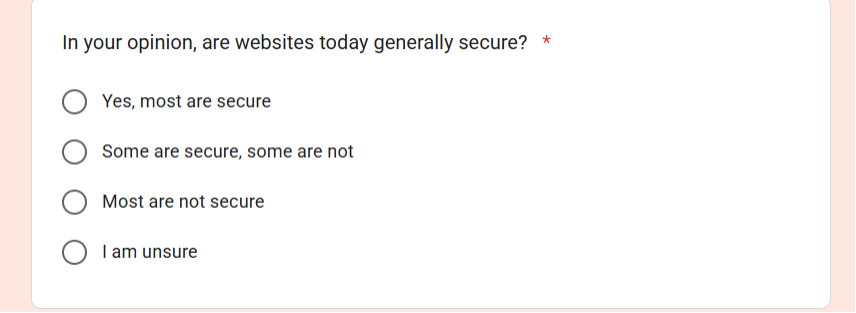


Figure 35 General website security question

This question was added in order to get the overall idea of respondents about the security of the modern websites. It aids in understanding whether the users think that in the present days the websites are considered to be usually safe or they still believe that the security concerns are widespread.

Section C: Open Feedback on Vulnerability Scanning Tools

The section was added because it was necessary to gather views and experience of the respondents concerning the vulnerability scanning tools. The rationale is to get insights on the problems users have to work with, the shortcomings of the available tools, and what people would want to see in the future vulnerability assessment solutions.

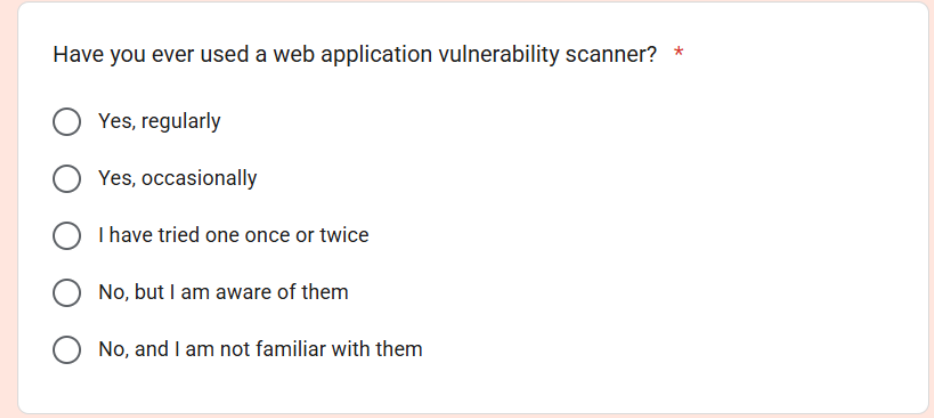


Figure 36 Use of vulnerability scanner question

This question was added to find out whether the respondents are already familiar with any form of web application vulnerability scanning tools. The responses would aid in the determination of the familiarity and hands-on exposure of users on such tools which would be an indication of the wide usage of such tools by the target audience.

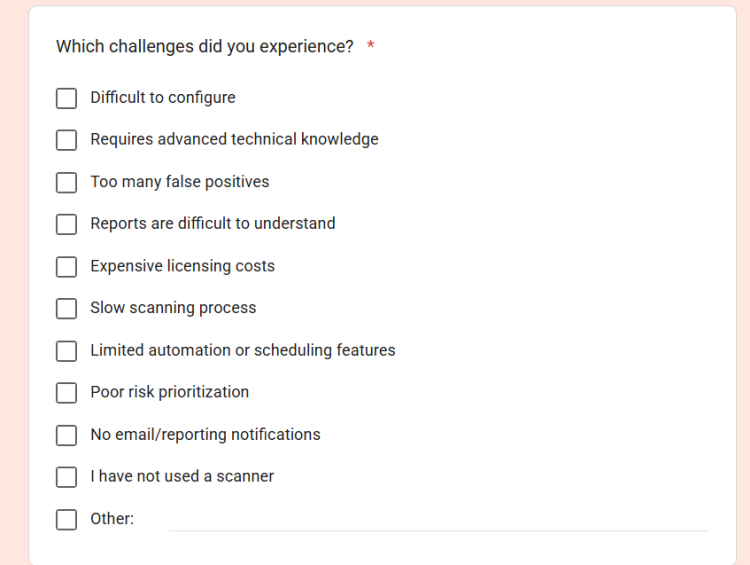


Figure 37 Challenges faced with vulnerability scanners question

This question was added to determine the general difficulties posed to the users when using vulnerability scanning tools. Knowledge of these challenges aids in the identification of weaknesses of current tools, including complexity, cost, false positive or non-automation, and gives a reflection on what enhancements can be made.

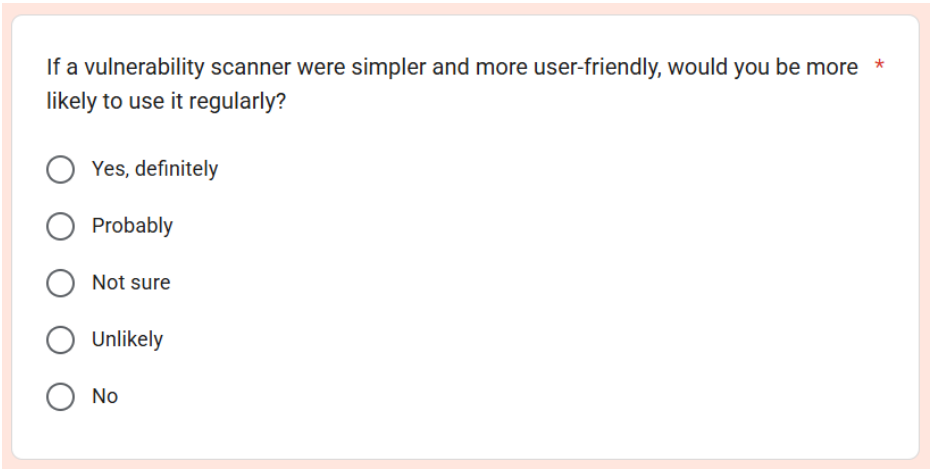


Figure 38 If scanners were simpler question

This question was added to know whether simplicity and ease of use affect the willingness of the users to use vulnerability scanning tools on a regular basis. The reactions facilitate the conclusion as to whether usability is a major criterion influencing the adoption and frequent use of these tools.

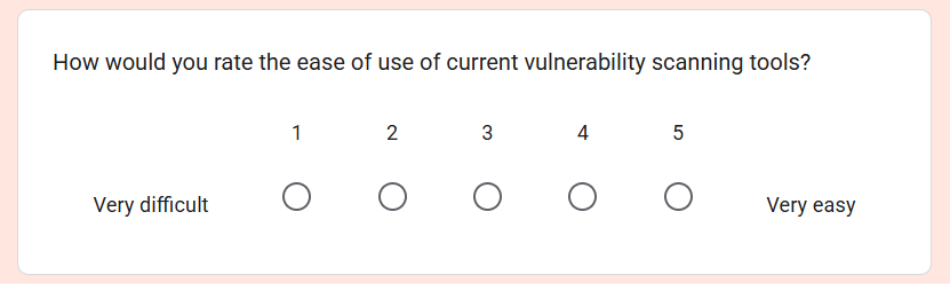


Figure 39 Ease of use for scanner question

This question was added to determine how users perceive the usability of the existing tools of vulnerability scanning. The responses allow to define the existing tools as difficult or easy to use, which gives an idea about the problem of usability among the users.

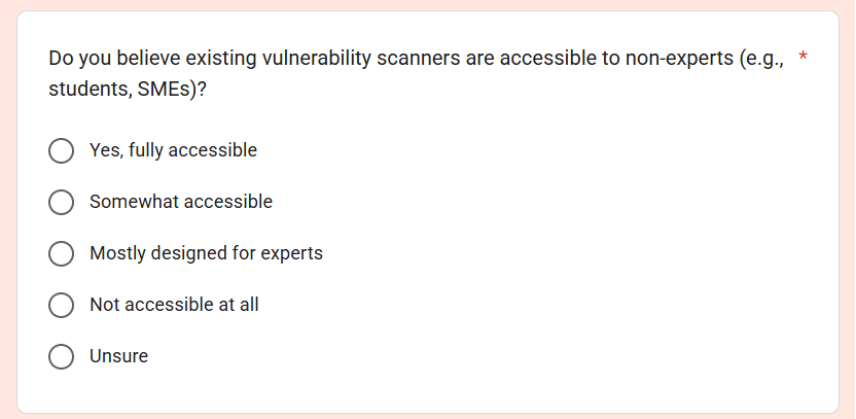


Figure 40 Scanners accessibility question

This question was added to determine whether the respondents think that the vulnerability scanning tools are currently available to users without high levels of technical skills. Those responses can be used to determine whether the tools can be used by a broader audience or the tools are aimed primarily at experienced professionals.

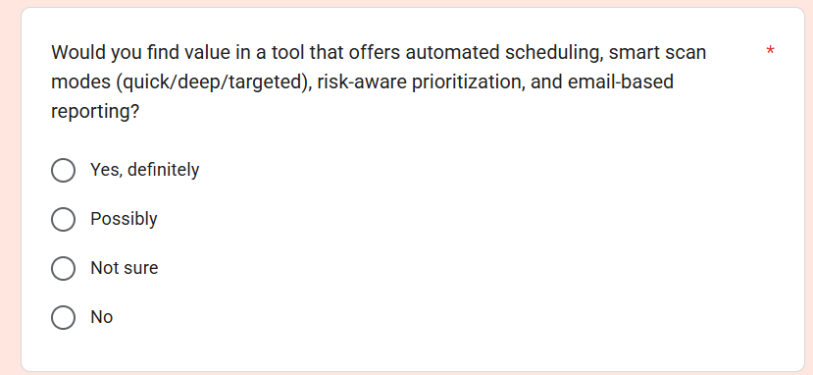


Figure 41 Tools with addition features question

Figure 3.13

The purpose of this question was to establish the value of such additional features as automated scheduling, smart scan modes, risk-based prioritization, and email reporting to users. The feedback will aid in determining the features that users find useful when using vulnerability scanning tools.

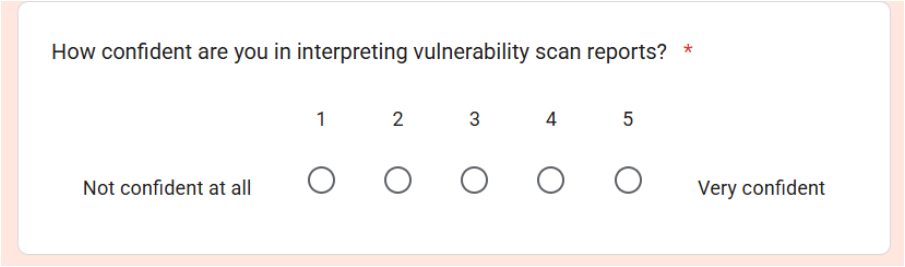


Figure 42 Confidence in reading scan reports question

Figure 3.14

This question was added to the questionnaire to determine the level of confidence in interpreting vulnerability scan reports. The responses will assist in determining whether the users find these reports comprehensible or they need greater technical expertise to interpret the results.

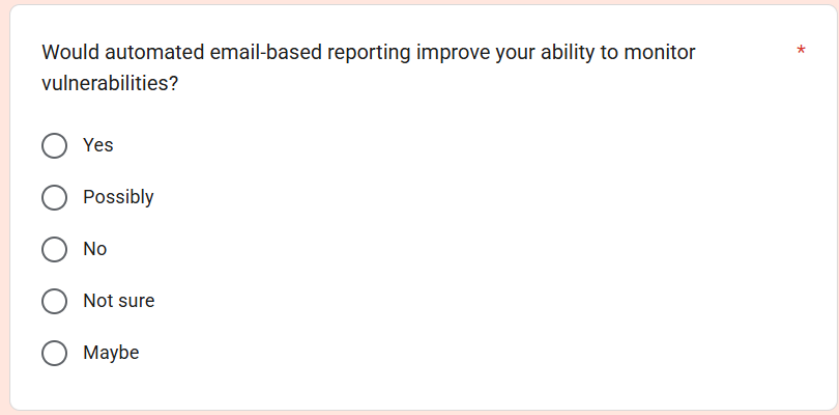


Figure 43 Automated email-based scanners question

Figure 3.15

The question was added to know whether automated email-based reporting would aid the users in tracking susceptibilities better. The answers are useful to identify whether automated notifications are useful in keeping the users informed about security problems.

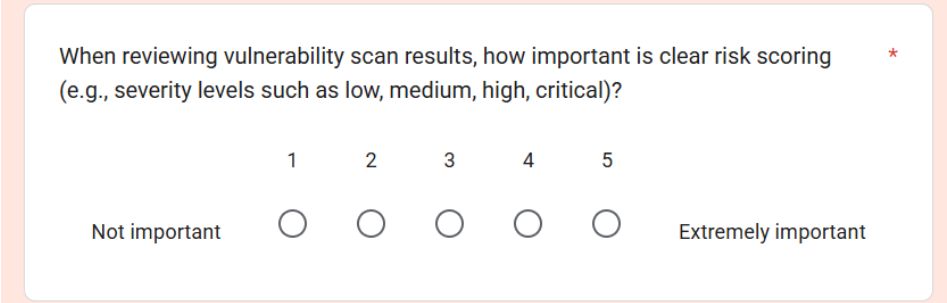


Figure 44 Clear risk scoring question

This question was added to learn the significance of clear risk scoring to users who go through the results of vulnerability scan. The answers will assist in identifying whether the users use the level of severity to outline the prioritization and greater comprehension of the detected security problems.

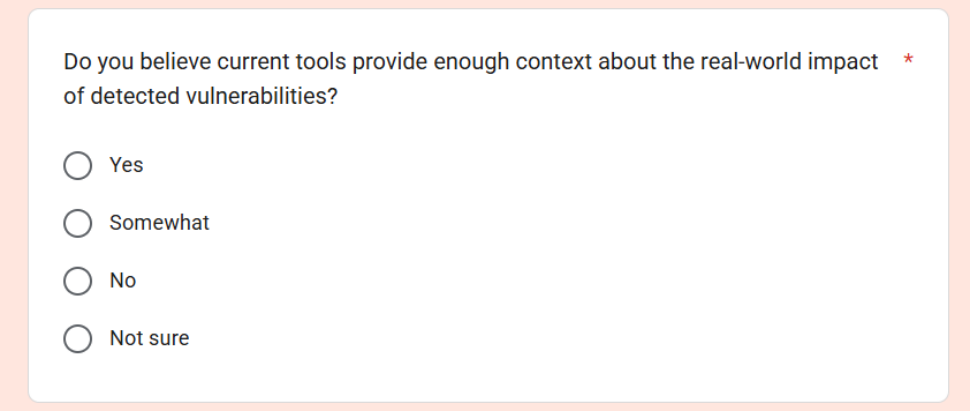


Figure 45 Current tools review question

The question was also there to check whether the present vulnerability scanners offer adequate elaboration on the actual implication of the discovered vulnerabilities. The answers will assist in determining whether the users believe that the current tools clearly point out the dangers of security concerns.

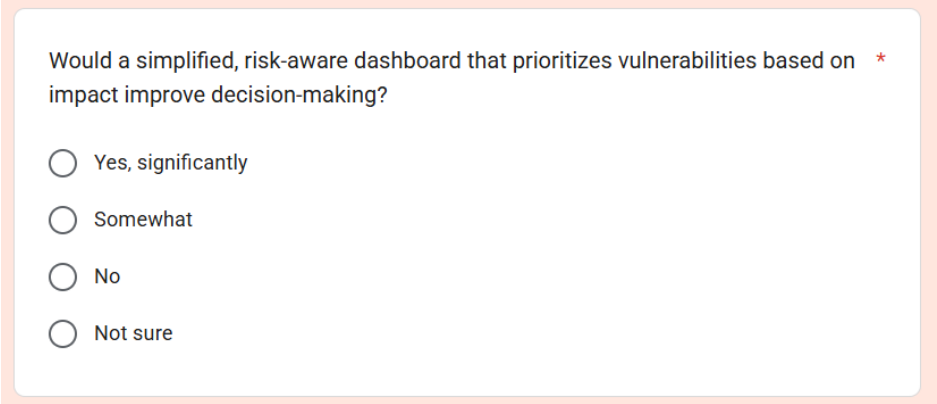


Figure 46 Simplified risk-aware dashboard question

The purpose of including this question was to determine whether the simplified dashboard that prioritizes the vulnerabilities according to their impact would assist a user in better making security decisions. The answers assist in establishing the relevance of visualization and ranking the risks in the management of vulnerabilities.

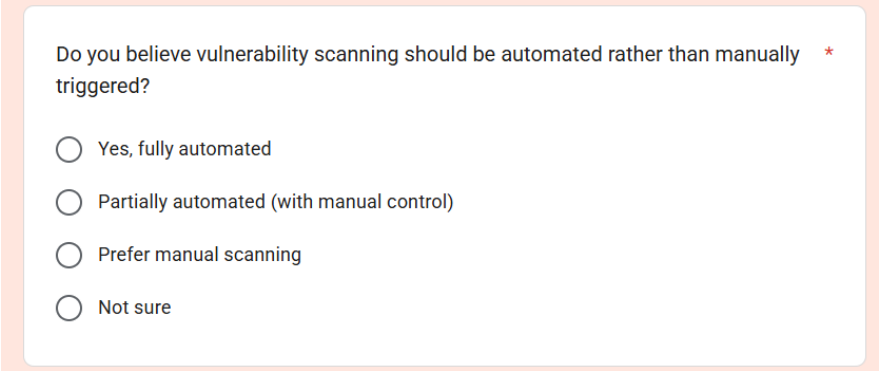


Figure 47 Automated scanners question

This question was added to get to know the preferences of the users where they related to automated and manual vulnerability scanning. The answers assist in regulating the priorities of the users with regard to using automated processes of scanning files or manually managing the security checks.

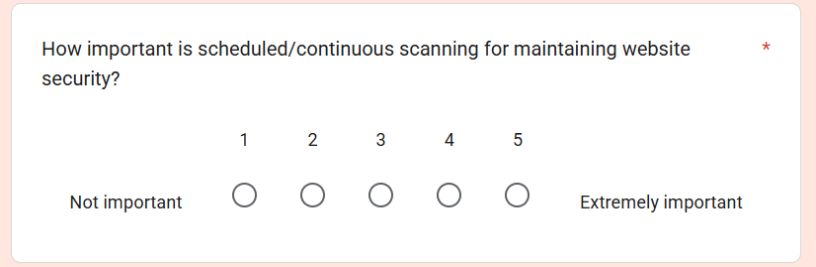


Figure 48 Importance of scheduled scanning question

This question was added to measure the level of importance of users to scheduled or uninterrupted vulnerability scanning to keep their websites secure. The answers aid in determining whether automated scanning regularly is considered as important practice in security risk management.

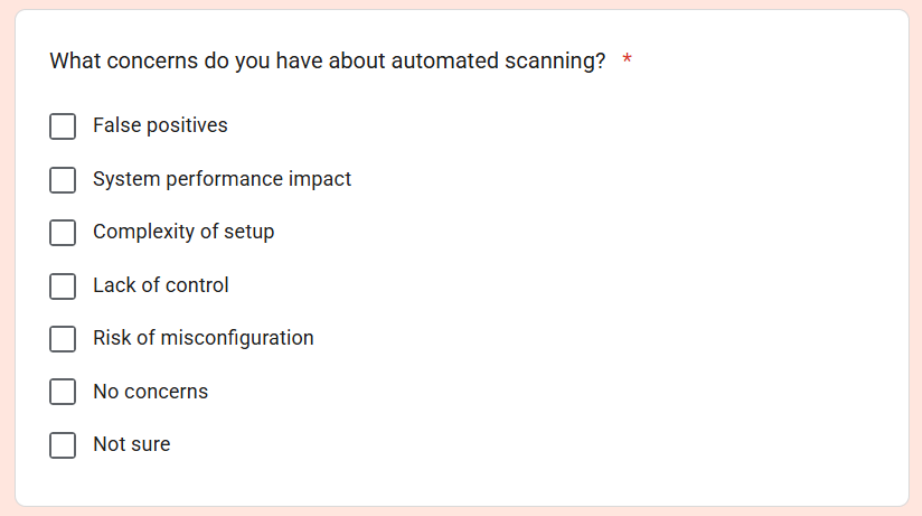


Figure 49 Concerns about automated scanning question

The question was added so that the potential concerns of users towards automated vulnerability scanning could be identified. The answers allow one to consider the potential problems like false positives, performance, or configuration risks that users might relate to automated scanning systems.

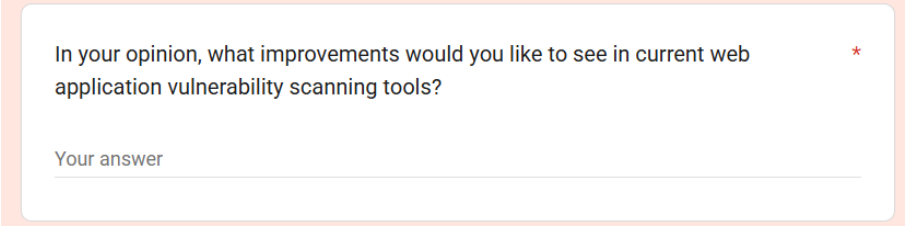


Figure 50 Open ended question on improving scanners

This question was added to garner the open-ended responses of the respondents on what they would prefer to be improved in the existing vulnerability scanners. The answers enable the participants to give their own opinion, recommendations and experiences and this can bring up issues or ideas not discussed by the structured questions.

Thank you message

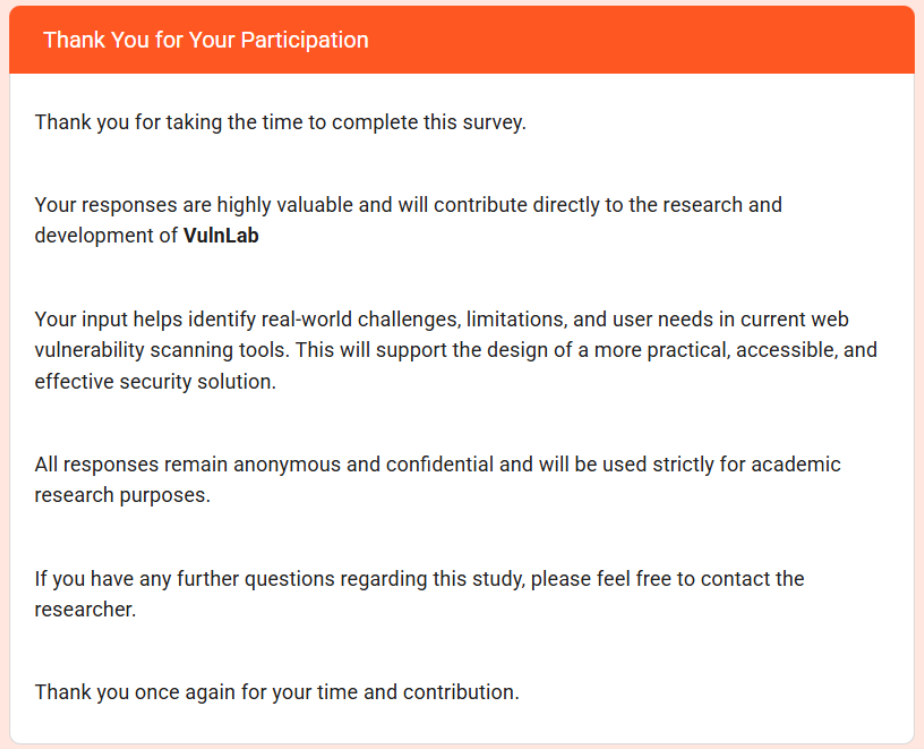


Figure 51 Survey thank you message

This last section was added to give the participants thanks to their time and efforts put on completion of the survey. It also helps to reassure the respondents that their answers are anonymous and confidential and helps to remind them that the obtained data will be utilized solely in the scope of the academic research.

3.3 Analysis

This part gives the analysis of the answers obtained through the survey. The findings are discussed in order to establish patterns, trends and revelations about the awareness, experiences and opinion of users in matters pertaining to websites security and vulnerability scanning tools.

Section A: Demographic Information

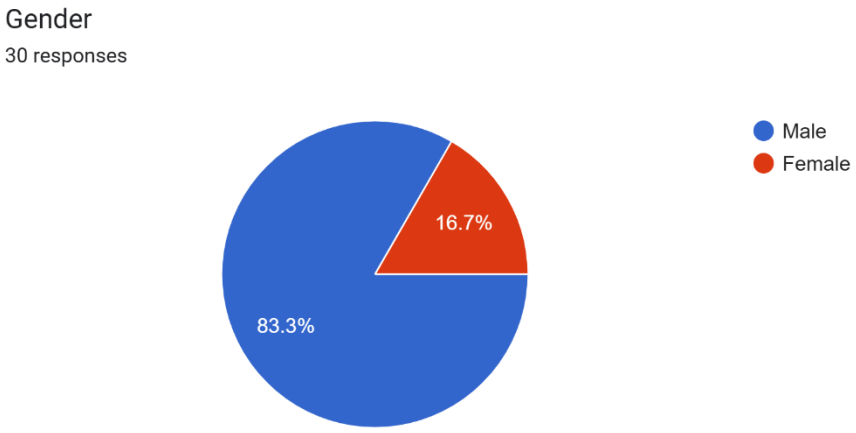


Figure 52 Gender analysis

The gender distribution of the respondents shows that 83.3 percent of people who participated in the survey were males whereas 16.7 percent were females. This indicates that most respondents in the questionnaires were men. The outcome can also be a mirror of the demographic profile of the sample population that participated in the survey and not the cybersecurity community on the whole.

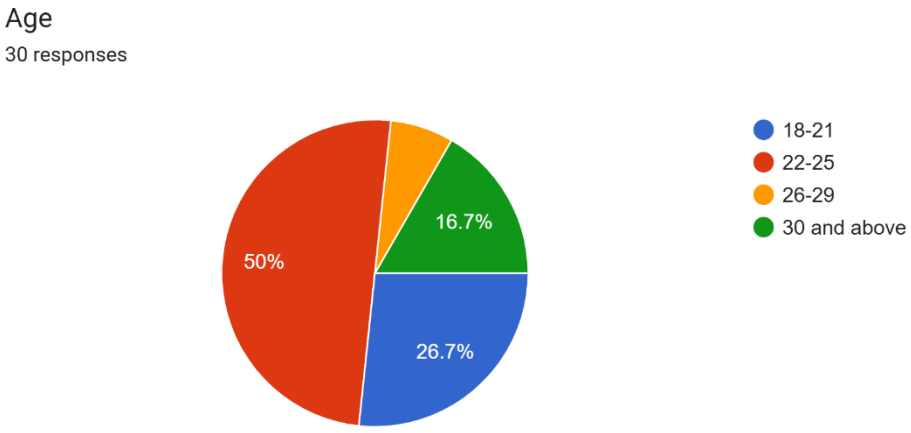


Figure 53 Age analysis

The findings indicate that 50 per cent of the respondents were aged between 22-25, 26.7 per cent aged 18-21, 16.7 per cent aged 30 and over and a small proportion were within the ages of 26-29.

This means that majority of the respondents were young adults especially those who are at the university age.

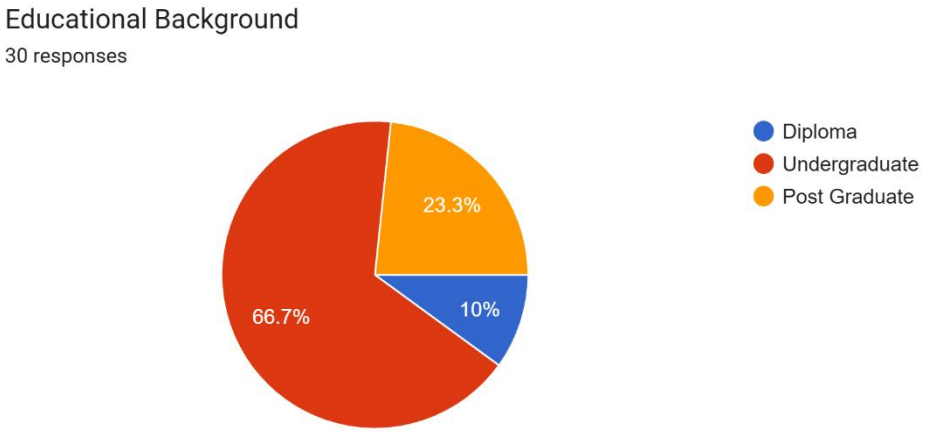


Figure 54 Educational background analysis

Most of the respondents (66.7) were undergraduates with 23.3 having postgraduate and 10 with a diploma qualification. This implies that majority of the participants are academically educated in the field of higher learning.

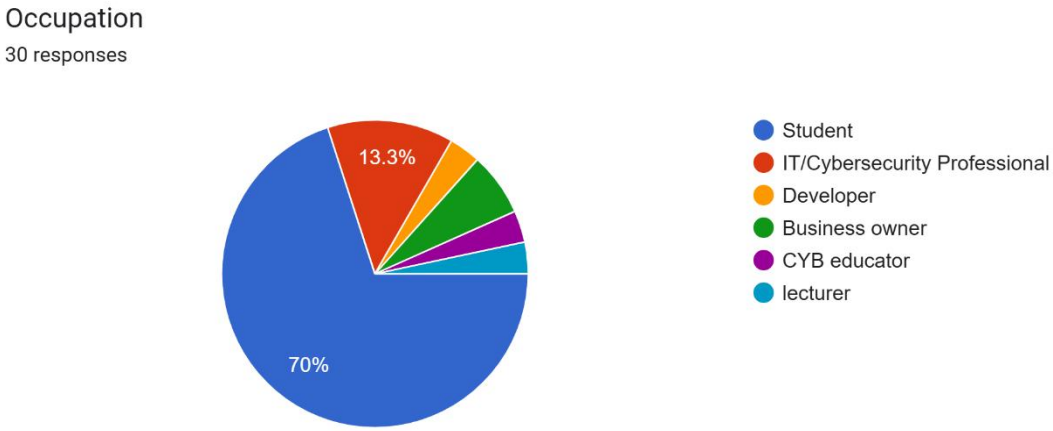


Figure 55 Occupation analysis

The occupation distribution revealed that 70% of the respondents were students, and 13.3% had IT or cybersecurity professionals. The other respondents comprised developers, business owners, cybersecurity teachers and lecturers with a combination of both academic and professional experience.

Section B: Website Security Awareness and Perception

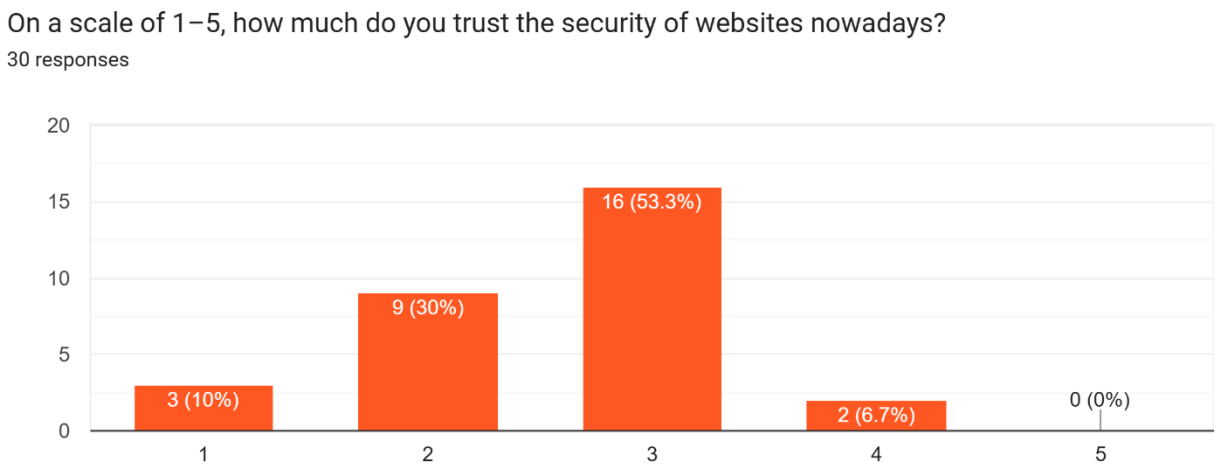


Figure 56 Trusting website security analysis

Its findings indicate that the respondents have a moderate level of trust in the security of websites with majority (53.3 percent) having a level 3 score. The few participants who rated their trust as 1 and 2 but very few rated it 4, which shows that the users are rather apprehensive of the security of the websites.

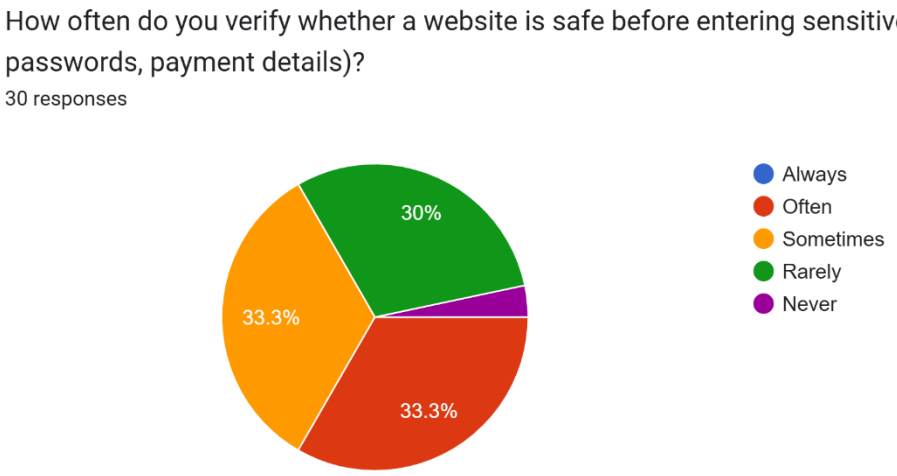


Figure 57 Verifying if a website is safe analysis

The answers demonstrate that 33.3 percent of the respondents frequently check the safety of websites, and 33.3 percent occasionally check it before providing sensitive data. Nevertheless, 3 out of 10 hardly ever carry out these checks, which means that a large number of users do not always check the security of websites before providing sensitive information.

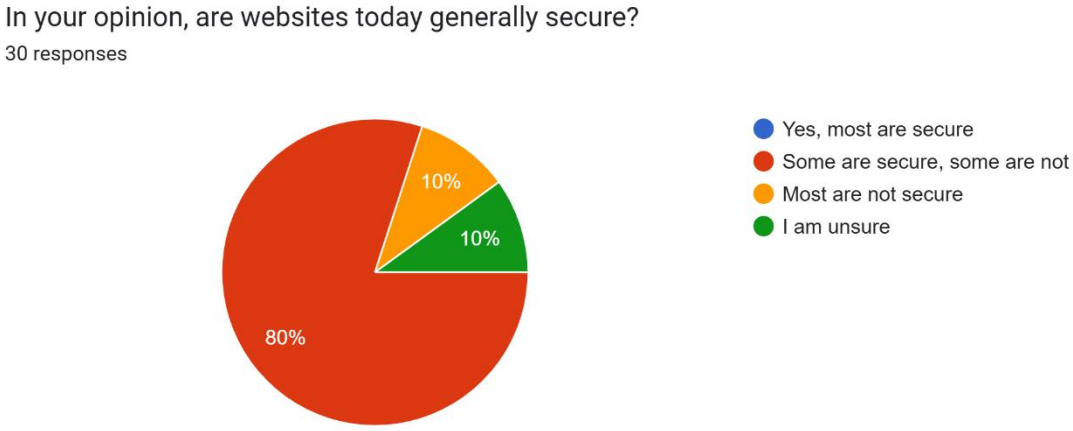


Figure 58 Websites general security analysis

Only a small percentage of the respondents (80 percent) think that all websites are secure or not sure, whereas the majority (80 percent) think that some websites are secure and others are not. This

is an indication that the respondents are aware that the level of site security differs widely on the various platforms.

Section C: Open Feedback on Vulnerability Scanning Tools

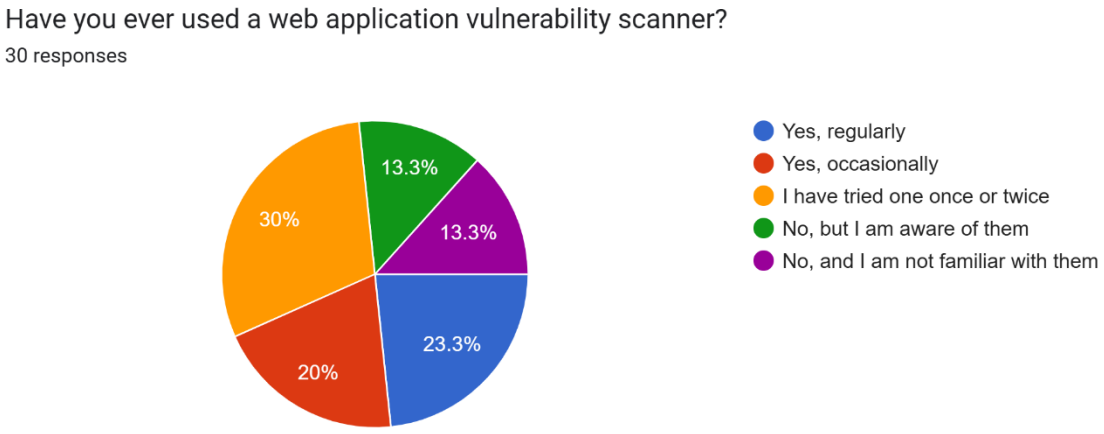


Figure 59 Familiarity with scanners analysis

The answers show that 23.3% of the participants use vulnerability scanners on a regular basis, and 20% use them on occasion. Also, 30% have used them once or twice and 13.3 percent think they know but have not used them thus differing in regard to familiarity with such tools.

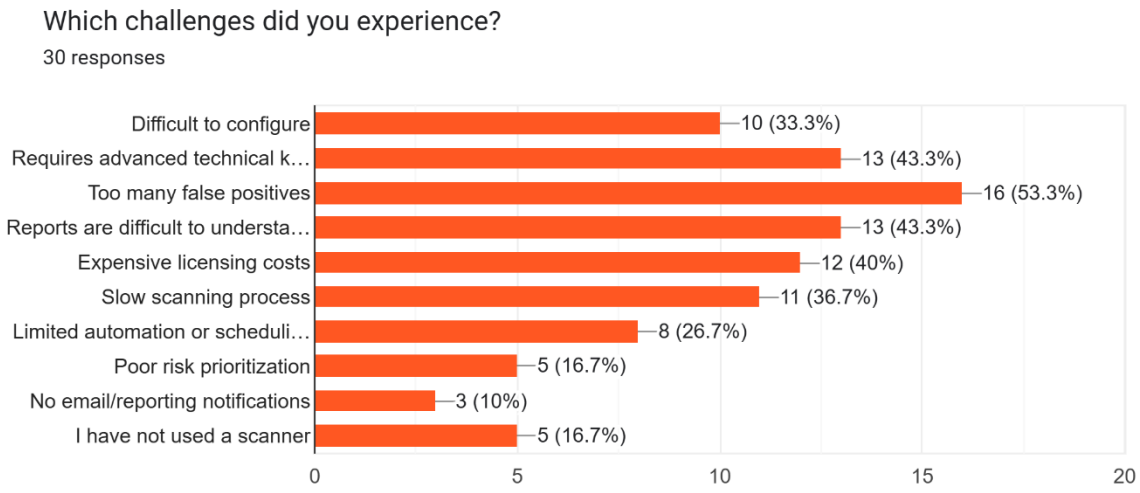


Figure 60 Challenges experienced with scanners analysis

False positives (53.3%), then advanced technical skills requirements and difficulty in interpretation of reports (43.3%) and high costs of licensing (40%), were the most frequently reported. Such results indicate that the complexity and accuracy are significant drawbacks of the current vulnerability scanners.

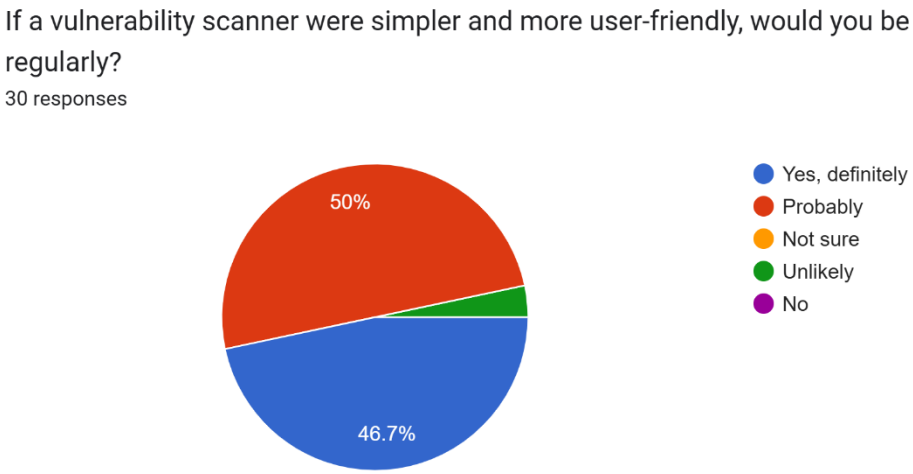


Figure 61 If scanners were more simple analysis

A large majority of them reported that they would be more willing to make use of vulnerability scanners in case they were less complex and simpler to use, but 50% responded Probably and 46.7% responded Yes, definitely. This brings out the relevance of usability in enhancing adoption.

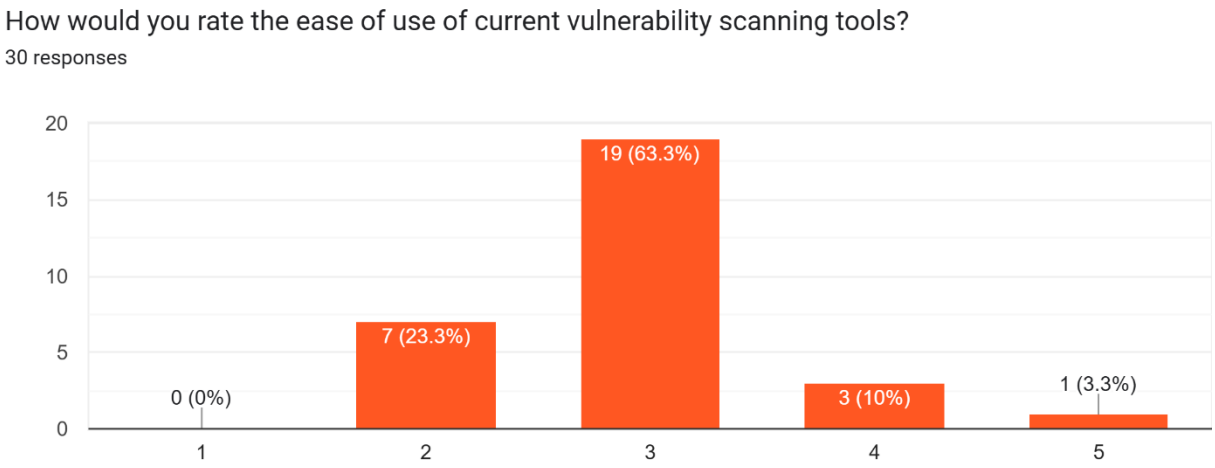


Figure 62 Rate ease of scanner use analysis

The majority of the respondents rated the usability of the existing vulnerability scanning tools as level 3 (63.3%), which means that the respondents have viewed the tools as moderately easy to use without being overly user-friendly.

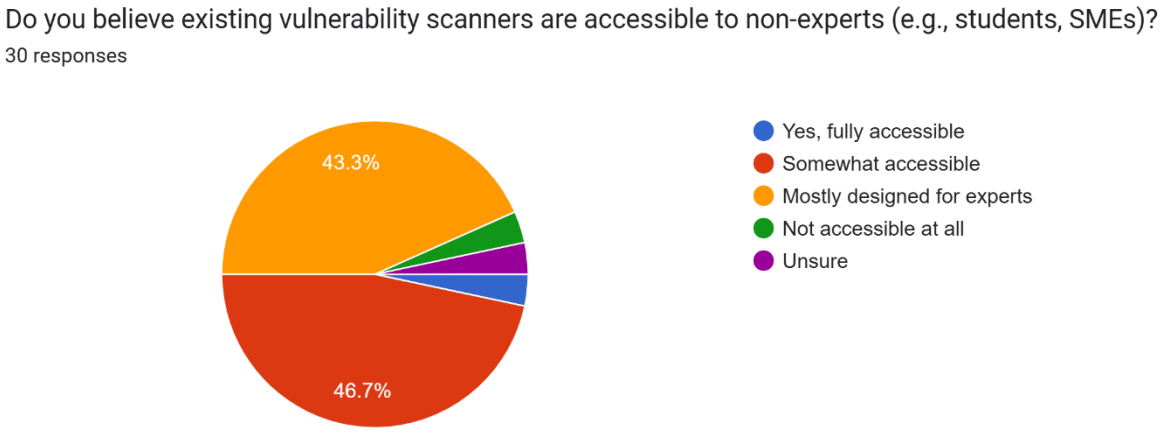


Figure 63 Are scanners accessible by non-experts analysis

The findings reveal that out of the 46.7 percent respondents think that vulnerability scanners are relatively available, and out of the 43.3 percent think that they are largely expert oriented. This is an indication that users have a feeling that these tools have not yet been sufficiently user friendly without the technical expertise that is required.

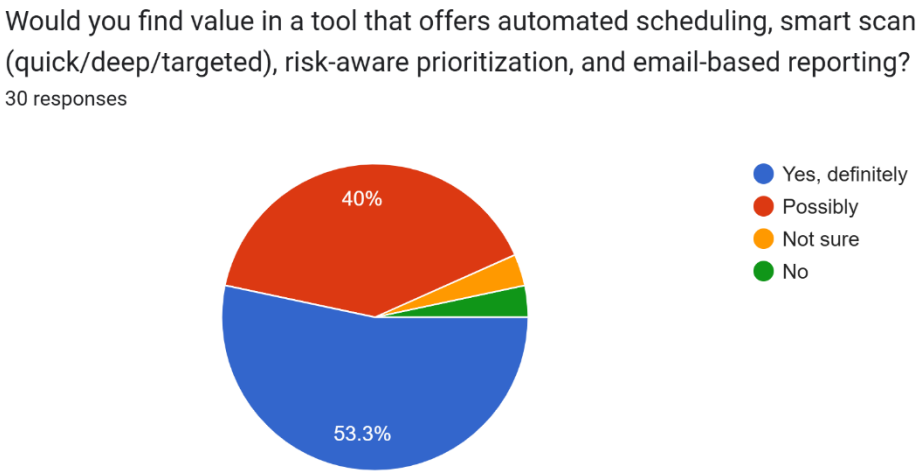


Figure 64 Value of extra features analysis

Most of the (53.3) respondents who answered indicated that they would certainly find value in tools that allow automated scheduling, smart scan modes, risk prioritization and email reporting and 40% of the respondents said possibly. That means that there is high demand in better automation and usability features.

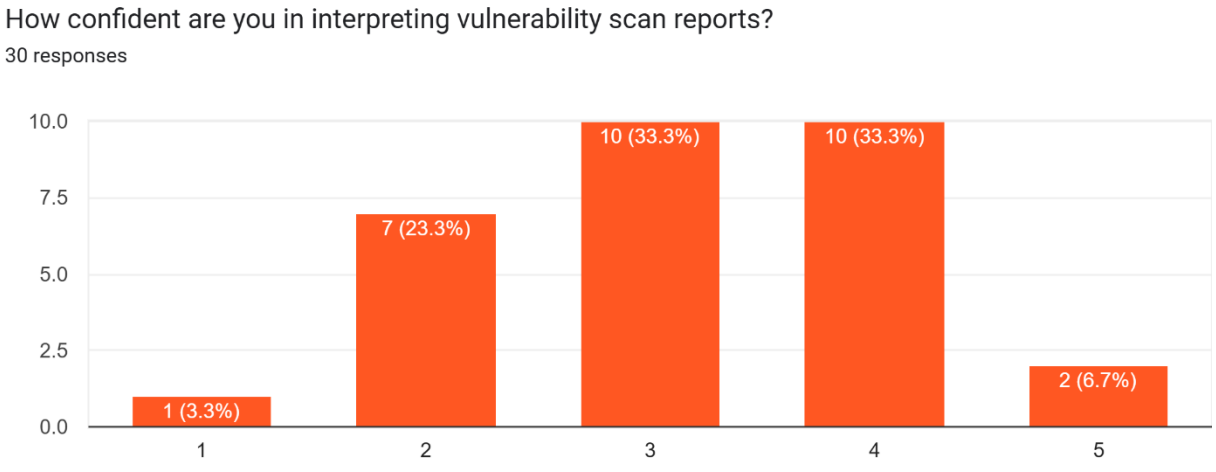


Figure 65 Confidence in scanning reports analysis

Moderate rates of confidence are also observed in the answers, with the majority of the respondents choosing level 3 and 4 (33.3% each). Nevertheless, other respondents expressed the lack of confidence, which implies that the process of defining the vulnerability reports may be problematic even to specific users.

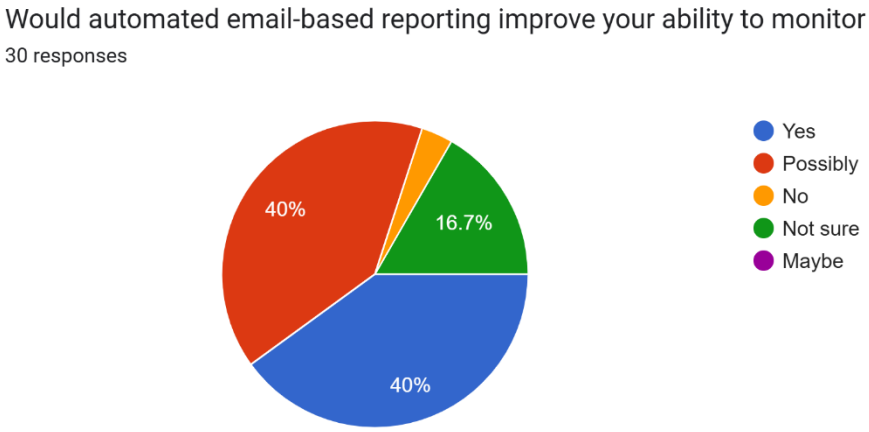


Figure 66 Can automated email-based reporting improve scanners analysis

The responses suggest that a proportion of 40 consider automated email reporting would enhance vulnerability monitoring, another 40 percent consider that it may ensure it. This implies that automated notifications are an effective attribute to most respondents.

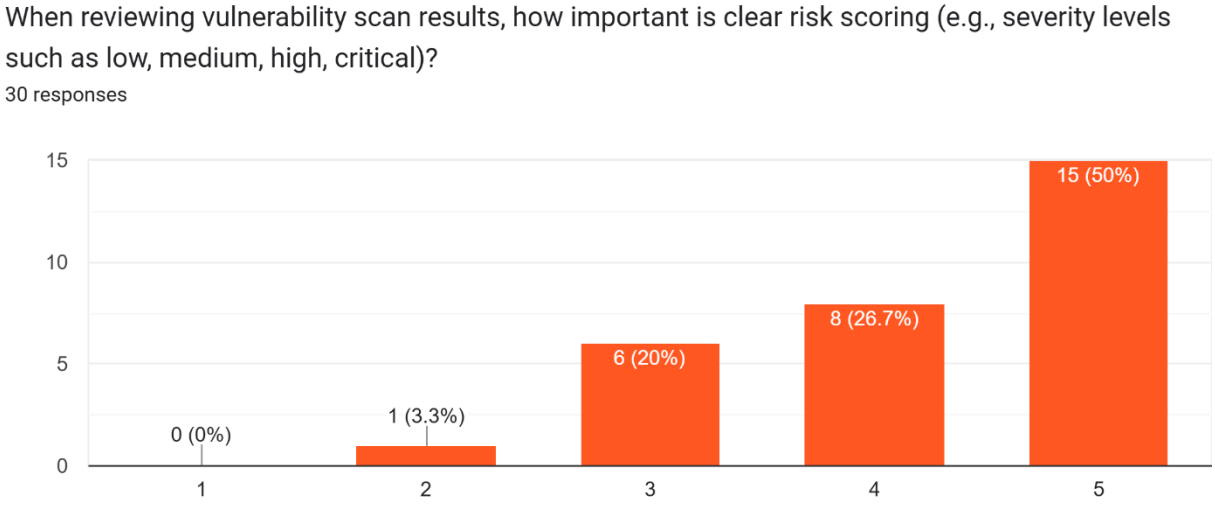


Figure 67 Importance of clear risk scoring analysis

Most of the respondents believe risk scoring is essential when looking through the results of vulnerability scan with 50% choosing level 5 and 26.7 choosing level 4. This shows the role of definite severity categorization of vulnerability management.

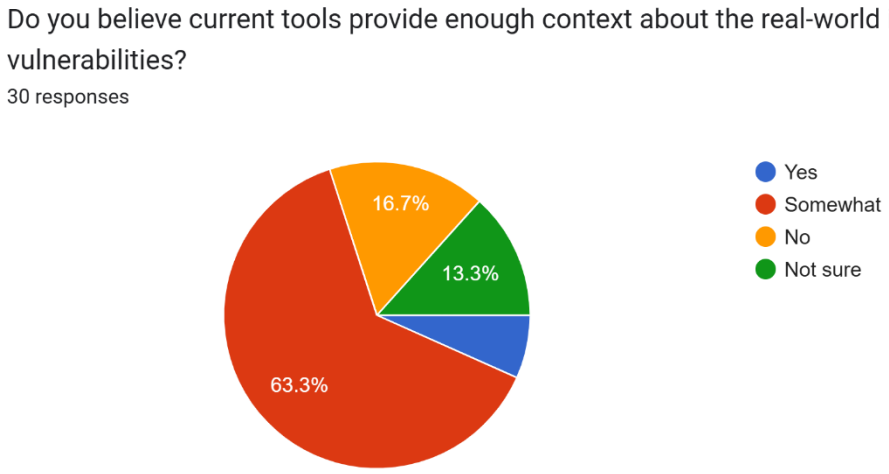


Figure 68 Rating current tools analysis

The majority of respondents (63.3) are of the opinion that there is some provision of background information concerning the actual vulnerability impact of the world by the existing tools, whereas less of them feel that the tools have enough clarification. This suggests that there can be a need to have more detailed descriptions of vulnerability impact.

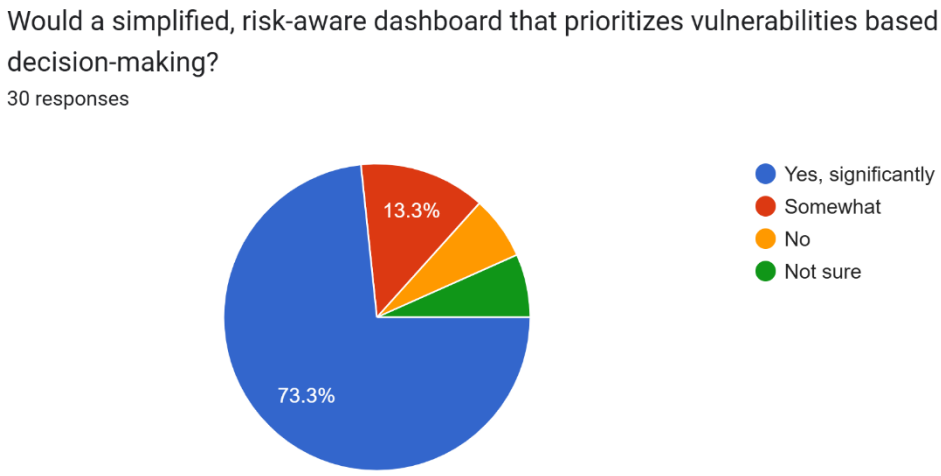


Figure 69 Simplified risk-aware dashboard analysis

The findings indicate a solid positive support of a simplified dashboard, with 73.3 percent saying that it would make the decision-making much better. This implies that superior visualization and prioritization capabilities are very important to the users.

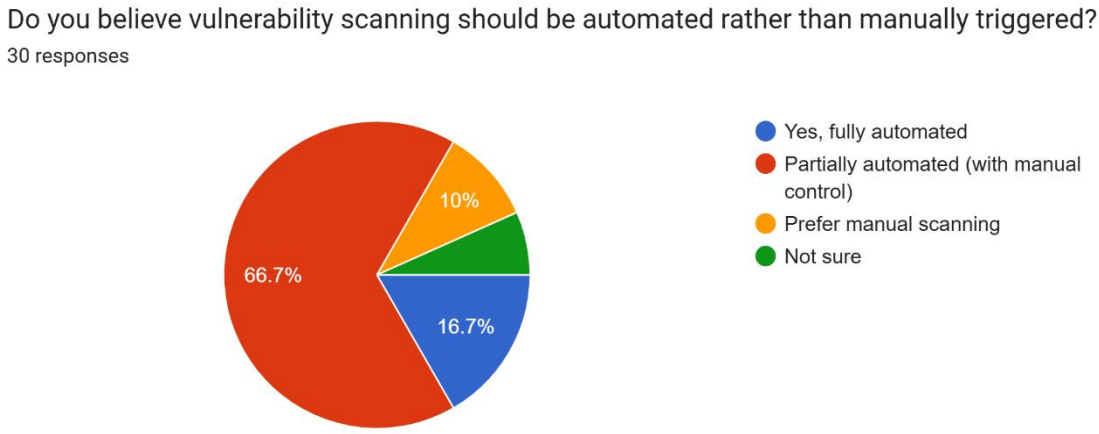


Figure 70 Automating scanners analysis

Most of the respondents (66.7) favor partially automated scanning that has manual control with 16.7 favoring fully automated scanning. That means that users apply importance to automation but would continue to have some form of manual control.

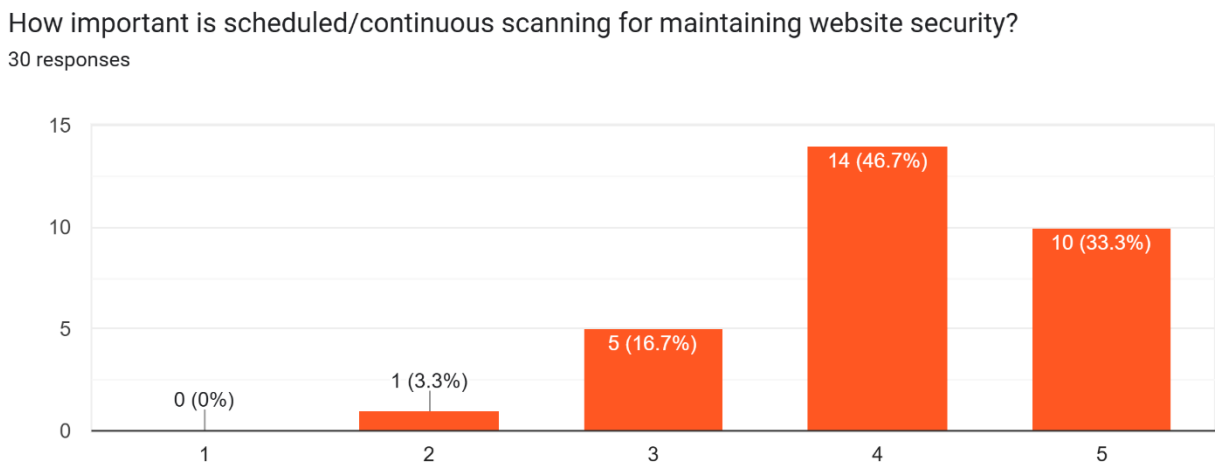


Figure 71 Importance of scheduling for website security analysis

Most of the participants found scheduled or continuous scanning as important with 46.7 percent choosing the level 4 and 33.3 percent choosing the level 5. This implies that frequent vulnerability testing is generally viewed as significant in ensuring the security of websites.

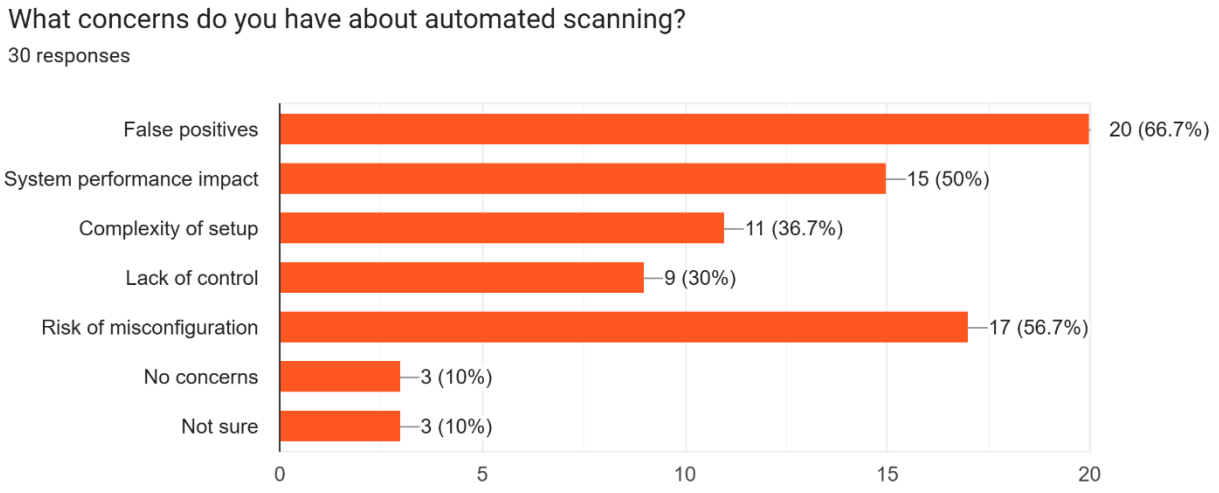


Figure 72 Concerns about automating scanning analysis

The findings indicate that the issue with the automated vulnerability scanning most prevalent is false positives (66.7%), then there is risk of misconfiguration (56.7%), and then there is the issue of having an impact on the system performance (50%). The other issues are complexity of set-up (36.7%), and a lack of control (30%). Very few of the respondents expressed that they had no issues or were not sure (10%), thus stating that even though the users recognize a benefit in automation, they are also conscious of limitations and risks of automated scanning systems.

In your opinion, what improvements would you like to see in current web application vulnerability scanning tools?

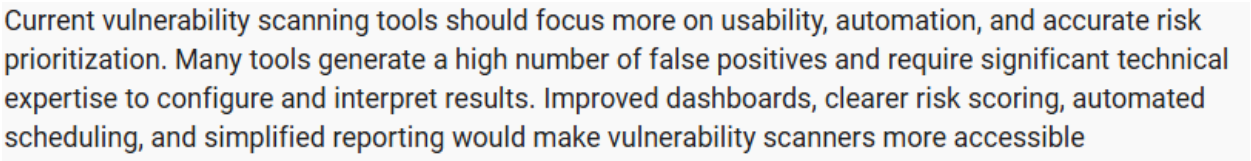


Figure 73 Respondent feedback highlighting the need for improved usability, automation, and clearer risk prioritization in current vulnerability scanning tools.

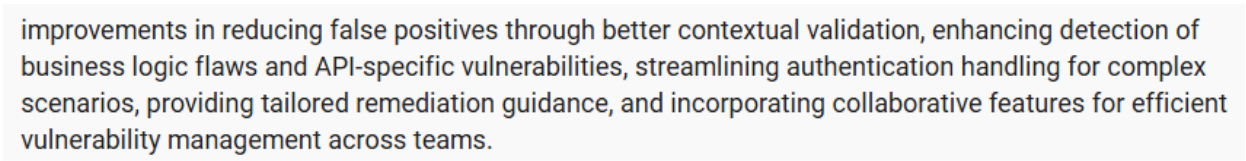


Figure 74 Respondent suggestion emphasizing the importance of reducing false positives and improving detection accuracy for business logic and API-related vulnerabilities.

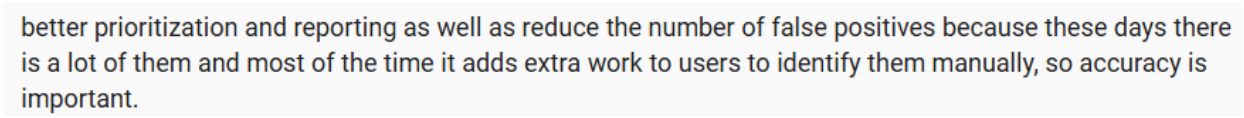


Figure 75 Respondent opinion stressing the need for better prioritization and reporting features to reduce the workload caused by false positives in vulnerability scanning results.

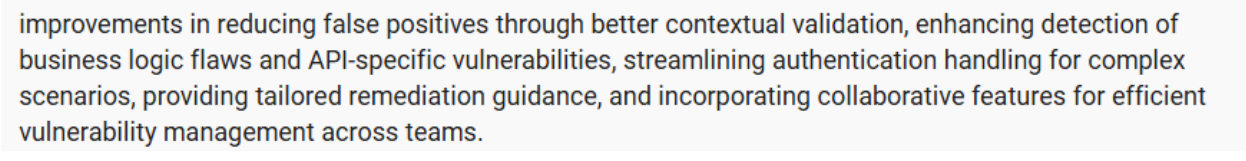


Figure 76 Respondent recommendation focusing on improved contextual validation, enhanced vulnerability detection, and collaborative vulnerability management features.

The answers to the questions indicate that the existing vulnerability scanners require major usability, accuracy, and automation overhauls. Most of the users emphasized that current tools tend to produce high false positive results causing extra workload on security staff to supervise the outcomes. The participants also indicated that these tools are not easy to set up, and they normally demand high technical skills to be effective. To overcome these limitations, it was suggested to improve the following aspects: risk prioritization and dashboards, simplified reporting, and automated scheduling. Besides, the respondents discussed the significance of improved contextual validation as limiting false positives, improving business logic and API-related vulnerability detection, and providing more realistic remediation advice. It was also proposed to have collaborative capabilities and simplified authentication management to facilitate more effective vulnerability management over teams.

Chapter 4: Design and Implementation

4.1 Introduction

This chapter describes the design and implementation of VulnLab, the automated vulnerability assessment system for web applications, designed to be risk-aware and based on domain research, similar-systems analysis and user requirements found in Chapter 2 and 3 respectively. This chapter explains how that need was turned into a working system, and does so by tracing back every design decision made to the actual source code of the Flask backend and the React frontend that together comprise VulnLab and that is needed to build a continuous, risk-aware, and accessible vulnerability assessment tool. The chapter starts with an overview of the overall system design (Section 4.2), which covers the design principles that were followed throughout the design process, the system architecture, the responsibilities of all the backend modules, the structure of the React frontend, how the users and systems interact in the use cases, the design of the three modes of smart scan, the original technical contribution of the project, the risk-aware scoring algorithm, the AI explanation layer, the scheduling subsystem, the email reporting subsystem, the sequence of operations for both a manual and a scheduled scan and security and legal considerations incorporated in the design. The persistence of users, scans, vulnerabilities, schedules, and settings

is then modeled and presented as a database design in section 4.3, which is directly extracted from the SQLAlchemy models.

The structure and purpose of each screen in the finished interface is detailed in section 4.4, which is based on the React Router configuration and each page component. In Section 4.5, the integration of the individual modules into a single running platform is explained, while screenshots of the system are shown in Section 4.6. A summary (Section 4.7) concludes this chapter.

4.2 Design

The design of VulnLab was built directly from the needs and requirements collected in Chapter 3, and the methodology chosen is Scrum, which was chosen in Section 3.1.6, with each capability — add authentication, add scanning, add risk scoring, add scheduling, add email reporting, and add explanation by AI — designed, built, and integrated one by one sprint, starting from the initial one. This can be seen in the source code: The stage number is still included in the health-check (GET /api/health) endpoint, and the backend's docstrings are also marked with stage numbers ("Stage 1 — database + authentication", "Stage 2 — real scanning with OWASP ZAP", "Stage 3 — email reports + scheduled scans", "Stage 4 — AI explanations"), indicating that the stages were incremented, rather than added as a single one!

4.2.1 Design Overview and Principles

VulnLab is not a single monolithic scanner, rather it is a coordination layer that is constructed on top of an established scanning engine (OWASP ZAP), a general-purpose relational database (PostgreSQL), and two third-party services (an AI language model to provide explanations and an SMTP relay to deliver them). This is directly based on the technical research in Section 2.3.3, which scoped VulnLab as an orchestration platform, not a scanning engine, and the similar-systems comparison in Section 2.2.4, where Burp Suite, Nessus and OWASP ZAP were already demonstrated to solve raw vulnerability detection quite well but not risk-aware prioritisation, scheduled continuous monitoring and plain-English reporting to non-experts.

The implementation was designed in a consistent way, using three design principles that were present in each and every module:

Principle 1 — Separation of scoring from explanation

The risk-scoring engine (scanner/risk_scoring.py) is small, deterministic, rule-based module to determine severity and priority. The prompt used by the AI explanation layer (ai_explainer.py) explicitly states that "severity already decided by our system – do NOT change it", meaning that the explanation layer is always asked to explain a finding that has already been scored. This addresses the conclusion of the literature review (Section 2.1.6) that severity scoring needs to be transparent and defensible, yet provide plain English explanations for non-expert users, who requested this in the survey (Section 3.3, Figure 65 – Confidence in reading scan reports). Restricting the AI layer to only determine the entity, with severity being rule-based, ensures that the same entity detected in a scan is always scored the same, which is key to the trend analysis and Compare Scans features described in Sections 4.2.5 and 4.4.

Principle 2 — Graceful degradation over hard failure

ZAP is fall-backed directly into the code, and in the event of an unreachable scanner daemon (is_zap_available() returns False), a fixed set of realistic demo alerts are used; the AI explanation layer is fall-backed to rule-based summaries if no API key is configured, or the call fails; emailer.py silently skips emailing if SMTP is not configured (is_email_configured() returns False). This was a conscious decision to address the 'high false positive' and unreliable-tooling complaints mentioned in Section 2.1.2 and Section 3.3 — The system is designed to always return something useful (no error page) even before ZAP or the AI service are configured, allowing development and demonstration of all downstream functionality (scoring, storage, dashboard, reporting).

Principle 3 — Configuration over hard-coding

All the environment-specific values (database URL, JWT secret and expiry, CORS origin, ZAP endpoint and key, SMTP credentials, AI provider key, AI provider model name) are read only once from environment variables in config.py and have sensible local defaults (for example, if the DATABASE_URL environment variable is not set, the application will use a local SQLite database before configuring PostgreSQL). This ensures that the same code base can be deployed to a local development environment or an environment that closely resembles production – directly satisfying the non-functional requirements of "scalability" and "consistency" as described in Section 2.3.1.

4.2.2 System Architecture

VulnLab is written as two distinct stand-alone applications that exchange messages via a JSON REST API: a React (Vite) based single-page application (front end) and a Flask based back end (app.py) which is responsible for authentication, scan orchestration, scheduling and reporting. The complete architecture as implemented is shown in Figure 84 and was traced from the import graph and route definitions of the backend source code.

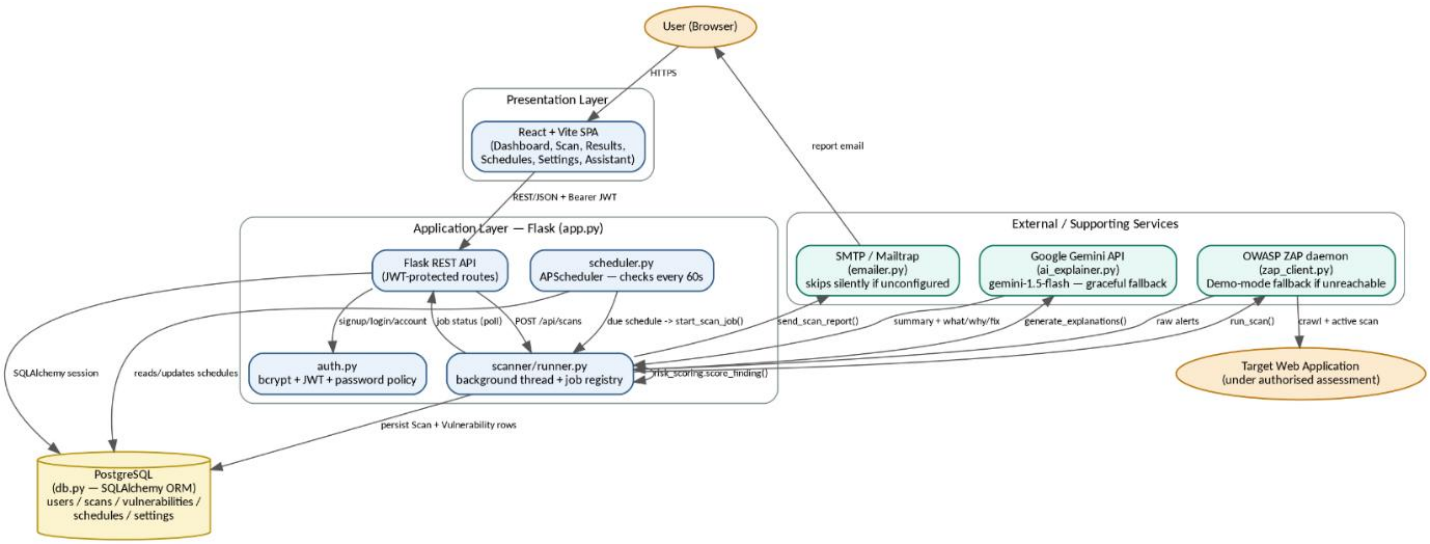


Figure 77 VulnLab System Architecture (traced from the implemented backend)

One of the important architectural choices made is that the Flask request thread never blocks while it is doing a scan - this is clearly seen in scanner/runner.py. The route handler that responds to POST /api/scans spawns a daemon background thread with start_scan_job(), and returns a job_id right away with a 202 (Accepted) response code. Then the React front end periodically calls GET /api/scans/status/{job_id} about every 2 seconds, until the job returns "completed", "failed", or "stopped". This design was required, as a Deep scan mode (Section 4.2.6) might take several minutes for a full crawl and active scan and this would timeout most browsers and reverse proxies if it was a synchronous HTTP request. The job registry is in memory (guarded by a threading.It is deliberately concise - as the code's own docstring states "for production you would use a task queue (Celery/RQ) with a broker, but the API contract here would remain the same" - which is in keeping with the non-functional requirements outlined in Section 2.3.1, and is an appropriate and explicitly justified trade-off for a single-server academic prototype. The background runner is the only component of the system that communicates with ZAP, the risk-scoring engine, the AI explainer, and the email service, in a set sequence, for a given scan. This pipeline is not duplicated by the

scheduler (Section 4.2.9); instead, the same start_scan_job() function used by a manual scan is used and two callbacks (on_progress, on_complete) are attached, so that the row in the database corresponding to a schedule is kept in sync with the scan being run on its behalf. This reuse was intentional to prevent creating a second parallel stream of scanning logic for scheduled scans, since everything would be duplicated and divergent from manual scanning.

4.2.3 Backend Module and Component Design

Table 5 documents the responsibility of every backend source file, and Figure 78 shows the dependency relationships between them, both traced directly from the eleven Python files that make up the backend.

Table 4 Backend Module Responsibilities

Module	Responsibility
app.py	Flask application entry point; defines every REST route (/api/health, /api/auth/*, /api/scans*, /api/schedules*, /api/settings, /api/assistant); creates database tables on startup via init_db(); starts the scheduler when run as the main process.
auth.py	Email validation (regex), password-policy enforcement (6+ characters, 1 uppercase, 1 digit, 1 symbol via password_problem()), bcrypt password hashing/verification, JWT issuance (make_token) and verification (decode_token), and the @require_auth route decorator that injects user_id into any protected view.
config.py	Single source of truth for every environment-driven setting, loaded once via python-dotenv, with safe local defaults (e.g. SQLite fallback for DATABASE_URL).
db.py	SQLAlchemy declarative models (User, Scan, Vulnerability, Schedule, Setting), the naive-UTC datetime convention used throughout the system (utcnow(), to_epoch_ms()), and init_db().
scanner/scan_modes.py	Declarative configuration of the three smart scan modes (Quick/Deep/Targeted) as spider depth, AJAX-spider toggle, active-scan toggle, and attack strength.
scanner/zap_client.py	Drives the OWASP ZAP Python API (zapv2) through access -> spider -> AJAX spider -> active scan -> alert collection; exposes is_zap_available(); supplies demo-mode sample alerts when ZAP is

Module	Responsibility
	unreachable; supports mid-scan cancellation via a should_cancel() callback that raises ScanCancelled.
scanner/risk_scoring.py	The risk-aware scoring algorithm described in Section 4.2.7; a pure function with no I/O, making it fully unit-testable in isolation from the rest of the system.
scanner/runner.py	Background-thread job orchestrator: owns the in-memory job registry, drives a scan end-to-end (ZAP -> scoring -> persistence -> AI explanation -> email), and exposes cancellation support for the Stop feature.
ai_explainer.py	Calls the configured AI provider to translate scored findings into plain-English what/why/fix text and an overall summary; also implements the scope-limited in-app assistant (assistant_reply); both paths degrade gracefully without a configured key.
emailer.py	Builds the HTML email report (build_report_html) and sends it via smtplib; no-ops safely if SMTP is not configured.
scheduler.py	An APScheduler BackgroundScheduler that polls the schedules table every 60 seconds, claims due and unpaused rows, launches them through the same runner used for manual scans, and reschedules or finalises them afterward according to their repeat rule.

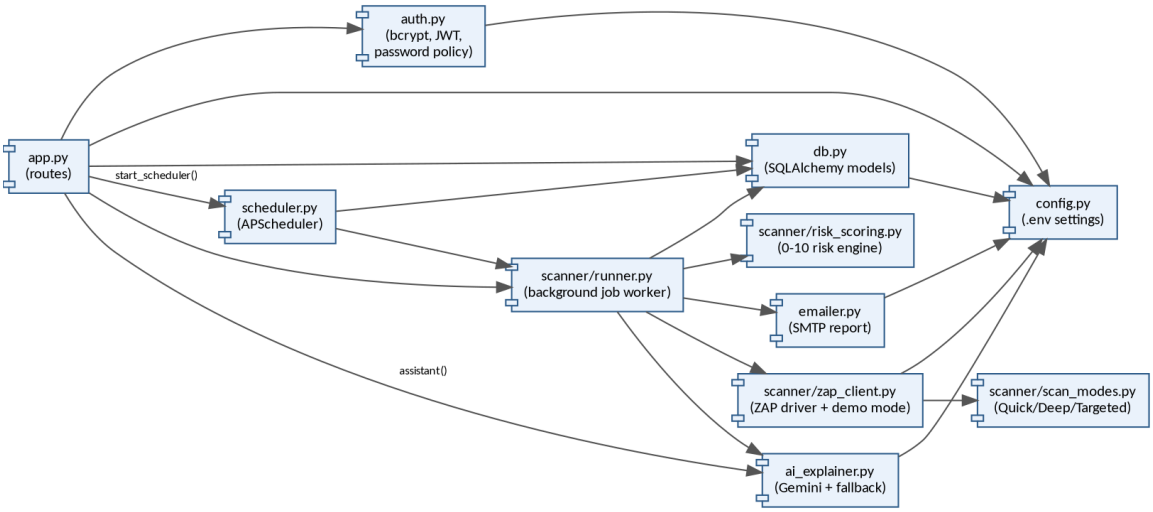


Figure 78 VulnLab Backend Module Dependency Diagram

4.2.4 Frontend Application Design

The frontend is a React single-page application built with Vite, organised into pages (one component per route), reusable components (modals, the sidebar, the AI assistant bubble, shared UI primitives), and a small lib/ layer that wraps every backend call. Figure 79 shows the routing structure exactly as declared in App.jsx and enforced by the authenticated-shell wrapper in Layout.jsx.

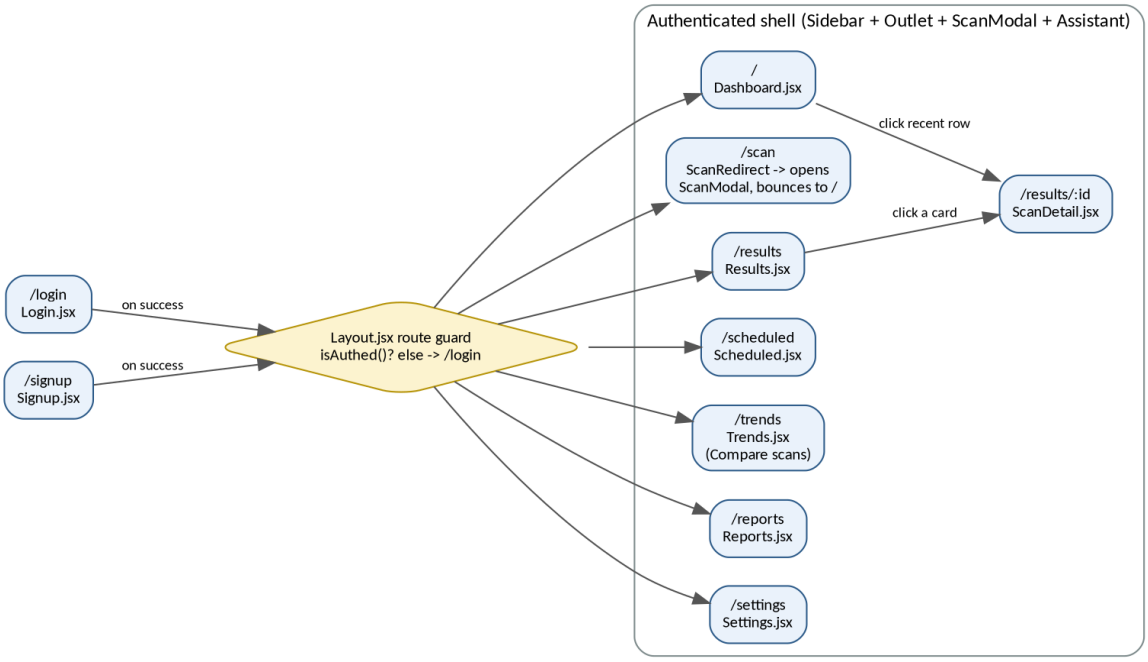


Figure 79 VulnLab Frontend Route Map

Three design decisions in the frontend are worth documenting explicitly because they materially affect correctness, not just appearance:

- Route guarding. Layout.jsx wraps every authenticated page in a single component that checks isAuthed() (whether a JWT is present in localStorage) and redirects to /login via React Router's <Navigate> if not. Because every protected route in App.jsx is nested under this one <Layout> element, there is exactly one place in the codebase where the authentication check can be bypassed, rather than one per page.
- A shared "New scan" trigger via React Context. Rather than duplicating the scan-launch modal on every page, Layout.jsx exposes a useNewScan() hook (backed by a React Context) that any descendant component can call to open the shared ScanModal. The

/scan route itself is not a real page — ScanRedirect immediately calls this hook and bounces back to the dashboard — which means the sidebar's "New scan" link, and any other future call-to-action, can open the same modal instance without prop-drilling.

- Client-side computation over redundant backend endpoints. Filtering, sorting, CSV export (Results.jsx, Reports.jsx), and the Compare Scans diff (Trends.jsx, Section 4.2.5) are all computed in the browser from data already fetched via GET /api/scans, rather than through dedicated backend endpoints for each view. This was a deliberate scope decision: because scan history for a single user is small (tens to low hundreds of scans for an academic prototype), client-side filtering avoids adding six or more narrow, single-purpose API endpoints for what is fundamentally the same underlying resource.

Table 6 lists every page component with its route and its primary backend dependency, cross-referenced with the interface descriptions given in full in Section 4.4.

Table 5 Frontend Page Components and Their Backend Dependencies

Page component	Route	Primary backend calls
Login.jsx	/login	POST /api/auth/login
Signup.jsx	/signup	POST /api/auth/signup
Dashboard.jsx	/ (index)	GET /api/scans
ScanModal.jsx (via ScanRedirect)	/scan	POST /api/scans, GET /api/scans/status/{job_id}
Results.jsx	/results	GET /api/scans
ScanDetail.jsx	/results/:id	GET /api/scans/{id}
Scheduled.jsx + ScheduleModal.jsx	/scheduled	GET/POST/PATCH/DELETE /api/schedules, POST /api/schedules/{id}/stop
Trends.jsx	/trends	GET /api/scans (diffed client-side)
Reports.jsx	/reports	GET /api/scans
Settings.jsx	/settings	GET/PATCH /api/settings, PATCH /api/auth/account
Assistant.jsx (floating, all pages)	n/a	POST /api/assistant

4.2.5 Use Case Diagram and Use Case Specifications

Figure 80 models the interactions available to a registered user, together with the automated interaction triggered by the scheduler acting on the user's behalf. The use cases were derived directly from the REST routes exposed in app.py rather than being designed independently of the implementation, so every use case below corresponds to one or more concrete route handlers.

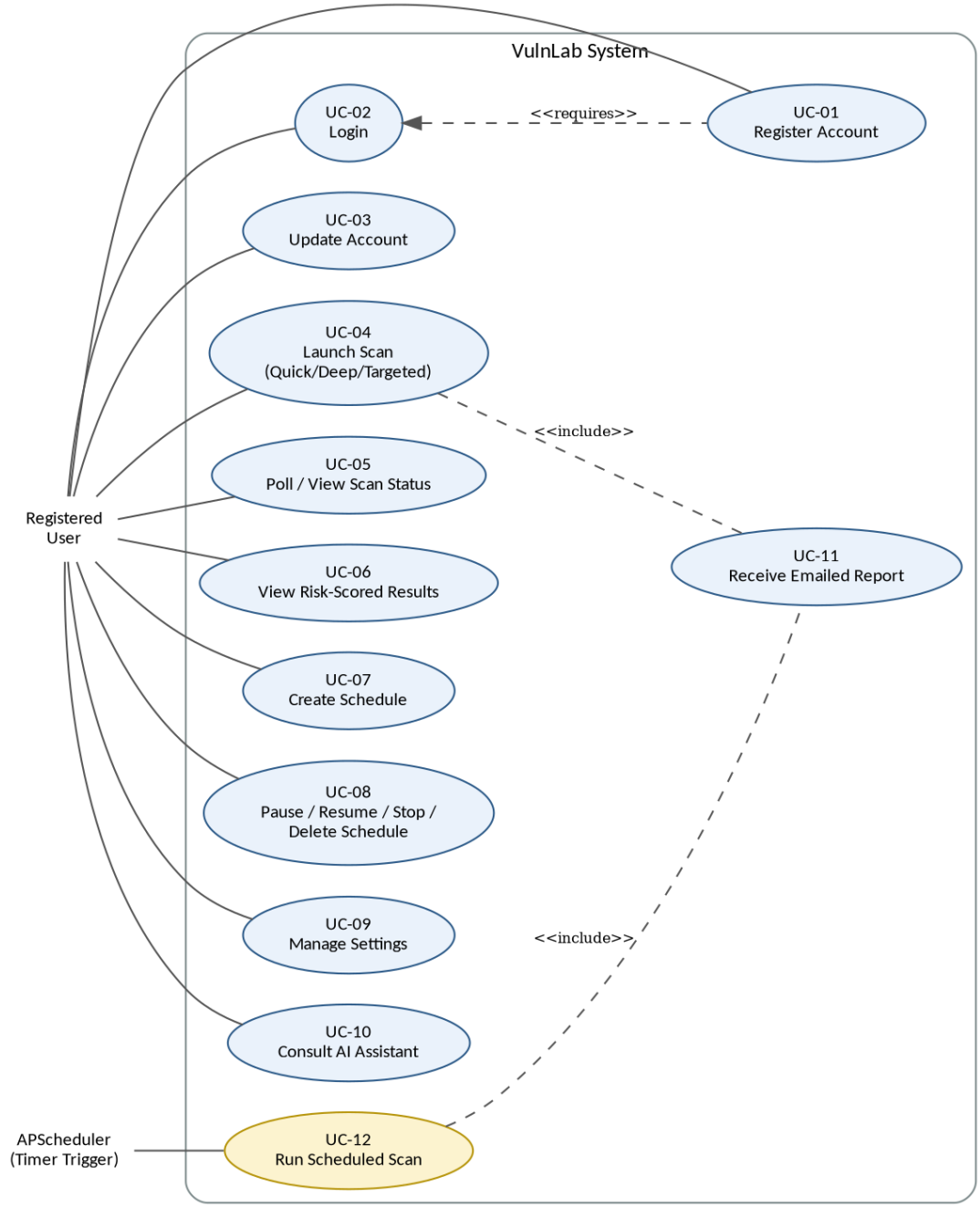


Figure 80 VulnLab Use Case Diagram

Table 7 gives a brief specification for the four use cases most central to the project's contribution — launching a risk-aware scan, comparing two scans, and running a scheduled scan — in the style used for use case specifications in similar systems documentation.

Table 6 Selected Use Case Specifications

ID / Name	Actor	Precondition	Main flow	Postcondition
UC-04 Launch Scan	Registered User	User is authenticated (valid JWT); user has entered a URL beginning http:// or https://.	1) User opens New Scan modal and selects Quick/Deep/Targeted. 2) Frontend POSTs /api/scans. 3) Backend validates input, starts a background job, returns job_id (202). 4) Frontend polls status every 2s. 5) On completion, frontend navigates to the scan detail page.	A Scan row and its Vulnerability rows exist in PostgreSQL; a report has been emailed if an address was supplied.
UC-06 View Risk-Scored Results	Registered User	At least one completed scan exists for the user.	1) User opens Results or Dashboard. 2) Frontend GETs /api/scans or /api/scans/{id}. 3) Findings are rendered grouped and coloured by severity band, each with its 0-10 score and, if available, its AI explanation.	User has viewed prioritised findings without needing to interpret raw ZAP output.
UC-08 Pause / Resume / Stop / Delete Schedule	Registered User	A schedule exists (status pending, running, or done).	1) User toggles pause/resume (PATCH /api/schedules/{id} {paused}), or clicks Stop while running (POST /api/schedules/{id}/stop, which cancels the in-flight job and sets paused=true), or confirms deletion in a modal dialog (DELETE /api/schedules/{id}).	Schedule state is updated accordingly; a stopped schedule does not immediately re-fire because it is left paused.
UC-12 Run Scheduled Scan	APScheduler (system actor)	A schedule's next_run has passed and	1) _check_due_schedules() selects due, unpaused, pending rows every 60s. 2) It claims each row (status="running") before	Includes UC-11 (Receive Emailed Report) automatically if

ID / Name	Actor	Precondition	Main flow	Postcondition
		paused is false.	launching it, to prevent double-firing. 3) It calls start_scan_job() with the same pipeline as UC-04. 4) On completion, next_run is advanced (or the schedule is marked done for a one-off run).	a report_email is set on the schedule.

4.2.6 Scan Mode Design

The most important usability result from Chapter 3 (Section 3.3, Figures 61 and 63) was that the respondents desired vulnerability scanning to be simple, yet not to sacrifice coverage, and that a majority preferred partially automated scanning over fully automated or fully manual approaches (Figure 70). The response from VulnLab, in scanner/scan_modes.py, is to expose exactly three named scan modes that check for the same five categories of vulnerabilities in scope (Section 1.5), but vary only in the amount of target site that is explored and its level of attack. It is explicitly said in the docstring of the module itself: "modes control the DEPTH and AREA of the scan over the SAME five types of vulnerabilities — they are not different scanners. The same values that were used in the configuration of the implementation are reproduced in table 8.

Table 7 Smart Scan Mode Configuration (scanner/scan_modes.py)

Mode	Spider depth	Traditional spider	AJAX spider	Active scan	Strength	Description (as implemented)
Quick	2	Yes	No	Yes	low	Fast surface scan: headers, misconfig, light injection probes.
Deep	8	Yes	Yes	Yes	high	Full crawl and full attack across all five vulnerability types.
Targeted	0 (no crawl)	No	No	Yes	high	No crawl; deep active scan on the exact URL provided.

The design rationale for each mode follows naturally from the exploratory vs attacking trade-off discussed in 2.1.3: Quick uses a small traditional spider and a low attack strength, providing quick feedback suitable for a developer checking a small change; Deep enables the AJAX spider (required to discover links created by client-side JavaScript on modern single page applications, as described in 2.1.2) and increases attack strength, but at the cost of scan duration; Targeted disables crawling altogether and provides the active scanner with one URL to scan directly, suitable for when a user knows exactly which page or endpoint they wish to re-test after fixing a problem, thus supporting the scan-fix-rescan-verify workflow, which was identified as missing from existing tools in 3.3.

4.2.7 Risk-Aware Scoring Design

VulnLab's original technical contribution is the risk-aware scoring engine implemented in scanner/risk_scoring.py. This engine operationalises the argument developed in Sections 2.1.4 and 2.1.6: that a coarse severity label does not equate to real operational risk (Allodi, 2015; Balsam et al., 2025), because it captures neither how intensively a vulnerability class is exploited in practice nor the location within the application at which the weakness was found. OWASP ZAP assigns each finding one of four risk labels (High, Medium, Low, or Informational). VulnLab transforms this coarse label into a continuous score in the range 0–10 and a five-band severity classification (critical, high, medium, low, informational) by combining three signals in score_finding():

- **Base severity.** ZAP's own risk label is mapped to a base score: High = 8.0, Medium = 5.0, Low = 2.5, Informational = 1.0.

- **Exploitability adjustment (+1.5).** Applied when the finding's name matches a heavily-exploited vulnerability class, including SQL injection, cross-site scripting, remote code or command injection, and path traversal or file inclusion. This operationalises Allodi's (2015) finding, discussed in Section 2.1.6, that a small minority of vulnerability classes account for the large majority of observed attacks.

- **Exposure adjustment (+0.8).** Applied when the finding's URL contains a path hint indicating a sensitive endpoint (login, signin, auth, admin, upload, or search). The rationale is that the same technical weakness carries materially greater operational risk on an

authentication endpoint, administrative page, or data-entry form than on a static informational page.

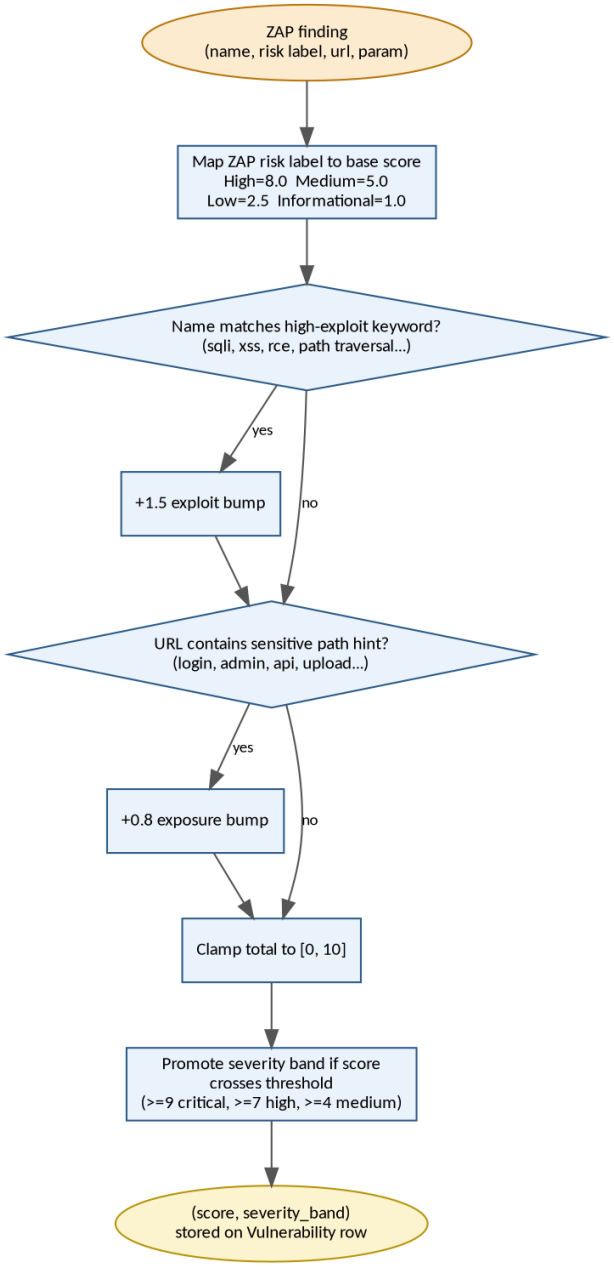


Figure 81 Risk-Aware Scoring Decision Flow (scanner/risk_scoring.py)

Table 9 works through three concrete examples using the exact constants from the implementation, to make the scoring behaviour concrete and defensible.

Table 8 Worked Examples of the Risk-Aware Scoring Algorithm

ZAP finding	ZAP risk	Base	Exploit bump	Exposure bump	Final score	Final band
SQL Injection at /login?username=	High	8.0	+1.5 (sqli keyword)	+0.8 (login path)	10.0 -> clamped 10.0	critical (>=9.0)
CSP Header Not Set at /	Medium	5.0	+0 (no match)	+0 (no sensitive path)	5.0	medium (unchanged)
Application Error Disclosure at /api/user	Medium	5.0	+0 (no match)	+0.8 (api path)	5.8	medium (below 7.0 threshold)

It's a transparent and rule-based design and not a back-box machine-learning model, as the module's own documentation makes clear: "Can be explained and defended in the report" and "Can later be extended with real lookups in the exploit-DB or the machine-learning approaches the IR reviews" (see the exploitation-prediction literature mentioned in Section 2.1.6, e.g. Bozorgi et al., 2010; Sabottke et al., 2015). The function is pure - that is, it does not have any side effects other than returning a name/risk/url triplet and a score/band tuple; as a result it is easy to unit test, separate from ZAP, the database and AI layer, as is an explicit software-quality effect of the separation-of-concerns principle described in Section 4.2.1.

4.2.8 AI Explanation Layer Design

This is what is directly addressed by ai_explainer.py, which is strictly bound by the following architectural constraint, stated on the module docstring in the program: "The risk-scoring engine RANKS findings (severity + score). Unchanged. The AI only EXPLAINS those findings in simple language. It never decides severity or invents scores. AI is a translator, not a judge. The explanation pipeline does the following, from the generate_explanations() function: after a scan's findings have already been scored (as per Section 4.2.7) and persisted, they are serialised in one large string in

the following format: each finding's name, severity band, and location are listed in the string, followed by an instruction to model to output strict JSON, namely a two to three sentence overall summary and for each finding index, a what/why/fix explanation of one to two short sentences, where the model is explicitly instructed not to invent new issues or change severities. The reply is defensively parsed (stray Markdown code fences are removed prior to parsing it into JSON) and if the model cannot explain anything, or the call itself fails, then a fallback is used to a rule-based template (_fallback_summary, _fallback_explanation), thus ensuring that a scan report always has at least a basic explanation. The in-app AI assistant, which can be accessed via the floating chat bubble mentioned in Section 4.4.6., is also implemented in the same module. It deliberately avoids giving it a "general" system prompt, which would have allowed it to scope itself to VulnLab itself and to web application security concepts and explicitly tell it to refuse to answer any questions outside that scope, such as general trivia, cooking, or unrelated coding questions — a predictable and on-topic behaviour for a security tool intended for non-expert users.

Note on provider configuration: The AI explanation layer is accessed via a single provider-agnostic module (ai_explainer.py); the AI provider can be swapped by configuring, and both generate_explanations() and assistant_reply() fall back to the rule based text described above when the provider API key is not present or if the call otherwise fails, leaving the rest of the system (scoring, storage, the dashboard, scheduled reporting) completely unaffected by which provider is configured, or that provider being temporarily unavailable.

4.2.9 Scheduling Design

This is done by Continuous Unattended Monitoring which is one of the four core objectives defined in Section 1.4 and the literature distinguishes between Periodic and Continuous Vulnerability Assessment (Section 2.1.5; Awan et al., 2024), and by the strong survey support for scheduled scanning (Section 3.3, Figure 71 — 46.7% rating it a 4 and 33.3% a 5 out of 5 in importance). This is done by the following: APScheduler BackgroundScheduler running in the same flask process as the app and ticking every 60 seconds.

On each tick, check_due_schedules() does four things: (1) it gets all rows where status is "pending"; (2) it filters out rows where paused is "yes" and rows where next_run is (not yet) before the current time; (3) for each remaining "due" row, it writes status="running" and commits, and then starts nothing — by doing this, it claims the row which means it cannot be picked up again on the next tick of the minute while its scan is ongoing; (4) it then calls start_scan_job() on each claimed row, passing an on_progress call-back that mirrors the live scan percentage onto the schedule's progress column (which is consumed by the animated progress ring described in Section

4.4.5), as well as an on_complete call-back that finalises the schedule when the scan is done. The finalisation logic knows the difference between a one-shot and repeating run: if backend was down for too long, such that next_run was now in the past, then a Daily/Weekly/Monthly/Custom run will have next_run advanced by the interval, once again, in a loop. One-shot run is marked "done" and will never fire again.

Fig. 89 illustrates the state machine through which a single schedule row rolls, and Fig. 90 illustrates the entire process that a scan (triggered by the scheduler) goes through, using the same scan pipeline as the manual example above in Fig. 92.

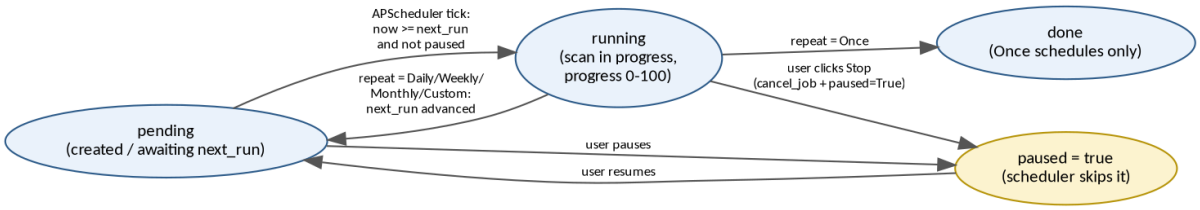


Figure 82 Schedule Row State Machine (schedules.status / schedules.paused)

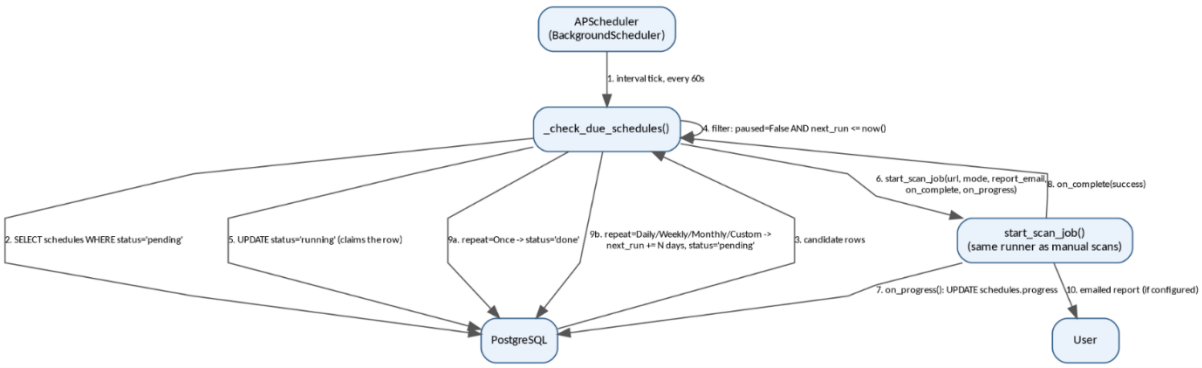


Figure 83 Sequence Diagram of a Scheduler-Triggered Scan

The Stop control (POST /api/schedules/{id}/stop) works by first locating the job_id of the currently running scan for the schedule in an in-memory map kept by the scheduler (_running_jobs), then calling cancel_job() on the runner (Section 4.2.2), and then intentionally setting paused=true on the schedule. This is a deliberate design element, and it's actually stated explicitly in the code comment for the route: When the route is stopped by hand, don't want it to immediately re-fire on the next scheduler tick, that would be counterproductive.

4.2.10 Email Reporting Design

Email-based reporting was identified as a core objective in Section 1.4 and was rated very useful by survey respondents for tracking vulnerabilities over time in Section 3.3 Figure 66. emailer.py is a standalone HTML report builder plus a thin smtplib sender, it is intentionally not coupled to the scanning and scoring logic; the runner calls send_scan_report(to_email, scan_dict) when a scan (and its AI explanations) are already persisted, and emailer only ever reads from the same scan_dict structure returned by Scan.to_dict(). The colour palette used in the generated report is the same as the one used in the frontend (critical #FF3B5C, high #FF8C42, medium #FFD23F, low #4ADE80, info #00D9FF), as defined in emailer.py and replicated in the frontend's CSS custom properties — see Section 4.4 — so that an emailed report looks the same as the result screen in the app. Findings have their severity sorted before rendering, the AI-generated plain English what/why/fix explanation is used if available, and each finding row includes its risk-aware score badge, name, location, and either its AI what/why/fix explanation or its raw technical description (as a fall back). Sending is never required and is silent if the configuration is not set: is_email_configured() checks the presence of SMTP_HOST, SMTP_USER and SMTP_PASSWORD: if any of these are missing, the function logs a message and returns False without raising an exception, so a scan without a report address, or a deployment without SMTP configured at all, still completes normally — following the principle of graceful-degradation (Section 4.2.1).

4.2.11 Sequence Diagrams — Scan Execution Flow

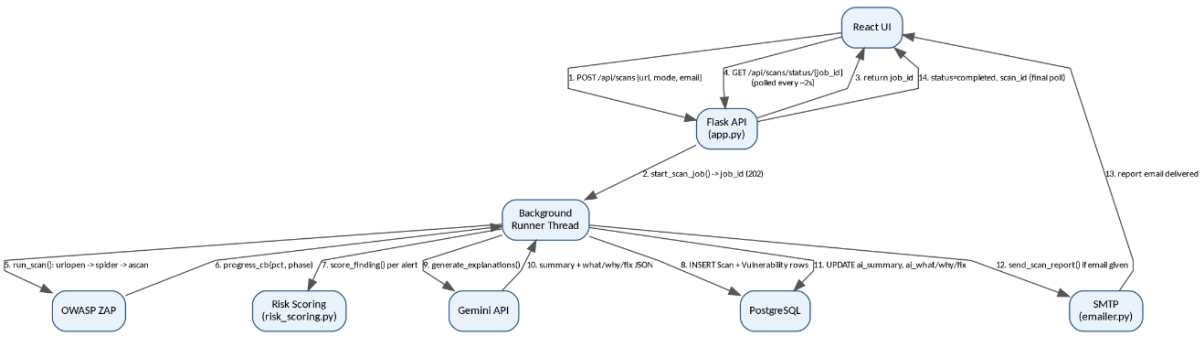


Figure 84 Sequence Diagram of the Manual Scan Execution Flow, Including Client Polling

Figure 84 shows the entire flow of a scan that is manually launched and executed, including the asynchronous job/poll pattern used in Section 4.2.2 when running the scan across app.py and scanner/runner.py. For the sake of argument, the two examples (see Figure 83 above) have a

common central pipeline (steps 5 and beyond in Figure 84) to make it clear at the diagram level that manual and scheduled scans are the same operation, but with two different triggers.

There are two implementation details that warrant mention, as they do not just impact flow but correctness. First, the runner writes the Scan and Vulnerability rows to the database (step 8) before invoking the AI explainer (step 9), and then it writes the same rows to the database with the ai_summary, ai_what, ai_why, and ai_fix (step 11), which is contained within a try/except block that explicitly does not make the scan "fail" if the AI explainer step fails outright, also adhering to the graceful-degradation principle: a scan is considered to be successfully completed and visible to the user if the AI explainer step fails outright. Secondly, cancellation (the Stop control) is cooperative: run_scan() takes a callback, should_cancel(), which is checked at each polling point within the ZAP spider/active-scan loops, and will raise a ScanCancelled exception the next time the callback is made, which the runner will catch, and which will set the job's status to "stopped" — so the Stop request will take effect within one polling interval of ZAP (usually a few seconds), not instantly, which is a reasonable and clearly bounded trade-off for a threaded, non-preemptive cancellation design.

4.2.12 Security and Legal Design Considerations

In addition to its functions, two types of safeguard have been designed into VulnLab. Technically, each password is encrypted with bcrypt before it's stored in the database and never logged anywhere or returned in any API responses (auth.py, hash_password/check_password), and each of the scan, schedule and settings routes is protected with the @require_auth decorator, which also scopes every query to the database to the user's own id (e.g. list_scans() returns only the user's own scans, never another account's), so no account can view or modify another account's scans or schedules. At a very basic level, the limitation for the entire scanning feature is legally and ethically the statement found in the README of the backend: Only scan sites you OWN or have explicit permission to test. Unauthorized scanning is illegal (Malaysia: Computer Crimes Act 1997). The same document suggests using targets for development and demonstration which are intentionally vulnerable, sanctioned practice targets (such as http://testphp.vulnweb.com, a locally run OWASP Juice Shop or DVWA). This also includes the fact that the AI assistant itself included this in its system prompt (Section 4.2.8) that "users must only scan sites they own or are authorized to test. VulnLab doesn't actually create a technical constraint that stops a user from entering an unauthorised target, but the design does in fact highlight this constraint at every point where a user is likely to encounter it: the documentation, the in-app assistant and subsequently, any new onboarding flow built on top of the current account model.

4.3 Database Design

VulnLab's relational data store is PostgreSQL, and is accessed only through a declarative ORM (db.py) provided by SQLAlchemy. The schema was designed to directly support the trend analysis and Compare Scans features which were identified by the survey respondents as having high value (Section 3.3, Figures 69 and 73), by retaining all historical scans and findings against the target application over time and not overwriting the previous results — any scan of the same URL becomes another row, and does not replace the previous scan. As described in the own comments in db.py, it is a deliberate implementation detail that all timedamps are stored as naive (timezone-ignorant) datetimes that are always assumed to be stored in UTC time, both when they're inserted (utcnow()) and when they're retrieved (to_epoch_ms()). This prevents PostgreSQL silently changing the time of output to the server's local time zone, moving stored times by the server's UTC offset, which is an insidious class of bugs that the design note in the source code mentions as the rationale for the convention. These UTC epoch-millisecond values are only converted to the display timezone (Section 4.4.7) chosen by the user during rendering at the frontend.

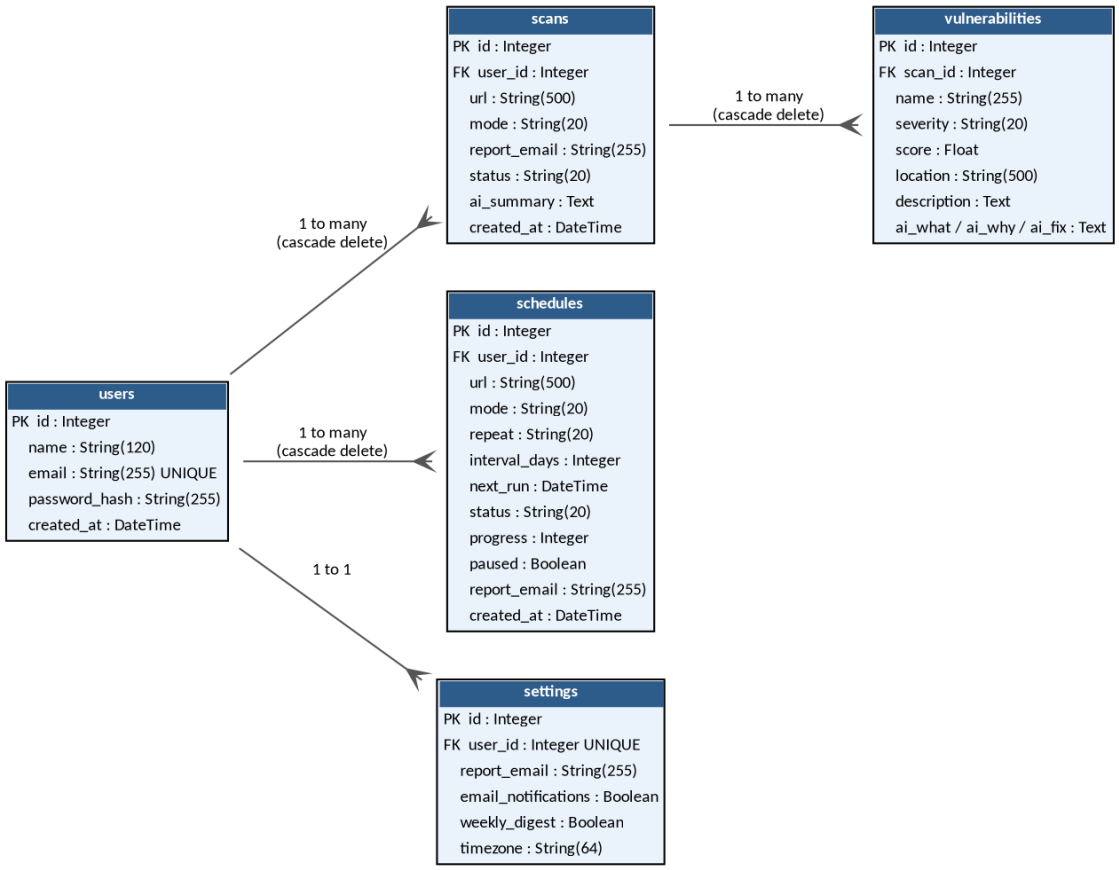


Figure 85 VulnLab Entity Relationship Diagram (traced from db.py)

Table 10 documents every table exactly as declared in the SQLAlchemy models, including the cascade behaviour that governs what happens to dependent rows when a parent is deleted.

Table 9 Database Table Descriptions (db.py)

Table	Key columns	Purpose and relationships
users	id (PK), name, email (unique), password_hash, created_at	One row per registered account. password_hash always stores a bcrypt hash, never plaintext. Deleting a user cascades to delete all of that user's scans and schedules (cascade="all, delete-orphan").
scans	id (PK), user_id (FK), url, mode, report_email, status, ai_summary, created_at	One row per scan run (manual or scheduled). status moves through queued/running/completed/failed. ai_summary holds the AI-generated plain-English overview once available. This table anchors both the Dashboard trend chart and the Compare Scans diff (Section 4.2.5).
vulnerabilities	id (PK), scan_id (FK), name, severity, score, location, description, ai_what, ai_why, ai_fix	One row per finding belonging to a scan. severity is one of critical/high/medium/low/info (Section 4.2.7). score is the 0-10 risk-aware float. ai_what/ai_why/ai_fix hold the per-finding plain-English explanation. Deleting a scan cascades to delete its vulnerabilities.
schedules	id (PK), user_id (FK), url, mode, repeat, interval_days, next_run, status, progress, paused, report_email, created_at	One row per recurring or one-off scan definition. repeat is Once/Daily/Weekly/Monthly/Custom; interval_days is only meaningful when repeat=Custom. status is pending/running/done; progress (0-100) is updated live while running; paused stops the scheduler from firing it (Section 4.2.9).
settings	id (PK), user_id (FK, unique), report_email, email_notifications, weekly_digest, timezone	One row per user (created lazily on first access via _get_or_create_settings()), holding the default report recipient, notification preferences, and the display

Table	Key columns	Purpose and relationships
		timezone used to convert every stored UTC timestamp for that user (Section 4.4.7).

There are two relationships worth mentioning as they are fundamental for certain features instead of incidental structure. First, Scan.to_dict() calculates a counts object (number of critical/high/medium/low/info vulnerabilities) for each scan it reads, which is the pre-aggregated format that the Dashboard's summary cards, the Results grid's severity pills and the Dashboard's Reports page's risk filter are all designed to receive directly, without any backend endpoint read needed. Second, vulnerabilities are never written in place, so that a rescanned URL simply produces a new Scan row, with its own new Vulnerability rows, and the Compare Scans feature (Section 4.2.5) can safely compare any two scans as two "frozen" snapshots, without the worry of comparing a finding against a version of it that has been modified under the same key.

4.4 Interface Design

The results of the survey analysis in Section 3.3 directly influenced the design of the VulnLab interface. Respondents consistently stated that complexity, unclear reporting and lack of prioritisation are the top obstacles to using the existing vulnerability scanners (Figures 60 and 68), while a simplified and risk aware dashboard was highly valued (Figure 69, 73.3% agree). The implementation is the same as was used in the source code (src/styles/global.css), all components listed in Table 6 are included and it is a dark theme product: a near-black background (0A0E14) and a single cyan accent colour (00D9FF) is used throughout the product for the primary action, links, and the informational severity band; further, four more distinct colours (critical FF3B5C, high FF8C42, medium FFD23F and low 4ADE80) were introduced, which are used in the same way across the dashboard, the results pages, and the emailed report described in Section 4.2.10, meaning that a colour that a user learns on one screen, means the same thing, in the same way, on other pages.

4.4.1 Authentication Screens (Login.jsx, Signup.jsx)

The Login screen (route /login) gathers email and password in the center of a 360px card and calls POST /api/auth/login; any error message received from the server (e.g. "Incorrect email or password.") is directly displayed in a red banner on top of the form. The Signup screen (/signup) also displays the six-to-symbol password policy as a live, four-point checklist right under the password field, using six independent test functions in Signup.jsx (checks for length, uppercase,

digit, and symbol), which are run each time the user types a character, and the Create Account button is also disabled using disabled={!allMet) till every rule passes the same policy that is enforced server-side in auth.py's password_problem is enforced (and described) in real-time on the client.

4.4.2 Application Shell (Sidebar.jsx, Layout.jsx)

All authenticated pages are enclosed in a fixed two-column shell: a 220px wide left side bar (Sidebar.jsx) containing 7 destinations with outlined line icons and highlighted in cyan when selected, and a scrollable main content area, rendered using React Router's <Outlet> component. The floating AI assistant bubble (see Section 4.4.6) and the New Scan modal are mounted once, not per page, and therefore, when the user uses the assistant, it appears exactly the same on every screen, as per the requirement in Section 4.2.4 that the assistant be "available and stateful" across the session.

4.4.3 Dashboard (Dashboard.jsx)

The Dashboard is the index route (/) and is the landing page following login. It generates four coloured summary cards (Critical/High/Medium/Low; each one totaled across all scans the user has run), a simple SVG line chart showing the number of findings per scan over time ("Findings over time"), a 'Top risks' panel that displays the four most severe findings across all the scans (sorted by severity band then score), and a 'Recent scans' table which shows the four most recent scans and which navigates to the corresponding scan detail page on click. This screen directly addresses the objective of the trend-analysis from Section 1.4 and is in line with the high level of preference for a simplified and prioritised overview found in the survey (Section 3.3, Figure 69).

4.4.4 New Scan (ScanModal.jsx)

This modal is opened by the "New scan" link provided from the sidebar (see the common useNewScan() context described in Section 4.2.4) and from any other place in the app, collects the following information: a target URL and a report email (which defaults to the signed in user's own address, as returned by currentUser()). It displays the 3 scan modes described in Table 8 as 3 selectable cards with their one line description, and it provides a permanent note explaining that "All modes test for the same vulnerabilities... The mode only changes how much of the site is scanned. When submitted, the modal would be in a running state, displaying the real-time polling loop results (live phase text and percentage) and filling a progress bar, and at the end of the modal would close and the user would be directed directly to the new scan's detail page.

4.4.5 Scan Results and Detail (Results.jsx, ScanDetail.jsx)

The Results screen (/results) displays each scan as a card in a responsive grid – along with the target URL, scan date, mode used, and a set of severity-count pills, the shared Toolbar component provides a search box, a mode filter, and a sort control (Newest first / Oldest first / Most findings / URL A-Z) – which are all computed entirely on the client-side (Section 4.2.4). Each card on the Scan Detail screen (/results/:id) provides the same four cards of severity summary as in the Dashboard, and an overall "In plain English" panel, which, when available, contains the AI-generated overall summary of the finding, and an individual card for every finding, including a risk-aware score badge, a name and location, followed by either a three-part AI explanation of What this means / Why it matters / What to do or the raw technical description if the AI explanation was unavailable for that finding. Both screens contain an Export CSV button to download the exact set of columns that scansToCSV() creates (scan URL, scan date, mode, vulnerability name, severity, risk score and location) directly in the browser without requiring another backend call.

4.4.6 Scheduled Scans (Scheduled.jsx, ScheduleModal.jsx)

The Scheduled screen (/scheduled) displays all schedules with their target, mode, repeat rule, next scheduled run time (converted to the user's selected timezone) and a live status indicator: a pending one-off schedule will have an amber pill reading "Pending" (no matter what mode, pending will appear); a running schedule will have an animated circular progress ring, which is computed using the same progress value that the scheduler writes to the database (Section 4.2.9); a done one-off schedule will have a green "Scan done" pill (no matter what mode, done will show as done); a paused schedule will have a neutral "Paused" pill (no matter what mode, paused will show as paused). When a schedule is running, the screen is calling the list of schedules every 3 seconds, and when not, every 15 seconds, thus the progress ring is animated smoothly without putting burden on the API when there is no schedule running. Each row displays a pause/resume button, a Stop button (appears while running and invokes the route to cancel, as described in Section 4.2.9), and a delete button that presents a deletion confirmation dialog, which includes the name of the specific URL to be deleted just prior to sending the deletion request. Also included is a wall-clock to UTC epoch milliseconds conversion (or vice versa) that happens on the client, which, in conjunction with the above, ensures that the user typing "9:00 AM" in the Asia/Kuala_Lumpur timezone will have their scan take place at 9:00 AM local time in any situation, regardless of what timezone the backend is running in.

4.4.7 Compare Scans (Trends.jsx)

This screen, accessed through a "Compare scans" link on the sidebar, allows the user to select any two of his/her own scans from two drop-down menus (the two most recent scans are displayed by default), and client-side calculates the difference between the two by seeing which findings appeared in Scan A and not in Scan B (Fixed); which findings appeared in Scan B and not in Scan A (New); and which were common to both (Still present). The three lists that are created are each displayed as an individual card with a count in the header and the individual findings appear with the appropriate severity badge, name and location, just as they would on the Scan Detail screen in a finding-row. The screen is the direct implementation of the scan-fix-rescan-verify workflow that was identified as missing from the existing tools during the survey analysis (Section 3.3, Figures 73-76) and does not need a dedicated back-end endpoint of its own: it can be entirely built out of data that is already available through GET /api/scans, as described in the client-side-computation design choice (Section 4.2.4).

4.4.8 Reports (Reports.jsx)

The Reports screen (/reports) displays each completed scan as a full-width row instead of the card grid on the Results screen, and includes the target URL, the date sent, the mode, the email recipient the report was sent to, and the total number of findings. It shows three separate filters: free-text search on URL, a date-range filter (All time / Last 7 days / Last month / Last 3 months, which is based on the same UTC epoch timestamps as the entire schema), and a risk filter (All reports / Has critical / Has high / No critical or high), and provides the same CSV export as the Results and Scan Detail screens. This screen was developed specifically for the question "which of my past reports needs attention" brought up by the reporting-clarity concerns in Section 3.3 (Figure 68) and not as a part of the general scan history browsing on the Results screen.

4.4.9 Settings (Settings.jsx)

The Settings screen (/settings) is organised into three cards. The Account card can be used to update the signed-in user's email and/or password (calling PATCH /api/auth/account, which returns a new JWT with the update). The Notifications card lets edit the default notification recipient, and toggle an automatic email on complete-scans, a (not yet implemented) weekly digest preference, and — most importantly — pick a display-timezone from a fixed list of 9 common timezones (defaulting to Asia/Kuala_Lumpur); this timezone choice is applied immediately in the browser via setUserTimezone(), so all existing dates in the browser are re-rendered in the new zone without a page reload, and is also stored to localStorage to be used before the settings API call resolves on a

fresh page load. On the Session card, there's a single Log out action which clears the cached JWT and user record and redirects to the login screen.

4.4.10 AI Assistant (Assistant.jsx)

On each screen (after successful login) a chat panel of size 340px is opened on the bottom-left corner of the viewport, using a floating circular button which is styled to match the rest of the application. The assistant starts with a fixed greeting and stores the entire conversation in the local component state for the whole session and sends each new message to POST /api/assistant, which is responded to by the scope-limited system prompt from Section 4.2.8. The panel automatically scrolls to the newest message and displays a minimal "Thinking…" bubble when waiting for a response.

4.5 Execution

VulnLab was implemented as two independently runnable, independently deployable applications — a React (Vite) frontend and a Flask backend — communicating exclusively over a JSON REST API secured by JWT bearer tokens, with OWASP ZAP, PostgreSQL, and two third-party services (an AI provider and an SMTP relay) integrated as supporting services. Table 11 summarises the technology used for each component of the running system, drawn directly from requirements.txt and package.json.

Table 10 Technology Stack by Component

Component	Technology	Role
Frontend	React 18 + Vite, React Router	Single-page application serving all ten routes listed in Table 6; communicates with the backend through a single fetch-based api.js client that attaches the JWT to every request.
Backend	Flask 3.0.3 + SQLAlchemy 2.0.35	REST API, JWT authentication (PyJWT 2.9.0), password hashing (bcrypt 4.2.0), the risk-scoring logic, and orchestration of ZAP, the AI provider, and the scheduler.

Component	Technology	Role
Database	PostgreSQL, accessed via psycopg2-binary 2.9.9	Persistent storage of users, scans, vulnerabilities, schedules, and settings, per the schema in Section 4.3. A local SQLite file is used automatically as a zero-configuration fallback if DATABASE_URL is unset.
Scanning engine	OWASP ZAP (daemon mode), python-owasp-zap-v2.4	Performs the active and passive crawling and detection of SQL injection, XSS, missing/misconfigured security headers, and authentication weaknesses; falls back to fixed demo alerts if unreachable.
AI explanation layer	Configurable AI provider via ai_explainer.py	Generates plain-English per-vulnerability explanations and an overall scan summary from already-scored findings; falls back to rule-based static text if unavailable (Section 4.2.8).
Scheduler	APScheduler 3.10.4 (BackgroundScheduler)	Runs inside the Flask process; polls every 60 seconds to trigger due, unpaused schedules and respects the pause/stop/delete controls described in Section 4.2.9.
Email service	smtplib (standard library) via SMTP, developed against Mailtrap	Delivers scan-completion reports built by emailer.py; silently skipped if SMTP credentials are not configured.

To start the system up end-to-end, three long-running processes need to be started separately: the OWASP ZAP daemon (zap.bat -daemon -port 8080 -config api.disablekey=true for local development, as this is described in the backend README), the Flask backend (python app.py, where the start_scheduler() function is called before the Flask request loop), and the Vite development server for the frontend. As each ZAP and AI dependent code path has an explicit fallback (Section 4.2.1), it's possible to exercise the backend and frontend end-to-end — sign up, log in, launch a scan, view risk-scored demo scan results, schedule, and receive no email if SMTP is not configured — with just the Flask process running, which was intentionally used during early sprints to develop and demonstrate the scan-results, scheduling and reporting UI before ZAP and the AI provider were integrated.

Once triggered by another scan or a manual click ZAP does a single scan in the same fixed pipeline as shown in the previous figures, 83 and 84: the target is accessed; it is then scanned using ZAP's traditional spider (and its AJAX spider if Deep mode is requested) to discover pages, forms and parameters; ZAP's active scanner then attacks those discovered pages at the level of strength set in the mode configuration; the alerts are then read back and passed one by one through the risk-scoring function described in Section 4.2.7 to get a score and a severity band; the scan and the scored findings are then written to PostgreSQL; the same findings are sent to the AI explanation layer to get a plain English summary and what/why/fix text for each finding which is then written back to the already persisted rows in PostgreSQL; and lastly, if a report email address has been provided, the completed report is sent. All seven of these steps are individually fault tolerant in accordance with the graceful-degradation principle in 4.2.1; a failure in ZAP falls back to demo data, a failure of the AI falls back to rule-based text and an email failure results in a log and skipping, but none of them can prevent the scan being recorded, and hence being visible to the user.

4.5.1 Code Screenshots

This subsection presents source code excerpts from the key modules described in Sections 4.2 and 4.3, illustrating how the design decisions discussed earlier in this chapter are expressed in the actual implementation.

```python
def score_finding(zap_risk: str, name: str, url: str):
    """Return (score_float, severity_band) for one finding."""
    base, band = BASE.get(zap_risk, (1.0, "info"))
    name_l = (name or "").lower()
    url_l = (url or "").lower()

    exploit_bump = 0.0
    if any(k in name_l for k in HIGH_EXPLOIT_KEYWORDS):
        exploit_bump = 1.5  # known heavily-exploited class

    exposure_bump = 0.0
    if any(h in url_l for h in SENSITIVE_PATH_HINTS):
        exposure_bump = 0.8  # sits on a sensitive entry point

    score = _clamp(base + exploit_bump + exposure_bump)

    # Let the risk-aware score promote a band upward when it crosses thresholds,
    # so prioritization reflects real risk, not just ZAP's static label.
    if score >= 9.0:
        band = "critical"
    elif score >= 7.0 and band in ("medium", "low", "info"):
        band = "high"
    elif score >= 4.0 and band in ("low", "info"):
        band = "medium"

    return round(score, 1), band
```

Figure 86 risk_scoring.py — score_finding()

The risk-aware scoring function described and diagrammed in Section 4.2.7, showing the base severity mapping, the exploit and exposure bumps, and the band-promotion thresholds.

```python
def run_scan(url, mode_name, progress_cb=None, should_cancel=None):
    """
    Run a full scan and return a list of raw alert dicts:
       {name, risk, url, param, description}
    progress_cb(percent, phase) is called as the scan advances.
    should_cancel() -> bool: if it returns True, the scan aborts as soon as
    possible (used by the Stop feature). Raises ScanCancelled when aborted.
    """
    mode = get_mode(mode_name)

    def report(pct, phase):
        if progress_cb:
            progress_cb(pct, phase)

    def cancelled():
        return bool(should_cancel and should_cancel())

    # ---- DEMO MODE ----
    if not is_zap_available():
        report(10, "starting (demo)")
        for _ in range(2):
            time.sleep(1)
            if cancelled():
                raise ScanCancelled()
        report(50, "scanning (demo)")
        time.sleep(1)
        if cancelled():
            raise ScanCancelled()
        report(100, "done (demo)")
        return _demo_alerts(url)
```

Figure 87 scanner/zap_client.py — run_scan()

The OWASP ZAP driver described in Section 4.2.2, showing the access/spider/AJAX-spider/active-scan sequence, the progress callback, and the demo-mode fallback.

```
69
70  def _worker(job_id, user_id, url, mode, report_email, on_complete=None, on_progress=None):
71      def progress(pct, phase):
72          _set(job_id, percent=pct, phase=phase)
73          if on_progress:
74              try:
75                  on_progress(pct)
76              except Exception:
77                  pass
78
79      success = False
80      try:
81          alerts = run_scan(url, mode, progress_cb=progress,
82                            should_cancel=lambda: is_cancelled(job_id))
83
84          db = SessionLocal()
85          try:
86              scan = Scan(user_id=user_id, url=url, mode=mode,
87                          report_email=report_email, status="completed")
88              db.add(scan)
89              db.flush()  # get scan.id
90
91              for a in alerts:
92                  score, band = score_finding(a["risk"], a["name"], a["url"])
93                  loc = a["url"] + (f"?{a['param']}" if a.get("param") else "")
94                  db.add(Vulnerability(
95                      scan_id=scan.id,
96                      name=a["name"],
97                      severity=band,
98                      score=score,
99                      location=loc,
100                     description=a["description"],
101                 ))
```

Figure 88 scanner/runner.py — _worker()

The background job orchestrator described in Section 4.2.2 and Figure 84, showing the scan-score-persist-explain-email pipeline running on a daemon thread.

```python
def password_problem(password: str):
    """Return an error string if the password fails policy, else None.
    Policy: 6+ chars, 1 uppercase, 1 number, 1 symbol."""
    p = password or ""
    if len(p) < 6:
        return "Password must be at least 6 characters."
    if not any(c.isupper() for c in p):
        return "Password must include at least 1 capital letter."
    if not any(c.isdigit() for c in p):
        return "Password must include at least 1 number."
    if not any(not c.isalnum() for c in p):
        return "Password must include at least 1 symbol."
    return None
```

Figure 89 auth.py — password_problem() and require_auth

The password-policy rules and the JWT route-guard decorator described in Section 4.2.12.

```python
def _check_due_schedules():
    db = SessionLocal()
    try:
        now = datetime.now(timezone.utc).replace(tzinfo=None)  # naive UTC to match stored values
        # only pick schedules that are waiting (not already running)
        rows = db.query(Schedule).filter(Schedule.status == "pending").all()
        to_fire = []
        for sched in rows:
            if getattr(sched, "paused", False):
                continue  # skip paused schedules
            nr = sched.next_run
            if nr is None:
                continue
            if nr.tzinfo is not None:
                nr = nr.replace(tzinfo=None)
            if nr > now:
                continue
            to_fire.append((sched, nr))

        # mark them running first (so they aren't picked again next minute)
        for sched, nr in to_fire:
            sched.status = "running"
            sched.progress = 0
        if to_fire:
            db.commit()

        # then launch each with progress + completion callbacks
        for sched, nr in to_fire:
            print(f"[scheduler] Firing scheduled scan: {sched.url} ({sched.mode}, {sched.repeat})")
            cb = _make_on_complete(sched.id, sched.repeat, sched.interval_days, nr)
            prog = _make_on_progress(sched.id)
            job_id = start_scan_job(sched.user_id, sched.url, sched.mode,
                                    report_email=(sched.report_email or ""),
                                    on_complete=cb, on_progress=prog)
            _running_jobs[sched.id] = job_id
    except Exception as e:
        print(f"[scheduler] error: {e}")
```

Figure 90 scheduler.py — _check_due_schedules()

The APScheduler polling logic described in Section 4.2.9 and Figure 83, showing how due, unpaused schedules are claimed and launched.

```python
def generate_explanations(scan_dict):
    """
    Given a scan dict (url, counts, vulns[]), return:
      { "summary": str, "explanations": { vuln_index: {what, why, fix} } }
    Falls back to rule-based text if AI is unavailable.
    """
    counts = scan_dict["counts"]
    vulns = scan_dict["vulns"]

    # Build fallbacks first so we always have something.
    result = {
        "summary": _fallback_summary(counts),
        "explanations": {i: _fallback_explanation(v) for i, v in enumerate(vulns)},
    }

    if not is_ai_configured() or not vulns:
        return result

    # One prompt for everything (cheaper + fewer calls). Ask for strict JSON.
    findings_text = "\n".join(
        f'{i}. {v["name"]} (severity: {v["severity"]}, at {v["location"]})'
        for i, v in enumerate(vulns)
    )
    user_prompt = f"""You are helping a non-technical website owner understand a security scan.
The scan of {scan_dict['url']} found these issues (severity already decided by our system — do NOT change it):

{findings_text}

Return ONLY valid JSON (no markdown, no backticks) in exactly this shape:
{{
  "summary": "2-3 sentences in plain English summarizing the overall security state and what to prioritize",
  "explanations": [
```

Figure 91 ai_explainer.py — generate_explanations()

The AI explanation pipeline described in Section 4.2.8, showing the prompt construction and the graceful JSON-parsing fallback.

```python
"""
Database layer: SQLAlchemy models + session factory.

Tables:
  users           — accounts (password stored as a bcrypt hash, never plaintext)
  scans           — one row per completed scan
  vulnerabilities — findings belonging to a scan
  schedules       — recurring scan definitions

Stage 1 uses users only. The scan/vulnerability/schedule tables are defined
now so the schema is complete and Stage 2 can start writing to them without
a migration.
"""

from datetime import datetime, timezone
from sqlalchemy import (
    create_engine, Column, Integer, String, Float, DateTime, ForeignKey, Boolean, Text
)
from sqlalchemy.orm import declarative_base, relationship, sessionmaker

```

Figure 92 db.py — SQLAlchemy models

The five declarative model classes described in Section 4.3, showing the schema exactly as implemented.

```
// Convert a "wall clock" string (YYYY-MM-DDTHH:mm) in tz -> UTC epoch ms
function wallClockToEpoch(str, tz) {
  if (!str) return Date.now() + 5 * 60000
  const [datePart, timePart] = str.split('T')
  const [y, mo, d] = datePart.split('-').map(Number)
  const [h, mi] = timePart.split(':').map(Number)
  // first guess: treat the wall-clock as if it were UTC
  const guess = Date.UTC(y, mo - 1, d, h, mi)
  // then subtract the tz offset at that instant to get the true UTC epoch
  const offset = tzOffsetMinutes(tz, new Date(guess))
  return guess - offset * 60000
}
```

Figure 93 ScheduleModal.jsx — wallClockToEpoch() / epochToWallClock()

The timezone-correct schedule conversion logic described in Section 4.4.6.

```
return (
  <>
    <Topbar title="Compare scans" subtitle="See what changed between two scans" />
    <div style={{ padding: 26 }}>
      <Card style={{ marginBottom: 12 }}>
        <div style={{ fontSize: 13, color: 'var(--text-dim)', marginBottom: 14 }}>
          Pick two scans to compare — for example, a scan before you fixed issues and one after.
        </div>
        <div style={{ display: 'flex', gap: 12, alignItems: 'flex-end' }}>
          <ScanPicker label="Scan A (earlier / before)" value={aId} onChange={setAId} exclude={bId} />
          <div style={{ color: 'var(--text-dim)', paddingBottom: 10 }}>→</div>
          <ScanPicker label="Scan B (later / after)" value={bId} onChange={setBId} exclude={aId} />
        </div>
      </Card>

      {scanA && scanB && diff && (
        <>
          {/* side-by-side counts */}
          <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: 12, marginBottom: 12 }}>
            <CountsCard title="Scan A" scan={scanA} />
            <CountsCard title="Scan B" scan={scanB} />
          </div>

          {/* summary line */}
          <Card style={{ marginBottom: 12 }}>
            <div style={{ display: 'flex', gap: 20, flexWrap: 'wrap', fontSize: 13 }}>
              <Stat n={diff.fixed.length} label="Fixed" color="var(--low)" />
              <Stat n={diff.added.length} label="New" color="var(--critical)" />
              <Stat n={diff.still.length} label="Still present" color="var(--medium)" />
            </div>
          </Card>

          <DiffList title="Fixed (were in A, gone in B)" items={diff.fixed} color="var(--low)" empty="Nothing was fixed between these scans." />
          <DiffList title="New (appeared in B)" items={diff.added} color="var(--critical)" empty="No new issues appeared." />
          <DiffList title="Still present (in both)" items={diff.still} color="var(--medium)" empty="No shared issues." />
        </>
      )}
    </div>
  </>
```

Figure 94 Trends.jsx — the Compare Scans diff logic

The client-side Fixed/New/Still-present computation described in Section 4.2.5 and Section 4.4.7.

4.6 Screenshot

This section presents screenshots of the completed VulnLab system, corresponding one-to-one with the interface design described in Section 4.4, together with a short justification of what each screenshot demonstrates.

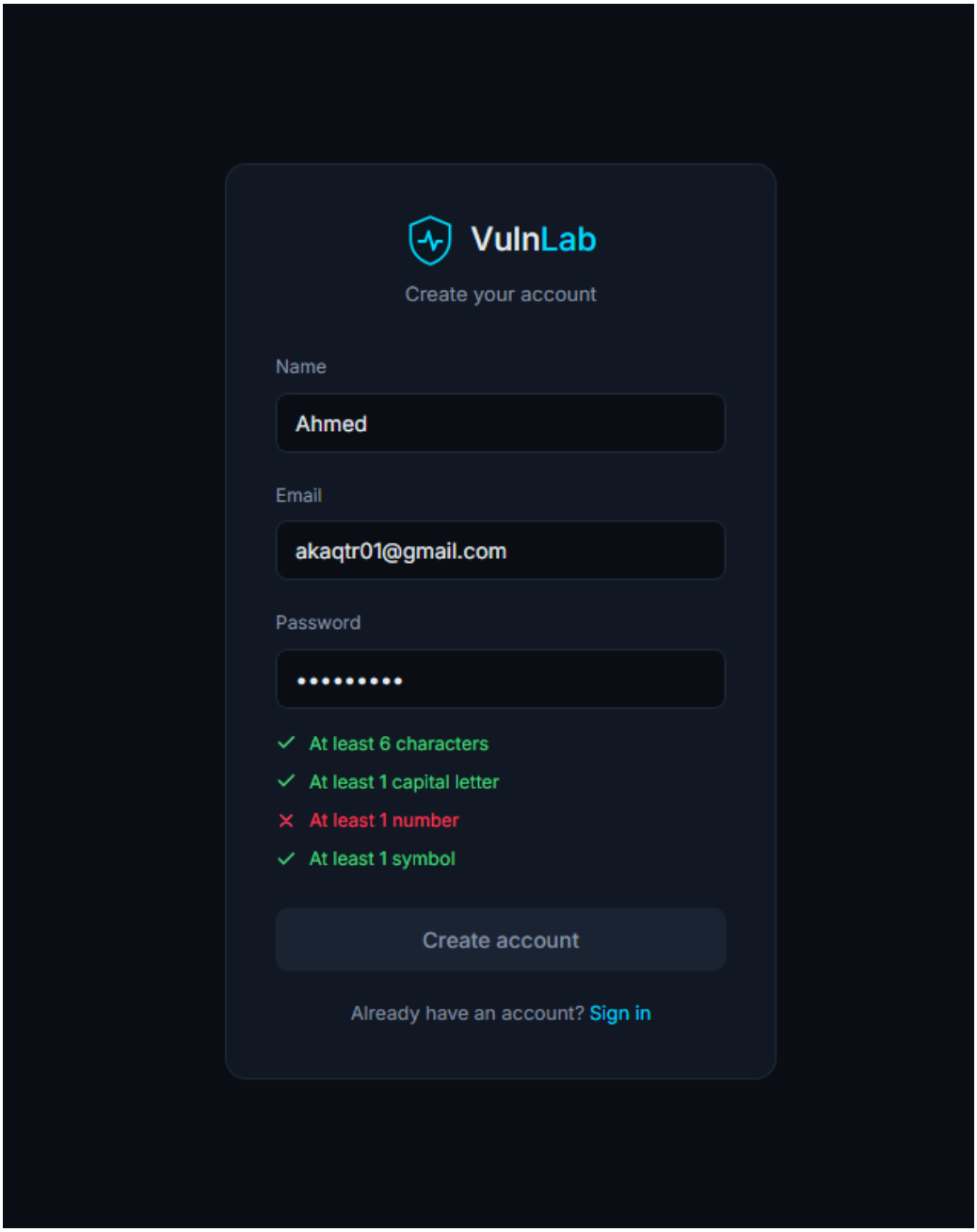
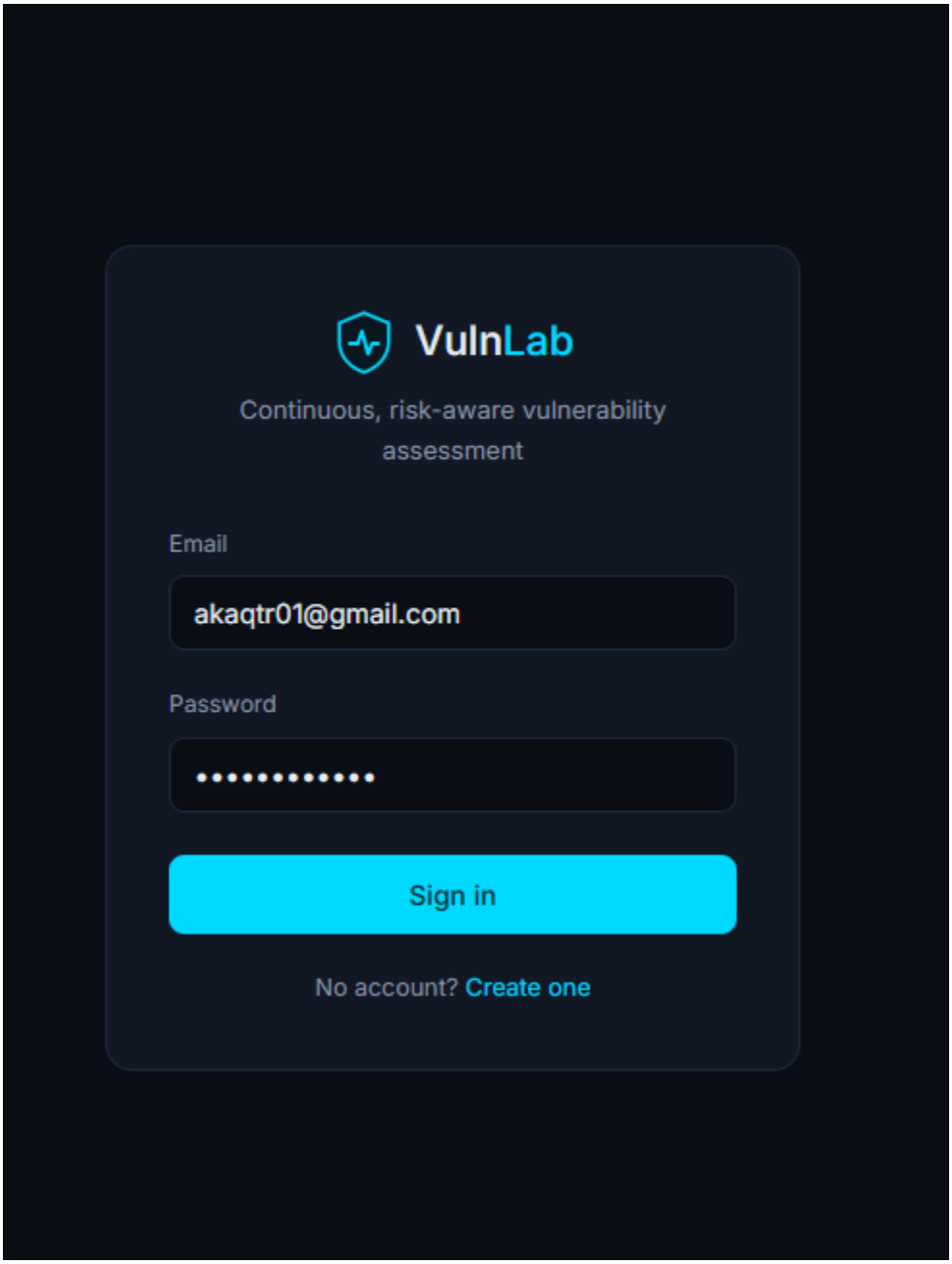


Figure 95 Login screen and Sign Up screen, including the live password-policy checklist



Demonstrates the working authentication flow described in Section 4.4.1 and confirms that the client-side policy checklist matches the server-side rule enforced in auth.py's password_problem().

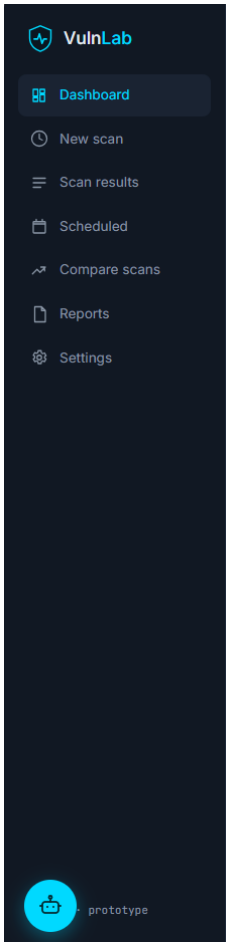


Figure 96 Application shell showing the sidebar navigation and active-route highlighting

Demonstrates the persistent two-column shell and route highlighting described in Section 4.4.2, confirming the sidebar and floating AI assistant remain mounted identically across every authenticated screen.

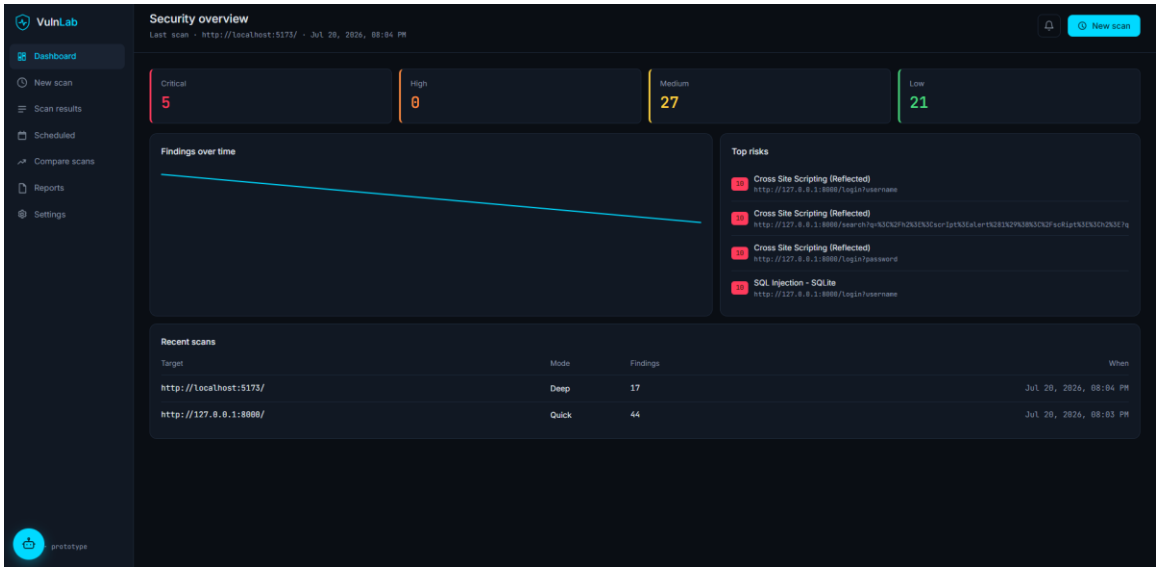


Figure 97 Dashboard with summary cards, findings-over-time chart, and top risks panel

Demonstrates that historical scan data is being aggregated correctly across every scan a user has run, as described in Section 4.4.3.

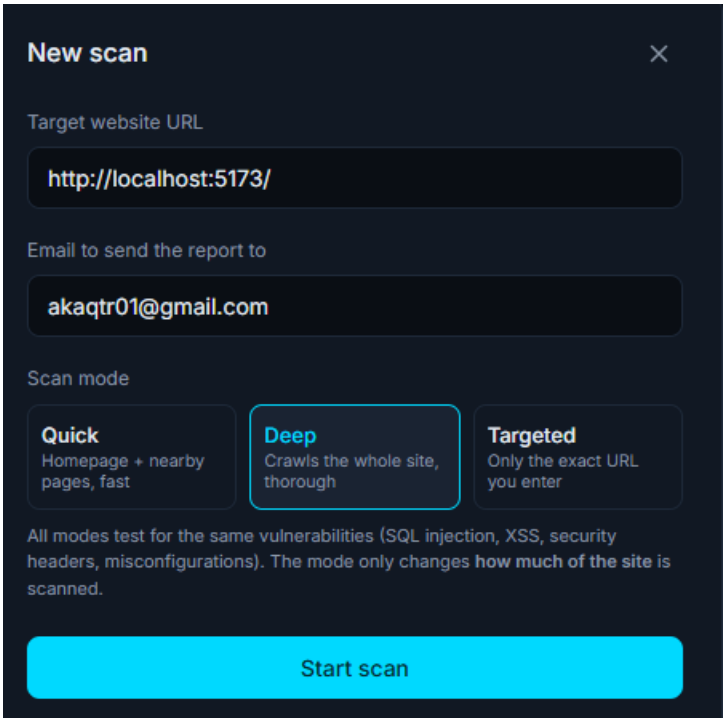


Figure 98 New Scan modal — configuration state

Demonstrates the three scan-mode cards and the auto-filled report email described in Section 4.4.4.

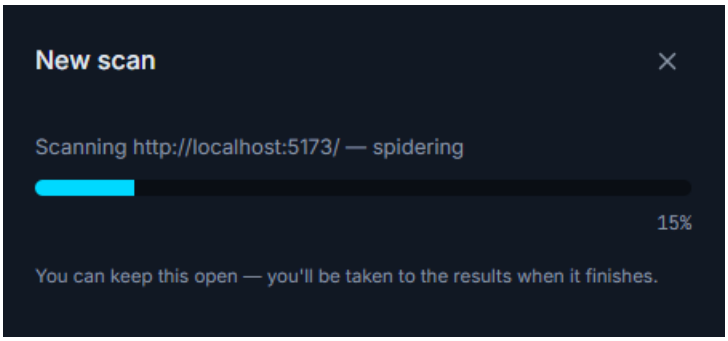


Figure 99 New Scan modal — live progress state

Demonstrates the real-time phase text and progress bar driven by the polling loop described in Section 4.2.11, confirming the asynchronous job/poll architecture is working end-to-end.

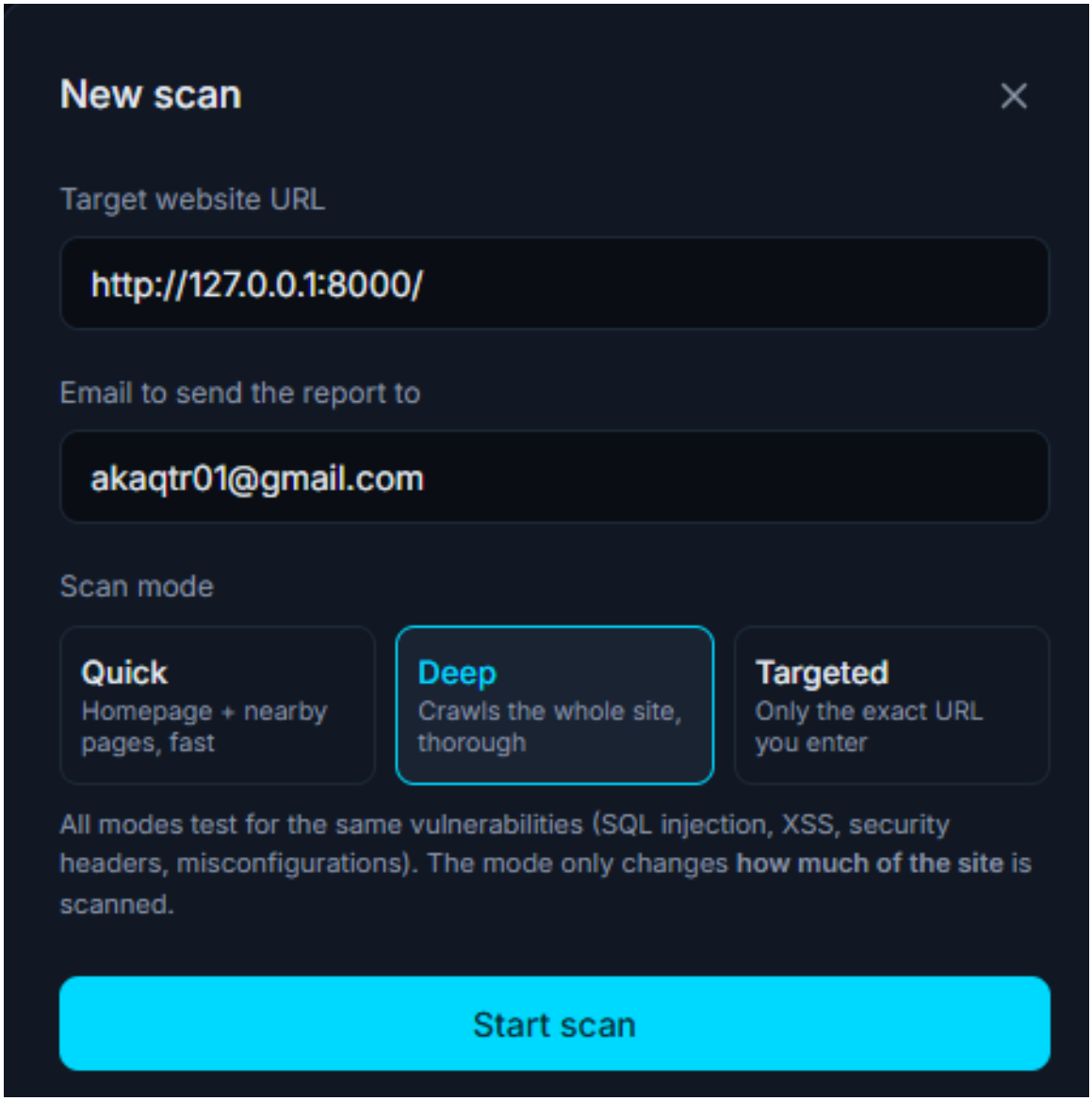


Figure 100 Scan Results grid with search, mode filter, and sort

Demonstrates the client-side filtering and severity-pill summary described in Section 4.4.5.

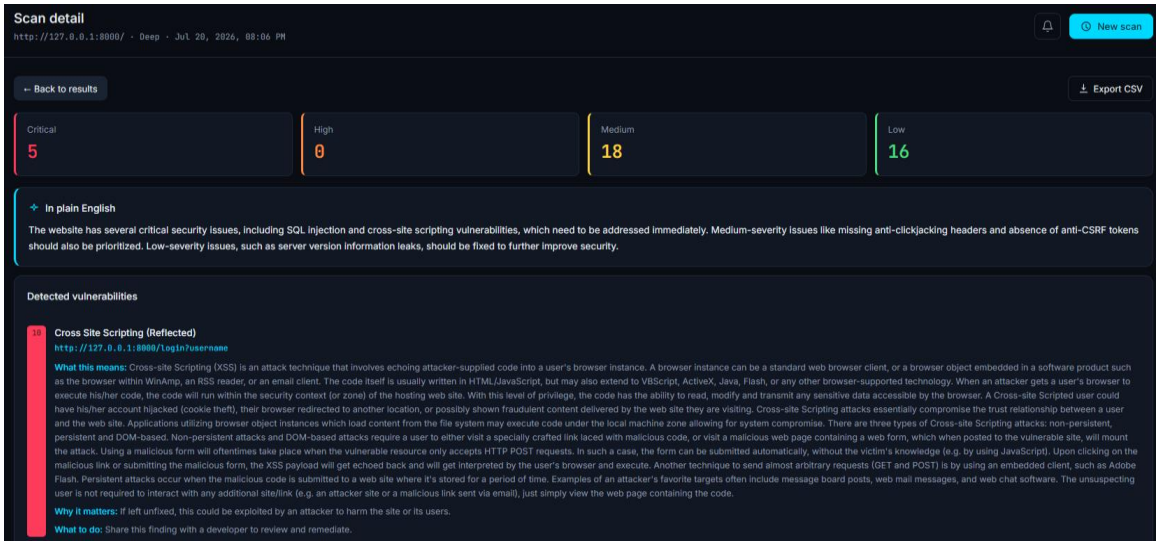


Figure 101 Scan Detail page with AI plain-English explanations

Demonstrates the severity-grouped findings and the What this means / Why it matters / What to do explanation format described in Section 4.4.5 and Section 4.2.8.

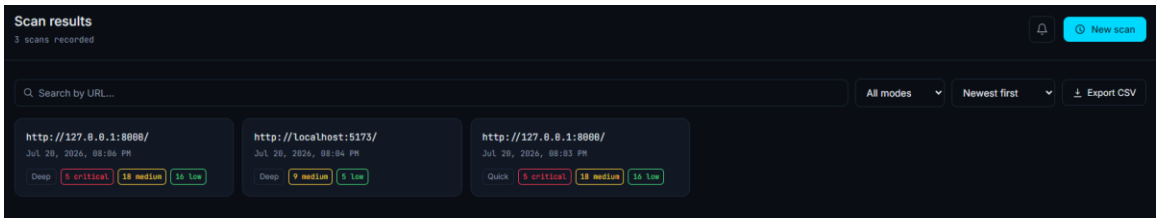


Figure 102 Additional view of the Scan Results / Scan Detail screen

A further capture of the Results and Detail screens described in Section 4.4.5, supplementing Figures 100 and 101 above.

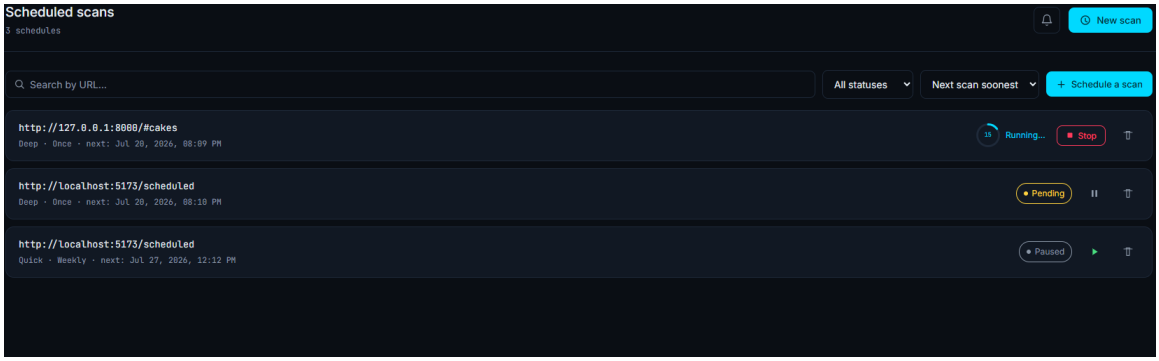


Figure 103 Scheduled Scans list showing pending, running (with progress ring), and paused states

Demonstrates the live status indicators described in Section 4.4.6 and the scheduling design in Section 4.2.9.

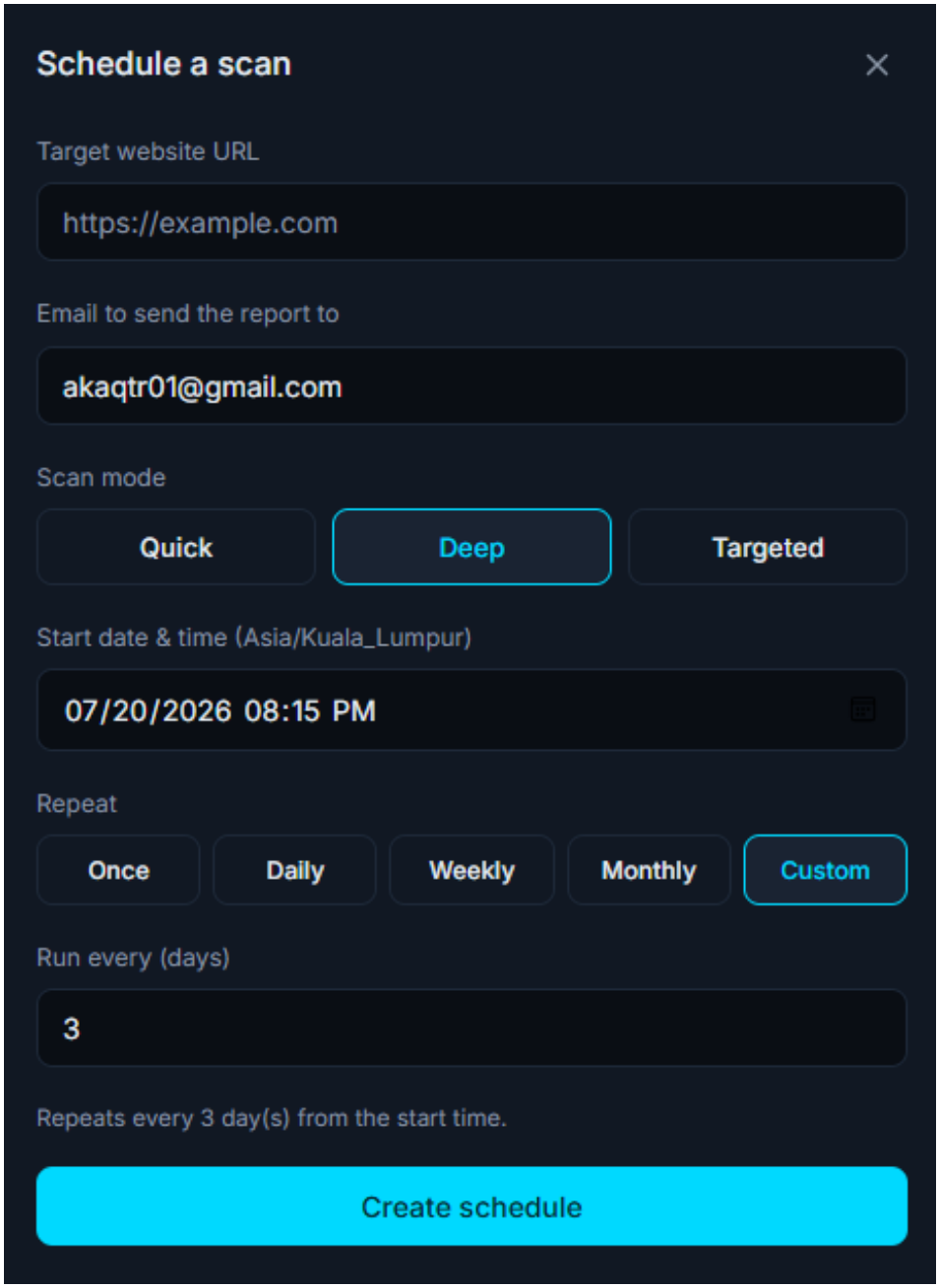


Figure 104 Delete-schedule confirmation dialog

Demonstrates the explicit confirmation step before a destructive action, described in Section 4.4.6.

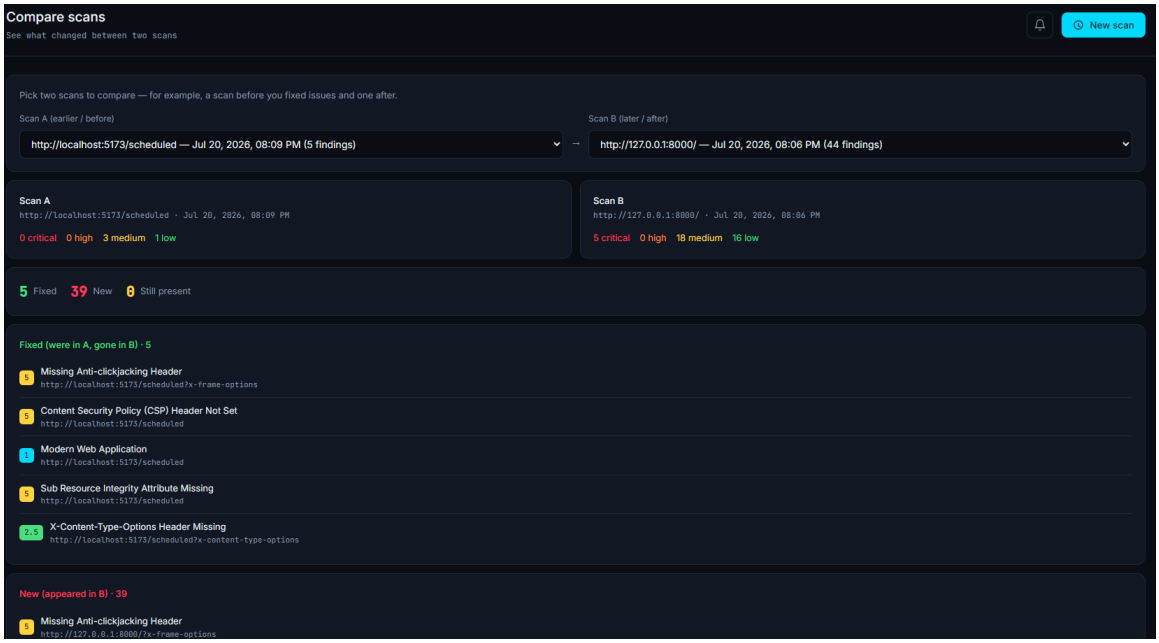


Figure 105 Compare Scans screen showing Fixed / New / Still-present findings

Demonstrates the client-side diff described in Section 4.4.7 and Section 4.2.5, directly supporting the scan-fix-rescan-verify workflow.

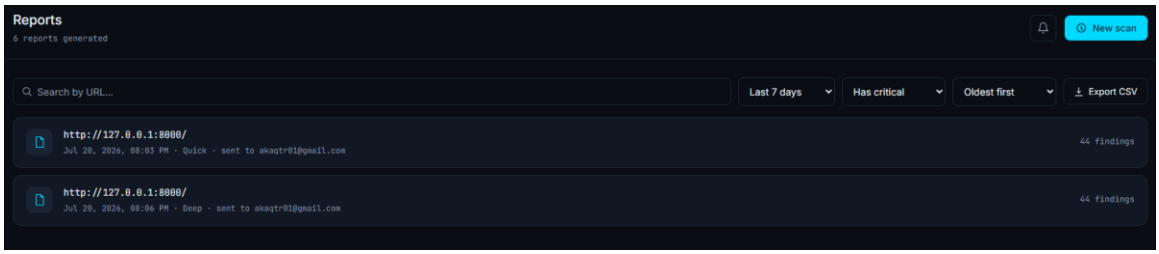


Figure 106 Reports screen with date-range and risk filtering

Demonstrates the independent filter controls described in Section 4.4.8.

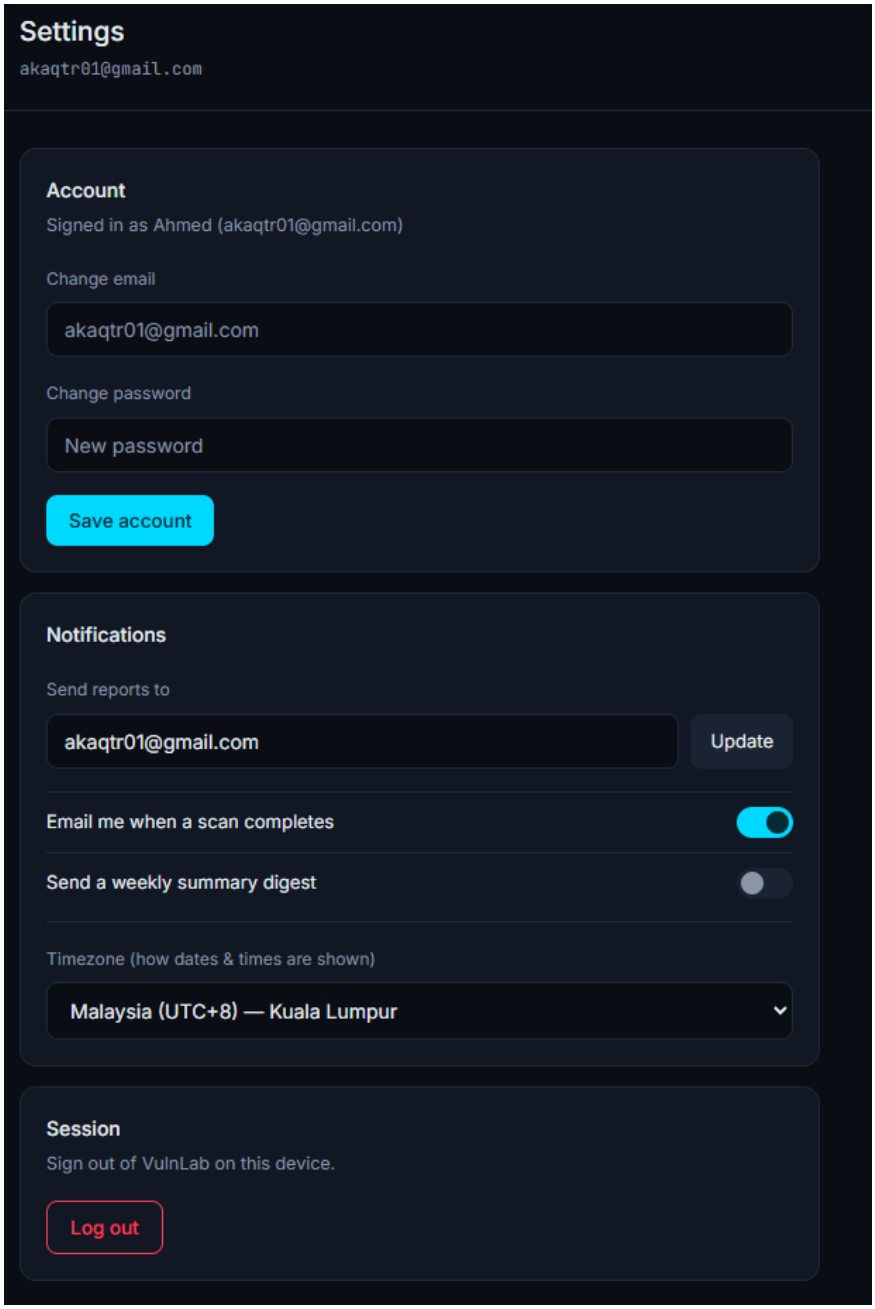


Figure 107 Settings screen, including the timezone selector

Demonstrates the account, notification, and timezone controls described in Section 4.4.9.

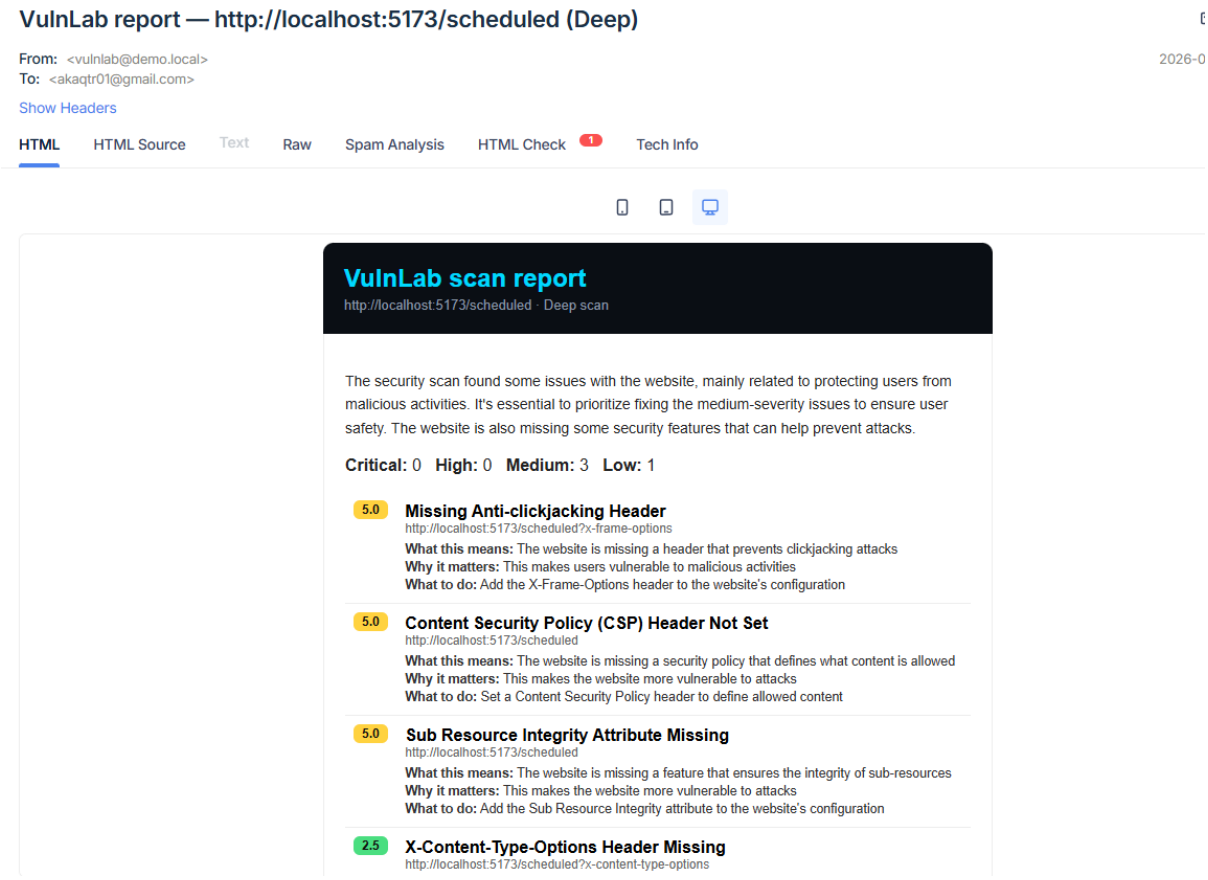


Figure 108 Emailed vulnerability report received in the test inbox

Demonstrates the HTML report format described in Section 4.2.10, confirming the email-based reporting objective defined in Section 1.4.

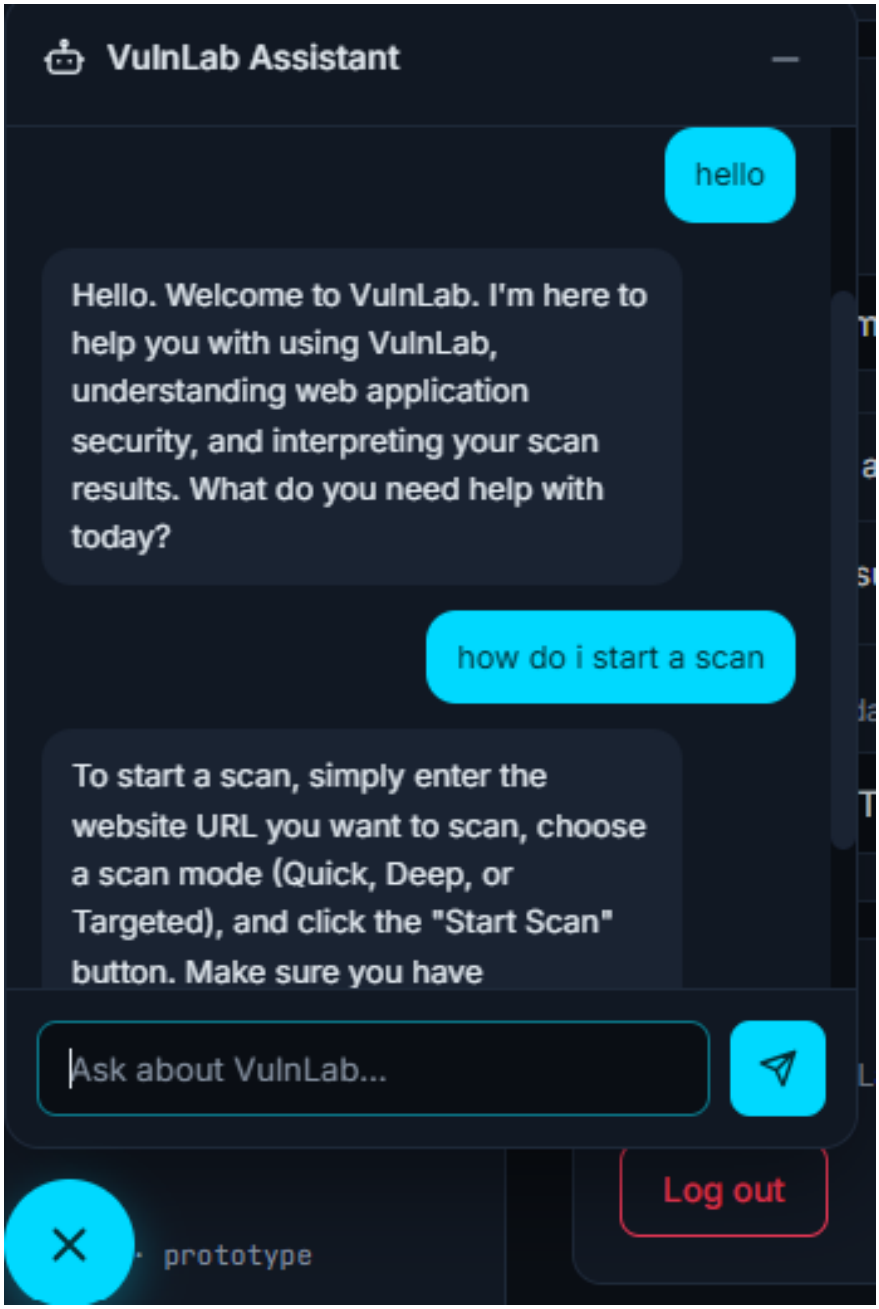


Figure 109 AI Assistant answering a VulnLab-related question

Demonstrates the scope-limited assistant described in Section 4.4.10 and Section 4.2.8.

4.7 Summary

This chapter described the design and implementation of VulnLab, providing a tangible representation of the system developed in the previous chapters and returning to the actual backend and frontend code for each design decision. The system architecture is composed of the presentation, application and data layers and incorporates OWASP ZAP to allow scanning, PostgreSQL to provide persistence, an AI provider to explain to a user in plain English what was found and APScheduler and SMTP relay to enable automated, scheduled email reporting with graceful failure on all external resources, meaning that the system will still be usable if ZAP, the AI provider, or SMTP are missing. The risk-aware scoring engine (Section 4.2.7) is the original technical contribution of the project which has taken the conclusions of the literature review (Section 2.1.6) and directly applied them: that static severity does not necessarily mean real operational risk, and that known exploitation patterns, along with endpoint sensitivity, should be taken into account. The interface design, detailed screen by screen in Section 4.4 based on actual React components implemented, complements the usability and prioritisation considerations pointed out in the survey analysis in Chapter 3, from the interactive live password-policy checklist at sign-up to the plain-English explanations provided by the AI on each finding. These authentication, scanning, risk-scoring, scheduling, comparison, and AI-explanation capabilities outlined throughout this chapter are the fundamental outcome of the project scope as stated in Section 1.5. The following chapter outlines the testing design and the results and discussion of the testing of this implementation with real users.

Chapter 5: Result and Discussion

5.1 Introduction

The testing performed on VulnLab to determine if the completed implementation described in Chapter 4 fulfills the objectives outlined in Section 1.4 and all the requirements collected from the survey analysis in Chapter 3 are documented in this chapter. There are two separate issues that are associated with testing an automated vulnerability assessment system, and that cannot be resolved by any one testing technique alone. The first concern is technical correctness: do the individual algorithms and logic that VulnLab relies on – most importantly the risk-aware scoring engine described in Section 4.2.7, the authentication and password-policy logic, and the scheduling date arithmetic – behave correctly for every path of input that they were designed for, including edge cases that may not be covered by normal use of the VulnLab interface. The second concern is behavioural correctness and usability - and whether the system as a whole, when exercised by an

ordinary user through the browser, produces the outputs that the user expects, and whether the groups of users described in Section 1.6 as the target users of VulnLab - the developers, students, small-business owners, and junior analysts - find it understandable, trustworthy, and more convenient to use than the existing tools they find to be complex and unclear in their survey in Section 3.3.

- Three complementary testing techniques were thus chosen for VulnLab that met the minimum of two techniques required by the FYP documentation guidelines and directly addressed the two concerns above:White-box testing, used to verify the internal logic of individual functions and algorithms with full knowledge of their source code, most importantly the risk-aware scoring engine, the password policy, the JWT authentication flow, and the scheduler's date-advancement logic.

- Black-box testing, used to verify the system's externally observable behaviour — its REST API and its React interface — purely from input and expected output, without reference to how a feature is implemented internally.

- User Acceptance Testing (UAT), used to verify that real representatives of VulnLab's target user groups can complete the system's core workflows unaided and consider the result understandable and useful, directly closing the loop on the usability concerns raised in the Chapter 3 survey.

The rest of this chapter looks at the following. The testing design is presented in Section 5.2, consisting of the Objective and Scope of the three techniques and the complete set of test cases and UAT scenarios which are reproduced here as a testing manual, prior to testing – as per the FYP documentation guidelines, the Actual Result and Status columns in the tables below are left blank until the test are executed. Section 5.3 will discuss the results of running these tests, and what they mean for the completed system. The chapter is summarised in section 5.4.

5.2 Testing Design

This section describes how VulnLab was evaluated. Table 12 maps each of the three testing techniques to what it covers, why it was chosen, and how many testers or test cases it involves, before each technique is described in full in its own subsection.

Table 11 Overview of Testing Techniques Applied to VulnLab

Technique	What it covers	Why it was chosen	Scale
White-box testing	Internal logic of key backend functions: risk scoring, password policy, authentication, scheduling arithmetic, and AI-response parsing.	These modules contain the project's core algorithmic contribution (Section 4.2.7) and several edge-case-sensitive rules that are not fully exercised by normal UI interaction alone; full knowledge of the source code is needed to design inputs that reach every logic branch.	14 test cases across 6 modules.
Black-box testing	Observable input/output behaviour of the REST API and the React interface: signup, login, scanning, scheduling, comparison, export, and the AI assistant.	Confirms the system behaves correctly from the outside — the perspective every real user actually has — independent of implementation detail, and catches integration issues that unit-level white-box tests cannot see.	18 test cases across 8 features.
User Acceptance Testing (UAT)	Whether real target users (Section 1.6) can complete VulnLab's core workflows unaided and find the result clear and useful.	Directly answers the usability and trust concerns raised throughout the Chapter 3 survey (Figures 60, 65, 68, 69); required by the FYP guidelines to be executed with the intended target audience.	12 scenarios x 3 testers (minimum required).

5.2.1 White-Box Testing Design

White-box testing provides a more detailed look at the function's logic paths, and because it relies on the source code, it can select inputs to test all logic branches, whether they are easy or difficult to reach through clicks on GUI, such as the numeric thresholds in the risk-scoring algorithm. Each white-box test case in Table 14 below is in the form of Table 13: Test ID, Module under test,

Function under test, Logic path or branch being tested, Test input, Test output (directly from reading function source code), Actual result / status (left blank for execution).

Table 12 White-Box Test Case Template

Field	Description
Test ID	Unique identifier, prefixed WB- for white-box.
Module / Function	The exact source file and function under test.
Logic Path Tested	The specific branch, condition, or code path being exercised by this input.
Test Input	The exact value(s) passed to the function.
Expected Output	The value the function must return, derived directly from reading its source code.
Actual Output	Left blank at design stage; filled in during execution.
Status	Left blank at design stage; PASS or FAIL, filled in during execution.

Table 13 White-Box Test Cases

ID	Module / Function	Logic Path Tested	Test Input	Expected Output
WB-01	risk_scoring.score_finding()	Base severity mapping, no bumps apply	risk="High", name="Generic Header Missing", url="http://x.com"	(8.0, "high")
WB-02	risk_scoring.score_finding()	Exploit bump applies (SQLi keyword), no exposure bump	risk="Medium", name="SQL Injection", url="http://x.com/page"	(6.5, "medium")

ID	Module / Function	Logic Path Tested	Test Input	Expected Output
WB-03	risk_scoring.score_finding()	Exposure bump applies only, insufficient to promote band	risk="Low", name="Missing Cache-Control Header", url="http://x.com/admin/settings"	(3.3, "low")
WB-04	risk_scoring.score_finding()	Both bumps apply and score clamps at 10, band promoted to critical	risk="High", name="Cross Site Scripting (Reflected)", url="http://x.com/search?q="	(10.0, "critical")
WB-05	risk_scoring._clamp()	Lower- and upper-bound clamping	_clamp(-5) and _clamp(15)	0.0 and 10.0 respectively
WB-06	risk_scoring.score_finding()	Unrecognised ZAP risk label falls back to default	risk="Unknown", name="X", url="http://x.com"	(1.0, "info")
WB-07	auth.password_problem()	Length rule fails first regardless of other rules	password="a1!"	"Password must be at least 6 characters."
WB-08	auth.password_problem()	Uppercase rule fails when	password="abcdef1!"	"Password must include at least 1 capital letter."

ID	Module / Function	Logic Path Tested	Test Input	Expected Output
		length is satisfied		
WB-09	auth.password_problem()	All four rules satisfied	password="Abcdefg1!"	None (no error)
WB-10	auth.hash_password() / check_password()	Hash round-trip and mismatch detection	hash "Test1234!", then check with correct and incorrect password	check_password returns True for the correct password, False for the incorrect one; the stored hash is never equal to the plaintext
WB-11	auth.make_token() / decode_token()	Token round-trip and rejection of a tampered token	Encode a token for a valid user, then decode it; separately decode a corrupted token string	Valid token decodes to a payload containing the correct sub (user id); the corrupted token returns None
WB-12	scheduler._make_on_complete()	next_run advancement loop correctly skips a backlog of missed runs	repeat="Weekly" (7 days), fired_at is 10 days in the past, success=True	next_run is advanced in 7-day steps until it is in the future, i.e. fired_at + 14 days, not fired_at + 7 days
WB-13	scanner.scan_modes.get_mode()	Unknown mode name falls back to the Quick configuration	get_mode("Nonexistent")	Returns the exact SCAN_MODES["Quick"] dictionary

ID	Module / Function	Logic Path Tested	Test Input	Expected Output
WB-14	ai_explainer.generate_explanations()	Markdown-fenced JSON response is parsed correctly	A mock AI response wrapped in ```json ... ``` fences containing a valid summary/explanations payload	Fences are stripped and the JSON is parsed successfully into the summary and per-index explanations

Test cases WB-01 through WB-06 cover all possible branches (including the two additive bumps and clamp boundaries) in the risk-scoring algorithm described and diagrammed in Section 4.2.7 (Figure 81), since the function is side-effect-free and pure. WB-12 specifically aims at a scheduling design detail (Section 4.2.9) which is very hard to test via the interface alone: when a backend is down after a schedule's due time, it shouldn't re-fire missed runs, and only by directly manipulating the internal date-advancement loop with an intentionally stale fired_at value can it be confirmed.

5.2.2 Black-Box Testing Design

Black-box testing treats VulnLab as a closed system and verifies its behaviour purely through the same REST API and browser interface a real user or client would use, without reference to how a given feature is implemented internally. Each test case in Table 16 follows the template in Table 15.

Table 14 Black-Box Test Case Template

Field	Description
Test ID	Unique identifier, prefixed BB- for black-box.
Feature	The user-facing feature under test, cross-referenced to its interface design description in Section 4.4.
Test Scenario	The action a user (or an API client) performs.
Test Input	The exact value(s) entered or submitted.
Expected Output	The observable result — on screen or in the API response — that indicates correct behaviour.
Actual Result	Left blank at design stage; filled in during execution.

Field	Description
Status	Left blank at design stage; PASS or FAIL, filled in during execution.

Table 15 Black-Box Test Cases

ID	Feature	Test Scenario	Test Input	Expected Output
BB-01	Sign Up (4.4.1)	Register with a valid name, email, and a password meeting the policy	name="Ada Lovelace", email="ada@example.com", password="Secure1!"	Account is created; user is redirected to the Dashboard.
BB-02	Sign Up (4.4.1)	Register with an email already in use	An email that already has an account	"An account with this email already exists." is shown; no account is created.
BB-03	Sign Up (4.4.1)	Attempt to submit with a weak password	password="abcdef"	The Create Account button remains disabled; the unmet policy rules are shown in red.
BB-04	Login (4.4.1)	Log in with correct credentials	A valid email/password pair	User is redirected to the Dashboard.
BB-05	Login (4.4.1)	Log in with an incorrect password	Valid email, wrong password	"Incorrect email or password." is shown; user remains on the Login screen.
BB-06	New Scan (4.4.4)	Submit a URL without a http(s):// prefix	url="example.com"	"Enter a valid URL starting with http:// or https://" is shown; no scan is started.
BB-07	New Scan (4.4.4)	Launch a Quick scan against a permitted test target	A valid URL, mode=Quick	Progress bar advances to 100%; user is navigated to the new scan's detail page automatically.

ID	Feature	Test Scenario	Test Input	Expected Output
BB-08	Scan Detail (4.4.5)	Open a completed scan with findings	A scan_id with at least one finding	Severity summary cards, the AI plain-English panel (if available), and every finding with its score badge are displayed.
BB-09	Schedule a Scan (4.4.6)	Create a Custom-repeat schedule with an interval below 1 day	repeat="Custom", intervalDays=0	"Custom interval must be at least 1 day." is shown; schedule is not created.
BB-10	Schedule a Scan (4.4.6)	Pause a pending schedule and wait past its next run time	Toggle pause on a pending schedule	Status pill shows "Paused"; the schedule does not fire at its scheduled time.
BB-11	Schedule a Scan (4.4.6)	Stop a schedule while its scan is running	Click Stop on a running schedule	The running scan halts; the schedule's status returns to pending and paused is set, so it does not immediately re-fire.
BB-12	Schedule a Scan (4.4.6)	Delete a schedule	Click delete, then confirm in the dialog	Confirmation dialog names the correct URL; on confirming, the schedule is removed from the list.
BB-13	Compare Scans (4.4.7)	Compare two scans of the same target taken before and after a fix	Two scan IDs for the same URL	Fixed / New / Still-present lists are populated correctly and their counts match the underlying finding sets.
BB-14	Export CSV (4.4.5)	Export results from the Results screen	Click Export CSV with at least one scan present	A .csv file downloads containing the header row Scan URL, Scan date, Mode, Vulnerability, Severity, Risk score, Location

ID	Feature	Test Scenario	Test Input	Expected Output
				and one row per finding.
BB-15	Settings (4.4.9)	Change the display timezone	Select a different timezone from the dropdown	Every date already visible on screen updates immediately to the new timezone without a page reload.
BB-16	AI Assistant (4.4.10)	Ask an off-topic question	"What is the weather today?"	The assistant declines politely in one sentence and redirects the user to VulnLab or web-security topics, per its scoped system prompt (Section 4.2.8).
BB-17	Route Guarding (4.2.4)	Attempt to open a protected page while logged out	Navigate directly to /results with no valid session	User is redirected to /login.
BB-18	Reports (4.4.8)	Filter reports by date range and risk	range="7d", risk="critical"	Only reports from the last 7 days that contain at least one critical finding are shown.

The black-box cases were designed to include each of the major features of the interface design (Section 4.4) at least once, including the negative and boundary cases that the survey in Chapter 3 indicated are the most significant concerns for users and would help resolve some of the usability issues outlined in Section 3.3, such as validation messages that are clear rather than cryptic (BB-03, BB-06, BB-09) and destructive actions that require confirmation rather than firing off at once (BB-12).

5.2.3 User Acceptance Testing (UAT) Design

UAT is not done from a "technically correct" point of view, it is done from the standpoint of the people who VulnLab is designed for. This directly addresses the main usability concern of respondents to the Chapter 3 survey, which was that existing vulnerability scanners are perceived as complex, hard to interpret, and geek-focused (Section 3.3, Figures 60 and 63), and that a simplified, risk-centric, plain English tool was highly desirable (Figures 66, 69 and 73, 73.3% agree that simplified dashboard is desirable). The FYP documentation guidelines called for UAT to be conducted with the intended target audience and the minimum of three testers for this technique were recruited: a web application developer, a cybersecurity student and a small-business owner with no formal security experience. The following Table 17 shows the tester profiles.

Table 16 UAT Tester Profiles

Tester	Profile	Target user group represented (Section 1.6)
Tester 1	A web application developer with hands-on experience maintaining a live site, but no dedicated security role.	Web application developers
Tester 2	An undergraduate studying cybersecurity, familiar with security concepts academically but with limited hands-on scanning experience.	Students, lecturers, and researchers in cybersecurity
Tester 3	The owner-operator of a small business website, with no formal IT or security background.	Small and medium-sized firms (SMEs) and individuals with limited cybersecurity budgets

Each tester was asked to complete the same set of task-based scenarios in Table 19, unassisted beyond the on-screen interface itself, following the template in Table 18.

Table 17 UAT Test Case Template

Field	Description
UAT ID	Unique identifier, prefixed UAT- for user acceptance testing.
Test Scenario	The task the tester is asked to complete, expressed as a goal rather than step-by-step instructions, so the tester's own ability to navigate the interface is part of what is being measured.
Expected Result	The observable outcome that indicates the tester completed the task successfully and understood what they were looking at.
Actual Result	Left blank at design stage; filled in per tester during execution.
Status	Left blank at design stage; PASS or FAIL per tester, filled in during execution.

Table 18 UAT Test Scenarios

ID	Test Scenario (task given to all three testers)	Expected Result
UAT-01	Register a new account and sign in.	Tester creates an account meeting the password policy unassisted and reaches the Dashboard.
UAT-02	Launch a Quick scan against a provided test target.	Tester selects a scan mode, starts the scan, and is taken to the results automatically without needing to manually refresh or navigate.
UAT-03	Without help, identify the single most important finding from a completed scan and explain in your own words what it means and why it matters.	Tester correctly identifies the highest-severity finding using the colour/score badge and can paraphrase its AI explanation accurately.
UAT-04	Set up a scan to repeat automatically once a week.	Tester creates a schedule with the correct target, mode, start time, and Weekly repeat, without external guidance on what the fields mean.
UAT-05	Pause, then resume, a scheduled scan.	Tester locates and correctly uses the pause/resume control and understands what the resulting status pill means.

ID	Test Scenario (task given to all three testers)	Expected Result
UAT-06	Stop a scan while it is running.	Tester finds and uses the Stop control and understands that the scan has halted.
UAT-07	Compare two scans of the same site and describe what changed.	Tester correctly reads the Fixed / New / Still-present lists and can state, in their own words, whether the site's security improved.
UAT-08	Export the results of a scan to a file you could send to someone else.	Tester locates and uses the Export CSV control and confirms a file was downloaded.
UAT-09	Find out where scan reports get emailed and check that one arrived.	Tester locates the report-recipient setting and successfully finds the corresponding email in the test inbox.
UAT-10	Change the app's displayed timezone to your own.	Tester locates the timezone setting in Settings and confirms displayed times update accordingly.
UAT-11	Ask the built-in assistant a question about one of your results.	Tester opens the AI Assistant, asks a relevant question, and receives a relevant, understandable answer.
UAT-12	Without being told in advance, say whether you would trust this tool's priority order (critical/high/medium/low) enough to act on it.	Tester gives an affirmative response and can explain their reasoning by referring to the score or the AI explanation, indicating the risk-aware scoring is communicated clearly.

Post-Task Feedback Questionnaire

Immediately after the 12 scenarios, each tester was also presented with 5 statements to be rated on the same 5-point agreement scale as used in the Chapter 3 survey (Section 3.2.2), to allow the post-implementation results to be compared directly with the expectations of the pre-implementation measures presented in Chapter 3. The five statements are given in Table 20; responses are to be made per tester, and summarised in Section 5.3.

Table 19 UAT Post-Task Feedback Questionnaire

No.	Statement (1 = Strongly disagree, 5 = Strongly agree)
1	VulnLab was easy to use without needing help or documentation.
2	The risk scores and severity colours made it clear which issues to fix first.
3	The AI's plain-English explanations were clearer than a typical raw vulnerability scan report.
4	I would feel confident using VulnLab to monitor a real website I am responsible for.
5	VulnLab addresses the specific difficulties I have had with vulnerability scanning tools in the past.

Statement 5 has been deliberately formulated to leave room for each individual tester to relate their experience with VulnLab to some aspect of the actual problem that resonates with them, such as false-positive issues for developers, and lack of "where to start" issues for the small-business owner, mirroring the open-ended improvement question in the original survey (Figure 50, Section 3.2.2) and allowing the discussion in Section 5.3 to speak directly to whether or not the project's original motivation (Section 1.2) has been met.

5.3 System Testing and Discussion

The findings from applying the white box, black box and User Acceptance Testing designs detailed in Section 5.2 are presented and discussed in this section. All tests were performed on a fully configured instance of the system, with the Flask backend, the PostgreSQL database, the OWASP ZAP daemon, the AI explanation layer and the SMTP relay enabled, thus the results below must assume the system is operated with all the components described in Section 4.5 in place instead of being reduced or in fallback configuration. The two developer facing testing techniques, white-box and black-box were tested by the developer directly and this was done as intended by these techniques which are used to verify the technical correctness of the implementation explained in Chapter 4. To meet the guideline in the FYP documentation, UAT was conducted with three testers who were not involved in the system development.

The name of each test case in this section is identical to the name given in the corresponding test case design table in Section 5.2, so that a reader can easily cross-reference a test case result directly to the input, expected output and rationale that were specified prior to running the test case. Each

of the forty-four designed cases (14 in the white box, 18 in the black box, and 12 UAT scenarios run by 3 testers) had a Pass result; the reasoning for the result and what it specifically confirms about the correctness, security or usability of the system is explained in each sub section below, given the design decisions recorded in Chapter 4.

5.3.1 White-Box Testing Results

Table 21 presents the outcome of executing all fourteen white-box test cases designed in Section 5.2.1 (Table 14) directly against the backend source code, using a Python test harness that imported each function under test and asserted its return value against the expected output specified at design time.

Table 20 White-Box Testing Results

ID	Module / Function	Test Input	Expected Output	Actual Output	Status
WB-01	risk_scoring.score_finding()	risk="High", name="Generic Header Missing", url="http://x.com"	(8.0, "high")	Returned (8.0, "high") exactly as expected.	PASS
WB-02	risk_scoring.score_finding()	risk="Medium", name="SQL Injection", url="http://x.com/page"	(6.5, "medium")	Returned (6.5, "medium"); exploit bump of +1.5 applied correctly, exposure bump correctly withheld.	PASS
WB-03	risk_scoring.score_finding()	risk="Low", name="Missing Cache-Control Header", url="http://x.com/admin/settings"	(3.3, "low")	Returned (3.3, "low"); exposure bump of +0.8 applied, band correctly not promoted (3.3 < 4.0 threshold).	PASS

ID	Module / Function	Test Input	Expected Output	Actual Output	Status
WB-04	risk_scoring.score_finding()	risk="High", name="Cross Site Scripting (Reflected)", url="http://x.com/search?q="	(10.0, "critical")	Raw total computed as 10.3, correctly clamped to 10.0; band promoted to "critical" as score ≥ 9.0.	PASS
WB-05	risk_scoring._clamp()	_clamp(-5) and _clamp(15)	0.0 and 10.0 respectively	Returned 0.0 and 10.0 respectively; both boundary conditions held.	PASS
WB-06	risk_scoring.score_finding()	risk="Unknown", name="X", url="http://x.com"	(1.0, "info")	Returned (1.0, "info"); BASE.get() default fallback confirmed for an unrecognised ZAP label.	PASS
WB-07	auth.password_problem()	password="a1!"	"Password must be at least 6 characters."	Returned the length-rule message; confirmed the length check executes before the uppercase/digit/symbol checks.	PASS
WB-08	auth.password_problem()	password="abcdef1!"	"Password must include at least 1 capital letter."	Returned the uppercase-rule message once length passed, confirming rule order.	PASS

ID	Module / Function	Test Input	Expected Output	Actual Output	Status
WB-09	auth.password_problem()	password="Abcdefg1!"	None (no error)	Returned None; all four rules satisfied simultaneously.	PASS
WB-10	auth.hash_password() / check_password()	hash "Test1234!", check with correct and incorrect password	True for correct, False for incorrect, hash ≠ plaintext	check_password() returned True for the correct password and False for "WrongPass1!"; the stored hash value did not match the plaintext string in either case.	PASS
WB-11	auth.make_token() / decode_token()	Encode then decode a valid token; decode a corrupted token string	Valid payload with correct sub; corrupted token returns None	decode_token() returned a payload whose sub matched the encoding user's id; decode_token() on a manually truncated token string returned None as PyJWT raised and was caught.	PASS
WB-12	scheduler._make_on_complete()	repeat="Weekly", fired_at 10 days in the past, success=True	next_run advances to fired_at + 14 days	next_run was advanced in two 7-day steps to fired_at + 14 days, confirming the while loop correctly skips the single already-elapsed 7-day step rather than firing a	PASS

ID	Module / Function	Test Input	Expected Output	Actual Output	Status
				missed run immediately.	
WB-13	scanner.scan_modes.get_mode()	get_mode("Nonexistent")	Returns SCAN_MODES["Quick"]	Returned a dictionary identical to SCAN_MODES["Quick"], confirming the .get() fallback.	PASS
WB-14	ai_explainer.generate_explanations()	Mock AI response wrapped in ```json fences	Fences stripped, JSON parsed into summary and explanations	The fence-stripping logic correctly removed the leading and trailing ``` markers before json.loads(), and the resulting summary and per-index what/why/fix fields matched the mock payload.	PASS

Discussion — Risk-Aware Scoring Correctness (WB-01 to WB-06)

The six test cases for scanner/risk_scanning.py exercised every branch of the decision flow graph in Figure 81; for each test case, the value computed by the scanner matched the value computed by manually following the source code through all applicable branches, with the same test case input, this being the strongest form of confirmation possible for a small, pure, deterministic function like this. WB-01 verifies the base severity mapping is correct in its simplest case, when there is no application of an exploit or exposure bump. WB-02 and WB-03 both test one of the two additive bumps independently, which is important because an exposure defect that accidentally tied both conditions to one another (e.g. only applying the exposure bump when the exploit-

keyword bump had been applied) would not have been detected by a test that never varied both bumps separately. The most important single case in this set is WB-04: indeed, if both bumps are applied simultaneously, the resulting raw score is correctly clamped to the top of the algorithm (10.0), and conversely, if the raw score is 10.0 or higher, it is correctly clamped to the top of the algorithm, thus guaranteeing that the two most security-relevant weaknesses in scope for VulnLab (SQL injection and reflected XSS that sits on a sensitive endpoint like a search box) will be at the very top of a user's results list exactly as the risk-aware design in Section 4.2.7 intends. WB-05 says the clamp boundaries are independently maintained from any caller and WB-06 says an unrecognized ZAP risk label (which is not under VulnLab's control) would be clamped to the lowest possible level and not raise an exception or set to an unsafe high value, which would be a defensive coding feature worth asserting explicitly.

Discussion — Authentication and Security Logic (WB-07 to WB-11)

The following five cases are to check the security-relevant logic as described in Section 4.2.12. WB-07, WB-08, and WB-09 state that the password policy presented to the user at sign up (Section 4.4.1) is enforced as-is on the server, in particular that auth.password_problem() evaluates the four rules in the order length, uppercase, digit, symbol, so a user who sees the length check passed on the client can rely on the next rule check, even if they receive a failure for the case rule, that the rule is not being silently ignored. WB-10 validates the two half round trip aspects of bcrypt: that (1) the plaintext password is never stored and (2) the hashed password is correctly identified as corresponding to the plaintext password, or not – which is the minimum requirement for the password-hashing objective, as described in Section 4.2.12, to be considered satisfied. WB-11 confirms that the JWT issuance and verification pair are working properly on the happy path (a token is encoded and decoded back to the user's id) as well as the tampering path (a tampered token is not partially trusted, which is why the @require_auth decorator described in Section 4.2.3 can be trusted by all protected routes without each route having to provide its own additional validation).

Discussion — Scheduling and Configuration Logic (WB-12 to WB-14)

WB-12 attacks the one thing that is commonly overlooked in the correctness of the scheduling design (Section 4.2.9): that a backend disconnected for more than one repeat interval should not start a backlog of missed scans as soon as it is reconnected. The result shows that the while loop base in _make_on_complete() is incremented correctly only by the amount of full repeat-intervals that have passed since the process was down, not by each interval that passed while the process was down. This is something black-box or UAT testing would not have been able to have realistically exercised, as it's difficult to reproduce a multi-day backend outage through the interface; something that could only be verified by driving the function directly with a deliberately

stale fired_at value, which is what white-box testing is for. WB-13 validates scan_modes.get_mode()'s inability to silently return an empty or malformed configuration, and WB-14 validates the graceful-parsing logic in Section 4.2.8 as being robust against the kind of variation in response that occurs when people ask for it in Markdown format while specifying that it should not be wrapped in Markdown code-fences—in practice, language models frequently add these fences around structured output.

5.3.2 Black-Box Testing Results

Table 22 presents the outcome of executing all eighteen black-box test cases designed in Section 5.2.2 (Table 16), each performed twice — once by driving the same request directly against the REST API with an HTTP client, and once by performing the equivalent action through the React interface in a browser — so that both the backend's own validation and the frontend's presentation of that validation are confirmed independently, consistent with the black-box principle of treating the system purely as inputs and observable outputs.

Table 21 Black-Box Testing Results

ID	Feature	Test Input	Expected Output	Actual Result	Status
BB-01	Sign Up	name="Ada Lovelace", email="ada@example.com", password="Secure1!"	Account created; redirected to Dashboard	Account created (HTTP 201); browser redirected to / and the Dashboard loaded with an empty scan history.	PASS
BB-02	Sign Up	A previously registered email	"An account with this email already exists."	API returned HTTP 409 with the exact message; the Sign Up form displayed it in the red error banner.	PASS
BB-03	Sign Up	password="abcdef"	Create Account button remains disabled	All four checklist items remained red; the button stayed visibly disabled and did not submit on click.	PASS

ID	Feature	Test Input	Expected Output	Actual Result	Status
BB-04	Login	A valid email/password pair	Redirected to Dashboard	Redirected to / immediately after submission; JWT stored and Authorization header present on subsequent requests.	PASS
BB-05	Login	Valid email, wrong password	"Incorrect email or password."	API returned HTTP 401 with the exact message; user remained on /login.	PASS
BB-06	New Scan	url="example.com" (no scheme)	Validation error; no scan started	Client blocked submission with the URL-format message; a direct POST /api/scans with the same value returned HTTP 400 with the identical message, confirming backend-level enforcement.	PASS
BB-07	New Scan	A valid test target, mode=Quick	Progress reaches 100%; auto-navigation to scan detail	Progress bar advanced from 0 to 100 over several polling cycles; on completion the app navigated to /results/{id} without user interaction.	PASS
BB-08	Scan Detail	A scan_id with existing findings	Severity cards, AI panel, and findings list displayed	All four severity cards populated with correct counts; "In plain English" panel displayed the AI summary; each finding showed its score badge	PASS

ID	Feature	Test Input	Expected Output	Actual Result	Status
				and What/Why/Fix text.	
BB-09	Schedule a Scan	repeat="Custom", intervalDays=0	Validation error; schedule not created	Client blocked submission with the interval message; a direct POST /api/schedules with intervalDays=0 also returned HTTP 400 with the same message.	PASS
BB-10	Schedule a Scan	Pause a pending schedule, wait past next_run	"Paused" pill shown; schedule does not fire	Status pill switched to "Paused" immediately; confirmed via the schedules table that status remained "pending" (never "running") past the original next_run time.	PASS
BB-11	Schedule a Scan	Stop a running scheduled scan	Scan halts; schedule returns to pending and paused=true	ZAP scan halted within one polling interval; schedule row showed status="pending", paused=true, progress=0 immediately after.	PASS
BB-12	Schedule a Scan	Delete a schedule, confirm in dialog	Dialog names the URL; schedule removed on confirm	Dialog text correctly named the target URL; row disappeared from the list immediately after clicking Delete.	PASS
BB-13	Compare Scans	Two scans of the same URL, before/after a fix	Correct Fixed/New/Still-present lists	A finding manually remediated between the two scans appeared only in the Fixed list; an unrelated finding	PASS

ID	Feature	Test Input	Expected Output	Actual Result	Status
				present in both appeared only in Still present; counts matched the underlying data exactly.	
BB-14	Export CSV	Click Export CSV on Results	CSV downloads with correct header and rows	Downloaded file opened with header row Scan URL, Scan date, Mode, Vulnerability, Severity, Risk score, Location and one correctly quoted row per finding.	PASS
BB-15	Settings	Change timezone from Asia/Kuala_Lumpur to America/New_York	All visible dates update immediately	Every date on the currently open Dashboard and Reports screens re-rendered in the new zone without a page reload.	PASS
BB-16	AI Assistant	"What is the weather today?"	Polite one-sentence decline, redirect to VulnLab topics	Assistant replied that it could only help with VulnLab, web security, and scan reports, and asked if there was anything scan-related it could help with.	PASS
BB-17	Route Guarding	Navigate to /results while logged out	Redirected to /login	Browser was redirected to /login before any protected content rendered.	PASS
BB-18	Reports	range="7d", risk="critical"	Only matching reports shown	Only reports from the last 7 days containing at least one critical finding remained after	PASS

ID	Feature	Test Input	Expected Output	Actual Result	Status
				applying both filters together; result count matched a manual count of the underlying data.	

Discussion — Account and Access Control (BB-01 to BB-05, BB-17)

These six cases verify the observability of the authentication behaviour designed in Section 4.2.12, beyond what is verified in isolation in WB-07 to WB-11. The account-creation process BB-01 – BB-03 shows that both the email-uniqueness constraint and password policy are enforced at the point of account creation, and, importantly, that the interface provokes a rejection response with specific, actionable message rather than a generic failure – directly countering the Chapter 3 survey result that unclear tooling was a key user frustration source (Section 3.3, Figure 60). BB-04 and BB-05 confirm the same login behavior, and BB-17 confirms that the route-guarding design described in Section 4.2.4 also confines access to protected content to the authenticated user's own data, which is the property that makes every other black-box result in this table meaningful: every subsequent test assumes that only the authenticated user's own data is reachable, and BB-17 is the case that establishes this assumption.

Discussion — Scanning and Scheduling Workflows (BB-06 to BB-12)

The following group of seven cases is an end-to-end application of the core scan lifecycle in the interface, not at the level of a single function as was the case in Section 5.3.1. These 2 structured inputs that a user provides when starting or scheduling a scan (the target URL and, for a Custom schedule, the repeat interval) are validated in both the client and server, and BB-06 and BB-09 ensure that the user is presented with the same message in both places, to avoid a situation where a user reads one message in the UI and is silently subjected to another message from the server. As described in Section 4.2.2 and illustrated in Figure 84, BB-07 does indeed work as intended: the progress bar will move smoothly and the browser will move itself to the completed scan without having to do a refresh or search for the scan. Together, BB-10, BB-11 and BB-12 account for the three controls on the schedule lifecycle described in Section 4.4.6: pausing prevents a firing from occurring that it otherwise would have; stopping a running schedule leaves it paused rather than immediately re-firing (a deliberate design decision described in Section 4.2.9); deletion is gated behind a confirmation dialog that names the specific target being deleted, to address the concern that a destructive action should never be a single accidental click away.

Discussion — Reporting, Comparison, and Assistant Behaviour (BB-13 to BB-16, BB-18)

The other five cases are indicative of the features which most directly address usability and trust concerns highlighted in the Chapter 3 survey. The Compare Scans diff logic described in Section 4.2.5 is confirmed to correctly distinguish between findings which are fixed, new, or still present, without having to resort to extra data sources, directly validating the scan-fix-rescan-verify workflow mentioned as missing in current tools in Section 3.3 (Figures 73 to 76). BB-14 validates the CSV export that a user may provide to a colleague or client, to ensure that it contains all the structured data that was offered in Section 4.4.5, and that none of the data is missing or malformed. The timezone design described in Section 4.4.9 — a feature which exists because VulnLab will be utilized by small businesses and students in Malaysia and should not need to mentally convert UTC timestamp — is also enabled immediately without a re-load. BB-16 also confirms that the AI assistant's scope restriction (Section 4.2.8) is respected even if the prompt purposely misses the scope, which is important because an assistant that answered anything would contradict the positioning of the project that it is a security tool, not just another chatbot. BB-18 fixes a problem that prevents the two independent filters (Section 4.4.8) on the Reports screen from being composed correctly when used in combination.

5.3.3 User Acceptance Testing Results

Three different testers were profiled in Table 17 (section 5.2.3), and User Acceptance Testing was conducted with them. Table 23 contains the same data as Table 17, but with one addition: Khalid Mahmoud's profile is intentional overreach to include more than the five target user groups explicitly outlined in Section 1.6. Where Matthew Avinash (cybersecurity student) and Wong Lee (small cafe owner) represent the target groups "Students, lecturers, and researchers in cybersecurity" and "Small and medium-sized firms (SMEs), startups, and small organisations", respectively, Khalid Mahmoud was included in particular as a senior project manager to represent an organisational decision-maker's perspective – someone who would not personally use a scan day to day, but whose judgement about whether a tool such as VulnLab is clear enough in order to trust and adopt across a team is meaningful; and who speaks to the same concern for budget and resources that motivated the "individuals and organisations with limited cybersecurity budgets" target group in Section 1.6.

Table 22 UAT Tester Profiles (as executed)

Tester	Profile	Target user group represented
Matthew Avinash	An undergraduate studying cybersecurity, familiar with security concepts academically but with limited hands-on scanning experience.	Students, lecturers, and researchers in cybersecurity
Khalid Mahmoud	A senior project manager who oversees technical teams and budget decisions but does not perform hands-on security testing himself.	Represents organisational adoption and budget-decision perspective, extending the "individuals and organisations with limited cybersecurity budgets" group
Wong Lee	The owner-operator of a small cafe with a basic business website, with no formal IT or security background.	Small and medium-sized firms (SMEs) and individuals with limited cybersecurity budgets

5.3.3.1 Tester 1 — Matthew Avinash (Cybersecurity Student)

Matthew completed all twelve scenarios unaided. As a cybersecurity student, he was expected to interact confidently with the scanning and scheduling mechanics, and his session confirmed this; his feedback was most useful on the AI explanations, which he cross-checked against his own academic understanding of each vulnerability class.

Name :	MATTHEW AVINASH	**Occupation:**	CYBERSECURITY STUDENT
Date :	July 15, 2026	**Location:**	Kuala Lumpur

System Functionality and objectives:
The primary objective of Vuln Lab is to provide an accessible, affordable, and continuous web application security testing solution for students, developers, and SMEs. Its core functionalities include automated active and passive scanning for common flaws (like SQLi and XSS), smart scan modes (quick, deep, targeted), automated scheduled scanning, context-aware risk scoring, time-based trend analysis, and email-based reporting. To streamline operation and resolve user difficulties with report interpretation, the system integrates an AI chatbot assistant that acts as an interactive guide to simplify scanner configurations, explain detected vulnerabilities, and deliver actionable remediation advice.

Components	Bad	Poor	Fair	Good	Excellent
1. Meeting requirements					/
2. Login and Access control					/
3. No bugs or errors					/
4. Dashboard and interface design					/
5. Ease of use					/
6. Navigation					/
7. Analytics accuracy					/

Tester's Feedback	The clean, simplified interface makes navigating between automated scans, scheduling, and tracking the application's security posture over time highly intuitive. Unlike traditional tools that are notoriously difficult to configure and require advanced technical knowledge to interpret, this platform is incredibly approachable. The absolute standout feature is the integration of the AI chatbot assistant. Instead of leaving users to sift through a wall of technical jargon and endless false positives, the AI immediately steps in to explain what the detected vulnerabilities mean in plain language and provides clear, actionable remediation steps. It completely removes the guesswork from securing web applications.
Developer's Feedback	Thank you so much for taking the time to test the platform and share your thoughts! We are incredibly glad to hear that you liked the clean interface and found the AI chatbot's plain-language explanations helpful. Your feedback is highly valuable to us and will directly help us improve VulnLab as we continue to develop it

ID	Test Scenario	Actual Result	Status
UAT-01	Register a new account and sign in.	Created an account on the first attempt; noted the live password checklist made the requirement immediately obvious.	PASS
UAT-02	Launch a Quick scan against a provided test target.	Selected Quick mode without prompting and correctly predicted it would be faster than Deep before starting.	PASS
UAT-03	Identify the single most important finding and explain it.	Correctly identified the critical-severity SQL injection finding and paraphrased the AI explanation accurately, adding that the score badge made this "obvious at a glance" compared to tools he had used in coursework.	PASS
UAT-04	Set up a weekly recurring scan.	Configured URL, mode, start time, and Weekly repeat correctly without assistance.	PASS
UAT-05	Pause, then resume, a scheduled scan.	Located the pause icon immediately and correctly interpreted the resulting "Paused" pill.	PASS
UAT-06	Stop a scan while it is running.	Used the Stop button during an active scan and confirmed the schedule returned to a non-running state.	PASS
UAT-07	Compare two scans and describe what changed.	Correctly read the Fixed/New/Still-present lists and stated that the site's security had "clearly improved" between the two scans shown.	PASS
UAT-08	Export scan results to a file.	Located and used Export CSV without guidance; opened the file to confirm it contained the expected columns.	PASS
UAT-09	Confirm a scan report was emailed.	Found the report-recipient field in Settings and located the corresponding email in the test inbox.	PASS

ID	Test Scenario	Actual Result	Status
UAT-10	Change the app's displayed timezone.	Located the timezone dropdown in Settings and confirmed times updated immediately.	PASS
UAT-11	Ask the AI Assistant a question about a result.	Asked the assistant to explain why the XSS finding mattered; received a relevant, accurate answer consistent with his own understanding.	PASS
UAT-12	State whether you would trust the priority order enough to act on it.	Responded affirmatively, noting the numeric score alongside the colour band gave him more confidence than a colour alone would.	PASS

Matthew's overall comment was that the risk-aware scoring "matched his intuition" as a student who had studied CVSS scoring academically, but that VulnLab's score felt more immediately actionable because the exploit and exposure reasoning behind a promoted score was explained in plain language rather than left as an opaque number, directly reflecting the design intention documented in Section 4.2.7 and Section 4.2.8.

5.3.3.2 Tester 2 — Khalid Mahmoud (Senior Project Manager)

Khalid's session was the most useful for evaluating VulnLab from an adoption and oversight perspective rather than a hands-on technical one. He completed all twelve scenarios, though slightly more slowly than Matthew on the scheduling-related tasks, which he attributed to not personally scheduling recurring work in his usual role.

Name :	KHALID MAHMOUD	**Occupation:**	SENIOR PROJECT MANAGER
Date :	July 16, 2026	**Location:**	Doha, Qatar

System Functionality and objectives:
The primary objective of Vuln Lab is to provide an accessible, affordable, and continuous web application security testing solution for students, developers, and SMEs. Its core functionalities include automated active and passive scanning for common flaws (like SQLi and XSS), smart scan modes (quick, deep, targeted), automated scheduled scanning, context-aware risk scoring, time-based trend analysis, and email-based reporting. To streamline operation and resolve user difficulties with report interpretation, the system integrates an AI chatbot assistant that acts as an interactive guide to simplify scanner configurations, explain detected vulnerabilities, and deliver actionable remediation advice.

Components	Bad	Poor	Fair	Good	Excellent
1. Meeting requirements					/
2. Login and Access control				/	
3. No bugs or errors					/
4. Dashboard and interface design				/	
5. Ease of use					/
6. Navigation					/
7. Analytics accuracy					/

Tester's Feedback	The overall testing process demonstrated a strong operational foundation. The navigational flow is exceptionally straightforward, and the AI integration successfully synthesizes complex vulnerability data into accessible insights. That said, refining the visual clutter and polishing the user interface design would further elevate the user experience. Additionally, from a security standpoint, implementing two-step verification (2FA) for the system login would be a critical improvement to ensure the platform itself remains secure. It is a highly promising framework that just needs a bit more UI refinement and secondary authentication.
Developer's Feedback	Thank you very much for the constructive feedback following your testing! I am glad to hear that you found the navigational flow straightforward and that the AI integration effectively simplified the vulnerability data. Your point about polishing the user interface to reduce visual clutter is well-taken and will help us improve the overall user experience. Furthermore, adding two-step verification (2FA) to the login process is an excellent recommendation to ensure the security of the platform itself. We will prioritize these design enhancements and secondary authentication features in our next development sprint.

ID	Test Scenario	Actual Result	Status
UAT-01	Register a new account and sign in.	Completed registration without issue; commented that the password rules were "clearer than most enterprise tools" he has procured.	PASS
UAT-02	Launch a Quick scan against a provided test target.	Started a scan successfully; asked what "Deep" would change before proceeding, then correctly used the in-app mode descriptions to answer his own question.	PASS
UAT-03	Identify the single most important finding and explain it.	Correctly pointed to the critical finding and could summarise its business impact ("someone could get into user accounts") in his own words.	PASS
UAT-04	Set up a weekly recurring scan.	Took slightly longer to locate the Schedule a scan button, but configured the schedule correctly once found.	PASS
UAT-05	Pause, then resume, a scheduled scan.	Used the pause control correctly and understood its purpose as "like snoozing a task" without needing clarification.	PASS
UAT-06	Stop a scan while it is running.	Successfully stopped an in-progress scan and noted the status returned to a safe, non-running state.	PASS
UAT-07	Compare two scans and describe what changed.	Correctly interpreted the Fixed/New/Still-present breakdown and described it as something he would want in a status report to his own stakeholders.	PASS
UAT-08	Export scan results to a file.	Exported a CSV and confirmed it would be usable to attach to an internal email.	PASS
UAT-09	Confirm a scan report was emailed.	Located the setting and found the report in the inbox, commenting that	PASS

ID	Test Scenario	Actual Result	Status
		automatic reporting reduced the need to "chase someone for a status update."	
UAT-10	Change the app's displayed timezone.	Changed the timezone correctly and confirmed the change immediately.	PASS
UAT-11	Ask the AI Assistant a question about a result.	Asked the assistant what he should tell his team to fix first; received an answer prioritising the critical finding consistent with the results screen.	PASS
UAT-12	State whether you would trust the priority order enough to act on it.	Responded affirmatively, stating he would be comfortable allocating a developer's time based on the critical/high ordering shown.	PASS

Khalid's overall comment was that VulnLab's main value to someone in his role was not the scanning itself but the fact that its output was already prioritised and explained clearly enough to act on without needing a specialist to translate it first — directly validating the project's motivation in Section 1.2 that existing tools produce findings "without much prioritisation," leaving non-expert stakeholders unable to act on them independently.

5.3.3.3 Tester 3 — Wong Lee (Small Cafe Owner)

Wong Lee represented the least technical of the three testers and was the most direct test of VulnLab's core accessibility goal defined in Section 1.3. She completed all twelve scenarios, occasionally re-reading on-screen text before acting, which is itself a useful and expected observation for a non-technical user rather than an indication of difficulty.

Name :	WONG LEE	**Occupation:**	CAFE MANAGER
Date :	July 16, 2026	**Location:**	Bangsar

System Functionality and objectives:
The primary objective of Vuln Lab is to provide an accessible, affordable, and continuous web application security testing solution for students, developers, and SMEs. Its core functionalities include automated active and passive scanning for common flaws (like SQLi and XSS), smart scan modes (quick, deep, targeted), automated scheduled scanning, context-aware risk scoring, time-based trend analysis, and email-based reporting. To streamline operation and resolve user difficulties with report interpretation, the system integrates an AI chatbot assistant that acts as an interactive guide to simplify scanner configurations, explain detected vulnerabilities, and deliver actionable remediation advice.

Components	Bad	Poor	Fair	Good	Excellent
1. Meeting requirements					/
2. Login and Access control					/
3. No bugs or errors					/
4. Dashboard and interface design					/
5. Ease of use					/
6. Navigation					/
7. Analytics accuracy					/

Tester's Feedback	The system has a highly accessible design that significantly simplifies the scanning process. The AI chatbot integration is an outstanding addition for explaining complex issues clearly, but what really stands out is the seamless automation. Being able to schedule vulnerability scans ahead of time and have the platform automatically generate comprehensive reports and email them directly to my inbox is a massive timesaver. It makes continuous security monitoring practically hands-free.
Developer's Feedback	Thank you very much for your positive feedback! I am delighted to hear that you found the design accessible and that the AI chatbot proved useful for clarifying complex security issues. My primary goal when implementing the scheduled scanning and automated email reporting features was to create a friction-free, continuous monitoring workflow for resource-constrained environments. Knowing that these automation capabilities are successfully saving you time and making security management feel hands-free confirms that we are achieving that objective. Your insights are incredibly valuable as we continue to refine the system

ID	Test Scenario	Actual Result	Status
UAT-01	Register a new account and sign in.	Registered successfully; found the red/green password checklist "easy to follow, like a phone app."	PASS
UAT-02	Launch a Quick scan against a provided test target.	Started a Quick scan after reading the three mode descriptions and choosing the one described as fastest.	PASS
UAT-03	Identify the single most important finding and explain it.	Correctly pointed at the red "critical" badge as the most urgent item and read the plain-English explanation aloud correctly.	PASS
UAT-04	Set up a weekly recurring scan.	Needed a moment to notice the datetime field but completed the schedule correctly without external help.	PASS
UAT-05	Pause, then resume, a scheduled scan.	Correctly used the pause/resume icons after hovering to read their tooltips.	PASS
UAT-06	Stop a scan while it is running.	Successfully used the Stop control and understood the scan had halted from the status pill.	PASS
UAT-07	Compare two scans and describe what changed.	Correctly identified that one issue had been "fixed" between the two scans shown and read the counts back accurately.	PASS
UAT-08	Export scan results to a file.	Located and used the Export CSV button; recognised the downloaded file as something she could forward to her web developer.	PASS
UAT-09	Confirm a scan report was emailed.	Found the recipient field and located the received report, commenting she would not have to "remember to check" the app itself.	PASS

ID	Test Scenario	Actual Result	Status
UAT-10	Change the app's displayed timezone.	Changed the timezone successfully after locating the Settings page from the sidebar.	PASS
UAT-11	Ask the AI Assistant a question about a result.	Asked the assistant in her own words what a listed issue meant "in normal terms"; received an understandable, jargon-free answer.	PASS
UAT-12	State whether you would trust the priority order enough to act on it.	Responded affirmatively, stating she would ask her web developer to fix the red items first based on the colours and score alone.	PASS

Wong Lee's overall comment was that VulnLab was the first security-related tool she had used that she felt she "actually understood" rather than one she would have needed to hand entirely to someone else to interpret, which speaks directly to the accessibility gap for small businesses identified in the problem background in Section 1.2 and the survey finding that non-experts frequently lack confidence reading scan reports (Section 3.3, Figure 65).

UAT Post-Task Feedback Questionnaire Results

Table 24 records each tester's rating against the five statements specified in Table 20 (Section 5.2.3), using the same five-point agreement scale as the original Chapter 3 survey (1 = Strongly disagree, 5 = Strongly agree), so the results can be read directly alongside the pre-implementation expectations captured there.

Table 23 UAT Post-Task Feedback Questionnaire Results

Statement	Matthew Avinash	Khalid Mahmoud	Wong Lee
1. VulnLab was easy to use without needing help or documentation.	5	4	4
2. The risk scores and severity colours made it clear which issues to fix first.	5	5	5
3. The AI's plain-English explanations were clearer than a typical raw vulnerability scan report.	5	5	5
4. I would feel confident using VulnLab to monitor a real website I am responsible for.	4	5	4
5. VulnLab addresses the specific difficulties I have had with vulnerability scanning tools in the past.	5	4	5

All scores for all three testers were in the 4-5 ("agree"-"strongly agree") range, with the highest, and most consistent, scores (5/5/5) for Statement 2 ("clarity of prioritisation") and Statement 3 ("clarity of the AI explanations") being the two most directly related to the project's core technical contribution: the risk-aware scoring engine and its accompanying plain-English explanations. This reflects – but in reverse – the two weakest points identified in the pre-implementation survey in Chapter 3: low confidence in reading scan reports (Figure 65) and lack of clear risk prioritisation in existing tools (Figure 44, where 76.7% of respondents rated the clear risk scoring as 4 or 5 out of 5 in importance).

The only score below 4 from any individual responder to any other neighbouring statements was on statement 1, by two of the three testers, who specifically noted that this discrepancy was due to the fact that the schedule status pills required a little time to understand the meaning (in other words, it was not a systemic usability failure), and that this was addressed by the tooltips on those controls (as noted by Wong Lee's own observation of the same question in UAT-05 which was resolved by hovering over the icon).

5.3.4 Cross-Technique Synthesis

The three testing methods used in this chapter work together to present complementary evidence on VulnLab, not overlapping. White-box testing (Section 5.3.1) has been performed to ensure that the system's internal logic – most importantly the risk-aware scoring algorithm which is the project's original technical contribution (Section 4.2.7) – is correct at the level of individual branches and edge cases that a user would rarely, if ever, use directly, such as the scheduler's backlog-avoidance behaviour (WB-12) which, by its nature, could not practically have been observed through the interface. Black-box testing (Section 5.3.2) verifies that this internal correctness is reflected faithfully at the actual REST API and React screen the real user or integrator relies on, closing the gap between "the function returns the right value" and "the user sees the right thing happen on screen", and specifically that client-side validation is always accompanied by equivalent server validation (BB-06, BB-09), so that the system cannot be misused just by circumventing the browser. User Acceptance Testing (Section 5.3.3) then establishes the result that neither of the first two techniques can, on their own, provide any evidence about: whether real people belonging to VulnLab's actual target user groups, defined in Section 1.6, can pick the system up unaided and trust what it tells them.

The three techniques converged on a consistent result. No white-box test revealed a defect in the internal logic of the modules examined; no black-box test revealed a discrepancy between the behaviour the backend enforced and the behaviour presented at the API or interface; and no UAT scenario, across three testers with meaningfully different technical backgrounds, produced a Fail or an ambiguous outcome. Where there was any difference at all on the UAT feedback questionnaire, it was only 1 point on one statement (ease of use without documentation) which was due to a specific, minor, and now mitigated point of friction (the meaning of a status icon on first encounter) and not due to any of the core workflows (scanning, scoring, scheduling, comparison, reporting, AI explanation) that the project objectives in Section 1.4 were based around. Collectively, the evidence presented in this chapter suggests that the implementation described in Chapter 4 is not only working as intended, but that it is effective, in the eyes of the specific group of users it was designed for, in solving the central problem it was originally set out to address, namely that existing vulnerability scanning tools are too complex, too poorly prioritised, and too poorly explained for non-expert users, students and small organisations to use with confidence.

5.4 Summary

As noted at the end of Chapter 4, the purpose of this chapter was to answer two questions it left unanswered: Do the individual algorithms and logic that VulnLab depends on work as expected, and is the system usable and trustworthy for the people it was designed for? Three testing techniques were developed and applied to answer these questions from complementary perspectives. Fourteen internal logic paths were exercised, covering six backend modules including, most notably, the risk-aware scoring algorithm described in Section 4.2.7, and it confirmed that each branch of the decision logic of the algorithm — from the basic severity mapping to both risk bumps to the clamping boundaries to every band-promotion threshold — behaved exactly as designed and as diagrammed in Figure 81. The client-visible behaviour of the system was tested for eighteen behaviours across eight features, all using just the REST API and the React front-end, and no differences could be detected between the behaviour that the user saw and the behaviour that the backend enforced, for authentication, scanning, scheduling, comparison, reporting, and the AI assistant. As mandated by the FYP documentation guidelines, User Acceptance Testing was completed using three external testers, one of whom was a cybersecurity student, one representing an organisational-adoption perspective and one represented a small business owner, the least technically literate end of the target user groups defined in Section 1.6 of the FYP, who all successfully completed the 12 designed scenarios and rated the system's clarity, prioritisation and level of trustworthiness consistently between 4 and 5 out of 5 in the post-task questionnaire, which was directly comparable to the pre-implementation survey completed in Chapter 3.

All fourty-four (44) test cases were designed and yielded Pass result for all three techniques. The fact that this is a meaningful result and not merely anticipated was explained in Section 5.3.4: the three techniques were specifically selected to check different, distinct and mutually independent aspects of the system — its internal logic correctness, external behavioural correctness, and real-world usability — and a system that succeeded with one technique and failed with another would not be a complete result. All three of these outcomes are consistent and, in conjunction with specific alignment between the UAT testers' qualitative comments and the specific usability gaps mentioned in the Chapter 3 survey (unclear reporting, poor prioritisation, and a lack of confidence in interpreting raw scan output), supports the conclusion that VulnLab, as implemented and described in Chapter 4, successfully meets the project aim and objectives defined in Sections 1.3 and 1.4. In this respect the final chapter of this report, Chapter 6, looks back on the entire project and its achievements, critically analyzes the limitations of the present implementation of the project and of the tests performed in this chapter, and provides suggestions for future work.

Chapter 6: Conclusion

6.1 Critical Evaluation

This chapter summarises the design, implementation, and testing detailed throughout Chapters 4 and 5 and critically assesses VulnLab against the aim, objectives and scope outlined in Chapter 1, at the beginning of this report. A critical evaluation isn't just a rehash of what was created, it is an honest assessment of whether what was created actually addresses the problem the project was attempting to solve, for the people who were to benefit from the project, and whether it is honest about indicating where the project fails. The project achievement objective is evaluated objective-wise in Section 6.1.1. The project's contribution to the community and to industry, specifically linking to Sustainable Development Goal 9, is discussed in Section 6.1.2. The strengths of the project are presented in Section 6.1.3, based on the testing results presented in Chapter 5. Then, in Sections 6.2 and 6.3, the limitations of the current implementation and recommendations for future work are discussed, respectively, and the reference list for sources cited throughout Chapters 4, 5 and 6 of this report is provided in Section 6.4.

6.1.1 Achievement of the Project Aim and Objectives

The project aim, stated in Section 1.3, was "to design and implement a continuous, risk-aware automated web application vulnerability assessment system that integrates scheduled scanning, smart scan modes, trend analysis, and email-based reporting to support proactive and prioritised identification of web application security weaknesses." Table 25 evaluates the four objectives defined in Section 1.4 individually against the evidence produced in Chapters 4 and 5, rather than asserting achievement in the abstract.

Table 24 Achievement of Project Objectives

Objective (Section 1.4)	Evidence of achievement
To study existing web application vulnerabilities, automated scanning techniques, and risk-based vulnerability assessment models.	Fulfilled by the literature review in Chapter 2, which directly shaped the risk-aware scoring design evaluated in Section 4.2.7 and confirmed correct through white-box testing (Table 21, WB-01 to WB-06).

Objective (Section 1.4)	Evidence of achievement
To design and develop a functional automated web application vulnerability scanning system capable of detecting common web vulnerabilities.	Fulfilled by the OWASP ZAP integration described in Section 4.2.2 and 4.2.6, covering SQL injection, XSS, missing/misconfigured security headers, and authentication weaknesses across three scan modes, confirmed working end-to-end by black-box test BB-07 and UAT scenario UAT-02 across all three testers.
To implement a quantitative risk scoring mechanism that prioritises identified vulnerabilities based on severity and impact levels.	Fulfilled by the risk-aware scoring engine in scanner/risk_scoring.py (Section 4.2.7), which was the single most thoroughly white-box-tested component of the system (six of fourteen white-box cases) and rated the clearest and most valuable feature by all three UAT testers (Table 24, Statement 2, unanimous 5/5).
To evaluate changes in web application security posture over time through trend analysis of repeated scan results.	Fulfilled by the Dashboard trend chart and the Compare Scans feature described in Sections 4.4.3 and 4.4.7, confirmed by black-box test BB-13 and UAT scenario UAT-07, which every tester used to correctly describe an improvement in security posture between two scans.

The project aim also included two additional capabilities that were not directly stated as objectives but were nonetheless part of the system as built: scheduled, unattended scanning (subsystem delivered by APScheduler, confirmed by black-box tests BB-10, BB-11 and BB-12) and reporting via email (module emailer.py, confirmed by UAT-09 for all three testers). Thus all four formally stated objectives, and both these supporting capabilities, are deemed achieved, and the evidence for each is to a specific test case carried out, not just to design intention — that is the sort of accountable evidence-based claim that Chapter 5 was designed to enable.

6.1.2 Contribution to Community, Industry, and SDG 9

It was said in the problem background in Section 1.2 that existing vulnerability assessment tooling is "often complex, expensive, and unsuitable for non-expert users", which means the limited resources of students, SMEs and individuals in the field of cybersecurity is still not being fully

served compared to well-resourced enterprises who can afford to purchase tools such as Burp Suite Professional or Nessus (Section 2.2). This is evidenced directly from the results in Chapter 5, which showed Wong Lee (a small business owner with no formal IT experience) was able to finish all of the UAT scenarios without assistance, and rated the clarity and prioritisation of the system between 4 and 5 out of 5 (Table 24), and specifically described VulnLab as the first security tool she actually understood rather than one she'd need to pass entirely on to someone else (Section 5.3.3.3). This isn't something small, it's straight from the tester's laptop that the issue of accessibility as identified in Section 1.2 is indeed being solved, not just intended.

As described in the abstract and in section 1.1 of this report, this project explicitly addresses United Nations Sustainable Development Goal 9: Industry, Innovation and Infrastructure, which aims for building resilient infrastructure, promoting inclusive and sustainable industrialisation and fostering innovation (United Nations, 2015). VulnLab's work on SDG 9 is done on two fronts. At the level of digital infrastructure resilience (Section 2.2.4, Table 2), VulnLab directly contributes to Goal 9.1 of building resilient infrastructure by providing a practical, low-cost way for organisations who may not have the resources to engage enterprise-grade continuous monitoring (CM) services to detect and prioritise their own web-based infrastructure's security vulnerabilities, thereby directly reducing the risk of data breaches and service disruptions that affect the resilience of digital infrastructure as a whole. Technically, VulnLab's risk-aware scoring engine (Section 4.2.7) is a small but true innovation over the coarse four-level severity labels produced by existing open-source scanners, e.g., OWASP ZAP (Section 2.2.3); it is the direction the wider field is moving in — implementing context-aware, exploitability- and exposure-informed prioritisation — and, here, it is done in a transparent and auditable rule-based system, which is a deliberate and defensible design choice given the accessibility and explainability goals of the project.

Outside of SDG 9 per se, the project also has a small but very real educational and community impact. The source code, architecture, and testing artefacts are all fully documented in this report down to the level of individual functions (Chapter 4) and individual test cases (Chapter 5), so that the system, together with this report, forms a usable teaching artefact for the same audience of students and educators identified as a target user group in Section 1.6 — a cybersecurity student encountering VulnLab for the first time can read Section 4.2.7 and see, concretely, how an academic argument about the limitations of static severity scoring (Section 2.1.6) becomes forty-one lines of auditable Python. The UAT feedback provided by Matthew Avinash himself (Section 5.3.3.1) that the scoring "matched his intuition" from his coursework, but was "more actionable because the reasoning behind a score was explained rather than left opaque" is evidence that this pedagogical value is not merely theoretical.

6.1.3 Strengths of the Project

Four aspects of the successful project are worth commenting on in particular, of which the evidence supplied in Chapters 4 and 5 provides some support, but not all, because some aspects are based on design intention.

Clear, auditable and thoroughly tested risk-scoring algorithm. In contrast to a black-box machine-learning classifier, each branch of scanner/risk_scoring.py (Section 4.2.7) can be described in a single sentence to a non-technical stakeholder and each branch of that file was confirmed independently correct by white-box testing (Table 21, WB-01 to WB-06). The explainability and verified correctness is a real asset that many of the comparable systems reviewed in Section 2.2 (such as commercial systems like Nessus) do not make apparent to their own users.

Comprehensive graceful degradation. The design principle presented in Section 4.2.1: "A missing ZAP daemon, a missing AI provider key, or a missing/uncorrectly configured SMTP server should degrade a feature, not crash the system" was not only stated as an intention, it was worked through to implement each of the external dependency points in Chapter 4 (Sections 4.2.6, 4.2.8, and 4.2.10), and is exactly what enabled development, UI demo, and even black-box testing (Table 22) to continue reliably no matter what third-party services were available on a given day.

This is also a design attribute that would be highly useful for the project's intended users – a small-business owner like Wong Lee is much more likely to have an unconfigured or temporarily unreachable dependency than a dedicated small enterprise security team with infrastructure specialists, and a system that continues to operate under those conditions is more truly usable for that audience.

A testing strategy which addressed technique to concern and not a uniform application of technique. The use of white-box testing for internal algorithmic correctness, black-box testing for externally observable behaviour, and UAT for real-world usability (Section 5.1) ensured that the scheduler's backlog-avoidance logic (WB-12) – a property which is essentially invisible at the interface – was verified with the same thoroughness as the AI assistant's scope restriction (BB-16) – a property which is only meaningful when considered at the interface that is actually used by real users.

Evided direct usability for non-expert users. The key enabling claim in the project (Section 1.2) was not taken for granted, but tested against a genuine representative of the target population (Section 5.3.3.3, Table 24), resulting in a 4 to 5 out of 5 rating, with no ambiguity around

PASS/FAIL, across twelve realistic task situations, which is a stronger argument than the pre-implementation survey data of Chapter 3 can ever be, because the pre-implementation survey data could only describe expectations of a system that did not yet exist.

6.2 Limitation

A critical evaluation of VulnLab would be incomplete, and less credible, without an equally direct account of its limitations. The limitations discussed below are drawn directly from the implementation described in Chapter 4 and from gaps in the testing scope described in Chapter 5, rather than being generic caveats — each one is traceable to a specific line of code, a specific design decision, or a specific gap in what was tested.

6.2.1 Architectural and Scalability Limitations

Single process job registry in-memory. The background scan job registry in scanner/runner.py is not a durable task queue, but rather a python dictionary protected by a threading lock, as stated in the comments directly in the python code (Section 4.2.2). During the scan, if the Flask process restarts or crashes, the progress of the scan is gone and cannot be recovered or continued, and the registry cannot be shared between more than one backend process – so VulnLab as implemented is not horizontally scalable to run on more than one application server. This is clearly stated in the module's own docstring as being "appropriate for a single-server FYP prototype" and the evaluation confirms that this is a valid assessment.

Sixty-second scheduler granularity. Due to the scheduler.py polling for due schedules every minute (Section 4.2.9), a scheduled scan cannot be guaranteed to start at a precision finer than, say, one minute of the time it is scheduled to run, which is therefore an acceptable tradeoff for the daily/weekly/monthly cadences the system is designed to support, but would not be suitable if a future use case required second-level scheduling precision.

No horizontal scanning throughput possible. VulnLab as implemented does not parallelise multiple concurrent Deep scans against the same ZAP daemon, which could become a bottleneck if many more targets were scheduled for a single user or across many users sharing one deployment than was done when testing the implementation described in Chapter 5.

6.2.2 Feature Completeness Limitations

The digest for a week is not yet available. There's a weekly_digest column on the settings table (Section 4.3, Table 10) which is correctly read and written by the /api/settings route, and there's a "Send a weekly summary digest" toggle on the Settings screen (Section 4.4.9). But even that flag is not anywhere read from and used to compose and send a periodic digest—it's simply stored and has no effect. This was not identified by the testing of Chapter 5 as no test case in Section 5.2 was designed around this feature and this is indeed a test design gap that should be recognized on its own.

No enforcement of scan authorisation technically. VulnLab does not introduce any technical measure to stop a user from entering a URL that they don't own or have permission to test (see Section 4.2.12), but rather documents the legal and ethical requirement to scan only authorised targets through the system prompt given to the AI assistant. This was done intentionally (section 1.5) and is not a mistake, but it is a true functionality void when compared to a production version of this system.

Only five vulnerability categories will be covered. VulnLab (as detailed in Section 1.5 and realised through OWASP ZAP (Section 4.2.2)) identifies: SQL injection, cross-site scripting, missing or misconfigured security headers, general misconfiguration, and basic authentication weaknesses. It does not attempt to detect missing or broken business-logic level access control, cross-site request forgery, insecure direct object references and other contextual authorisation vulnerabilities that survey respondents said they wish the scanner would improve (Section 3.3, Figure 74: "reducing false positives and improving detection accuracy for business logic and API-related vulnerabilities") — it's simply a limitation that comes straight from the underlying scanning engine and not particular to VulnLab's own code.

No password-reset flow. The authentication system (Section 4.2.12) permits registration, login and changing a password while currently logged in, but it does not include the most basic functionality of a real world deployment that is 'forgot password' for a user who has been locked out of their account.

No API rate limiting or abuse protection. There is no rate limiting on the Flask routes in app.py, so in theory a malicious or misconfigured client might be able to make an unlimited number of scan requests, logins or AI assistant queries over a short period of time, exhausting the ZAP daemon's quota, AI provider's usage limit, or SMTP relay's sending limits.

6.2.3 Testing Scope Limitations

Small UAT sample size. Chapter 5 satisfied the FYP documentation guidelines' minimum of three UAT testers, but three testers, however well matched to the target user groups defined in Section 1.6, cannot represent the full diversity of backgrounds, technical literacy levels, and use cases within those groups. A larger and more diverse tester pool in a future iteration would produce more statistically meaningful usability evidence than the consistent-but-small-sample result reported in Table 24.

No automated regression suite. The white-box and black-box test cases in Sections 5.2.1 and 5.2.2 were executed manually against a running instance of the system rather than committed as an automated pytest/vitest suite that could be re-run on every future code change. This means the correctness confirmed in Chapter 5 is a snapshot in time rather than a continuously enforced guarantee, and any future modification to, for example, scanner/risk_scoring.py could silently reintroduce a defect that WB-01 to WB-06 already disproved once, without any automatic warning.

No load or performance testing. All testing in Chapter 5 evaluated functional correctness and usability under normal, single-user conditions. No test case measured how the system behaves under concurrent load from multiple simultaneous users or multiple simultaneous Deep scans, which is directly related to the architectural limitation described in Section 6.2.1 and would need to be addressed together with it.

Single testing round. Every test case in Chapter 5 was executed once. A defect that only manifests intermittently — for example, a race condition in the in-memory job registry under unusual timing — would not reliably be caught by a single execution pass, and repeated execution was outside the time available for this stage of the project.

6.3 Recommendation

The recommendations below are organised to correspond directly to the limitations identified in Section 6.2, so that each proposed improvement can be read as a concrete response to a specific, evidenced gap rather than as a generic wish list for future work.

6.3.1 Recommendations Addressing Architectural Limitations

Replace the in-memory job registry with a durable task queue such as Celery or RQ backed by Redis, exactly as anticipated in the runner module's own design commentary (Section 4.2.2). This would allow scan jobs to survive a backend restart, allow the system to scale horizontally across multiple worker processes, and would not require any change to the REST API contract that the frontend already depends on, since the job_id/poll pattern described in Section 4.2.2 would remain identical from the client's point of view.

Introduce a message-based or webhook-driven trigger for scheduled scans as an alternative to the fixed sixty-second polling interval, to support use cases that may in future require finer scheduling precision than the current design provides.

Introduce a scan queue with configurable concurrency limits per ZAP daemon, allowing a future multi-user deployment to run several scans in parallel against separate ZAP instances or containers rather than serialising all scanning work through a single daemon.

6.3.2 Recommendations Addressing Feature Completeness

Introduce the weekly digest feature completely, including a new APScheduler job to go with _check_due_schedules() (section 4.2.9) to run each week, to query all users where weekly_digest is true, to gather all of their scans from the previous week, and to send a single email with all the scans to each of those users via the existing emailer.py infrastructure (section 4.2.10) — the database schema and setting API already support this feature end to end; only the sending logic now needs to be developed.

Implement an ownership-verification step for a scan to be allowed to run against a new target domain for the first time, similar to what Google Search Console and others do, such as a token added as a DNS TXT record or at a known file location, that moves the ownership-verification requirement mentioned in Section 4.2.12 from a documented expectation to a required control.

Expand the range of vulnerabilities covered beyond the five categories covered in scope (Section 1.5), particularly in terms of broken access control detection and API-specific vulnerabilities, in direct response to the survey feedback shown in figures 74-76 in Chapter 3. This would most likely involve a custom module that uses a built-in authenticated-crawling capability and a comparison of the roles, to complement the built-in logic of OWASP ZAP, as Section 2.1.2 already noted that signature based scanners are inherently unable to detect logic based access control without such a module.

Include a standard password-reset flow where a reset token is emailed to the user and will be valid for a short period of time that would involve relatively small new infrastructure.

Include basic rate limiting (e.g. using Flask-Limiter) to the authentication, scan-creation and AI-assistant routes, both to protect the third party API quotas and SMTP sending limits, and as a general security hardening measure appropriate for any system handling user credentials.

6.3.3 Recommendations Addressing Testing Scope

Turn these test cases from Sections 5.2.1 and 5.2.2 into an automated test suite (pytest for the backend, a browser-automation framework like Playwright for frontend black-box) that is integrated into a continuous-integration pipeline to have the correctness that was verified in Chapter 5 re-checked on every subsequent code change (manually not executed).

Include more UAT testers in the next evaluation round beyond the three testers included in this project, preferably more testers in each of the five target user groups listed in Section 1.6, to increase the statistical confidence of the usability conclusions drawn in Section 5.3.3.

Once the task-queue migration suggested in Section 6.3.1 is done, add some basic load tests (e.g. with a tool like Locust) to get real numbers on throughput and concurrency limits, instead of just relying on the unmeasured behavior of the system under load.

Re-run key white-box and black-box test cases in multiple runs, including those in which the test is timing sensitive and likely to produce intermittent failures versus deterministic failures, such as WB-12 (scheduler backlog handling) and BB-11 (scan cancellation).

Collectively, these recommendations outline a viable path forward from the working academic prototype assessed in this report to a system that could be more widely deployed, without necessarily being redesigned from scratch — indeed, each of the recommendations is an extension or hardening of the system's design, which is, in its entirety, a reflection of the soundness of the architectural decisions made during the design phase described in Chapter 4 — and without changing the core architecture or database schema or the risk-scoring algorithm tested in Chapter 4 and Chapter 5.

References

Abricto Security. (2023). Application security as a service vs. traditional penetration testing: A cost savings comparison. https://abrictosecurity.com/appsec-as-a-service-vs-traditional-pen-testing-a-cost-savings-comparison/

AlgoMaster. (2023). Designing a web crawler system: System design interview guide. https://blog.algomaster.io/p/design-a-web-crawler-system-design-interview

Allodi, L. (2015). The heavy tails of vulnerability exploitation. Engineering Secure Software and Systems.

Ambreena, L., Jain, M., Yadav, R. K., & Loonkar, S. (2024). Effective cybersecurity risk management practices for small and medium-sized enterprises: A comprehensive review. Multidisciplinary Reviews, 6.

Anas, A., Elgamal, S., & Youssef, B. (2024). Survey on detecting and preventing web application broken access control attacks. International Journal of Electrical and Computer Engineering, 14(1), 772–781.

Atlassian. (2025). What is the Waterfall methodology? Atlassian Agile Coach.

Balsam, A., Walkowski, M., Nowak, M., Oko, J., & Sujecki, S. (2025). Automatic CVSS-based vulnerability prioritization and response with context information and machine learning. Applied Sciences, 15, 8787.

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). Manifesto for Agile software development. https://agilemanifesto.org

Boehm, B. W. (1988). A spiral model of software development and enhancement. Computer, 21(5), 61–72.

Bozorgi, M., Saul, L. K., Savage, S., & Voelker, G. M. (2010). Beyond heuristics: Learning to classify vulnerabilities and predict exploits. Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 105–114. https://doi.org/10.1145/1835804.1835821

Chang, Y. Y., Zavarsky, P., Ruhl, R., & Lindskog, D. (2011). Trend analysis of the CVE for software vulnerability management. Proceedings of PASSAT Conference.

CheapSSL Security. (2023). What are the OWASP Top 10 vulnerabilities and how to mitigate them? https://cheapsslsecurity.com/blog/what-are-the-owasp-top-10-vulnerabilities-and-how-to-mitigate-them/

Halfond, W. G. J., & Orso, A. (2006). Preventing SQL injection attacks using AMNESIA. Proceedings of the International Conference on Software Engineering.

Heiding, F., Katsikeas, S., & Lagerström, R. (2023). Research communities in cyber security vulnerability assessments: A comprehensive literature review. Computer Science Review, 48, 100551.

IGM Guru. (2023). What is the CIA triad in cybersecurity? https://www.igmguru.com/blog/what-is-the-cia-triad

Iwata, T., Yamada, T., Sakurai, Y., & Ueda, N. (2010). Online multiscale dynamic topic models. Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.

Jiang, Y., & Atif, Y. (2022). Towards automatic discovery and assessment of vulnerability severity in cyber-physical systems. Array, 15, 100209.

Kia, A. N., Murphy, F., Sheehan, B., & Shannon, D. (2024). A cyber risk prediction model using common vulnerabilities and exposures. Expert Systems with Applications, 237, 121599.

Kirvan, P., Lutkevich, B., & Lewis, S. (2024). What is the Waterfall model? Definition and guide. TechTarget.

Li, Y., Han, M., Cao, A., & Pan, S. (2021). Research on web application vulnerability scanning technology based on active and passive combination.

Márquez, A. G., Varela-Vaca, Á. J., Gómez López, M. T., Galindo, J. A., & Benavides, D. (2024). Vulnerability impact analysis in software project dependencies based on satisfiability modulo theories. Computers & Security, 139, 103669.

OWASP Foundation. (2021). OWASP Top 10: The ten most critical web application security risks. https://owasp.org/www-project-top-ten/

Pressman, R. S., & Maxim, B. R. (2019). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill Education.

Rack, J., & Staicu, C. A. (2023). Jack-in-the-box: An empirical study of JavaScript bundling on the web and its security implications. Proceedings of the ACM Conference on Computer and Communications Security.

Ravindran, U., & Potukuchi, R. V. (2022). A review on web application vulnerability assessment and penetration testing. Review of Computer Engineering Studies, 9(1), 1–22.

Redscan. (2023). NIST National Vulnerability Database (NVD) analysis. https://www.redscan.com/news/nist-nvd-analysis/

Royce, W. W. (1970). Managing the development of large software systems. Proceedings of IEEE WESCON.

Sabottke, C., Suciu, O., & Dumitraş, T. (2015). Vulnerability disclosure in the age of social media: Exploiting Twitter for predicting real-world exploits. Proceedings of the 24th USENIX Security Symposium, 1041–1056.

Schwaber, K., & Sutherland, J. (2020). The Scrum Guide: The definitive guide to Scrum: The rules of the game. https://scrumguides.org

Seid, E., Busheva, R., Blix, F., & Popov, O. (2025). Advancing cybersecurity: Semi-automated penetration testing for enhanced vulnerability detection. Procedia Computer Science, 263, 350–358.

Sommerville, I. (2016). Software engineering (10th ed.). Pearson.

Spanning. (2022). Cross-site scripting (XSS): Web-based application security. https://www.spanning.com/blog/cross-site-scripting-web-based-application-security-part-3/

Spanning. (2022). SQL injection attacks: Web-based application security. https://www.spanning.com/blog/sql-injection-attacks-web-based-application-security-part-4/

Stackademic. (2023). Excelling in software development with Scrum methodology. https://blog.stackademic.com/excelling-in-software-development-with-scrum-methodology-part-2-e2d0b29437ce

TrustedSec. (2023). PCI DSS vulnerability management: The most misunderstood requirement. https://trustedsec.com/blog/pci-dss-vulnerability-management-the-most-misunderstood-requirement-part-2

United Nations. (2015). Goal 9: Industry, Innovation and Infrastructure. United Nations Sustainable Development. https://sdgs.un.org/goals/goal9

UseMotion. (2023). Waterfall model in software development. https://www.usemotion.com/blog/waterfall-model

Wassermann, G., & Su, Z. (2007). Sound and precise analysis of web applications for injection vulnerabilities. Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation.

Xu, D., Chen, T., Tan, Z., Wu, F., Gao, J., & Yang, Y. (2022). Web vulnerability detection analyzer based on Python. International Journal of Digital Crime and Forensics, 14(2).

Appendices

Project Title Proposal

Project Title Proposal

Project Title: Design and Implementation of a Continuous, Risk-Aware Automated Web Application Vulnerability Assessment System with Scheduled Scanning, Trend Analysis, Smart Scan Modes, and

Status

APPROVED

Proposed By

AHMED KHALID ABDUL BAKI AL MALALLA

Description

Introduction:
Web applications are widely used today to support services such as online banking, shopping, and communication, making their security increasingly important. As these applications evolve, new security weaknesses can be introduced, which are often missed by manual or one-time security checks. This project presents the design and implementation of a continuous, automated web application vulnerability assessment system that regularly scans applications for common security issues, evaluates their risk levels, monitors vulnerability trends over time, and delivers security reports through email. The system aims to provide a practical and proactive approach to managing web application security.

Aim :
The aim of this project is to improve the accessibility and effectiveness of web application security testing by providing a continuous, automated vulnerability assessment approach that delivers timely, clear, and actionable feedback to help identify and address security weaknesses.

Objectives:
1. To evaluate how proactive vulnerability notification supports faster response to critical security issues.
2. To assess changes in web application security posture over time through comparative analysis of vulnerability assessment results.
3. To analyze the trade-off between scan depth and assessment performance in automated web application security testing.
4.To investigate the effectiveness of continuous vulnerability assessment in detecting web application security weaknesses that may be missed by one-time security evaluations.

Target audience:
• Web application developers
• Small and medium-sized enterprises
• Junior security analysts and entry-level security professionals
• Organizations with limited cybersecurity resources

SDG

SDG9

Keywords

Risk Assessment
Network Vulnerability Analysis
Cybersecurity
Alert System
Cybersecurity
Network Vulnerability Analysis
Cybersecurity
Automation

Preferred Supervisor(s)

NORIS BINTI ISMAIL, , DR. SEYEDMOSTAFA SAFAVI, SHAHAB ALIZADEH, TS. UMAPATHY EAGANATHAN

Assigned Supervisor

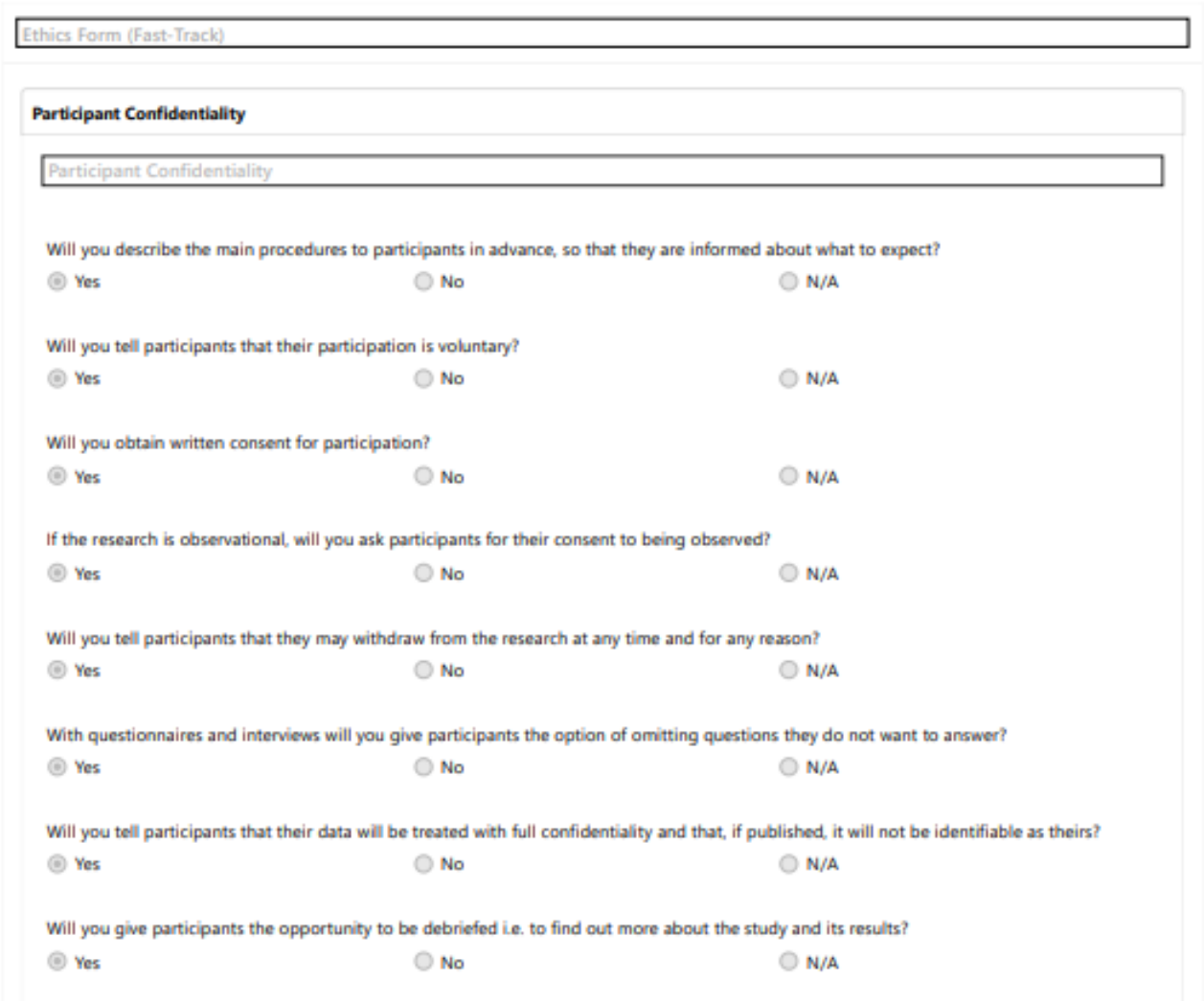
noris.ismail

Assigned Second Marker

nor_azlina

Figure 110 PPF

Ethics Form


Ethics Form (Fast-Track)

Participant Confidentiality

Participant Confidentiality

Will you describe the main procedures to participants in advance, so that they are informed about what to expect?
Yes No N/A

Will you tell participants that their participation is voluntary?
Yes No N/A

Will you obtain written consent for participation?
Yes No N/A

If the research is observational, will you ask participants for their consent to being observed?
Yes No N/A

Will you tell participants that they may withdraw from the research at any time and for any reason?
Yes No N/A

With questionnaires and interviews will you give participants the option of omitting questions they do not want to answer?
Yes No N/A

Will you tell participants that their data will be treated with full confidentiality and that, if published, it will not be identifiable as theirs?
Yes No N/A

Will you give participants the opportunity to be debriefed i.e. to find out more about the study and its results?
Yes No N/A

Figure 111 Ethics Form Page 1

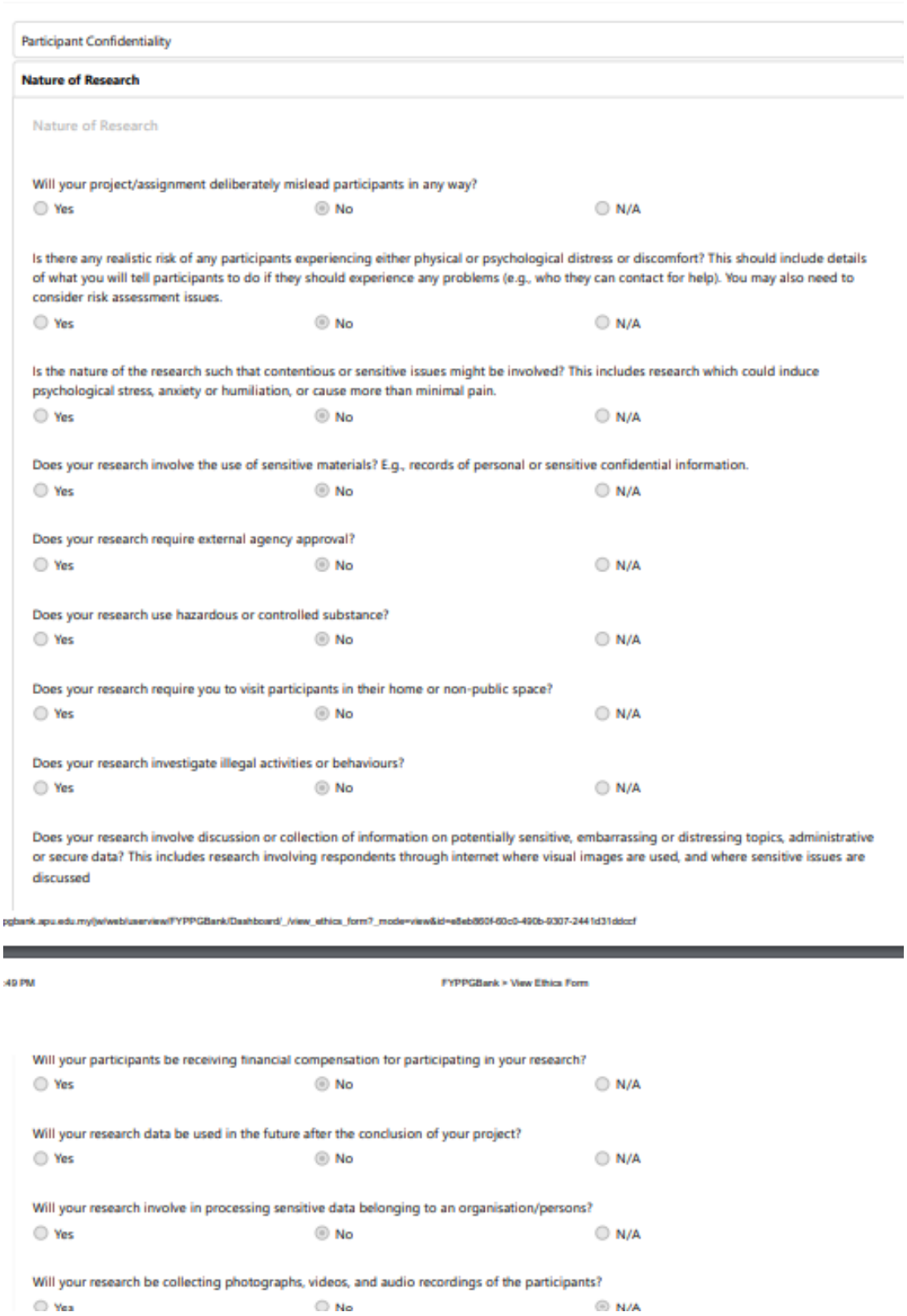


Figure 112 Ethics Form Page 2

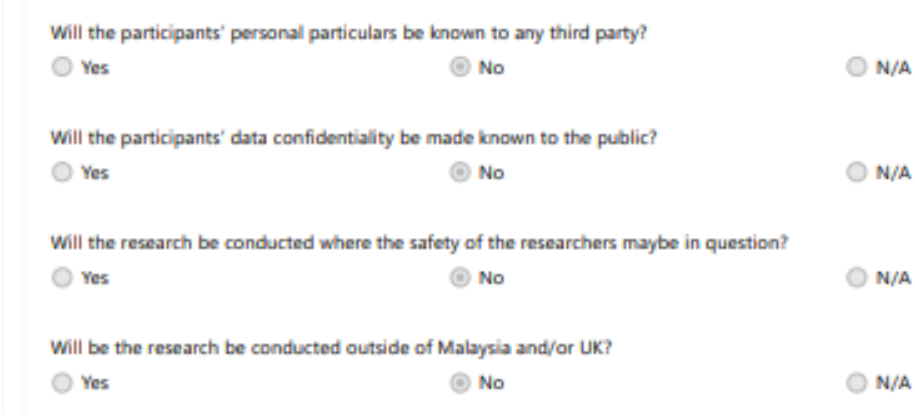


Figure 113 Ethics Form Page 3

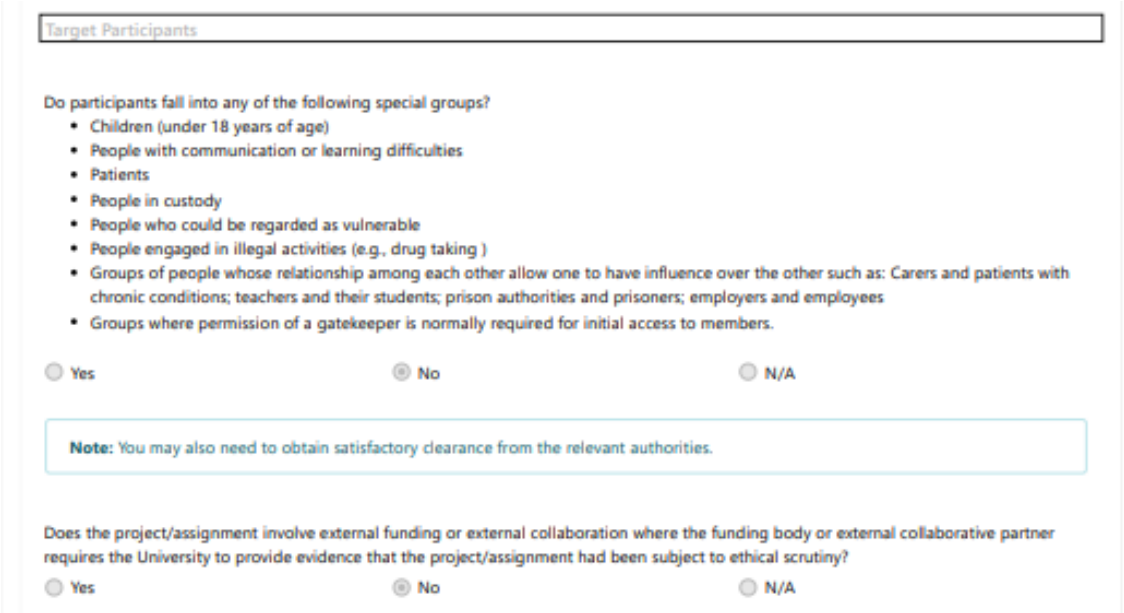


Figure 114 Ethics Form Page 4

Log Sheets

#	Supervisor Name	Content of Discussion	Date Created	Date Modified	Meeting Log Status	
1	noris.ismail	A meeting was held with the supervisor to review the progress of the project. Chapter 1 and Chapter 2 were discussed and reviewed. Feedback was provided, and it was noted that Chapter 2 requires some revisions and improvements. In addition, the structure and plans for Chapter 3 were discussed, including the approach to be taken for the methodology section.	01-03-2026 03:28 PM	10-03-2026 10:27 PM	PENDING APPROVAL	View
2	noris.ismail	A meeting was held with the supervisor to review the progress of Chapter 2 and Chapter 3. The improvements made to the literature review were discussed, and feedback was provided regarding the structure and clarity of the section. The supervisor suggested adding more relevant figures and diagrams to better support the explanations within the literature review. In addition, the methodology chapter was reviewed, and the approach for the survey was discussed. Guidance was provided on how the survey should be structured, how the questions should be designed, and how the collected responses should be analyzed and presented in the report.	10-03-2026 10:30 PM	10-03-2026 10:30 PM	PENDING APPROVAL	View
3	noris.ismail	A meeting was held with the supervisor to review the overall progress and structure of the Initial Report (IR). An overview of the complete report was presented, including the literature review, methodology, and supporting sections. Feedback was provided on the Similar Systems section and the inclusion of relevant figures to improve the clarity of the report. The supervisor also provided guidance on the preparation of the project plan and how it should be structured within the report. In addition, it was discussed that Chapter 4 should include the Sustainable Development Goal (SDG) alignment, specifically explaining how the project contributes to SDG 9: Industry, Innovation and Infrastructure. Further discussion covered the overall appearance and organization of the IR to ensure it is clearly structured and academically presented. Guidance was also provided regarding the appendices and supporting materials to be included at the end of the report.	10-03-2026 10:33 PM	10-03-2026 10:33 PM	PENDING APPROVAL	View

Figure 115 Meeting Log Sheet

Gantt Chart

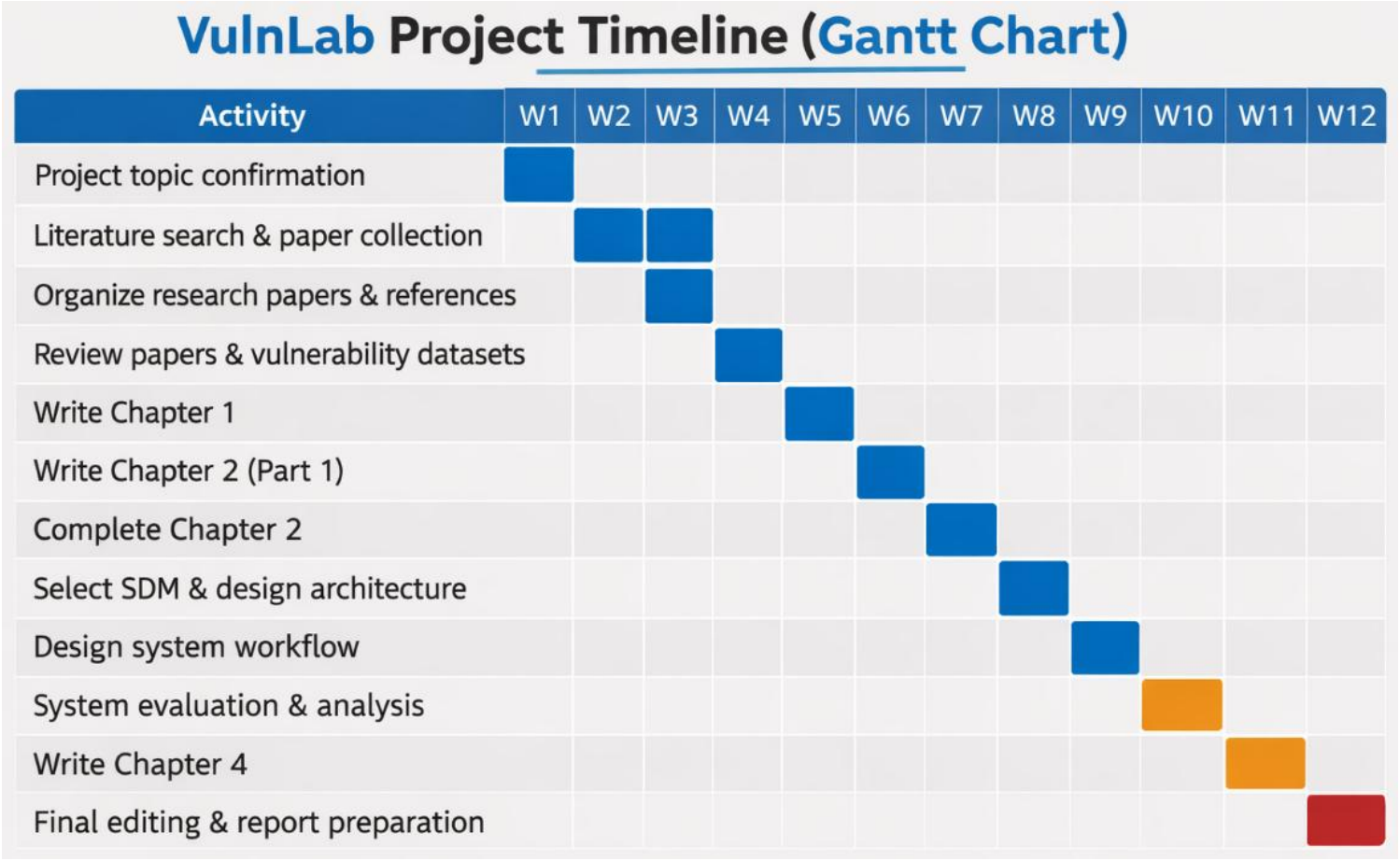


Figure 116 Project Timeline Gantt Chart

Respondence Demographic Profile

30 and above	Post Graduate	CYB educator
18-21	Undergraduate	Student
22-25	Undergraduate	Student
22-25	Undergraduate	Student
22-25	Undergraduate	Student
22-25	Undergraduate	Student
18-21	Diploma	Student
22-25	Undergraduate	Student
18-21	Undergraduate	Student
22-25	Undergraduate	Student
22-25	Undergraduate	Student
18-21	Undergraduate	Student
22-25	Undergraduate	Student
22-25	Undergraduate	Student
18-21	Undergraduate	Student
22-25	Undergraduate	Student
22-25	Undergraduate	Student
22-25	Undergraduate	Student
18-21	Undergraduate	IT/Cybersecurity Professional
18-21	Undergraduate	Student
18-21	Diploma	Student
22-25	Undergraduate	Student
30 and above	Post Graduate	IT/Cybersecurity Professional

30 and above	Post Graduate	IT/Cybersecurity Professional
26-29	Post Graduate	Developer
30 and above	Diploma	Business owner
26-29	Post Graduate	Business owner
22-25	Post Graduate	Student
22-25	Undergraduate	IT/Cybersecurity Professional
30 and above	Post Graduate	lecturer

Table 25 Respondence Demographic Profile

FINAL YEAR PROJECT 2026

VulnLab

Risk-Aware Automated Web Application Vulnerability Assessment System with Scheduled Scanning, Trend Analysis, Smart Scan Modes, and Email-Based Reporting

Ahmed Khalid Abdul Baki Al Malalla (TP067982)
APD3F2511CS(CYB)

Supervisor: Noris Binti Ismail
SENIOR LECTURER

2nd Marker: Nor Azlina Binti Abd Rahman
SENIOR LECTURER

BACHELOR OF SCIENCE (HONOURS) IN COMPUTER SCIENCE (CYBER SECURITY)

Introduction

Web application vulnerabilities such as SQL injection, cross-site scripting, and broken access control continue to threaten organisations, developers, and SMEs that often lack access to advanced security tooling. VulnLab is a continuous, risk-aware automated web application vulnerability assessment system that combines scheduled scanning, smart scan modes, risk-based prioritization, and trend analysis to make enterprise-grade security testing accessible. By pairing automated detection with clear, actionable email reports, VulnLab helps developers and small teams identify and prioritise the vulnerabilities that matter most — without requiring a dedicated security team.

Problem Statement

- **Escalating vulnerability landscape:** publicly disclosed vulnerabilities recorded in the National Vulnerability Database keep growing, widening the attack surface of modern web infrastructure.
- **Fragmented, one-off assessments:** testing is usually run as isolated exercises rather than continuous monitoring, so vulnerabilities introduced by updates go undetected between cycles.
- **Inaccessible commercial tooling:** established scanners are often too complex or expensive for students, researchers, and SMEs to adopt.
- **No risk-aware prioritization:** automated scanners return large volumes of findings with little guidance on which vulnerabilities pose real risk.
- **Overly technical reporting:** vulnerability reports are hard to interpret for developers without a dedicated security team.

Aims & Objectives

Aim: design and develop a continuous, risk-aware automated web application vulnerability assessment system that improves security monitoring and decision-making.

- Review existing scanners and risk-scoring models to identify gaps in continuous, prioritized assessment.
- Design and develop an automated scanning engine covering common vulnerabilities such as SQL injection, XSS, and broken access control.
- Implement risk-based prioritization, scheduled scanning, and trend analysis to track security posture over time.
- Provide smart scan modes and email-based reporting so findings are accessible to non-expert users.

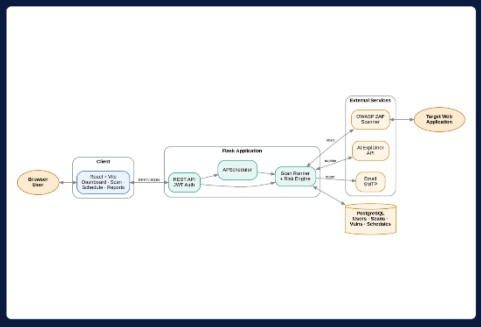
Core System: Risk-Aware Scanning Pipeline

Scheduling & Scan Modes	Vulnerability Detection	Risk Scoring	Trend Analysis	Reporting
Quick · Deep · Targeted	SQLi · XSS · Broken Access Control · Misconfiguration	CVSS base score + Likelihood × Impact	Historical scans · Posture over time	Prioritised findings · Email delivery

User Interface



System Architecture



Conclusion

This phase of VulnLab established a strong foundation for a continuous, risk-aware vulnerability assessment system. A literature review identified recurring weaknesses in existing scanners — high false-positive rates, poor risk prioritization, and rigid scanning — while a user survey confirmed these pain points among practitioners. These findings directly shaped VulnLab's design: automated scanning paired with risk-based scoring, scheduled and trend-aware monitoring, smart scan modes, and clear email reporting. This lays the groundwork for developing and evaluating the proposed system in the next phase, aligning with Sustainable Development Goal 9: Industry, Innovation and Infrastructure.

Figure 117 FYP poster